Java Frameworks (Spring and Hibernate)

Where Java Becomes Enterprise-Ready

You can write a REST API in raw Servlets or JDBC - but when your system grows, you'll quickly hit walls:

- Too much boilerplate.
- Hard-to-manage dependencies.
- No unified transaction control.
- A spaghetti mess of database code.

That's exactly why **Spring and Hibernate** exist.

They took what was painful in traditional Java web development and built **a complete ecosystem** around simplicity, scalability, and maintainability.

I still remember my first migration from raw Servlets to Spring MVC - it felt like magic. Suddenly, dependency injection handled object creation, transactions were declarative, and REST controllers were just a few annotations away.

But that magic only makes sense when you know how it works inside. Because when Spring fails, it fails hard - and debugging a misconfigured bean or lazy-loading exception in Hibernate can break your weekend.

This chapter teaches you what every real backend engineer needs to know - **how frameworks work under the hood** and how to use them like a pro, not a beginner.

What You'll Master in This Chapter

- **Spring Framework Basics**

  → Dependency Injection (IoC), ApplicationContext, and Bean lifecycle.
- **Aspect-Oriented Programming (AOP)**

  → How cross-cutting concerns like logging, transactions, and security are handled cleanly.
- **Spring Boot**

  → Opinionated configuration, embedded servers (Tomcat/Jetty), and auto-configuration magic.
- **Creating REST APIs with Spring Boot**

  → @RestController, @GetMapping, @PostMapping, and handling requests/responses elegantly.
- **Hibernate ORM**

  → Object-Relational Mapping, entity management, and how Hibernate connects objects to database tables.
- **Spring Data JPA**

  → Repository abstraction, pagination, query derivation, and how it simplifies data access.
- **Transaction Management**

  → Declarative transactions using @Transactional and how rollback/commit works behind the scenes.
- **Best Practices for Frameworks in Production**

  → Profiles, configuration management, error handling, and lazy vs eager loading strategies.

Famous Free Documents & Resources

1. **Spring Core Concepts**
    - Spring Official Docs – Core Overview
    - Baeldung – Spring IoC Container
    - GeeksforGeeks – Spring Framework Basics
2. **Spring Boot and REST APIs**
    - Spring Official Guide – Building a RESTful Service
    - Baeldung – Spring Boot Tutorial
    - Programiz – Spring Boot Basics
3. **Hibernate ORM**
    - Hibernate Official Docs – Getting Started
    - Baeldung – Hibernate Annotations
    - GeeksforGeeks – Hibernate ORM Explained
4. **Spring Data JPA**
    - Spring Docs – Spring Data JPA Reference
    - Baeldung – Spring Data JPA Repositories
5. **Transaction Management & AOP**
    - Baeldung – @Transactional Deep Dive
    - Spring Docs – AOP Concepts

Expert Notes & Gotchas

- **Spring's core power is Inversion of Control (IoC).**

  You don't "create" objects - you "declare" them, and Spring wires them automatically.

  That's how dependency injection eliminates tight coupling.
- **Every annotation you use (@RestController, @Service, @Repository) is built on Reflection + Metadata.**

  Understanding Chapter 8 (Annotations & Reflection) will help you master this.
- **Avoid "field injection" (@Autowired on fields)** - prefer constructor injection.

  It's cleaner, testable, and avoids proxy issues.
- **Hibernate's LazyInitializationException** happens when you try to access an uninitialized relationship outside the session.

  Solution: Use DTOs, fetch join, or OpenSessionInView (carefully).
- **@Transactional is powerful but tricky**
  - It only works on public methods.
  - Self-invocation skips proxy-based interception.
  - Use it at the service layer, not controllers.
- **Spring Boot Auto-Configuration ≠ Magic.**

  It uses classpath scanning and conditional beans (@ConditionalOnClass, @ConditionalOnMissingBean) to wire things intelligently.
- **Use Profiles wisely.**

  application-dev.yml vs application-prod.yml - separate config, same code.
- **Hibernate Tip:**

  Always use batch fetching or pagination for large datasets.

  Fetching 100K records in one go is the fastest way to OOM a JVM.

Example: REST API with Spring Boot and Hibernate

```
@Entity
public class Product {
   @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String name;
   private double price;
}

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {}

@Service
public class ProductService {
   private final ProductRepository repo;
   public ProductService(ProductRepository repo) { this.repo = repo; }

   public List<Product> getAllProducts() { return repo.findAll(); }
   public Product saveProduct(Product p) { return repo.save(p); }
}

@RestController
@RequestMapping("/api/products")
public class ProductController {
   private final ProductService service;
   public ProductController(ProductService service) { this.service = service; }

   @GetMapping
   public List<Product> getAll() { return service.getAllProducts(); }

   @PostMapping
   public Product save(@RequestBody Product p) { return service.saveProduct(p); }
}
```

What this demonstrates:

- Entity management via Hibernate.
- Repository abstraction (no boilerplate JDBC).
- Clean REST API layer using Spring Boot annotations.
- Constructor-based dependency injection - testable and clean.

Once you master Spring and Hibernate, you've essentially mastered **the language of enterprise Java.**
From microservices to monoliths, everything relies on these frameworks.

Question Categorization Overview

To make this guide more practical for different experience levels and roles, I've categorized every question in this chapter based on:

1. **Experience Level (L1–L4)** – representing how deep the understanding required is.
2. **Role Focus** – identifying which job profile or area the question is most relevant to.

This way, you can easily filter and prepare according to your **career stage** and **target role**.

Level	Experience Range	Description
L1	0–2 years	Foundational Java concepts; syntax, data types, control flow, OOP basics
L2	2–5 years	Intermediate backend engineer; memory, exceptions, collections, concurrency, and frameworks
L3	5–9 years	Senior engineer; architecture, performance, multithreading, and optimization
L4	9+ years	Tech Lead / Architect; deep JVM internals, design trade-offs, scalability, and tuning

Role	Focus Area
Backend Core	Core Java, Collections, Multithreading, Exception Handling
Full-Stack Developer	Java backend + some frontend integration (REST APIs, JSON, etc.)
Microservices Engineer	Spring Boot, REST, concurrency, distributed systems
System Design Engineer	Architecture, caching, GC tuning, concurrency
Data/Integration Engineer	File I/O, serialization, streaming, data handling
Security Engineer	Encryption, transient fields, safe serialization

How to Use This Categorization

- **If you're a beginner:** Focus on **L1 questions** – they form the foundation for every backend interview.
- **If you're preparing for mid-level roles (SDE-1/2):** Prioritize **L2 questions**, especially on collections, exceptions, and concurrency.
- **If you're moving toward senior roles:** Practice **L3–L4** – JVM tuning, concurrency patterns, and architectural decisions.
- **If you're targeting a specific role (like Microservices or Security):** Use the *Role Focus* column to zero in on your domain.

1. Core Java Fundamentals (1 - 44)

#	Question (Short)	Level	Role Focus
1	What is Java & why popular	L1	Backend Core
2	JVM vs JRE vs JDK	L1	Backend Core
3	Data types & why	L1	Backend Core
4	Variables & naming rules	L1	Backend Core
5	Operators & types	L1	Backend Core
6	Purpose of control statements	L1	Backend Core
7	if–else with example	L1	Backend Core, Full-Stack
8	switch & when to use	L1	Backend Core
9	for loop	L1	Backend Core
10	while loop	L1	Backend Core
11	do-while & diff from while	L1	Backend Core
12	break (usage)	L1	Backend Core
13	continue (usage)	L1	Backend Core
14	Arrays in Java	L1	Backend Core
15	Strings & immutability	L1	Backend Core
16	StringBuilder vs StringBuffer	L2	Backend Core
17	Wrapper Classes	L1	Backend Core
18	Autoboxing & Unboxing	L2	Backend Core
19	Access Modifiers	L1	Backend Core
20	Static keyword	L2	Backend Core
21	Final keyword	L2	Backend Core
22	this vs super	L2	Backend Core
23	Method Overloading vs Overriding	L2	Backend Core
24	Constructors & types	L1	Backend Core
25	Inheritance	L1	Backend Core
26	Encapsulation	L1	Backend Core
27	Polymorphism	L1	Backend Core
28	Abstraction	L1	Backend Core
29	Interfaces & default methods	L2	Backend Core
30	Abstract classes	L1	Backend Core
31	Object class methods	L2	Backend Core
32	equals() vs ==	L2	Backend Core
33	hashCode() importance	L2	Backend Core
34	Exception handling hierarchy	L2	Backend Core
35	Checked vs Unchecked Exceptions	L2	Backend Core
36	try-catch-finally	L1	Backend Core
37	throw vs throws	L2	Backend Core

38	Custom exceptions	L2	Backend Core
39	Packages & imports	L1	Backend Core
40	Java 8 features overview	L1	Backend Core
41	Lambda expressions	L2	Backend Core
42	Functional Interfaces	L2	Backend Core
43	Stream API	L2	Backend Core
44	Optional class	L2	Backend Core

2. Java Advanced & OOP Deep Dive (45–75)

#	Question (Short)	Level	Role Focus
45	Generics in Java	L2	Backend Core
46	Collections hierarchy	L2	Backend Core
47	List vs Set vs Map	L2	Backend Core
48	HashMap vs Hashtable	L2	Backend Core
49	ConcurrentHashMap	L3	Advanced Backend
50	Fail-fast vs Fail-safe	L3	Advanced Backend
51	Comparator vs Comparable	L2	Backend Core
52	Java Streams intermediate ops	L3	Backend Core
53	allMatch, anyMatch, noneMatch	L2	Backend Core
54	Optional usage	L2	Backend Core
55	Predicate, Function, Supplier	L2	Backend Core
56	Default & Static in interfaces	L2	Backend Core
57	Immutable classes	L3	Backend Core
58	Deep copy vs shallow copy	L3	Backend Core
59	Serialization & Deserialization	L2	Backend Core
60	transient keyword	L2	Backend Core
61	volatile keyword	L3	Concurrency
62	synchronized & locks	L3	Concurrency
63	Executor framework	L3	Advanced Backend
64	Callable vs Runnable	L2	Concurrency
65	Thread lifecycle	L2	Concurrency
66	wait(), notify(), notifyAll()	L3	Concurrency
67	Deadlock, Livelock	L3	Advanced Backend
68	ThreadLocal	L3	Advanced Backend
69	CompletableFuture	L3	Backend Core
70	ForkJoinPool	L3	Advanced Backend
71	ReentrantLock	L3	Advanced Backend
72	Atomic classes	L3	Advanced Backend
73	JVM memory areas	L3	Performance
74	Garbage Collection	L3	Performance
75	ClassLoader mechanism	L3	Advanced Backend

3. Spring Framework (76–125)

#	Question (Short)	Level	Role Focus
76	What is Spring Framework	L1	Backend Core
77	Inversion of Control	L1	Backend Core
78	Dependency Injection	L1	Backend Core
79	@Component, @Service, @Repository	L2	Backend Core
80	@Autowired usage	L2	Backend Core
81	Bean lifecycle	L2	Backend Core
82	@Bean annotation	L2	Backend Core
83	@Configuration	L2	Backend Core
84	@Qualifier	L2	Backend Core
85	ApplicationContext	L2	Backend Core
86	XML vs Annotation config	L2	Backend Core
87	Singleton vs Prototype scope	L2	Backend Core
88	Lazy Initialization	L2	Backend Core
89	@Value and @PropertySource	L2	Backend Core
90	AOP in Spring	L3	Advanced Backend
91	@Aspect, @Around	L3	Advanced Backend
92	Transaction Management	L3	Advanced Backend
93	@Transactional	L3	Backend Core
94	Exception Handling in Spring	L2	Backend Core
95	@ControllerAdvice	L2	Backend Core
96	@RestController	L1	Backend Core
97	@ResponseBody vs @RequestBody	L2	Backend Core
98	@PathVariable vs @RequestParam	L2	Backend Core
99	Spring MVC Flow	L2	Backend Core
100	DispatcherServlet	L2	Backend Core
101	ViewResolver	L2	Backend Core
102	ModelAndView	L1	Backend Core
103	ExceptionResolver	L2	Backend Core
104	Spring Filters	L2	Backend Core
105	Spring Security overview	L3	Security
106	CSRF, XSS Protection	L3	Security
107	OAuth2 in Spring	L3	Security
108	JWT Integration	L3	Backend Core
109	Spring Boot Starter	L1	Backend Core
110	Auto-Configuration	L2	Backend Core
111	application.properties	L1	Backend Core
112	Spring Boot CLI	L1	Backend Core

113	Embedded Server (Tomcat)	L2	Backend Core
114	Profiles & Environments	L2	Backend Core
115	Logging (SLF4J)	L2	Backend Core
116	Actuator & Health Checks	L3	DevOps
117	Custom Starter	L4	Advanced Backend
118	Conditional Beans	L3	Advanced Backend
119	Meta Annotations	L3	Advanced Backend
120	DevTools	L1	Backend Core
121	Externalized Config	L2	Backend Core
122	Exception Handling in Boot	L2	Backend Core
123	REST Template vs WebClient	L3	Microservices
124	Feign Client	L3	Microservices
125	Circuit Breaker Pattern	L3	Microservices

4. Cloud & Microservices with Spring Boot (126–175)

#	Question (Short)	Level	Role Focus
126	What is Spring Boot and why it was introduced	L1	Backend Core
127	Spring Boot Architecture overview	L2	Backend Core
128	Spring Boot Starters and Auto-Configuration flow	L2	Backend Core
129	How Dependency Injection works in Boot	L2	Backend Core
130	What is SpringApplication.run()	L1	Backend Core
131	@SpringBootApplication annotation internals	L2	Backend Core
132	Embedded Servers (Tomcat, Jetty, Undertow)	L2	Backend Core
133	Starter Dependency Resolution	L2	Backend Core
134	Spring Boot DevTools and Hot Reload	L1	Backend Core
135	Spring Profiles and Environment Configs	L2	Backend Core
136	Externalized Configuration – .properties / .yml	L2	Backend Core
137	@ConfigurationProperties vs @Value	L2	Backend Core
138	CommandLineRunner and ApplicationRunner	L2	Backend Core
139	REST APIs in Spring Boot	L1	Backend Core
140	Exception Handling in Boot (@RestControllerAdvice)	L2	Backend Core
141	Validation with Hibernate Validator (@Valid)	L2	Backend Core
142	Logging (SLF4J + Logback)	L2	Backend Core
143	Spring Boot Testing (@SpringBootTest, MockMvc)	L2	Backend Core
144	REST Template vs WebClient	L3	Microservices
145	Reactive Programming with Spring WebFlux	L3	Microservices
146	Building a Reactive REST API with Mono/Flux	L3	Microservices
147	Difference between Reactive and Blocking IO	L3	Microservices
148	Spring Data JPA overview	L2	Backend Core
149	JpaRepository vs CrudRepository	L2	Backend Core
150	Derived Query Methods (findBy...)	L2	Backend Core
151	@Query and Native Query	L2	Backend Core
152	Pagination and Sorting (Pageable)	L2	Backend Core
153	EntityManager and PersistenceContext	L2	Backend Core
154	Transactions (@Transactional)	L2	Backend Core
155	Transaction Propagation Types	L3	Advanced Backend
156	Exception Handling in JPA	L2	Backend Core
157	Database Connection Pooling (HikariCP)	L2	Backend Core
158	Spring Boot + MySQL/PostgreSQL setup	L1	Backend Core
159	Spring Boot Actuator and Metrics	L3	DevOps
160	Custom Actuator Endpoints	L3	DevOps
161	Spring Boot Security basics	L2	Security

162	Authentication vs Authorization	L2	Security
163	Implementing Basic Auth & Form Login	L2	Security
164	JWT Authentication in Spring Boot	L3	Security
165	OAuth2 and OpenID Connect	L3	Security
166	Spring Cloud Config Server	L3	Microservices
167	Eureka Service Registry and Discovery	L3	Microservices
168	Feign Client for Inter-Service Calls	L3	Microservices
169	API Gateway (Spring Cloud Gateway)	L3	Microservices
170	Circuit Breaker with Resilience4J	L3	Microservices
171	Distributed Tracing (Sleuth + Zipkin)	L3	Microservices
172	Centralized Logging (ELK Stack)	L3	DevOps
173	Configuring Async Communication (Kafka, RabbitMQ)	L3	Microservices
174	Dockerizing a Spring Boot App	L3	DevOps
175	Deploying Spring Boot on Cloud (AWS / Azure / GCP)	L3	DevOps

5. Hibernate / JPA Core (176 – 225)

#	Question (Short)	Level	Role Focus
176	What is JPA and how Hibernate implements it	L1	Backend Core
177	ORM concept explained	L1	Backend Core
178	Entity Life Cycle states in JPA	L2	Backend Core
179	@Entity and @Id annotations	L1	Backend Core
180	@GeneratedValue strategies	L2	Backend Core
181	@Column and @Table	L1	Backend Core
182	@Transient and @Enumerated	L2	Backend Core
183	@Temporal and LocalDate mapping	L2	Backend Core
184	@Embeddable and @Embedded	L2	Backend Core
185	One-to-One mapping	L2	Backend Core
186	One-to-Many and Many-to-One mapping	L2	Backend Core
187	Many-to-Many mapping	L2	Backend Core
188	Cascade types in JPA	L2	Backend Core
189	Fetch types (LAZY vs EAGER)	L2	Backend Core
190	Orphan Removal vs CascadeType.REMOVE	L2	Backend Core
191	Entity Manager and Persistence Context	L2	Backend Core
192	JPQL basics	L2	Backend Core
193	Criteria Builder API	L3	Advanced Backend
194	Native SQL Queries	L2	Backend Core
195	Pagination and Sorting with JPA	L2	Backend Core
196	@NamedQuery vs Dynamic Query	L2	Backend Core
197	Parameter Binding in Queries	L2	Backend Core
198	Projection and DTO Queries	L2	Backend Core
199	Criteria vs JPQL vs Native SQL	L3	Advanced Backend
200	First-Level Cache (Session Cache)	L2	Backend Core
201	Second-Level Cache overview	L2	Backend Core
202	@Cacheable annotation	L2	Backend Core
203	N+1 Problem and Fetch Joins	L3	Performance
204	Dirty Checking and Flush Modes	L3	Advanced Backend
205	Persistence Context Propagation	L3	Advanced Backend
206	Transaction Management with JPA	L2	Backend Core
207	@Transactional and rollback rules	L2	Backend Core
208	Entity Listeners (@PrePersist, @PostUpdate)	L3	Advanced Backend
209	Optimistic Locking (@Version)	L3	Advanced Backend
210	Pessimistic Locking (LockModeType)	L3	Advanced Backend
211	Fetch Graphs and Entity Graphs	L3	Advanced Backend
212	@SQLDelete and @Where for Soft Deletes	L2	Backend Core

213	Audit Fields (createdAt, updatedAt)	L2	Backend Core
214	@MappedSuperclass vs @Inheritance	L3	Advanced Backend
215	Single Table vs Joined vs Table-per-Class	L3	Advanced Backend
216	Batch Insert and Update Basics	L3	Performance
217	StatelessSession for bulk ops	L3	Data Engineering
218	Query Cache and Region Config	L3	Performance
219	Schema Auto-Generation (hbm2ddl.auto)	L2	Backend Core
220	Logging SQL Statements	L2	Backend Core
221	@JoinColumn and @JoinTable	L2	Backend Core
222	LazyInitializationException handling	L3	Advanced Backend
223	Detached vs Persistent Entities	L2	Backend Core
224	Entity Lifecycle Callbacks	L2	Backend Core
225	JPA Best Practices for Production	L3	Advanced Backend

6. Hibernate Advanced & Production (226 – 301)

#	Question (Short)	Level	Role Focus
Batch Processing & Performance (226 – 235)			
226	Optimize batch inserts / updates in Hibernate	L3	Backend Core, Performance
227	Event Listeners vs Interceptors	L3	Advanced Backend
228	Second-Level Cache Providers overview	L3	Advanced Backend, Microservices
229	Fetch Joins to avoid N + 1 queries	L2	Backend Core
230	Criteria API and dynamic queries	L2	Backend Core
231	Using Native SQL with Hibernate	L2	Backend Core
232	Query Caching mechanism	L3	Backend Core, Performance
233	Enable and manage SQL logging	L1	Backend Core
234	StatelessSession use cases	L3	Data Engineering / Batch Jobs
235	Batch Fetching optimization	L2	Backend Core, Performance
Event System / Auditing / Filters (236 – 241)			
236	Hibernate Event System and callbacks	L3	Advanced Backend
237	Interceptors vs Listeners difference	L3	Advanced Backend
238	Multi-Tenancy strategies	L4	Architect, SaaS Design
239	Hibernate Envers for auditing	L3	Backend Core, Compliance
240	Implement Soft Deletes (@SQLDelete, @Where)	L2	Backend Core
241	Hibernate Filters and use cases	L3	Backend Core, Multi-Tenant
Statistics / Caching / Optimization (242 – 250)			
242	Hibernate Statistics API	L3	Performance, DevOps
243	Caching Best Practices	L3	Advanced Backend
244	Batch Fetching vs N + 1 problem	L2	Backend Core
245	StatelessSession (lightweight session)	L3	Data ETL Jobs
246	Fetch Joins and performance	L2	Backend Core
247	Entity Lifecycle States	L2	Backend Core
248	Dirty Checking mechanism	L2	Backend Core
249	Automatic Schema Generation (hbm2ddl.auto)	L1	Backend Core
250	Hibernate Proxies & Lazy Loading	L2	Backend Core
Caching / Integration / Advanced Queries (251 – 259)			
251	Query Caching in Hibernate	L3	Backend Core, Performance
252	Executing Native SQL Queries	L2	Backend Core
253	Integrating Hibernate with Spring Data JPA	L2	Backend Core
254	Connection Pooling (HikariCP, C3P0)	L2	Backend Core
255	Multi-Tenancy implementation example	L4	SaaS Architect
256	Interceptors use cases (auditing, logging)	L3	Advanced Backend
257	Event Listeners vs Interceptors (differences)	L3	Advanced Backend
258	Hibernate Session vs JPA EntityManager	L2	Backend Core
259	Hibernate Validator (Bean Validation)	L1	Backend Core

Fetching / Concurrency / Locking (260 – 268)			
260	Lazy vs Eager vs Subselect fetching	L2	Backend Core
261	Named Queries usage and benefits	L2	Backend Core
262	Criteria API vs JPQL vs Native SQL	L2	Backend Core
263	Optimistic vs Pessimistic Locking	L3	Advanced Backend
264	Interceptors vs Event Listeners comparison	L3	Advanced Backend
265	Second-Level Cache providers comparison	L3	Performance Tuning
266	Batch Fetching vs Fetch Joins	L2	Backend Core
267	Query Cache vs L2 Cache	L3	Advanced Backend
268	Pros & Cons of Native SQL	L2	Backend Core
Advanced Operations / Spring Integration (269 – 277)			
269	Named vs Dynamic Queries	L2	Backend Core
270	StatelessSession usage and limitations	L3	Data Engineering
271	Proxies & LazyInitializationException	L3	Backend Core
272	Event Listeners and Interceptors overview	L3	Advanced Backend
273	Multi-Tenancy Strategies (DB/Schema/Discriminator)	L4	Architect, SaaS
274	Batch Processing (Session vs StatelessSession)	L3	Performance Optimization
275	Fetch Plans and Performance Optimization	L3	Backend Core, Performance
276	Second-Level Cache Providers (Ehcache, Redis, Hazelcast)	L3	Advanced Backend
277	Hibernate + Spring Data JPA integration benefits	L2	Backend Core
Production / Reactive / Microservices (278 – 283)			
278	Performance Tuning Best Practices	L3	Advanced Backend
279	Hibernate in Microservices architecture	L4	Architect, Cloud
280	Hibernate Reactive and R2DBC	L4	Modern Backend
281	Hibernate Envers auditing example	L3	Compliance, Audit
282	Interceptors and Listeners for lifecycle events	L3	Advanced Backend
283	Integrating Hibernate with Legacy Databases	L3	Migration, Backend
Database Migrations / Tenancy / DTO (284 – 289)			
284	Hibernate with Flyway or Liquibase migrations	L3	DevOps, Backend
285	Multi-Tenancy support overview	L4	SaaS Architect
286	Implementing Soft Deletes (@SQLDelete @Where)	L2	Backend Core
287	Working with Stored Procedures	L3	Advanced Backend
288	DTO Projections in Hibernate	L2	Backend Core
289	Batch Fetching vs JOIN FETCH comparison	L2	Backend Core
Caching Strategies / Auditing / Best Practices (290 – 301)			
290	Natural IDs and usage	L2	Backend Core
291	Envers Auditing deep-dive	L3	Compliance
292	Cache Concurrency Strategies (READ_ONLY, READ_WRITE...)	L3	Advanced Backend
293	Multi-Tenancy revisited (architectural design)	L4	SaaS Architect
294	Interceptors & Event Listeners practical use	L3	Advanced Backend
295	JPA vs Native Hibernate APIs	L3	Advanced Backend
296	StatelessSession (high-performance mode)	L3	Performance
297	Query Hints and optimizations	L3	Performance

298	Performance Tuning Best Practices (summary)	L3	Advanced Backend
299	Envers Auditing use cases	L3	Compliance
300	Natural IDs best practices	L2	Backend Core
301	Hibernate Projections overview	L2	Backend Core

#	Question (Short)	Level	Role Focus
Projections & Criteria (301–307)			
301	What are Projections in Hibernate and when are they used	L2	Backend Core, Performance
302	Hibernate Event Listeners and how to use them	L3	Advanced Backend, Auditing
303	Hibernate Statistics API – purpose and usage	L3	Performance, Monitoring
304	Hibernate Validation and integration with JPA	L2	Backend Core, Spring Boot
305	Using Native SQL queries in Hibernate	L2	Backend Core, Data Engineering
306	Differences between Criteria API and JPQL	L2	Backend Core
307	Hibernate Performance Tuning Best Practices	L3	Advanced Backend, Architect
Session, Interceptors & Event System (308–312)			
308	Session vs StatelessSession	L3	Data Engineering, Bulk Ops
309	Interceptors vs Event Listeners	L3	Advanced Backend
310	Multi-Tenancy in Hibernate (DB, Schema, Discriminator)	L4	SaaS Architect
311	Hibernate Envers for Auditing	L3	Compliance, Banking
312	UserType vs AttributeConverter	L3	Backend Core, Custom Types
Integration & Best Practices (313–317)			
313	Hibernate Best Practices with Spring Boot	L2	Backend Core, Spring Integration
314	Hibernate with NoSQL (Hibernate OGM)	L4	Cloud Architect, Polyglot Persistence
315	DTO Projections and their benefits	L2	Backend Core, Performance
316	Hibernate with QueryDSL for Dynamic Queries	L3	Backend Core, Enterprise Apps
317	Hibernate Multitenancy in SaaS Apps	L4	SaaS Architect
DB Operations & Stored Procedures (318–320)			
318	Hibernate with Stored Procedures	L3	Enterprise Backend
319	Event Listeners vs Interceptors differences	L3	Advanced Backend
320	Dirty Checking mechanism	L2	Backend Core
DTO, Envers & JSON (321–324)			
321	DTO Projections in Hibernate	L2	Backend Core
322	Hibernate Envers for Auditing and History	L3	Compliance, Audit
323	Mapping JSON Columns	L3	Modern Backend
324	Streaming & Pagination for Large Datasets	L3	Data Engineering
Testing & Schema Evolution (325–328)			
325	Testing Hibernate Apps (H2, Testcontainers)	L2	QA, Integration
326	Multi-Tenancy Implementation	L4	SaaS Architect
327	Hibernate with Flyway/Liquibase	L3	DevOps, Production

328	Hibernate in Microservices Architecture	L3	Microservices, Event-driven Design
Performance & Caching (329–333)			
329	Hibernate Performance Pitfalls & Fixes	L3	Performance Optimization
330	Batch Processing Tuning	L3	Data Engineering
331	Analyzing Slow Hibernate Queries	L3	Advanced Backend
332	Effective Caching in Hibernate	L3	Backend Core, Performance
333	Fetch Profiles, Entity Graphs & DTO Projections	L3	Advanced Backend
Multi-tenancy & Stored Procedures (334–335)			
334	Hibernate Multi-Tenancy (DB/Schema/Discriminator)	L4	SaaS Architect
335	Calling Stored Procedures	L3	Enterprise Backend
Cache & Interceptors (336–341)			
336	Caching Concurrency Strategies	L3	Performance, Production
337	Role of Interceptors & Event Listeners	L3	Advanced Backend
338	Hibernate Tuning for High-Performance Systems	L3	Architect
339	Hibernate Integration with Spring Boot	L2	Backend Core
340	Auditing with Hibernate Envers	L3	Compliance
341	Interceptor vs EventListener Differences	L3	Advanced Backend
Schema Evolution & Stateless Sessions (342–343)			
342	Hibernate + Flyway/Liquibase	L3	DevOps
343	StatelessSession and its Use Cases	L3	Data Engineering, Bulk Processing
Reactive & Connection Pooling (344–345)			
344	Hibernate Reactive Overview	L4	Cloud Architect
345	Connection Pooling Best Practices (HikariCP)	L3	Performance, DevOps
Event-Driven & Thread Safety (346–347)			
346	Hibernate Integration with Kafka (Transactional Outbox)	L4	Microservices Architect
347	Hibernate Multi-threading Pitfalls & Solutions	L3	Backend Core
Fetching & Microservices (348–349)			
348	Fetch Joins vs Subselects vs Projections	L3	Performance
349	Hibernate Best Practices in Microservices	L3	Cloud, Architect
Monitoring & Diagnostics (350–353)			
350	Debugging Hibernate Performance in Production	L3	DevOps, Monitoring
351	Interceptors vs Event Listeners Differences	L3	Advanced Backend
352	StatelessSession vs Session Comparison	L2	Backend Core
353	Stored Procedures and Functions	L3	Enterprise Backend
Caching & Multi-tenancy (354–355)			
354	Caching Concurrency Strategies	L3	Performance, Enterprise
355	Multi-Tenancy Strategies in Hibernate	L4	SaaS Architect

Schema Management & Envers (356–357)			
356	Schema Generation and Validation	L2	Backend Core
357	Auditing / Versioning with Hibernate Envers	L3	Compliance
Query Optimization & Filters (358–359)			
358	Query Optimization Strategies	L3	Performance
359	Hibernate Filters vs WHERE Clauses	L3	Backend Core, Security
Concurrency & Locking (360–362)			
360	Concurrency Control in Hibernate	L3	Enterprise Backend
361	Query Cache in Hibernate	L2	Performance
362	Natural IDs – Use and Benefits	L2	Backend Core
Enhancement & DB Specifics (363–364)			
363	Bytecode Enhancement	L3	Advanced Backend
364	Database-specific Stored Procedures Integration	L3	Enterprise Backend
Interceptors, Batch & Cache (365–368)			
365	Event Listeners and Interceptors	L3	Advanced Backend
366	Hibernate with Spring Boot	L2	Backend Core
367	Efficient Batch Processing	L3	Data Engineering
368	Hibernate Caching Layers (L1, L2, Query)	L2	Backend Core
Concurrency, Detached & Lazy Loading (369–372)			
369	Concurrency Control Approaches	L3	Backend Core
370	Detached Entities & Reattachment	L2	Backend Core
371	Lazy Loading Proxies vs Bytecode Enhancement	L3	Advanced Backend
372	Preventing N+1 Query Problem	L3	Performance
Tenancy & Auditing (373–374)			
373	Hibernate Multi-Tenancy Implementation	L4	SaaS Architect
374	Hibernate Auditing using Envers	L3	Compliance
Interceptors & Cache (375–376)			
375	Hibernate Interceptors and Event Listeners	L3	Backend Core
376	Second-Level Cache Concurrency Strategies	L3	Performance
Statistics, Deadlocks & Native Queries (377–379)			
377	Statistics API for Monitoring	L3	DevOps, Performance
378	Handling Deadlocks	L3	Advanced Backend
379	Native SQL Queries – When and How	L2	Backend Core
Stored Procedures & Schema Evolution (380–381)			
380	Calling Stored Procedures	L2	Backend Core
381	Schema Evolution (DDL Auto-Generation)	L2	Backend Core
Session & DB Specific Features (382–384)			
382	StatelessSession and Use Cases	L3	Data Engineering
383	Database-Specific Features (Dialect, SQL Functions)	L3	Advanced Backend
384	Dynamic Update vs Dynamic Insert	L2	Performance
Reactive, Soft Deletes & Multi-tenancy (385–387)			
385	Hibernate Reactive Programming	L4	Modern Backend

386	Implementing Soft Deletes	L2	Backend Core
387	Multi-Tenancy Strategies Overview	L4	SaaS Architect
Performance, Sharding & Redis (388–390)			
388	Tuning Hibernate for High Performance	L3	Architect
389	Database Sharding with Hibernate	L4	Cloud Architect
390	Hibernate + Redis Cache Integration	L3	Performance, Microservices
Concurrency, Query Plans & GraphQL (391–393)			
391	Concurrency in Distributed Systems	L4	Cloud Architect
392	Query Plan Cache – Optimization	L3	Performance
393	Hibernate with GraphQL APIs	L3	Modern Backend
Timeouts, Auditing & Write-heavy Systems (394–396)			
394	Query Timeouts in Hibernate	L3	Performance
395	Hibernate Envers – Auditing Implementation	L3	Compliance
396	Hibernate for Write-Heavy Applications	L3	Data Engineering
Entity Lifecycle & Lazy Loading (397–398)			
397	Entity Lifecycle States	L2	Backend Core
398	Preventing LazyInitializationException	L2	Backend Core
Immutable & Natural IDs (399–400)			
399	Immutable Entities in Hibernate	L2	Backend Core
400	Natural IDs vs Primary Keys	L2	Backend Core
Event Listeners Recap (401)			
401	Hibernate Event Listeners and When to Use	L2	Backend Core, Auditing

HIBERNATE DEEP DIVE & ENTERPRISE READINESS (402 – 519)

#	Question (Short)	Level	Role Focus
Schema Management & Configuration (402 – 406)			
402	Schema Validation & Auto-Generation (hbm2ddl.auto)	L2	Backend Core
403	Interceptors vs Listeners	L3	Advanced Backend
404	Multi-Tenancy Strategies (DB/Schema/Discriminator)	L4	SaaS Architect
405	Optimizing Hibernate for Read-Heavy Systems	L3	Performance Engineer
406	Cache Concurrency Strategies (READ_ONLY, READ_WRITE...)	L3	Backend Core, Performance
Lazy Loading, Caching & Proxies (407 – 415)			
407	Preventing LazyInitializationException	L3	Backend Core
408	Hibernate Envers Auditing	L3	Compliance / Audit
409	Session vs StatelessSession	L3	Data Engineering
410	Event Listeners & Use Cases	L3	Advanced Backend
411	Projections & DTO Queries	L2	Backend Core
412	Native SQL vs HQL vs Criteria API	L2	Backend Core
413	save() vs persist()	L2	Backend Core
414	Composite Primary Keys (@EmbeddedId vs @IdClass)	L2	Backend Core
415	get() vs load()	L1	Backend Core
Entity Lifecycle & Cascading (416 – 421)			
416	Dirty Checking Mechanism	L2	Backend Core
417	Cascade Types & Usage	L2	Backend Core
418	Entity States (Transient, Persistent, Detached, Removed)	L2	Backend Core
419	merge() vs update()	L3	Backend Core
420	Transaction Management (JDBC & Spring)	L2	Backend Core
421	Fetching Strategies (Lazy, Eager, Join Fetch)	L2	Backend Core
Advanced ORM & Composite Keys (422 – 431)			
422	Mapping Composite Keys	L2	Backend Core
423	load() vs get()	L1	Backend Core
424	Native SQL Queries – When to Use	L2	Backend Core
425	save() vs persist() vs merge()	L2	Backend Core
426	Role of SessionFactory	L2	Backend Core
427	openSession() vs getCurrentSession()	L2	Backend Core
428	Lazy Loading Internals	L3	Advanced Backend
429	get() vs load() (differences & usage)	L1	Backend Core
430	save() vs persist() vs saveOrUpdate()	L2	Backend Core
431	Dirty Checking Detailed	L2	Backend Core

Session Management & Lifecycle (432 – 442)			
432	merge() vs update() (behavior)	L3	Backend Core
433	Session vs SessionFactory	L2	Backend Core
434	Entity Lifecycle States	L2	Backend Core
435	get() vs load() (deep comparison)	L1	Backend Core
436	Hibernate Proxies & Lazy Mechanism	L3	Advanced Backend
437	First-Level vs Second-Level Cache	L2	Backend Core
438	Query Cache vs L2 Cache	L3	Performance
439	get() vs load() recap	L1	Backend Core
440	save() vs persist()	L1	Backend Core
441	update() vs merge() vs saveOrUpdate()	L2	Backend Core
442	flush() – Purpose & Timing	L2	Backend Core
Batch Processing & Memory Optimization (443 – 447)			
443	clear() – When to Use	L2	Data Engg
444	refresh() vs clear()	L2	Backend Core
445	get() vs load() (final recap)	L1	Backend Core
446	merge() vs update() (advanced)	L3	Advanced Backend
447	Dirty Checking Mechanism (How It Works)	L2	Backend Core
Object States & SessionFactory (448 – 450)			
448	Transient vs Persistent vs Detached	L1	Backend Core
449	SessionFactory Responsibilities	L2	Backend Core
450	Interceptors & Use Cases	L3	Advanced Backend
Bytecode Enhancement & Statistics (451 – 456)			
451	Hibernate Proxies Explanation	L2	Backend Core
452	@Immutable Entities – Usage	L2	Backend Core
453	Bytecode Enhancement Concept	L3	Advanced Backend
454	hibernate.bytecode.use_reflection_optimizer	L3	Performance
455	@Immutable Deep Dive (Performance Gains)	L2	Backend Core
456	Hibernate Statistics API & Monitoring	L3	DevOps / Performance
Interceptors & EventListeners (457 – 459)			
457	Interceptor Implementation Example	L3	Advanced Backend
458	EventListener vs Interceptor (Comparison)	L3	Advanced Backend
459	StatelessSession – When to Use	L3	Data Engg / Batch
Factories & Multi-Tenancy (460 – 462)			
460	SessionFactory vs EntityManagerFactory	L3	Architect
461	Multi-Tenancy Support (Setup & Code)	L4	SaaS Architect
462	Bytecode Enhancement (Gradle Example)	L3	Advanced Backend
Locking & Optimizations (463 – 466)			
463	Dirty Checking Mechanics	L2	Backend Core
464	Proxies & Lazy Support	L2	Backend Core
465	Optimistic vs Pessimistic Locking	L3	Backend Core, Architect
466	Dynamic Insert/Update Optimization	L3	Performance
Natural IDs & Batch Ops (467 – 472)			
467	Natural IDs (@NaturalId)	L2	Backend Core

468	StatelessSession (Revisited)	L3	Data Engg
469	Immutable Entities	L2	Backend Core
470	Naming Strategies (Implicit/Physical)	L2	Backend Core
471	Transient Persistent Detached States	L1	Backend Core
472	Polymorphic Queries (Inheritance)	L3	Advanced Backend
Security & Injection Prevention (473 – 474)			
473	Preventing SQL Injection in HQL	L2	Backend Core, Security
474	@Immutable Usage Summary	L1	Backend Core
Proxies & Query Execution (475 – 477)			
475	Entity Proxies with Lazy Loading	L2	Backend Core
476	getSingleResult() vs getResultList()	L2	Backend Core
477	Show SQL Logs (hibernate.show_sql etc.)	L1	DevOps
StatelessSession & Locking (478 – 480)			
478	StatelessSession vs Session Recap	L3	Data Engg
479	Configuring Optimistic/Pessimistic Locking	L3	Backend Core
480	Delete Associations – Cascade & Orphan Removal	L2	Backend Core
Detached Objects & Natural IDs (481 – 484)			
481	save() vs persist() vs saveOrUpdate()	L2	Backend Core
482	Managing Detached Objects	L3	Backend Core
483	Natural IDs – Caching & Usage	L3	Backend Core
484	merge() vs update() Revisited	L2	Backend Core
Caching & Batch Performance (485 – 495)			
485	Proxy Objects & Lazy Behavior	L2	Backend Core
486	hbm2ddl.auto Property Explained	L1	Backend Core
487	get() vs load() (Quick Recap)	L1	Backend Core
488	save() vs persist() vs saveOrUpdate() (Recap)	L2	Backend Core
489	openSession() vs getCurrentSession()	L2	Backend Core
490	Database Connection Pooling (HikariCP, c3p0)	L3	DevOps
491	SessionFactory vs EntityManagerFactory (Revisited)	L3	Architect
492	Debugging SQL Queries (N+1 Detection)	L3	DevOps
493	get() vs byId().load()	L2	Backend Core
494	Composite Primary Keys (@Embeddable/@IdClass)	L2	Backend Core
495	JDBC Batch Size Configuration	L3	Data Engg / Performance
Queries & Logging (496 – 499)			
496	merge() vs update() Comparison	L2	Backend Core
497	show_sql vs format_sql vs use_sql_comments	L1	DevOps
498	Optimistic vs Pessimistic Locking (Revisited)	L3	Backend Core
499	createQuery() vs createSQLQuery()	L2	Backend Core
Named Queries & Lifecycle (500 – 519)			
500	Named Queries & Advantages	L2	Backend Core
501	save() vs persist() (Quick Recap)	L1	Backend Core

502	merge() vs update() (Deep Comparison)	L2	Backend Core
503	Entity States (Transient/Persistent/Detached)	L1	Backend Core
504	flush() vs clear()	L2	Backend Core
505	get() vs load() (Final Recap)	L1	Backend Core
506	save() persist() saveOrUpdate() (Recap)	L2	Backend Core
507	openSession() vs getCurrentSession() (Final)	L2	Backend Core
508	merge() vs update() (Safe Usage)	L3	Backend Core
509	Session vs EntityManager	L3	Architect
510	SessionFactory vs EntityManagerFactory	L3	Architect
511	load() vs get() (Behavior Summary)	L1	Backend Core
512	save() vs persist() (Behavior Summary)	L1	Backend Core
513	update() vs saveOrUpdate()	L2	Backend Core
514	merge() vs update() (Safe Practice)	L2	Backend Core
515	flush() vs clear() (Revisited)	L2	Backend Core
516	close() vs evict()	L2	Backend Core
517	Session vs EntityManager (Feature Comparison)	L3	Architect
518	get() vs byId().load() (Advanced Use)	L2	Backend Core
519	ScrollableResults (Cursor Processing)	L3	Data Engg / Performance

1. What is the Spring Framework, and why is it used in Java development?

Answer

Spring is the foundation many Java teams use to build production services. At its core, it's an IoC/DI container that manages object lifecycles, but it also gives you a batteries-included ecosystem: web (Spring MVC/WebFlux), data access (Spring Data/JPA/JDBC), security, AOP, messaging, and cloud integrations. Teams choose Spring because it keeps code loosely coupled, testable, and easier to evolve. With Spring Boot you get auto-configuration, opinionated starters, and Actuator endpoints, which means you focus on business logic instead of wiring.

Quick mental model: think of a banking app with modules for KYC, payments, and notifications. Spring manages the wiring between these modules, transactions across them, and exposes HTTP endpoints-without you hand-coding factories and glue.

Minimal example (Boot-style, constructor DI):

```
// build.gradle: implementation("org.springframework.boot:spring-boot-starter-web")

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;

@SpringBootApplication
public class App {
  public static void main(String[] args) {
    SpringApplication.run(App.class, args);
  }
}

@RestController
@RequestMapping("/products")
class ProductController {
  private final ProductService service;
  ProductController(ProductService service) { this.service = service; }

  @GetMapping("/{id}")
  ProductDto get(@PathVariable long id) { return service.getProduct(id); }
}

@org.springframework.stereotype.Service
class ProductService {
  private final ProductRepository repo;
  ProductService(ProductRepository repo) { this.repo = repo; }
```

```
  ProductDto getProduct(long id) {
    var p = repo.findById(id).orElseThrow();
    return new ProductDto(p.getId(), p.getName());
  }
}

// In real apps, ProductRepository would be a Spring Data JPA interface.
record ProductDto(long id, String name) {}
```

Why this works well in practice:

- Loose coupling via interfaces and DI
- Cross-cutting concerns handled centrally (security, metrics, tracing)
- Production readiness from day one (health checks, config, logging)

2. What is Dependency Injection (DI) in Spring Framework?

Answer

DI means your classes declare what they need, and Spring provides those dependencies. Instead of an OrderService creating a PaymentGateway with new, Spring constructs both and injects one into the other. That keeps construction logic out of business code and makes testing easy.

Types and when to use them:

- Constructor injection (preferred): enforces required dependencies and supports immutability.
- Setter injection: good for truly optional dependencies.
- Field injection: avoided in modern code because it hinders testing and immutability.

E-commerce example-placing an order:

```
interface PaymentGateway { boolean charge(String customerId, long amountPaise); }

@org.springframework.stereotype.Component
class RazorpayGateway implements PaymentGateway {
 public boolean charge(String customerId, long amountPaise) {
  // call external API
  return true;
 }
}

@org.springframework.stereotype.Service
class OrderService {
 private final PaymentGateway gateway;
 OrderService(PaymentGateway gateway) { this.gateway = gateway; }

 public String placeOrder(String customerId, long amountPaise) {
  if (!gateway.charge(customerId, amountPaise)) throw new
RuntimeException("Payment failed");
  return "ORDER-" + System.currentTimeMillis();
 }
}

Unit testing becomes trivial:

class FakeGateway implements PaymentGateway {
 public boolean charge(String c, long a) { return true; }
```

```
}

@Test
void placesOrderWhenPaymentSucceeds() {
  OrderService svc = new OrderService(new FakeGateway());
  String orderId = svc.placeOrder("CUST1", 9999);
  assertNotNull(orderId);
}
```

Key benefits you can articulate in an interview:

- Easier testing (swap real implementations with fakes/mocks)
- Clear separation of construction vs. behavior
- Flexible configuration (profiles, properties) without touching code

3. What is Aspect-Oriented Programming (AOP) in Spring?

Answer

AOP lets you implement cross-cutting concerns-logging, metrics, retries, transactions-in one place and apply them across many methods. Instead of sprinkling try/finally and timers everywhere, you write an aspect once and target methods via pointcuts. Spring weaves this at runtime using proxies.

Concepts in 20 seconds:

- Aspect: the class that holds cross-cutting logic
- Advice: the code to run (@Before, @AfterReturning, @Around, etc.)
- Join point: a method execution
- Pointcut: an expression selecting join points
- Weaving: applying advice to targets (Spring uses proxies: JDK or CGLIB)

Example-timing service methods in a ticket-booking system:

```
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Aspect
@Component
class TimingAspect {

 @Around("execution(* com.acme.booking..*(..))")
 public Object time(ProceedingJoinPoint pjp) throws Throwable {
  long start = System.nanoTime();
  try {
   return pjp.proceed();
  } finally {
   long ms = (System.nanoTime() - start) / 1_000_000;
   String sig = pjp.getSignature().toShortString();
   System.out.println("[Timing] " + sig + " took " + ms + " ms");
  }
 }
}

Transactions are also AOP-backed:

import org.springframework.transaction.annotation.Transactional;

@org.springframework.stereotype.Service
class BookingService {
```

```
  @Transactional
  public void book(String userId, String showId) {
    // reserve seat, charge card, issue ticket
  }
}
```

Practical nuances (interviewer-level detail):

- Self-invocation isn't advised: calling a proxied method from within the same class skips the proxy, so the advice (e.g., @Transactional) won't run. Solution: move the method to another bean or use AspectJ compile-time weaving.
- Final/private methods can't be proxied by JDK dynamic proxies; Spring switches to CGLIB when needed.
- Keep aspects small and focused; too-broad pointcuts can degrade performance or hide bugs.

4. What is Spring Boot, and how does it simplify application development?

Answer

Spring Boot builds on top of the Spring Framework but focuses on removing the setup pain. Instead of writing lengthy XML, configuring beans, and setting up servers manually, you just add starter dependencies and let Boot auto-configure most of it. It comes with embedded servers like Tomcat/Jetty, production-ready metrics via Actuator, and sensible defaults, which means you can literally start coding business features on day one.

Think of it like ordering a pre-configured laptop: the OS, drivers, and basic tools are already installed-you just start working. That's how Boot treats enterprise app scaffolding.

Simple example:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;

@SpringBootApplication
@RestController
public class Application {

  @GetMapping("/greet")
  public String greet() {
    return "Hello, Spring Boot!";
  }

  public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
  }
}
```

Running this spins up an embedded Tomcat and exposes /greet immediately-no external WAR deployment. This "convention over configuration" approach is why Boot is the default choice for microservices.

5. What are annotations in Spring Boot, and how are they useful?

Answer

Annotations in Boot replace XML and boilerplate code. Instead of declaring beans in XML, you just annotate classes, and Spring's classpath scanning plus Boot's auto-configuration wires everything together. This keeps the code more declarative and readable.

Some key ones:

- @SpringBootApplication: bundles configuration, auto-configuration, and component scanning.
- @RestController: marks a class as a REST endpoint.
- @Service, @Repository, @Component: mark beans for DI.
- @Configuration and @Bean: for manual bean definitions if needed.

Example in a simple service:

```
import org.springframework.web.bind.annotation.*;

@RestController
public class HelloController {

  @GetMapping("/hello")
  public String sayHello() {
    return "Hello, World!";
  }
}
```

Here @RestController is actually shorthand for @Controller + @ResponseBody. Without it, you'd manually configure a controller and map responses, but this one annotation does all of it, making REST APIs quick to build.

6. What is Hibernate, and what role does it play in Java development?

Answer

Hibernate is an ORM framework that takes care of mapping Java classes to database tables. Instead of writing SQL manually for every insert, update, or fetch, you work with Java objects, and Hibernate translates that into SQL behind the scenes. It reduces boilerplate, manages transactions, and also gives caching and lazy-loading for efficiency.

Analogy: think of it like Google Translate between two languages-Java objects and relational tables. You speak in Java, Hibernate handles the SQL conversation with the DB.

Example: persisting an employee record.

```
import jakarta.persistence.*;

@Entity
@Table(name = "employee")
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;
 private String name;
 private String department;

 // getters and setters
}
```

Basic usage:

```
import org.hibernate.*;
import org.hibernate.cfg.Configuration;

public class HibernateDemo {
 public static void main(String[] args) {
  SessionFactory factory = new Configuration().configure().buildSessionFactory();
  Session session = factory.openSession();
  Transaction tx = session.beginTransaction();

  Employee emp = new Employee();
  emp.setName("John Doe");
  emp.setDepartment("IT");

  session.save(emp);
  tx.commit
```

```
    session.close();
  }
}
```

This saves a row into the employee table without you writing INSERT INTO employee….

In interviews, it’s also good to mention: Hibernate supports different fetching strategies (lazy/eager), caching levels, and integrates tightly with JPA (Java Persistence API), which is the standard.

7. What are JPA annotations in Hibernate, and why are they important?

Answer

JPA annotations standardize how Java classes are mapped to relational database tables. Instead of writing XML mapping files, you use annotations directly on your model classes. This makes the mapping more readable and keeps it close to the domain model. Hibernate, as a JPA provider, understands these annotations and generates the SQL automatically.

They are important because they handle both structural mapping (class → table, field → column) and relationship mapping (one-to-one, one-to-many, many-to-many). That way, developers focus on business logic instead of repetitive SQL and schema management.

Key annotations:

- @Entity: marks the class as a persistent entity.
- @Table: specifies table details like name or schema.
- @Id: declares the primary key.
- @GeneratedValue: tells Hibernate how to generate IDs.
- @Column: customizes column properties.
- @OneToMany, @ManyToOne, @ManyToMany: define relationships.

Example with Student and Course:

```
@Entity
@Table(name = "student")
public class Student {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;

  @Column(name = "student_name", nullable = false)
  private String name;

  @OneToMany(mappedBy = "student", cascade = CascadeType.ALL)
  private List<Course> courses;
}

@Entity
@Table(name = "course")
public class Course {
  @Id
```

```
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;

  @Column(name = "course_name", nullable = false)
  private String name;

  @ManyToOne
  @JoinColumn(name = "student_id", nullable = false)
  private Student student;
}
```

Here, @OneToMany and @ManyToOne define the relationship between students and courses. Hibernate takes care of generating the foreign keys and handling joins when you query.

8. What is Spring Data JPA, and how does it relate to Hibernate?

Answer

Spring Data JPA sits on top of JPA (and Hibernate under the hood, in most cases). It removes boilerplate DAO code by giving you repository interfaces with ready-to-use CRUD methods. Instead of writing EntityManager queries or Hibernate sessions manually, you just declare repository methods, and Spring generates the queries.

Features:

- Prebuilt repository interfaces like JpaRepository and CrudRepository
- Query derivation: method names like findByDepartment auto-generate SQL
- Support for JPQL, native queries, and custom queries
- Integration with pagination, sorting, and specifications

Example for Employee:

```
@Entity
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;
 private String name;
 private String department;
}

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
 List<Employee> findByDepartment(String department);
}

@Service
public class EmployeeService {
 private final EmployeeRepository repo;
 public EmployeeService(EmployeeRepository repo) { this.repo = repo; }

 public List<Employee> getEmployeesByDepartment(String dept) {
  return repo.findByDepartment(dept);
 }
}

@RestController
@RequestMapping("/employees")
public class EmployeeController {
```

```
  private final EmployeeService service;
  public EmployeeController(EmployeeService service) { this.service = service; }

  @GetMapping("/department/{dept}")
  public List<Employee> getByDept(@PathVariable String dept) {
    return service.getEmployeesByDepartment(dept);
  }
}
```

Here, there's no EntityManager, no explicit SQL. Spring Data JPA builds the query for you using the method name. Hibernate is still doing the ORM work underneath.

9. What is the difference between @Entity and @Table annotations in JPA?

Answer

@Entity is the fundamental annotation. Without it, Hibernate won't even recognize your class for persistence-it tells the provider that this class should be mapped to a table.

@Table is optional and only used when you want to customize table properties. If omitted, the class name becomes the table name by default.

Key differences:

- @Entity: declares that the class is persistent
- @Table: configures the table details like name, schema, and constraints

Example:

```
@Entity
@Table(name = "employees", schema = "hr",
    uniqueConstraints = {@UniqueConstraint(columnNames = {"email"})})
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 @Column(name = "employee_name", nullable = false)
 private String name;

 @Column(name = "email", unique = true, nullable = false)
 private String email;
}
```

Here:

- @Entity makes Employee a persistent class.
- @Table changes the default table name to employees, assigns schema hr, and enforces uniqueness on email.

In practice: use @Entity always, and add @Table only when you need more control.

10. What are some best practices for using Spring and Hibernate together?

Answer

When combining Spring and Hibernate, the goal is to keep the architecture clean and the performance solid. A few key practices:

1. Use Spring's transaction management: annotate service methods with @Transactional instead of manually managing Session and Transaction. This keeps your service layer declarative and reduces boilerplate.
2. Keep entities and business logic separate: entities should be plain models, not mixed with service code. Use repositories or DAOs to encapsulate persistence, and services to orchestrate logic.
3. Prefer lazy loading by default: relationships should usually be FetchType.LAZY, so you only load what's needed. For example, when fetching a user, don't immediately pull all orders unless explicitly required.
4. Use DTOs for API responses: never expose entities directly in REST controllers. DTOs decouple persistence models from API contracts, preventing unintended updates or over-fetching.
5. Optimize queries: use Spring Data JPA derived queries, JPQL, or specifications. For heavy read scenarios, use projections or native queries.

Example-transaction management in a service:

```
@Service
public class EmployeeService {

 private final EmployeeRepository repo;

 public EmployeeService(EmployeeRepository repo) {
  this.repo = repo;
 }

 @Transactional
 public Employee saveEmployee(Employee employee) {
  return repo.save(employee);
 }
}
```

Here, Spring manages the Hibernate session lifecycle and rolls back if any unchecked exception occurs. This avoids boilerplate try/catch/finally with transactions.

11. What is the purpose of @Repository in Spring?

Answer

@Repository is a stereotype annotation that marks a class as a Data Access Object (DAO). It does two main things:

1. Registers the class as a Spring bean during component scanning.
2. Provides exception translation: database-specific exceptions (like SQLException) are converted into Spring's DataAccessException hierarchy, which is consistent across vendors.

This ensures the rest of the codebase doesn't have to worry about low-level DB errors-Spring standardizes them.

Example with a custom DAO:

```
@Repository
public class EmployeeDao {

 @PersistenceContext
 private EntityManager em;

 public Employee findById(int id) {
  return em.find(Employee.class, id);
 }

 public void save(Employee emp) {
  em.persist(emp);
 }
}
```

Here, EmployeeDao is clearly identified as the DB layer, and Spring automatically applies exception translation if persistence errors occur. This is especially useful when switching from MySQL to Oracle or Postgres-you don't rewrite exception handling logic.

12. What is @Transactional in Spring, and why is it important?

Answer

@Transactional is used to mark methods (or classes) as transactional. It ensures that all database operations inside that scope are treated as one unit of work-either all succeed (commit) or all fail (rollback). Without it, developers would have to manage transactions manually, which is error-prone and verbose.

By default:

- It rolls back on unchecked exceptions (RuntimeException, Error).
- It does not roll back on checked exceptions unless configured.

Real-world analogy: think of placing an e-commerce order. If payment succeeds but inventory update fails, you don't want the order half-processed. @Transactional ensures either everything is done, or nothing is.

Example:

```
@Service
public class OrderService {

 private final OrderRepository repo;

 public OrderService(OrderRepository repo) {
  this.repo = repo;
 }

 @Transactional
 public void placeOrder(Order order) {
  repo.save(order);
  updateInventory(order);
 }

 private void updateInventory(Order order) {
  // simulate error
  throw new RuntimeException("Inventory update failed");
 }
}
```

If updateInventory throws an exception, Spring rolls back the save(order) automatically, leaving the database consistent.

13. What is the difference between @Controller and @RestController in Spring?

Answer

Both are used to define controllers in Spring, but the main difference is in how they return responses.

- @Controller: designed for traditional web apps. It returns a view name (like home.jsp or dashboard.html) that the view resolver picks up and renders. Typically used when building server-side rendered HTML pages.
- @RestController: introduced for REST APIs. It's a shorthand for @Controller + @ResponseBody, meaning every method directly returns data (usually JSON or XML) instead of a view. This is the go-to annotation for microservices or API-first apps.

Example-traditional MVC controller:

```
@Controller
public class HomeController {
 @GetMapping("/home")
 public String home() {
  return "home"; // View resolver maps to home.html or home.jsp
 }
}

Example-REST controller:

@RestController
public class ApiController {
 @GetMapping("/api/message")
 public String getMessage() {
  return "Hello, REST!"; // Returned as JSON response
 }
}
```

In practice:

- Use @Controller for apps where you serve HTML pages (like an admin panel).
- Use @RestController for building REST APIs that return JSON/XML.

14. What are Spring Boot starters, and how do they simplify development?

Answer

Starters are curated sets of dependencies that cover common use cases. Instead of manually figuring out which versions of Spring MVC, Jackson, and Tomcat work together, you add one starter dependency and Boot brings in the right set with compatible versions.

For example, spring-boot-starter-web includes:

- Spring MVC
- Jackson (for JSON serialization)
- An embedded Tomcat server

This drastically cuts down setup time and reduces version conflicts.

Example in Maven:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Now you can immediately write a REST API without adding any other library for web, JSON, or server.

Advantages in practice:

- No “jar hell” (Boot manages compatible versions).
- Quick project bootstrapping-just add the right starter.
- Consistency across teams (everyone uses the same curated set).

15. What is @Service in Spring, and how does it differ from @Component?

Answer

Both @Service and @Component register a class as a Spring bean. The difference is semantic and intent.

- @Component: a generic stereotype, can be used for any bean.
- @Service: a specialized stereotype that clearly indicates the bean holds business logic and belongs to the service layer.

This improves readability, helps with code organization, and also allows Spring (or third-party frameworks) to apply additional processing to services in the future.

Example:

```
@Service
public class UserService {

 private final UserRepository repo;

 public UserService(UserRepository repo) {
  this.repo = repo;
 }

 public User getUserById(int id) {
  return repo.findById(id).orElse(null);
 }
}
```

Here, UserService sits between the controller and repository layers, handling business rules like validations or transformations.

So, while @Component could technically be used here, @Service makes the role of the class explicit-this helps when scanning large codebases.

16. What is Lazy Loading in Hibernate, and when should it be used?

Answer

Lazy loading means Hibernate delays fetching associated data until it's actually accessed. So when you load an Employee, the projects list isn't fetched immediately. Instead, Hibernate creates a proxy and only hits the database when you call employee.getProjects().

This is the default for collections and many relationships in Hibernate, and it's particularly useful when:

- You're dealing with large datasets and don't want unnecessary data in memory.
- You don't always need the child entities. For example, when listing employees in a payroll system, you may not need their projects unless you open the detail page.

Configuration example:

```
@Entity
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 @OneToMany(mappedBy = "employee", fetch = FetchType.LAZY)
 private List<Project> projects;
}
```

Here, projects will only be queried when accessed.

Caution: if you access a lazy collection outside of a transaction (e.g., in a closed session), you'll hit the LazyInitializationException. That's why lazy loading must be combined with proper transaction management or DTO mapping.

17. What is Eager Loading in Hibernate, and how does it differ from Lazy Loading?

Answer

Eager loading fetches the associated entities immediately, along with the parent entity. So when you fetch an Employee, all its projects are also loaded in the same query (or via additional queries).

The difference:

- Lazy: fetch data only when required.
- Eager: fetch everything upfront, even if not needed.

Eager loading can simplify code when you know you'll always need the related data, but it can lead to performance issues (N+1 query problem or heavy joins) when applied carelessly.

Example:

```
@Entity
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 @OneToMany(mappedBy = "employee", fetch = FetchType.EAGER)
 private List<Project> projects;
}
```

In this case, fetching a single employee will also load all their projects right away.

Rule of thumb: prefer lazy loading as default, use eager loading only when you're certain the association is always required.

18. What is the purpose of @Query in Spring Data JPA?

Answer

Spring Data JPA lets you define queries by method names (findByName, findByDepartment), but sometimes those are too limiting. @Query gives you the flexibility to write custom queries directly on repository methods.

It supports:

- JPQL (portable, object-oriented queries using entity names and fields)
- Native SQL (for DB-specific queries or performance tuning)
- Parameter binding (positional ?1, ?2 or named :dept)

Example:

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

 // JPQL query
 @Query("SELECT e FROM Employee e WHERE e.department = :dept")
 List<Employee> findByDepartment(@Param("dept") String dept);

 // Native query
 @Query(value = "SELECT * FROM employee WHERE department = ?1", nativeQuery
= true)
 List<Employee> findByDepartmentNative(String dept);
}
```

Advantages:

- Lets you handle complex queries directly in the repository.
- Reduces boilerplate DAOs-no need to manage EntityManager manually.
- Works seamlessly with pagination (Pageable) and sorting.

Best practices:

- Prefer JPQL because it's database-independent.
- Use native queries only for performance-critical cases or DB-specific features.
- Always parameterize queries to avoid SQL injection.

19. What is the difference between CrudRepository and JpaRepository in Spring Data JPA?

Answer

Both are repository interfaces in Spring Data JPA, but they differ in the level of functionality they offer.

- CrudRepository: the most basic. It gives you essential CRUD methods like save(), findById(), findAll(), and delete(). It's fine for simple use cases where you don't need advanced JPA features.
- JpaRepository: extends CrudRepository and PagingAndSortingRepository. It adds JPA-specific methods such as flush(), saveAndFlush(), and query support with pagination and sorting. In practice, most projects use JpaRepository because it's more powerful and flexible.

Example:

```
public interface EmployeeCrudRepository extends CrudRepository<Employee,
Integer> {
  List<Employee> findByDepartment(String department);
}

public interface EmployeeJpaRepository extends JpaRepository<Employee, Integer> {
  List<Employee> findByDepartment(String department);
  List<Employee> findByDepartment(String department, Pageable pageable);
}
```

Key takeaway:

- Use CrudRepository only for very lightweight, minimal projects.
- Prefer JpaRepository for real-world applications-it covers everything CrudRepository does plus pagination, batch operations, and better integration with JPA.

20. What is the EntityManager in JPA?

Answer

The EntityManager is the heart of JPA-it manages entities and acts as a bridge between the application and the database. Think of it like Hibernate's Session. It operates within a persistence context, meaning it tracks the state of your entities (new, managed, detached, removed) during a transaction.

Responsibilities of EntityManager:

- CRUD operations: persist(), merge(), remove(), find()
- Transaction handling: commits or rollbacks changes
- Query execution: supports JPQL (createQuery()) and native SQL (createNativeQuery())

Example:

```
@Entity
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;
 private String name;
 private String department;
}

Using EntityManager directly:

public class EmployeeService {

 @PersistenceContext
 private EntityManager em;

 public void saveEmployee(Employee emp) {
  em.persist(emp); // Inserts a new record
 }

 public Employee findEmployeeById(int id) {
  return em.find(Employee.class, id); // Fetches by primary key
 }
}
```

Best practices:

- Always use it within a transaction; otherwise, changes won’t be persisted.
- Avoid keeping persistence contexts open too long (can lead to memory leaks).
- In most Spring apps, you use Spring Data JPA repositories which internally rely on EntityManager.

21. How does the Spring Framework manage transactions, and what are the propagation levels?

Answer

Spring manages transactions using @Transactional (declarative) or TransactionTemplate (programmatic). With @Transactional, Spring opens a transaction before the method runs and commits/rolls it back afterward. If an exception occurs, the transaction is rolled back by default for runtime exceptions.

Propagation levels define how a method behaves if there's already an active transaction. The most common ones:

- REQUIRED: joins an existing transaction or starts a new one if none exists (default).
- REQUIRES_NEW: always creates a new transaction, suspending any existing one.
- NESTED: starts a nested transaction (supported by certain databases).
- SUPPORTS: runs in a transaction if one exists, otherwise non-transactional.

Example:

```
@Service
public class PaymentService {

 private final OrderService orderService;

 public PaymentService(OrderService orderService) {
  this.orderService = orderService;
 }

 @Transactional(propagation = Propagation.REQUIRED)
 public void processPayment(Order order) {
  orderService.saveOrder(order);
  // Payment logic here
 }
}
```

If orderService.saveOrder() is already transactional, processPayment joins that transaction. If not, Spring creates a new one.

In real-world systems (like booking tickets or processing payments), picking the right propagation is critical-you don't want partial commits if part of the workflow fails.

22. What is the N+1 Select problem in Hibernate, and how can it be avoided?

Answer

The N+1 problem happens when Hibernate fetches a parent entity list in one query, but then fires one additional query for each child collection. So you end up with 1 (for parents) + N (for children) queries, which kills performance when N is large.

Example of the issue:

```
List<Department> departments = session.createQuery("FROM Department",
Department.class).list();
for (Department dept : departments) {
 System.out.println(dept.getEmployees()); // triggers another select per department
}
```

If there are 20 departments, Hibernate fires 1 query for departments + 20 queries for employees.

Ways to avoid:

1. **JOIN FETCH**: fetch parent and child in one query.
2. List<Department> departments = session.createQuery(
3. "SELECT d FROM Department d JOIN FETCH d.employees",

   Department.class).list();
4. **@BatchSize**: tells Hibernate to fetch associations in batches.
5. @BatchSize(size = 10)
6. private List<Employee> employees;
7. **Entity Graphs**: define fetch plans explicitly using JPA entity graphs.

In practice: use JOIN FETCH when you always need the children, @BatchSize when you sometimes need them. This balances performance with flexibility.

23. What is the difference between @Entity and @MappedSuperclass in JPA?

Answer

Both are used to mark classes in ORM mapping, but their purpose is different:

- @Entity: the class itself is a JPA entity and maps to a table.
- @MappedSuperclass: not an entity, not mapped to a table; it just provides common mappings for subclasses. Fields are inherited by child entities.

Example:

```
@MappedSuperclass
public abstract class BaseEntity {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(name = "created_at")
  private LocalDateTime createdAt;
}

@Entity
public class Employee extends BaseEntity {
  private String name;
}

@Entity
public class Department extends BaseEntity {
  private String departmentName;
}
```

Here, BaseEntity is never a table, but its id and createdAt columns appear in both Employee and Department tables.

Rule of thumb:

- Use @Entity for tables.
- Use @MappedSuperclass for reusable fields across entities.

24. How does Hibernate handle caching, and what are the levels of caching?

Answer

Hibernate reduces database hits by using caching. It has two main cache levels:

1. **First-Level Cache (Session Cache)**
   - Built-in and always enabled.
   - Scoped to a Hibernate Session.
   - If you fetch the same entity twice in one session, Hibernate serves it from the cache, not the DB.

   Example:

```
Session session = factory.openSession();
Employee e1 = session.get(Employee.class, 1); // hits DB
Employee e2 = session.get(Employee.class, 1); // comes from cache
```

2. **Second-Level Cache (SessionFactory Cache)**
   - Optional, must be configured.
   - Shared across sessions.
   - Stores entities, collections, or queries using a cache provider (Ehcache, Redis, Infinispan).
   - Useful for read-heavy apps where the same data is accessed repeatedly.

   Example configuration:

```
<property name="hibernate.cache.use_second_level_cache" value="true" />
<property name="hibernate.cache.region.factory_class"
    value="org.hibernate.cache.ehcache.EhCacheRegionFactory" />
```

In practice:

- First-level cache is always there, you don't manage it.
- Second-level cache is powerful but should be used carefully-best for reference data (like product catalog, lookup tables), not for rapidly changing transactional data.

25. What is @OptimisticLocking in Hibernate, and how does it work?

Answer

Optimistic locking is a concurrency control strategy in Hibernate that prevents two users from overwriting each other's changes without explicit locking. Instead of locking rows in the database, it relies on a version check at commit time.

Hibernate supports this through the @Version annotation. When an entity is updated, Hibernate checks the version column. If another transaction has already modified the row and incremented the version, Hibernate throws an OptimisticLockException.

This is best used in systems where conflicts are rare but possible-for example, updating customer addresses in a banking system.

Example:

```
@Entity
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 private String name;

 @Version
 private int version;
}
```

Here's what happens:

- Transaction A reads Employee(id=1, version=1).
- Transaction B also reads the same Employee (version=1).
- A updates and commits → version becomes 2.
- B tries to update → Hibernate sees version mismatch (expected 1, found 2) and throws OptimisticLockException.

This way, you ensure data integrity without using heavy database locks.

26. How can you write custom SQL queries in Spring Data JPA?

Answer

Spring Data JPA lets you define custom queries with the @Query annotation when query derivation by method name isn't enough. You can write them in JPQL (object-oriented) or native SQL (DB-specific).

Example with JPQL:

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

  @Query("SELECT e FROM Employee e WHERE e.department = :dept")
  List<Employee> findEmployeesByDepartment(@Param("dept") String department);
}
```

Here, JPQL is used-it refers to entity and field names (Employee, department) instead of table/column names.

Example with native SQL:

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

  @Query(value = "SELECT * FROM employee WHERE department = ?1", nativeQuery
= true)
  List<Employee> findEmployeesByDepartmentNative(String department);
}
```

When to use:

- JPQL: default choice, portable across databases.
- Native SQL: for complex queries, vendor-specific optimizations, or performance tuning.

Best practice: always parameterize queries (?1, :dept) to prevent SQL injection.

27. What is the purpose of Entity Graphs in JPA?

Answer

Entity Graphs allow you to control how related entities are fetched, without changing your mappings. Instead of relying on default LAZY/EAGER fetch strategies, you can define fetch plans dynamically at runtime. This helps solve the N+1 problem and optimizes queries for specific use cases.

Example definition:

```
@Entity
@NamedEntityGraph(
  name = "Employee.detail",
  attributeNodes = @NamedAttributeNode("department")
)
public class Employee {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;

  private String name;

  @ManyToOne
  private Department department;
}
```

Using the entity graph:

```
EntityGraph<?> graph = em.getEntityGraph("Employee.detail");
Map<String, Object> hints = new HashMap<>();
hints.put("jakarta.persistence.fetchgraph", graph);

Employee emp = em.find(Employee.class, 1, hints);
```

Here, when fetching an Employee, the associated Department is loaded eagerly as part of the query-without you needing to change the @ManyToOne mapping.

This is powerful when you want different fetch strategies in different use cases-for example, loading only employee basics for a list view, but loading department details for a profile page.

28. What is the difference between JPQL and Criteria API in JPA?

Answer

Both JPQL and Criteria API are ways to write queries in JPA, but they serve different purposes:

- **JPQL (Java Persistence Query Language)**
  - String-based, looks similar to SQL but operates on entities and their fields instead of tables/columns.
  - Quick to write and very readable.
  - Downside: prone to runtime errors, because the compiler doesn't check field names inside strings.
- **Criteria API**
  - Programmatic, type-safe way of building queries using Java objects.
  - Eliminates runtime syntax errors, since field names and conditions are checked at compile time.
  - Downside: more verbose and harder to read for simple queries.

Example with JPQL:

```
@Query("SELECT e FROM Employee e WHERE e.name = :name")
List<Employee> findByName(@Param("name") String name);

Example with Criteria API:

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Employee> query = cb.createQuery(Employee.class);
Root<Employee> root = query.from(Employee.class);
query.select(root).where(cb.equal(root.get("name"), "John"));
List<Employee> result = em.createQuery(query).getResultList();
```

When to use:

- JPQL for simple queries (like fetching by name).
- Criteria API for dynamic queries built at runtime (like complex search filters in an e-commerce site).

29. What is the purpose of @JoinTable in JPA, and when is it used?

Answer

@JoinTable is used to define a join table for many-to-many relationships. Since relational databases don't support direct many-to-many mappings, JPA introduces a middle join table to store the relationships.

Example:

```
@Entity
public class Student {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 private String name;

 @ManyToMany
 @JoinTable(
  name = "student_course",
  joinColumns = @JoinColumn(name = "student_id"),
  inverseJoinColumns = @JoinColumn(name = "course_id")
 )
 private List<Course> courses;
}
```

Here:

- student_course is the join table.
- student_id is the foreign key to Student.
- course_id is the foreign key to Course.

This mapping allows one student to enroll in multiple courses, and one course to have many students.

When to use: any many-to-many mapping, like students ↔ courses, users ↔ roles, or actors ↔ movies.

30. What are the main components of a Spring Boot application?

Answer

A Spring Boot application typically has five main components:

1. **Starter Dependencies** – opinionated dependencies like spring-boot-starter-web, spring-boot-starter-data-jpa, which pull in everything you need for that module.
2. **Embedded Server** – Boot bundles Tomcat, Jetty, or Undertow so you don't need an external container.
3. **Auto-Configuration** – Boot inspects the classpath and auto-configures beans, reducing setup boilerplate.
4. **Application Properties/YAML** – centralized configuration using application.properties or application.yml.
5. **Annotations** – like @SpringBootApplication, @RestController, @Service, which glue everything together.

Example minimal Boot app:

```
@SpringBootApplication
public class Application {
  public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
  }
}
```

Running this single class launches an embedded Tomcat server and exposes your endpoints immediately.

In real projects, Boot apps typically follow a layered structure:

- Controller layer → handles web requests
- Service layer → contains business logic
- Repository layer → interacts with DB (via Spring Data JPA/Hibernate)

31. What is @Cascade in Hibernate, and how does it differ from CascadeType in JPA?

Answer

JPA defines a standard set of cascade operations with CascadeType (like PERSIST, MERGE, REMOVE, ALL). These cover common cases where operations on a parent should propagate to children.

Hibernate goes further with its own @Cascade annotation, which introduces additional options such as SAVE_UPDATE and DELETE_ORPHAN. These give you finer-grained control over entity lifecycle.

- **JPA CascadeType examples:**
    - PERSIST: saving a parent also saves its children.
    - REMOVE: deleting a parent also deletes its children.
- **Hibernate-specific examples:**
    - SAVE_UPDATE: ensures child entities are either saved or updated along with the parent, regardless of state.
    - DELETE_ORPHAN: deletes a child when it's removed from the parent's collection (similar to orphanRemoval = true).

Example:

```
@Entity
public class Parent {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 @OneToMany(mappedBy = "parent", orphanRemoval = true)
 @Cascade(org.hibernate.annotations.CascadeType.DELETE_ORPHAN)
 private List<Child> children;
}
```

Here, removing a Child from the children list deletes it from the database automatically.

Key difference:

- CascadeType (JPA) is portable and standardized.
- @Cascade (Hibernate) is vendor-specific, with extra capabilities.=

32. How does Hibernate handle bidirectional relationships, and what are the common pitfalls?

Answer

Bidirectional relationships let you navigate both ways between entities. For example, a Department can access its employees, and each Employee can access its department.

How it works:

- **Owning side:** the side that actually maintains the foreign key in the DB (usually @ManyToOne).
- **Inverse side:** the side mapped with mappedBy (usually @OneToMany). It doesn't own the foreign key.

Example:

```
@Entity
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 @ManyToOne
 private Department department;
}

@Entity
public class Department {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 @OneToMany(mappedBy = "department")
 private List<Employee> employees;
}
```

Common pitfalls:

1. **Infinite recursion:** frameworks like Jackson may serialize both sides endlessly.
    - Fix: use @JsonIgnore, @JsonManagedReference/@JsonBackReference, or map DTOs.

2. **Orphan removal issues:** if you only update one side (like adding an Employee to a Department list but not setting employee.setDepartment()), Hibernate may leave the relationship inconsistent.
    - Fix: always update both sides in code.

In practice: keep the @ManyToOne as the owning side and ensure your code synchronizes both ends.

33. What are Fetch Profiles in Hibernate, and how are they used?

Answer

Fetch Profiles are a Hibernate-specific feature that lets you define custom fetching strategies and enable them dynamically. They're useful when you want to optimize certain queries without globally changing mappings.

Why use them:

- Avoid loading unnecessary data.
- Improve query performance by overriding defaults for specific scenarios.

Example:

```
@Entity
@FetchProfile(
 name = "employeeWithDepartment",
 fetchOverrides = {
  @FetchProfile.FetchOverride(
   entity = Employee.class,
   association = "department",
   mode = FetchMode.JOIN
  )
 }
)
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 @ManyToOne
 private Department department;
}
```

Usage:

```
Session session = sessionFactory.openSession();
session.enableFetchProfile("employeeWithDepartment");
Employee emp = session.get(Employee.class, 1);
```

Here, enabling the profile makes Hibernate join-fetch the department with the employee in a single query.

Advantages:

- Flexible-you can define multiple profiles and enable them only when needed.

- Helps fine-tune performance for specific use cases (like API endpoints that need richer data).

34. How does Spring Boot handle externalized configuration?

Answer

Spring Boot externalizes configuration so you don't hardcode values in your application. Instead, you define them outside your code and override them depending on the environment (dev, test, prod). Boot reads properties in a specific order (command-line > env variables > property files > defaults), so you can override values without changing code.

Common sources:

1. **application.properties / application.yml** → default configuration files in the classpath.
2. **Environment variables** → can override properties at OS or container level.
3. **Command-line arguments** → useful when running jars with --server.port=9000.
4. **Profile-specific files** → application-dev.yml, application-prod.yml.

Example:

```
application.properties

server.port=8081
app.name=MyApplication

Injecting properties in a bean:

@Component
public class AppConfig {
 @Value("${app.name}")
 private String appName;

 public void printConfig() {
  System.out.println("App Name: " + appName);
 }
}
```

Advantages:

- Centralized config management.
- Profiles make it easy to switch between environments.
- Supports YAML for hierarchical config.

In real-world setups, you can also use **Spring Cloud Config** or **Vault** for secure centralized configuration.

35. What is @EntityGraph in JPA, and how is it used?

Answer

@EntityGraph lets you define fetch plans at query time, specifying which associations should be eagerly fetched. It's similar to fetch profiles in Hibernate but standardized in JPA. The main goal is performance tuning: load exactly what you need, avoid N+1 queries, and skip unnecessary joins.

Example definition:

```
@Entity
@NamedEntityGraph(
 name = "Employee.detail",
 attributeNodes = {
  @NamedAttributeNode("department"),
  @NamedAttributeNode("address")
 }
)
public class Employee {
 @Id
 private int id;

 @ManyToOne
 private Department department;

 @OneToOne
 private Address address;
}
```

Using it:

```
EntityGraph<?> graph = em.getEntityGraph("Employee.detail");
Map<String, Object> hints = new HashMap<>();
hints.put("jakarta.persistence.fetchgraph", graph);

Employee emp = em.find(Employee.class, 1, hints);
```

Advantages:

- Optimizes performance by customizing fetch behavior per use case.
- Avoids writing complex queries with multiple joins.
- Keeps entity mappings clean while still allowing flexibility.

In practice, you might use it in service methods that need richer data (like an employee profile API) while keeping defaults lazy for list endpoints.

36. What is Spring AOP, and how does it handle cross-cutting concerns?

Answer

Spring AOP (Aspect-Oriented Programming) allows you to separate cross-cutting concerns-logic that cuts across multiple modules, like logging, security, or transaction management. Instead of duplicating the same code in multiple places, you define it once in an aspect and let Spring apply it where needed.

Key concepts:

- **Aspect** → a module that encapsulates cross-cutting concern (e.g., logging).
- **Join Point** → a point during execution, usually a method call.
- **Advice** → the code you want to run (before, after, around the join point).
- **Pointcut** → the expression defining where to apply the advice.

Example logging aspect:

```
@Aspect
@Component
public class LoggingAspect {

 @Before("execution(* com.example.service.*.*(..))")
 public void logBefore(JoinPoint joinPoint) {
  System.out.println("Method called: " + joinPoint.getSignature().getName());
 }
}
```

Here, before any service method executes, the aspect logs the method name.

How it works internally: Spring creates proxies around your beans. When a method is called, the proxy intercepts it, applies the advice, and then delegates to the actual method.

Real-world use cases:

- Logging method execution.
- Declaring transactions with @Transactional.
- Enforcing security checks.
- Performance monitoring.

37. How does Spring Boot handle application profiling?

Answer

Spring Boot provides profiles so you can define environment-specific configurations (dev, test, prod, staging, etc.). Instead of hardcoding values, you create separate property or YAML files for each profile, and Boot activates the right one at runtime.

How it works:

- Default file: application.properties or application.yml.
- Profile-specific files: application-dev.properties, application-prod.yml, etc.
- Activation: set spring.profiles.active=dev in properties, environment variable, or pass it as a command-line arg.

Example:

```
application-dev.properties

server.port=8081

application-prod.properties

server.port=8080

Running with profile:

java -jar myapp.jar --spring.profiles.active=dev
```

This way, the same app can run in different environments with different configs (DB URLs, ports, logging levels) without code changes.

38. How does Hibernate implement inheritance mapping?

Answer

Hibernate supports object-oriented inheritance mapping to relational tables. JPA defines three main strategies:

1. **Single Table**
    - All classes in the hierarchy stored in one table.
    - Uses a discriminator column to distinguish types.
    - Fastest but wastes space with nulls.
2. **Table per Class**
    - Each class has its own table with all fields (including inherited ones).
    - No joins needed but duplicates common columns in each table.
3. **Joined Strategy**
    - Parent class has one table, child classes have their own, joined with foreign keys.
    - Normalized schema, but queries involve joins.

Example with joined strategy:

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Employee {
  @Id
  private int id;
}

@Entity
public class Manager extends Employee {
  private String department;
}
```

When you fetch a Manager, Hibernate joins Employee and Manager tables.

Rule of thumb:

- **Single Table** for small hierarchies.
- **Joined** when normalization and integrity matter.
- **Table per Class** is rarely used in production due to duplication.

39. What is Spring Boot Actuator, and how is it useful?

Answer

Spring Boot Actuator adds production-ready features for monitoring, health checks, and metrics without writing extra code. It exposes HTTP endpoints that ops teams or monitoring tools can use.

Key endpoints:

- /actuator/health: shows app health status (DB up, disk space, etc.).
- /actuator/info: app metadata.
- /actuator/metrics: exposes JVM, memory, HTTP metrics.

Setup:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Enable specific endpoints:

```
management.endpoints.web.exposure.include=health,info,metrics

Example: hitting http://localhost:8080/actuator/health returns:

{"status":"UP"}
```

Use cases:

- Integration with Prometheus, Grafana, or CloudWatch for monitoring.
- Liveness/readiness probes in Kubernetes.
- Quick visibility into runtime behavior without custom controllers.

40. How can you optimize batch processing in Hibernate?

Answer

Hibernate lets you optimize bulk inserts/updates by batching SQL calls instead of sending them one by one. This is crucial when working with thousands of rows.

Techniques:

1. **Batch Size**
    - Configure hibernate.jdbc.batch_size=20 in properties.
    - Groups 20 statements into a single batch.
2. **StatelessSession**
    - Use when caching isn't needed. Faster for massive bulk operations because it skips first-level cache overhead.
3. **Flush and Clear**
    - Flush changes and clear the session periodically to prevent memory buildup.

Example:

```
for (int i = 0; i < employees.size(); i++) {
  session.save(employees.get(i));
  if (i % 20 == 0) {
    session.flush();
    session.clear();
  }
}
```

Here, Hibernate inserts employees in batches of 20. This drastically improves performance compared to one SQL per insert.

In practice: combine batching with connection pool tuning and turn off auto-flush for best results.

41. What is Spring Security, and how does it integrate with Spring Boot?

Answer

Spring Security is a framework that handles authentication, authorization, and protection against common security attacks (CSRF, session fixation, etc.). In a Spring Boot app, you just add the dependency spring-boot-starter-security, and by default, every endpoint is secured with HTTP Basic authentication.

Integration steps:

1. Add the starter dependency.
2. Define a security configuration class using SecurityFilterChain.
3. Provide user credentials via in-memory config, DB, or external providers.

Example minimal setup:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

 @Bean
 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  http
    .authorizeHttpRequests(auth -> auth
       .requestMatchers("/admin/**").authenticated()
       .anyRequest().permitAll()
    )
    .httpBasic();
  return http.build();
 }

 @Bean
 public UserDetailsService users() {
  UserDetails admin = User.withDefaultPasswordEncoder()
     .username("admin")
     .password("password")
     .roles("ADMIN")
     .build();
  return new InMemoryUserDetailsManager(admin);
 }
}
```

Now, /admin/** endpoints require authentication, while others are open.

In real apps, credentials usually come from a database or an identity provider (OAuth2, LDAP).

42. How does JWT authentication work in Spring Boot?

Answer

JWT (JSON Web Token) is a compact token used for stateless authentication. Instead of storing session state on the server, the client holds the token, and every request includes it in the header. Spring Security validates the token to identify the user.

Flow:

1. Client logs in with username/password.
2. Server generates a JWT signed with a secret key and sends it back.
3. Client includes the token in the Authorization: Bearer <token> header for each request.
4. Server validates the signature and extracts claims (like roles, username).

Example filter in Spring Boot:

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
 @Override
 protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
FilterChain chain)
    throws IOException, ServletException {
  String authHeader = req.getHeader("Authorization");
  if (authHeader != null && authHeader.startsWith("Bearer ")) {
   String token = authHeader.substring(7);
   // validate token, extract username & roles
   UsernamePasswordAuthenticationToken auth =
      new UsernamePasswordAuthenticationToken("user", null, List.of(new
SimpleGrantedAuthority("ROLE_USER")));
   SecurityContextHolder.getContext().setAuthentication(auth);
  }
  chain.doFilter(req, res);
 }
}
```

Advantages:

- Stateless (no server-side session).
- Works well in distributed microservices.
- Can store extra claims like user roles.

In interviews, highlight JWT is widely used for **microservices APIs** and integrates with API gateways easily.

43. What is OAuth2, and how is it implemented in Spring Security?

Answer

OAuth2 is an authorization framework that lets users grant limited access to their resources without sharing credentials. For example, logging into an app using Google or GitHub.

Spring Security provides full OAuth2 support via spring-boot-starter-oauth2-client and spring-boot-starter-oauth2-resource-server.

Two common use cases:

1. **OAuth2 Login (SSO)** → users log in with Google, GitHub, etc.
2. **OAuth2 Resource Server** → validating JWT access tokens issued by an auth server (e.g., Keycloak, Okta).

Example – enabling Google login:

```
spring:
 security:
  oauth2:
   client:
    registration:
     google:
      client-id: your-client-id
      client-secret: your-client-secret
      scope: email,profile
```

With this config, Boot auto-generates the login flow. When a user hits a protected endpoint, they're redirected to Google for authentication.

Why it matters:

- Industry standard for securing APIs.
- Used in microservices for Single Sign-On (SSO).
- Often combined with JWT in distributed systems.

44. What is CSRF protection in Spring Security, and how does it work?

Answer

CSRF (Cross-Site Request Forgery) is an attack where a malicious site tricks a logged-in user into performing unintended actions (like transferring money).

Spring Security enables CSRF protection by default for stateful apps (those using sessions and cookies). It does this by:

- Generating a CSRF token for each session.
- Requiring that token in each modifying request (POST, PUT, DELETE).
- Rejecting requests with missing or invalid tokens.

Example: in a form-based login app, the token is added automatically to HTML forms as a hidden field.

In REST APIs (stateless, using JWTs), CSRF protection is usually disabled since there are no server-side sessions.

Config example:

```
http
  .csrf(csrf -> csrf.disable()) // disable for stateless APIs
  .authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
```

Rule of thumb:

- Keep CSRF on for session-based web apps.
- Disable for token-based APIs.

45. How does Spring Security handle role-based and method-level security?

Answer

Spring Security allows access control not just at URL level, but also at **method level** inside services or controllers.

Enabling it:

```
@EnableGlobalMethodSecurity(prePostEnabled = true)
@Configuration
public class SecurityConfig { }

Then you can annotate methods:

@Service
public class AdminService {

 @PreAuthorize("hasRole('ADMIN')")
 public void deleteUser(int userId) {
  // only ADMIN can do this
 }

 @PreAuthorize("#userId == authentication.principal.id")
 public User getUser(int userId) {
  // user can only view their own profile
  return ...
 }
}
```

Advantages:

- Fine-grained control (business rules at method level).
- Supports SpEL (Spring Expression Language) for custom conditions.

In practice: use URL-based security for broad endpoints, and method-level security for sensitive business rules.

46. What are filters in Spring Security, and how does the filter chain work?

Answer

Spring Security is built on top of a **filter chain**. Each filter intercepts requests and performs specific tasks (like authentication, authorization, logging).

Some key filters:

- UsernamePasswordAuthenticationFilter → handles login forms.
- BasicAuthenticationFilter → processes HTTP Basic headers.
- BearerTokenAuthenticationFilter → validates JWTs.
- ExceptionTranslationFilter → handles access denied exceptions.

The order matters: filters are applied in sequence, and once a request is authenticated, the SecurityContext holds the user's details.

Custom filters: you can add your own filter (e.g., for JWT validation) by extending OncePerRequestFilter.

Example:

```
http.addFilterBefore(new JwtAuthFilter(),
UsernamePasswordAuthenticationFilter.class);

This ensures JWT validation happens before username/password login.
```

Real-world analogy: think of the filter chain like airport security - bag check, body scan, passport control. Each layer checks something different before letting you through.

47. What is Spring Cloud Config Server, and why is it used?

Answer

In microservices, each service needs environment-specific configuration (DB URLs, API keys, feature flags). Managing these configs inside each service is messy.

Spring Cloud Config Server solves this by providing a **centralized configuration service**:

- Stores configs in Git, SVN, or a file system.
- Clients (microservices) fetch their configs at startup or even refresh at runtime.
- Supports profile-specific configs (application-dev.yml, application-prod.yml).

Setup:

1. Add spring-cloud-config-server dependency.
2. Annotate main app with @EnableConfigServer.
3. Point it to a Git repo containing configs.

Example client config (bootstrap.yml):

```
spring:
 application:
  name: orderservice
 cloud:
  config:
   uri: http://localhost:8888
```

When orderservice starts, it fetches orderservice-dev.yml or orderservice-prod.yml from the config server.

This makes config changes easy to manage across **dozens of microservices** without redeploying each one.

48. What is Eureka in Spring Cloud, and how does service discovery work?

Answer

In microservices, services need to find and talk to each other. Hardcoding hostnames and ports doesn't work because instances scale dynamically.

Eureka is a **service registry** from Netflix OSS.

- Each service registers itself with Eureka at startup.
- Other services query Eureka to discover instances.
- Eureka also supports load balancing between instances.

Setup:

1. Add spring-cloud-starter-netflix-eureka-server dependency.
2. Start Eureka server with @EnableEurekaServer.
3. Each service registers with @EnableEurekaClient.

Example client (application.yml):

```
spring:
 application:
  name: orderservice
eureka:
 client:
  service-url:
   defaultZone: http://localhost:8761/eureka/
```

Now, orderservice registers itself with Eureka. Other services (like paymentservice) can discover it using the name ORDERSERVICE.

This avoids hardcoded URLs and makes scaling services seamless.

49. What is Spring Cloud Gateway, and how is it different from Zuul?

Answer

Spring Cloud Gateway is an API gateway that routes requests to backend microservices. It also provides cross-cutting features like authentication, logging, and rate limiting.

Why needed:

- Clients don't directly call microservices.
- Gateway acts as a single entry point.
- Helps with load balancing, monitoring, and security.

Differences from Zuul (Netflix OSS):

- Gateway is built on **Reactor (non-blocking, reactive)**, while Zuul 1 was servlet-based (blocking).
- Gateway supports advanced features like rate limiting, path rewriting, and circuit breakers more efficiently.
- It's now the preferred solution in Spring Cloud.

Example config (application.yml):

```
spring:
 cloud:
  gateway:
   routes:
    - id: orderservice
      uri: lb://ORDERSERVICE
      predicates:
       - Path=/orders/**
```

Here, any request to /orders/** is routed to the ORDERSERVICE discovered via Eureka (lb:// means load-balanced).

50. What is a Circuit Breaker pattern, and how does Spring Cloud support it?

Answer

In microservices, if one service is down or slow, repeated calls from other services can cause cascading failures. The **Circuit Breaker pattern** prevents this by "opening the circuit" when too many failures happen, so calls fail fast instead of overloading the system.

Spring Cloud supports this using **Resilience4j** (Hystrix was older Netflix OSS).

Flow:

- Circuit is closed by default → requests flow normally.
- If failures exceed a threshold, the circuit opens → requests fail immediately.
- After a wait period, it moves to half-open → lets a few test requests through.
- If they succeed, the circuit closes again.

Example:

```
@CircuitBreaker(name = "orderService", fallbackMethod = "orderFallback")
public String placeOrder() {
  // Call payment service
  return restTemplate.getForObject("http://PAYMENTSERVICE/pay", String.class);
}

public String orderFallback(Exception e) {
  return "Payment service is down. Please try later.";
}
```

This avoids cascading failures and provides a graceful fallback to clients.

51. How can you implement rate limiting in Spring Cloud Gateway?

Answer

Rate limiting ensures a client doesn't overload a service by sending too many requests. Spring Cloud Gateway provides a filter RequestRateLimiter that uses Redis for token-bucket-based limiting.

Example (application.yml):

```
spring:
 cloud:
  gateway:
   routes:
    - id: orderservice
      uri: lb://ORDERSERVICE
      predicates:
       - Path=/orders/**
      filters:
       - name: RequestRateLimiter
         args:
          redis-rate-limiter.replenishRate: 10
          redis-rate-limiter.burstCapacity: 20
```

Here:

- replenishRate=10 → allows 10 requests per second.
- burstCapacity=20 → allows short spikes.

This protects downstream services from being overloaded by clients.

In interviews, it's good to mention you'd combine rate limiting with **API keys or user roles** for fairness.

52. What is distributed tracing, and how is it implemented in Spring Cloud?

Answer

In microservices, a single client request may span multiple services. Debugging issues requires tracing the request across services. **Distributed tracing** solves this by assigning a unique trace ID to each request and propagating it downstream.

Spring Cloud uses **Spring Cloud Sleuth** + **Zipkin/Jaeger**:

- Sleuth adds trace IDs and span IDs to logs automatically.
- Trace IDs are passed via headers (X-B3-TraceId, X-B3-SpanId).
- Zipkin or Jaeger collect and visualize traces across services.

Example log output:

```
2025-09-29 14:25:12 [orderservice,traceId=4f5e1a,spanId=9c2d7b] Placing order
2025-09-29 14:25:12 [paymentservice,traceId=4f5e1a,spanId=3a7b1c] Processing payment
```

Benefits:

- You can trace where latency occurs.
- Helps identify bottlenecks in a chain of microservices.
- Critical for production monitoring.

53. How does Spring Boot handle logging?

Answer

Spring Boot uses **Commons Logging** internally but defaults to **Logback** for logging. It supports out-of-the-box integration with SLF4J, Log4j2, and Logback.

Key points:

- Default logging format includes timestamp, log level, process ID, thread, and logger name.
- Default level: INFO.
- Can be customized via application.properties or logback-spring.xml.

Example (application.properties):

```
logging.level.root=INFO
logging.level.com.example=DEBUG
logging.file.name=app.log
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} - %msg%n

Usage in code:

private static final Logger logger = LoggerFactory.getLogger(OrderService.class);

public void placeOrder() {
  logger.info("Order placed successfully");
  logger.debug("Debugging order placement...");
}
```

In real-world apps, logs are often exported to monitoring systems like ELK stack (Elasticsearch + Logstash + Kibana) or Splunk.

54. What is Spring Boot DevTools, and why is it useful?

Answer

Spring Boot DevTools is a developer-friendly module that improves productivity during development.

Features:

- **Automatic restart**: restarts the app when classpath files change.
- **LiveReload support**: refreshes browser on static resource changes (HTML, CSS).
- **Config defaults**: sets cache off for templates, enables debug logging.

To enable, just add dependency:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-devtools</artifactId>
 <scope>runtime</scope>
</dependency>
```

Example: change a controller class, save → DevTools restarts app automatically within a second.

This eliminates the need to restart the entire application manually after every code change.

55. How does Spring Boot support scheduling and asynchronous tasks?

Answer

Spring Boot provides built-in support for task scheduling (@Scheduled) and asynchronous execution (@Async).

1. **Scheduled Tasks**:
    - Enable with @EnableScheduling.
    - Annotate methods with @Scheduled(fixedRate = ...) or cron expressions.

Example:

```
@Component
public class ReportScheduler {

 @Scheduled(fixedRate = 5000)
 public void generateReport() {
  System.out.println("Report generated at: " + new Date());
 }
}
```

This runs the method every 5 seconds.

2. **Asynchronous Tasks**:
    - Enable with @EnableAsync.
    - Annotate methods with @Async.

Example:

```
@Service
public class EmailService {

 @Async
 public void sendEmail(String to) {
  System.out.println("Sending email to: " + to);
 }
}
```

This method runs in a separate thread, so the caller doesn't wait for it.

Real-world usage:

- Scheduling cron jobs (e.g., nightly DB cleanup).
- Sending emails, SMS, or notifications asynchronously.

56. How does Spring MVC handle request mapping?

Answer

Spring MVC maps HTTP requests to controller methods using annotations. The core annotation is @RequestMapping, while @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping are shortcuts for specific HTTP methods.

Example:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

 @GetMapping("/{id}")
 public Order getOrder(@PathVariable int id) {
  return new Order(id, "Laptop", 1);
 }

 @PostMapping
 public Order createOrder(@RequestBody Order order) {
  return order; // simulate save
 }
}
```

Here:

- /orders/{id} maps to getOrder() using a path variable.
- /orders (POST) maps to createOrder(), which takes a JSON request body.

Spring automatically converts JSON to Java objects using Jackson and vice versa.

57. How do you handle exceptions globally in Spring Boot?

Answer

Instead of writing try-catch blocks in every controller, Spring provides @ControllerAdvice and @ExceptionHandler for centralized exception handling.

Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {

 @ExceptionHandler(ResourceNotFoundException.class)
 public ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {
  return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
 }

 @ExceptionHandler(Exception.class)
 public ResponseEntity<String> handleGeneral(Exception ex) {
  return new ResponseEntity<>("Something went wrong",
HttpStatus.INTERNAL_SERVER_ERROR);
 }
}
```

Now, if any controller throws ResourceNotFoundException, the handler catches it and returns a proper 404 response.

This ensures consistent error responses across all APIs.

58. What is the difference between Filters and Interceptors in Spring MVC?

Answer

Both can process requests before reaching controllers, but they work at different layers.

- **Filters** (Servlet API):
  - Run before Spring MVC.
  - Suitable for authentication, logging, request validation.
  - Work on all requests, including static resources.
- **Interceptors** (Spring-specific):
  - Run within Spring MVC, before and after controller execution.
  - Suitable for cross-cutting concerns like measuring execution time or modifying model attributes.

Example Interceptor:

```
@Component
public class LoggingInterceptor implements HandlerInterceptor {
 @Override
 public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
  System.out.println("Request URI: " + req.getRequestURI());
  return true;
 }
}
```

Registered via WebMvcConfigurer.

In practice:

- Use filters for generic request handling.
- Use interceptors for controller-specific concerns.

59. How does Spring Boot handle graceful shutdown?

Answer

Graceful shutdown means allowing ongoing requests to complete while preventing new ones when the application is stopping. This is important in production (e.g., Kubernetes deployments) so users don't face abrupt failures.

Spring Boot (2.3+) supports graceful shutdown out of the box:

- When the app receives a shutdown signal, the embedded server (Tomcat, Jetty, Undertow) stops accepting new requests.
- It waits for in-flight requests to finish before terminating.

Enable it in application.properties:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

Here, Boot will wait up to 30 seconds for active requests to complete before shutting down.

This ensures smoother deployments with zero downtime.

60. How can you customize Actuator endpoints in Spring Boot?

Answer

Spring Boot Actuator provides endpoints like /actuator/health, /actuator/metrics. These can be enabled, disabled, or secured as needed.

Example configuration:

```
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always
management.endpoints.web.base-path=/manage
```

Here:

- Only health, info, and metrics endpoints are exposed.
- Health details are always shown (useful in monitoring).
- Endpoints are available under /manage instead of /actuator.

You can also create custom endpoints:

```
@Component
@Endpoint(id = "customstatus")
public class CustomEndpoint {
  @ReadOperation
  public String status() {
    return "All systems running fine!";
  }
}
```

Now /actuator/customstatus gives a custom health-like response.

This is often used for exposing business-level health checks (e.g., “is payment gateway reachable?”).

61. How does Spring Boot support metrics and monitoring?

Answer

Spring Boot integrates with **Micrometer**, a metrics collection facade. Micrometer can export metrics to monitoring systems like Prometheus, Grafana, Datadog, CloudWatch.

Default metrics include:

- JVM metrics (memory, GC, threads).
- HTTP request metrics (count, latency, status).
- Database connection pool metrics (HikariCP).

Example:

management.endpoints.web.exposure.include=health,metrics,prometheus

Now /actuator/prometheus exposes metrics in Prometheus format.

Custom metrics can be added via MeterRegistry:

```
@Component
public class OrderMetrics {
 private final Counter orderCounter;

 public OrderMetrics(MeterRegistry registry) {
  this.orderCounter = Counter.builder("orders_placed").register(registry);
 }

 public void recordOrder() {
  orderCounter.increment();
 }
}
```

Now orders_placed is tracked and visible in monitoring dashboards.

In real-world microservices, Actuator + Micrometer + Prometheus/Grafana is a very common stack.

62. How does Spring Boot handle asynchronous processing?

Answer

Spring Boot supports asynchronous method execution using @Async. This allows tasks to run in a separate thread without blocking the main flow.

Steps:

1. Enable async with @EnableAsync in the config class.
2. Annotate methods with @Async.

Example:

```
@Configuration
@EnableAsync
public class AsyncConfig { }

@Service
public class EmailService {

  @Async
  public void sendEmail(String to) {
    System.out.println("Sending email to " + to + " in thread " +
Thread.currentThread().getName());
  }
}
```

Now when sendEmail() is called, it runs asynchronously in another thread.

Use cases: sending emails, notifications, processing background jobs.

Best practice: always configure a custom thread pool (ThreadPoolTaskExecutor) instead of relying on default.

63. What is event-driven programming in Spring, and how are events handled?

Answer

Spring provides an **application event mechanism** where components can publish and listen to events asynchronously or synchronously.

- **Publisher**: uses ApplicationEventPublisher to publish events.
- **Listener**: methods annotated with @EventListener.

Example:

```
public class OrderPlacedEvent {
  private final String orderId;
  public OrderPlacedEvent(String orderId) { this.orderId = orderId; }
  public String getOrderId() { return orderId; }
}

@Service
public class OrderService {
  @Autowired
  private ApplicationEventPublisher publisher;

  public void placeOrder(String orderId) {
    publisher.publishEvent(new OrderPlacedEvent(orderId));
  }
}

@Component
public class NotificationListener {
  @EventListener
  public void handleOrder(OrderPlacedEvent event) {
    System.out.println("Notifying for order: " + event.getOrderId());
  }
}
```

Here, when an order is placed, a notification event is triggered without tight coupling between services.

This is widely used in microservices for decoupled communication.

64. How does Spring Boot integrate with messaging systems like Kafka and RabbitMQ?

Answer

Spring Boot integrates with messaging systems via **Spring Kafka** and **Spring AMQP**. This supports event-driven and asynchronous communication between services.

Example with Kafka:

1. Add spring-kafka dependency.
2. Define producer and consumer.

Producer:

```
@Service
public class OrderProducer {
 @Autowired
 private KafkaTemplate<String, String> kafkaTemplate;

 public void sendOrder(String order) {
  kafkaTemplate.send("orders", order);
 }
}
```

Consumer:

```
@Service
public class OrderConsumer {
 @KafkaListener(topics = "orders", groupId = "order_group")
 public void consume(String order) {
  System.out.println("Received order: " + order);
 }
}
```

Example with RabbitMQ:

```
@Service
public class RabbitMqProducer {
 @Autowired
 private AmqpTemplate amqpTemplate;

 public void sendMessage(String msg) {
  amqpTemplate.convertAndSend("orderExchange", "orderKey", msg);
 }
}
```

Messaging is crucial in microservices for:

- Decoupling services.
- Handling spikes in load (queueing).
- Event-driven architectures.

65. How do you handle file upload and download in Spring Boot?

Answer

Spring Boot provides built-in support for multipart file uploads using MultipartFile.

File upload example:

```
@RestController
@RequestMapping("/files")
public class FileController {

 @PostMapping("/upload")
 public String uploadFile(@RequestParam("file") MultipartFile file) throws
IOException {
  String path = "uploads/" + file.getOriginalFilename();
  file.transferTo(new File(path));
  return "File uploaded: " + path;
 }

 @GetMapping("/download/{filename}")
 public ResponseEntity<Resource> downloadFile(@PathVariable String
filename) throws IOException {
  Path path = Paths.get("uploads/" + filename);
  Resource resource = new UrlResource(path.toUri());
  return ResponseEntity.ok()
     .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=" +
filename)
     .body(resource);
 }
}
```

- MultipartFile handles uploads.
- ResponseEntity<Resource> handles downloads.

In real-world apps, uploads are usually stored in cloud storage (AWS S3, GCP Storage) instead of local disk.

66. What are REST API best practices in Spring Boot?

Answer

When building REST APIs with Spring Boot, follow these best practices:

1. **Use proper HTTP methods**:
    - GET → fetch data
    - POST → create
    - PUT → update
    - DELETE → delete
2. **Meaningful endpoints**:
    - /users/1/orders instead of /getOrdersForUser.
3. **Return proper status codes**:
    - 200 OK, 201 Created, 404 Not Found, 400 Bad Request.
4. **Use DTOs**:
    - Don't expose entity classes directly.
5. **Validation**:
    - Use @Valid with @NotNull, @Size, etc.
6. **Error handling**:
    - Use global @ControllerAdvice.
7. **Pagination and filtering**:
    - Expose query params (/orders?page=0&size=10).

Example with validation:

```
public class UserDto {
 @NotBlank private String name;
 @Email private String email;
}

@PostMapping("/users")
public ResponseEntity<UserDto> createUser(@Valid @RequestBody UserDto user) {
 return ResponseEntity.status(HttpStatus.CREATED).body(user);
}
```

These practices make APIs robust, user-friendly, and production-ready.

67. How do you handle CORS in Spring Boot?

Answer

CORS (Cross-Origin Resource Sharing) is a browser mechanism that blocks cross-domain requests unless allowed. In microservices + frontend apps, this needs to be configured.

Spring Boot ways to handle:

1. **Global config**:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
 @Override
 public void addCorsMappings(CorsRegistry registry) {
  registry.addMapping("/**")
      .allowedOrigins("http://localhost:3000")
      .allowedMethods("GET", "POST", "PUT", "DELETE");
 }
}
```

2. **At controller level**:

```
@CrossOrigin(origins = "http://localhost:3000")
@RestController
@RequestMapping("/orders")
public class OrderController {
 @GetMapping
 public List<Order> getOrders() { return List.of(new Order(1, "Laptop")); }
}
```

3. **Properties (Spring Security 5.4+)** → configure inside HttpSecurity with .cors().

Best practice:

- Allow only trusted origins, not * in production.
- Handle preflight (OPTIONS) requests properly.

68. How do you implement API versioning in Spring Boot?

Answer

API versioning ensures backward compatibility when APIs evolve. Spring Boot supports multiple strategies:

1. **URI versioning**

```
@GetMapping("/api/v1/orders")
public List<Order> getOrdersV1() { ... }

@GetMapping("/api/v2/orders")
public List<OrderDto> getOrdersV2() { ... }
```

Simple, clear, but URLs change.

2. **Request parameter versioning**

```
@GetMapping(value = "/api/orders", params = "version=1")
public List<Order> getOrdersV1() { ... }

@GetMapping(value = "/api/orders", params = "version=2")
public List<OrderDto> getOrdersV2() { ... }
```

3. **Header versioning**

```
@GetMapping(value = "/api/orders", headers = "X-API-VERSION=1")
public List<Order> getOrdersV1() { ... }
```

4. **Content negotiation (media type)**

```
@GetMapping(value = "/api/orders", produces =
"application/vnd.company.app-v1+json")
public List<Order> getOrdersV1() { ... }
```

Best practice: start with URI versioning (easy to maintain), move to header/content-based versioning for mature APIs.

69. What is HATEOAS, and how is it used in Spring Boot?

Answer

HATEOAS (Hypermedia as the Engine of Application State) is a REST principle where responses include **links to related actions**. Instead of just data, clients also get navigational links.

Spring provides support via spring-boot-starter-hateoas.

Example:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

 @GetMapping("/{id}")
 public EntityModel<Order> getOrder(@PathVariable int id) {
  Order order = new Order(id, "Laptop", 1);
  return EntityModel.of(order,
     linkTo(methodOn(OrderController.class).getOrder(id)).withSelfRel(),
     linkTo(methodOn(OrderController.class).getAllOrders()).withRel("all-orders"));
 }

 @GetMapping
 public List<Order> getAllOrders() { ... }
}
```

Response:

```
{
 "id": 1,
 "item": "Laptop",
 "quantity": 1,
 "_links": {
  "self": { "href": "http://localhost:8080/orders/1" },
  "all-orders": { "href": "http://localhost:8080/orders" }
 }
}
```

This makes APIs more **discoverable** and **self-documented**.

Real-world: used in hypermedia-driven APIs (HAL format, HAL Explorer).

70. How do you document APIs in Spring Boot using Swagger/OpenAPI?

Answer

Swagger (OpenAPI) provides interactive documentation for REST APIs. In Spring Boot, it's implemented using **springdoc-openapi**.

Setup:

```
<dependency>
 <groupId>org.springdoc</groupId>
 <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
 <version>2.2.0</version>
</dependency>
```

By default, docs are available at:

- http://localhost:8080/swagger-ui.html
- http://localhost:8080/v3/api-docs

Example controller with Swagger annotations:

```
@RestController
@RequestMapping("/users")
public class UserController {

 @Operation(summary = "Get user by ID", description = "Fetch a single user by their ID")
 @GetMapping("/{id}")
 public User getUser(@PathVariable int id) {
  return new User(id, "John Doe");
 }
}
```

Swagger UI then displays API endpoints with descriptions and allows testing them directly.

Benefits:

- Interactive testing for developers and QA.
- Self-updating docs as code changes.
- Widely adopted in microservices environments.

71. How do you implement pagination and sorting in Spring Boot REST APIs?

Answer

Spring Data JPA provides built-in support for pagination and sorting using Pageable and Page interfaces.

Repository example:

public interface EmployeeRepository extends JpaRepository<Employee, Integer> { }

Controller example:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

 @Autowired
 private EmployeeRepository repo;

 @GetMapping
 public Page<Employee> getEmployees(Pageable pageable) {
  return repo.findAll(pageable);
 }
}
```

Now clients can request:

GET /employees?page=0&size=5&sort=name,asc

Response includes:

- Content (list of employees).
- Metadata (totalPages, totalElements, pageSize).

This ensures APIs remain scalable when dealing with large datasets.

72. How do you perform input validation in Spring Boot?

Answer

Spring Boot integrates with Hibernate Validator (JSR 380/Bean Validation). You annotate fields in DTOs/entities with constraints and validate them using @Valid.

DTO example:

```
public class UserDto {
 @NotBlank(message = "Name is required")
 private String name;

 @Email(message = "Invalid email")
 private String email;

 @Min(18)
 private int age;
}
```

Controller example:

```
@PostMapping("/users")
public ResponseEntity<String> createUser(@Valid @RequestBody UserDto user) {
 return ResponseEntity.ok("User created: " + user.getName());
}
```

If validation fails, Spring automatically returns a 400 Bad Request with error details.

You can also create custom validators using @Constraint for business-specific rules.

73. What is internationalization (i18n) in Spring Boot, and how is it implemented?

Answer

Internationalization (i18n) allows apps to support multiple languages by externalizing messages into resource bundles.

Steps in Spring Boot:

1. Create message files → messages.properties, messages_fr.properties, messages_de.properties.

```
# messages.properties
welcome.message=Hello, Welcome!

# messages_fr.properties
welcome.message=Bonjour, Bienvenue!
```

2. Configure a message source:

```
@Bean
public ResourceBundleMessageSource messageSource() {
ResourceBundleMessageSource source =
newResourceBundleMessageSource();
 source.setBasename("messages");
 source.setDefaultEncoding("UTF-8");
 return source;
}
```

3. Use in controllers:

```
@Autowired
private MessageSource messageSource;

@GetMapping("/welcome")
public String welcome(Locale locale) {
 return messageSource.getMessage("welcome.message", null, locale);
}
```

4. Call API with header:

GET /welcome
Accept-Language: fr

Response → Bonjour, Bienvenue!

This is especially useful in **e-commerce apps** serving global customers.

74. How do you use Spring Boot profiles for multi-environment configuration?

Answer

Profiles let you define environment-specific configurations (dev, test, prod) and activate them dynamically.

1. **Profile-specific property files**:

   application-dev.properties

   application-prod.properties

Example:

```
# application-dev.properties
server.port=8081

# application-prod.properties
server.port=8080
```

2. **Activate profile**:

- In application.properties:

  spring.profiles.active=dev

- Or via command line:

  java -jar app.jar --spring.profiles.active=prod

3. **Profile-specific beans**:

```
@Configuration
@Profile("dev")
public class DevConfig {
 @Bean
 public String demoBean() {
  return "Dev Bean";
 }
}
```

This makes switching between environments seamless without code changes.

75. How does Spring Boot support configuration properties binding?

Answer

Instead of using scattered @Value annotations, Spring Boot allows binding properties to POJO classes using @ConfigurationProperties.

Example:

app.name=MyApp
app.version=1.0

POJO:

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppConfig {
  private String name;
  private String version;

  // getters and setters
}
```

Usage:

```
@Autowired
private AppConfig config;

public void printConfig() {
  System.out.println(config.getName() + " - " + config.getVersion());
}
```

Advantages:

- Groups related configs together.
- Type-safe (maps directly to Java types).
- Easy to validate using @Validated and annotations like @NotNull.

76. What is a custom Spring Boot starter, and why would you create one?

Answer

A Spring Boot starter is a pre-packaged set of dependencies and auto-configuration for a specific feature. Boot itself provides starters like spring-boot-starter-web.

You can create custom starters when your organization has common functionality shared across projects.

Steps:

1. Create a new module mycompany-spring-boot-starter.
2. Add dependencies you want to include (e.g., logging, monitoring).
3. Provide auto-configuration class:

```
@Configuration
@ConditionalOnClass(MyService.class)
public class MyServiceAutoConfiguration {
 @Bean
 public MyService myService() {
  return new MyService();
 }
}
```

4. Register in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports.

Now, when another project adds your starter dependency:

```
<dependency>
 <groupId>com.mycompany</groupId>
 <artifactId>mycompany-spring-boot-starter</artifactId>
</dependency>
```

Spring Boot auto-configures everything without extra setup.

This reduces boilerplate and enforces standards across multiple projects.

77. What is Spring Boot CLI, and how is it used?

Answer

Spring Boot CLI (Command Line Interface) is a tool that lets you quickly prototype and run Spring apps using **Groovy scripts** instead of full Java boilerplate.

Key features:

- Auto-imports common Spring classes.
- Supports running apps with just a few lines of code.
- Useful for demos, quick POCs, or scripting tasks.

Example: app.groovy

```
@RestController
class HelloController {
   @RequestMapping("/")
   String home() {
      "Hello Spring Boot CLI"
   }
}
```

Run it:

spring run app.groovy

This starts a Spring Boot app with an embedded Tomcat server.

In practice:

- Mostly used for quick prototypes.
- For production, developers stick to Java + Maven/Gradle builds.

78. How do you create custom Actuator endpoints in Spring Boot?

Answer

Spring Boot Actuator exposes built-in endpoints, but you can also create custom ones for application-specific monitoring.

Steps:

1. Add spring-boot-starter-actuator dependency.
2. Create a custom endpoint class.

Example:

```
@Component
@Endpoint(id = "customstatus")
public class CustomEndpoint {

 @ReadOperation
 public String status() {
  return "All systems operational";
 }

 @WriteOperation
 public String updateStatus(@Selector String newStatus) {
  return "Status updated to: " + newStatus;
 }
}
```

Now /actuator/customstatus is available.

- @ReadOperation → GET endpoint.
- @WriteOperation → POST/PUT.

Use cases: exposing internal business checks like “is the payment gateway alive?” or “queue depth of Kafka topics.”

79. How do you deploy a Spring Boot app with Docker and Kubernetes?

Answer

Spring Boot apps package into fat jars, which are easily containerized with Docker.

Docker deployment

1. Create a Dockerfile:

```
FROM openjdk:17-jdk-slim
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

2. Build & run:

docker build -t myapp .
docker run -p 8080:8080 myapp

Kubernetes deployment

1. Create a Deployment + Service manifest:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: myapp
spec:
 replicas: 2
 selector:
  matchLabels:
   app: myapp
 template:
  metadata:
   labels:
    app: myapp
  spec:
   containers:
    - name: myapp
      image: myapp:latest
      ports:
       - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: myapp-service
spec:
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 8080
  type: LoadBalancer
```

2. Deploy:

kubectl apply -f deployment.yaml

Now your app runs on Kubernetes with **replicas for scaling** and a **service for load balancing**.

Real-world: combine with **ConfigMaps/Secrets** for configs and **Ingress** for external routing.

80. How does Spring Boot integrate with Kubernetes health checks?

Answer

Kubernetes uses **liveness** and **readiness** probes to check if a container is healthy and ready to serve traffic. Spring Boot's Actuator endpoints map directly to these probes.

Setup in application.properties:

management.endpoint.health.probes.enabled=true
management.endpoints.web.exposure.include=health

Now Actuator exposes:

- /actuator/health/liveness → checks if the app is running.
- /actuator/health/readiness → checks if the app is ready to accept traffic.

Kubernetes config example:

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 5
```

This ensures:

- Pods failing readiness probes won't receive traffic.
- Pods failing liveness probes are restarted automatically.

81. How do you set up centralized logging for Spring Boot in production?

Answer

In microservices, logs from different containers need to be centralized for monitoring. Common approach: **ELK/EFK stack**.

- **ELK** → Elasticsearch, Logstash, Kibana.
- **EFK** → Elasticsearch, Fluentd, Kibana (more cloud-native).

Spring Boot outputs logs in JSON format for easier parsing:

```
logging.pattern.console={"timestamp":"%d{yyyy-MM-dd HH:mm:ss}", "level":"%p", "logger":"%c", "message":"%m"}%n
```

Logs are collected by Fluentd/Logstash and shipped to Elasticsearch, where Kibana visualizes them.

Best practices:

- Use correlation IDs (Spring Sleuth) to trace logs across microservices.
- Avoid writing to local disk in containers; write to stdout/stderr for Docker/Kubernetes to capture.

82. How do you deploy Spring Boot apps to cloud platforms like AWS/GCP/Azure?

Answer

Spring Boot apps can run as JARs, WARs, or containers, so they integrate well with cloud services.

AWS

- Deploy as container → Amazon ECS/EKS.
- Deploy JAR → Elastic Beanstalk.
- Use RDS for DB, CloudWatch for monitoring.

GCP

- Cloud Run → serverless containerized Spring Boot apps.
- GKE → Kubernetes.
- Cloud SQL → managed database.

Azure

- Azure App Service → direct deployment of JARs/Docker images.
- AKS → Kubernetes service.
- Application Insights → monitoring.

CI/CD pipelines (Jenkins, GitHub Actions, GitLab CI) are often used to automate builds, Dockerization, and deployments.

Real-world scenario:

- Build → Dockerize → Push to registry (ECR/GCR/ACR).
- Deploy → Kubernetes (EKS/GKE/AKS).
- Monitor → Prometheus + Grafana, or native cloud monitoring.

83. What is the lifecycle of a Spring Bean?

Answer

A Spring bean goes through multiple stages, managed by the Spring IoC container.

1. **Instantiation** – Spring creates the bean instance, either via a constructor or a static factory method.
2. **Populate Properties** – Dependencies are injected (constructor, setter, or field injection).
3. **Aware Interfaces (optional)** – If the bean implements special interfaces like BeanNameAware or ApplicationContextAware, Spring calls those to pass metadata.
4. **BeanPostProcessors (before init)** – Any registered BeanPostProcessor gets a chance to modify the bean before initialization.
5. **Initialization** – If the bean has an @PostConstruct method or implements InitializingBean, those are invoked.
6. **BeanPostProcessors (after init)** – Post-processors can again modify the bean after initialization.
7. **Bean is Ready for Use** – Application code can now use it.
8. **Destruction** – When the container shuts down, Spring calls @PreDestroy or DisposableBean.destroy().

Example:

```
@Component
public class PaymentService implements InitializingBean, DisposableBean {

   @Override
   public void afterPropertiesSet() {
      System.out.println("PaymentService initialized");
   }

   @Override
   public void destroy() {
      System.out.println("PaymentService destroyed");
   }
}
```

In a real-world banking app, imagine a PaymentService bean:

- At init → it connects to the payment gateway.
- At destroy → it closes the connection pool.

84. What is the difference between BeanFactory and ApplicationContext?

Answer

Both are Spring IoC containers but with different capabilities.

- **BeanFactory**
  - The most basic container.
  - Provides dependency injection.
  - Lazily initializes beans (creates bean only when requested).
  - Lightweight and useful in resource-constrained environments.
- **ApplicationContext**
  - Built on top of BeanFactory.
  - Eagerly initializes singleton beans by default.
  - Provides enterprise-level features:
    - Internationalization (i18n)
    - Event publishing (ApplicationEventPublisher)
    - AOP support
    - BeanPostProcessor and BeanFactoryPostProcessor auto-detection
  - Common implementations: ClassPathXmlApplicationContext, AnnotationConfigApplicationContext.

Example:

```
ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
OrderService orderService = context.getBean(OrderService.class);
```

In practice, developers almost always use **ApplicationContext** since it provides all the features of BeanFactory plus more.

85. What is ApplicationEventPublisher in Spring, and how are events handled?

Answer

Spring provides an **event-driven programming model** where beans can publish and listen to events without being tightly coupled.

- **Publisher**: Publishes an event using ApplicationEventPublisher.
- **Event**: A simple POJO extending ApplicationEvent or any custom class.
- **Listener**: Annotated with @EventListener or implementing ApplicationListener.

Example:

```
public class OrderPlacedEvent {
   private final String orderId;
   public OrderPlacedEvent(String orderId) { this.orderId = orderId; }
   public String getOrderId() { return orderId; }
}

@Service
public class OrderService {
   @Autowired private ApplicationEventPublisher publisher;

   public void placeOrder(String orderId) {
      // save order logic
      publisher.publishEvent(new OrderPlacedEvent(orderId));
   }
}

@Component
public class NotificationListener {
   @EventListener
   public void onOrderPlaced(OrderPlacedEvent event) {
      System.out.println("Sending notification for order: " + event.getOrderId());
   }
}
```

Real-world analogy:

- When an order is placed in an e-commerce system, multiple things need to happen-send email, reduce stock, trigger shipment.
- Instead of hardcoding everything in OrderService, Spring events let other beans react asynchronously.

86. What are circular dependencies in Spring, and how are they handled?

Answer

A circular dependency happens when two or more beans depend on each other, forming a loop.

Example:

```
@Component
class A {
  @Autowired private B b;
}

@Component
class B {
  @Autowired private A a;
}
```

Here, when Spring tries to create bean A, it also needs bean B. But B again requires A, so the container doesn't know which to build first.

- **By default (constructor injection)** → This fails with a BeanCurrentlyInCreationException.
- **With setter/field injection** → Spring can resolve it by creating one bean, injecting the reference later.

Ways to handle:

1. **Use setter injection instead of constructor injection** if circular dependency is unavoidable.

   Use @Lazy to delay injection until it's actually needed.

   ```
   @Component
   class A {
     @Autowired @Lazy private B b;
   }
   ```

2. **Refactor design** – Circular dependencies are often a code smell. Introduce a new service to break the cycle.

Real-world example:

- If PaymentService depends on OrderService and OrderService depends on PaymentService, better extract a PaymentValidator that both can use instead of wiring them into a loop.

87. How does Spring Boot's auto-configuration work internally?

Answer

Auto-configuration is what makes Spring Boot "just work" without manual setup.

- Spring Boot uses the annotation @EnableAutoConfiguration (included in @SpringBootApplication).
- At startup, Boot loads spring.factories (or AutoConfiguration.imports in newer versions) from all JARs in the classpath.
- Each entry points to an auto-configuration class.
- Boot checks **conditions** (@ConditionalOnClass, @ConditionalOnProperty, etc.) to decide whether to apply that configuration.

Example:
If spring-boot-starter-web is on the classpath:

- WebMvcAutoConfiguration is loaded.
- It auto-configures a DispatcherServlet, RequestMappingHandlerMapping, and Jackson for JSON.
- If you define your own beans, Boot backs off ("auto-configuration is smart defaults, not hard rules").

Analogy:

Think of it like a hotel buffet → dishes are prepared automatically if the ingredients are present. If you bring your own dish (custom bean), the chef doesn't prepare that item.

88. What is the difference between @Configuration and @Component in Spring?

Answer

Both register beans with the Spring container, but they serve different purposes.

- **@Component**
  - Generic stereotype for any Spring bean.
  - Detected during component scanning.
  - Typically used for services, repositories, controllers.
- **@Configuration**
  - A specialized form of @Component.
  - Used for defining beans explicitly with @Bean methods.
  - Spring ensures such classes are **CGLIB proxied**, meaning each @Bean method always returns the same singleton instance (instead of creating new objects).

Example:

```
@Configuration
public class AppConfig {
   @Bean
   public PaymentService paymentService() {
      return new PaymentService();
   }
}

@Component
public class NotificationService {
   public void notifyUser(String msg) {
      System.out.println("Notifying: " + msg);
   }
}
```

Key difference:

- If you use @Component with @Bean methods, multiple calls may create new instances (not recommended).
- @Configuration ensures proper singleton behavior.

Real-world analogy:

- @Component → a regular employee in the company.
- @Configuration → the HR manager who hires employees (creates beans) and ensures there's only one employee per position.

89. What are @Conditional annotations in Spring, and when are they useful?

Answer

Spring's @Conditional annotations let you create beans only if certain conditions are met. This makes applications flexible and environment-aware.

- **@ConditionalOnClass** – bean created only if a specific class is on the classpath.
- **@ConditionalOnMissingBean** – bean created only if no other bean of that type exists.
- **@ConditionalOnProperty** – bean created only if a property has a specific value.

Example:

```
@Configuration
public class PaymentConfig {

   @Bean
   @ConditionalOnProperty(name = "payment.gateway", havingValue = "razorpay")
   public PaymentService razorpayService() {
      return new RazorpayService();
   }

   @Bean
   @ConditionalOnProperty(name = "payment.gateway", havingValue = "stripe")
   public PaymentService stripeService() {
      return new StripeService();
   }
}
```

Now, based on payment.gateway property in application.properties, Spring decides which bean to load.

Real-world use case:
In an e-commerce app, you might want to switch between different payment providers (Razorpay in India, Stripe internationally) without changing code - only configs.

90. How do Spring Profiles differ from @Conditional, and when should each be used?

Answer

- **Spring Profiles** are for environment-level decisions (dev, test, prod).
- **@Conditional** is for fine-grained bean loading decisions based on properties, class availability, etc.

Example with Profiles:

```
@Configuration
@Profile("dev")
class DevDataSourceConfig {
   @Bean DataSource devDataSource() { return new H2DataSource(); }
}

@Configuration
@Profile("prod")
class ProdDataSourceConfig {
   @Bean DataSource prodDataSource() { return new MySQLDataSource(); }
}
```

Activate with:
--spring.profiles.active=prod

Example with Conditional:
Use @ConditionalOnProperty to toggle a feature flag (featureX.enabled=true).

Rule of thumb:

- Use **Profiles** when beans differ by environment.
- Use **@Conditional** when beans differ by configuration or dependencies.

91. How does Spring resolve bean ambiguity when multiple candidates exist?

Answer
If multiple beans of the same type exist, Spring needs a way to decide which one to inject. It provides several mechanisms:

1. **@Primary** – marks a bean as the default choice.

```
@Component
@Primary
class EmailNotificationService implements NotificationService { ... }
```

2. **@Qualifier** – explicitly specify which bean to inject.

```
@Component("smsService")
class SmsNotificationService implements NotificationService { ... }

@Component
class AlertService {
  @Autowired
  @Qualifier("smsService")
  private NotificationService notificationService;
}
```

3. **Custom annotations** – you can create your own meta-annotations for clarity.

Real-world analogy:
Imagine multiple couriers (DHL, FedEx, UPS). If you don't specify, Spring is confused.

- @Primary is like marking FedEx as the default courier.
- @Qualifier("DHL") is like saying "use DHL specifically for this package."

92. What are the different bean scopes in Spring?

Answer

Bean scope defines the lifecycle and visibility of a Spring bean. The main scopes are:

1. **singleton (default):**
    - One instance per Spring container.
    - All requests for the bean return the same object.
2. @Component
3. public class AppConfig { }

   → Useful for stateless services (e.g., service classes).

4. **prototype:**
    - A new instance every time the bean is requested.

   @Scope("prototype")

   @Component

   public class TempService { }

   → Useful when you need stateful or short-lived objects.

5. **request:**
    - One instance per HTTP request (web applications).
6. **session:**
    - One instance per user session.
7. **application:**
    - One instance per ServletContext.

Interview Tip:

Always mention that by default, beans are **singleton**, but this is *Spring's singleton* (per container, not JVM-wide).

93. What is the difference between Spring's singleton bean and the Singleton Design Pattern?

Answer

- **Singleton Design Pattern:**
  - Enforced at the **class level** using private constructors and static instance.
  - Ensures only one object exists per JVM.
- **Spring Singleton Bean:**
  - Enforced at the **container level**.
  - Each Spring ApplicationContext creates **one bean instance per container**.
  - If you create multiple contexts, each has its own bean instance.

Example:

```
@Component
@Scope("singleton") // Spring singleton
public class MyService { }
```

Key difference:

- Design Pattern = one instance per JVM.
- Spring = one instance per container.

94. What is @Lazy in Spring, and when should it be used?

Answer

By default, Spring creates singleton beans at container startup (eager loading).
@Lazy delays bean creation until it's actually requested.

Usage:

```
@Component
@Lazy
public class HeavyService {
   public HeavyService() {
      System.out.println("HeavyService initialized");
   }
}
```

With @Lazy, the bean is only created when it's first accessed.

When to use:

- For beans that are expensive to create (large caches, network clients).
- When a bean might not always be needed.
- To reduce startup time in large applications.

Pro Tip:

- You can also use @Lazy on @Configuration classes to lazy-load all beans inside them.
- In microservices, combine with profiles (@Profile) for environment-based loading.

95. What is Auto-Configuration in Spring Boot?

Answer

Auto-Configuration is one of Spring Boot's core features. It automatically configures beans and settings based on the dependencies on your classpath.

- Example:
    - If spring-boot-starter-web is on the classpath → Boot auto-configures a DispatcherServlet, Jackson for JSON, and an embedded Tomcat.
    - If spring-boot-starter-data-jpa is present → it configures an EntityManagerFactory, DataSource, and JpaTransactionManager.

How it works:

- Driven by @EnableAutoConfiguration (included in @SpringBootApplication).
- Looks at spring.factories (Spring Boot < 3) or AutoConfiguration.imports (Spring Boot 3+).
- Applies conditional annotations like @ConditionalOnClass, @ConditionalOnMissingBean.

Benefits:

- Zero setup for common use cases.
- You only override configurations you need.

96. What is @EnableAutoConfiguration in Spring Boot?

Answer

@EnableAutoConfiguration tells Spring Boot to automatically configure beans based on the dependencies on the classpath.

- It's part of @SpringBootApplication, so you don't need to declare it explicitly.
- It uses conditional logic to configure only what's relevant.

Example:

```
@SpringBootApplication
public class MyApp {
   public static void main(String[] args) {
      SpringApplication.run(MyApp.class, args);
   }
}
```

Here, @SpringBootApplication =

- @Configuration
- @EnableAutoConfiguration
- @ComponentScan

Key Point:

- Auto-configuration is applied **after** your explicit configuration.
- If you define your own bean, Boot won't override it (@ConditionalOnMissingBean).

97. How do you create custom Auto-Configuration in Spring Boot?

Answer

You can create your own auto-configuration module for reusable libraries.

Steps:

1. Create a configuration class with conditional annotations:

```
@Configuration
@ConditionalOnClass(MyService.class)   // load only if MyService is present
public class MyServiceAutoConfiguration {

  @Bean
  @ConditionalOnMissingBean
  public MyService myService() {
    return new MyService();
  }
}
```

2. Register it in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (Spring Boot 3+).

com.example.config.MyServiceAutoConfiguration

3. Add it as a dependency in another project → Spring Boot auto-configures it.

When to use:

- Internal libraries shared across teams (logging, monitoring, auditing).
- Company-wide starters (mycompany-starter-logging).

Interview Tip:

Mention that this is exactly how Spring Boot's own starters (e.g., spring-boot-starter-data-jpa) work internally.

98. What is the role of ApplicationContext in Spring and Spring Boot?

Answer

ApplicationContext is the **central interface** in Spring for managing beans, configuration, and the application lifecycle.

- It builds on top of BeanFactory, adding:
    - Bean lifecycle management.
    - Dependency injection.
    - Event publication.
    - Internationalization (i18n).
    - AOP support.

In Spring Boot:

- SpringApplication.run() creates an ApplicationContext (usually AnnotationConfigServletWebServerApplicationContext for web apps).
- All beans (from auto-config + your code) live inside this context.

Example:

```
@SpringBootApplication
public class App {
   public static void main(String[] args) {
      ApplicationContext ctx = SpringApplication.run(App.class, args);
      System.out.println("Beans count: " + ctx.getBeanDefinitionCount());
   }
}
```

Key Point:

- ApplicationContext = Container of everything (your beans + Boot auto-config beans).
- Without it, dependency injection, auto-configuration, and events wouldn't work.

99. Explain the lifecycle of a Spring Boot application.

Answer

Spring Boot applications follow a clear lifecycle when you call SpringApplication.run().

Steps:

1. **Bootstrap**
    - Create and configure SpringApplication.
    - Prepare environment (system properties, env vars, application.properties).
2. **Create ApplicationContext**
    - Choose context type (AnnotationConfigApplicationContext for non-web, AnnotationConfigServletWebServerApplicationContext for web).
3. **Load Beans**
    - Component scan + auto-configuration.
    - Register and instantiate beans.
4. **Refresh Context**
    - Initialize beans, call @PostConstruct, run bean lifecycle callbacks.
5. **Run Application Runners**
    - Executes CommandLineRunner and ApplicationRunner beans.
6. **Application Ready**
    - All beans available, server started, events published.
7. **Shutdown**
    - On JVM shutdown → graceful shutdown of context.
    - Calls @PreDestroy and DisposableBean.destroy().

Tip for interviews:

You can hook into lifecycle using:

- ApplicationListener<ApplicationReadyEvent>
- @PostConstruct and @PreDestroy
- SmartLifecycle

100. What are Conditional Annotations in Spring Boot?

Answer

Conditional annotations control **when auto-configurations or beans should load**, based on classpath, properties, or environment.

Examples:

1. @ConditionalOnClass → load bean only if a class is present.

```
@Bean
@ConditionalOnClass(DataSource.class)
DataSourceInitializer initializer() { ... }
```

2. @ConditionalOnMissingBean → don't override user-defined beans.

```
@Bean
@ConditionalOnMissingBean
MyService myService() { return new MyService(); }
```

3. @ConditionalOnProperty → enable/disable features with config.

```
@Bean
@ConditionalOnProperty(name = "feature.x.enabled", havingValue = "true")
FeatureX featureX() { return new FeatureX(); }
```

4. @ConditionalOnExpression → load if SpEL expression is true.

Why important?

- They make auto-configuration flexible.
- Ensure Boot never forces configs - you stay in control.

Interview Angle:

When asked *"How does Boot know what to auto-configure?"* → Answer: *It uses @Conditional... annotations inside auto-config classes.*

101. What are Application Events in Spring?

Answer

Spring has an **event-driven model** inside the ApplicationContext. Beans can publish events, and other beans can listen to them.

- **Publisher**: uses ApplicationEventPublisher.
- **Listener**: methods annotated with @EventListener or beans implementing ApplicationListener<E>.
- Events propagate inside the same ApplicationContext.

Example:

```
// Event
public class OrderPlacedEvent extends ApplicationEvent {
   private final String orderId;
   public OrderPlacedEvent(Object source, String orderId) {
      super(source);
      this.orderId = orderId;
   }
   public String getOrderId() { return orderId; }
}

// Publisher
@Service
public class OrderService {
   @Autowired private ApplicationEventPublisher publisher;

   public void placeOrder(String id) {
      // business logic
      publisher.publishEvent(new OrderPlacedEvent(this, id));
   }
}

// Listener
@Component
public class NotificationService {
   @EventListener
   public void handleOrderPlaced(OrderPlacedEvent event) {
      System.out.println("Sending notification for order: " + event.getOrderId());
   }
}
```

Analogy:
Think of an e-commerce site → Placing an order triggers multiple listeners: inventory update, payment processing, email notification. Each module reacts without tight coupling.

102. What built-in events does Spring Boot provide during startup?

Answer

Spring Boot automatically publishes **lifecycle events** you can listen to:

- **ApplicationStartingEvent** → before context is created.
- **ApplicationEnvironmentPreparedEvent** → environment ready but context not loaded.
- **ApplicationPreparedEvent** → context created, but not refreshed.
- **ApplicationStartedEvent** → context refreshed, application started but not ready.
- **ApplicationReadyEvent** → application fully up and running.
- **ApplicationFailedEvent** → startup failed.

Example Listener:

```
@Component
public class StartupListener {

   @EventListener(ApplicationReadyEvent.class)
   public void onAppReady() {
      System.out.println("App is ready, start background jobs!");
   }
}
```

Tip:

- Use ApplicationReadyEvent for initialization tasks (like warm-up caches, connecting to external systems).
- Use ApplicationFailedEvent to clean up or alert on startup failure.

103. How do CommandLineRunner and ApplicationRunner fit into the lifecycle?

Answer

Both are callback interfaces to execute logic **just after the context is ready**.

- **CommandLineRunner** → exposes command-line arguments as a String[].
- **ApplicationRunner** → exposes arguments as ApplicationArguments (parsed key/value).

Example:

```
@Component
public class DataSeeder implements CommandLineRunner {
   @Override
   public void run(String... args) {
      System.out.println("Seeding initial data into DB...");
   }
}

@Component
public class ArgPrinter implements ApplicationRunner {
   @Override
   public void run(ApplicationArguments args) {
      System.out.println("Non-option args: " + args.getNonOptionArgs());
   }
}
```

When to use:

- CommandLineRunner → quick tasks, e.g., load config, clean temp files.
- ApplicationRunner → better when parsing command-line options.

In interviews:

If asked, emphasize these run **after ApplicationReadyEvent** - i.e., beans are loaded, DB connections ready, transactions can run

104. How does Spring Boot Auto-Configuration work internally?

Answer

Spring Boot's magic comes from **Auto-Configuration**, which wires beans for you based on what's on the classpath and environment properties.

Steps:

1. When your app starts with @SpringBootApplication, it implicitly includes @EnableAutoConfiguration.
2. That annotation triggers Spring's AutoConfigurationImportSelector.
3. This selector looks into:
    - META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (since Boot 2.7+, earlier it was spring.factories).
    - It finds all auto-config classes provided by starters. Example: WebMvcAutoConfiguration, JpaRepositoriesAutoConfiguration, etc.
4. Each auto-config class is annotated with conditions (@ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty) to ensure it only activates if:
    - The classpath has certain libraries.
    - No conflicting bean is already defined.
    - Certain properties are enabled.

Example:
DataSourceAutoConfiguration → creates a DataSource bean if:

- javax.sql.DataSource is on classpath.
- You didn't define your own DataSource.
- Properties like spring.datasource.url exist.

```
@Configuration
@ConditionalOnClass(DataSource.class)
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {
  @Bean
  @ConditionalOnMissingBean
```

```
    public DataSource dataSource(DataSourceProperties props) {
      return DataSourceBuilder.create()
          .url(props.getUrl())
           .username(props.getUsername())
           .password(props.getPassword())
           .build();
    }
}
```

So Boot **creates a datasource bean automatically** if the right conditions are met.

105. How do you customize or disable auto-configuration?

Answer

You rarely need to write XML or manual wiring - Boot lets you override auto-config easily.

Options:

1. **Define your own bean** → If you provide a @Bean of the same type, Boot's

```
@ConditionalOnMissingBean prevents duplication.

@Configuration
public class CustomDataSourceConfig {
  @Bean
  public DataSource customDataSource() {
    return new HikariDataSource(); // your custom config
  }
}
```

2. This disables Boot's default datasource auto-config.
3. **Disable specific auto-config classes**:

```
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
public class MyApp { }
```

4. **Use properties**:
   Many auto-configs can be toggled via application.properties. Example:

```
spring.jpa.open-in-view=false
spring.datasource.initialize=false
```

106. How can you write your own custom auto-configuration?

Answer

Sometimes you want a **shared starter** across teams (e.g., custom logging, tracing, metrics).

Steps:

1. Create a library project (e.g., mycompany-spring-boot-starter).

   Add auto-config class:

```
@Configuration
@ConditionalOnClass(MyService.class)
public class MyServiceAutoConfiguration {
  @Bean
  @ConditionalOnMissingBean
  public MyService myService() {
    return new MyService();
  }
}
```

2. Register it in:

   META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

   Content:

   com.mycompany.autoconfig.MyServiceAutoConfiguration

3. Now any app including this starter automatically gets MyService unless overridden.

Analogy:

Think of auto-configuration like a **"default recipe book"**. If you don't bring your own recipe (bean), Spring Boot cooks one for you based on ingredients in your kitchen (classpath + properties). But the moment you say "No thanks, I'll make my own," Boot steps back.

107. What is @ConditionalOnClass in Spring Boot?

Answer

This condition activates a bean or configuration **only if a given class is on the classpath**.

For example, WebMvcAutoConfiguration is only active if Spring MVC is available:

```
@Configuration
@ConditionalOnClass(DispatcherServlet.class)
public class WebMvcAutoConfiguration {
   @Bean
   public SomeWebBean webBean() {
      return new SomeWebBean();
   }
}
```

If you remove spring-boot-starter-web (which brings DispatcherServlet), this config won't load.

Real-world analogy:
Imagine a car factory. If the factory detects "GPS chip is available," then it installs navigation software. If the chip isn't there, no need to load useless software.

108. What is @ConditionalOnMissingBean in Spring Boot?

Answer

This condition prevents Boot from overwriting your custom beans. It creates a bean **only if the developer hasn't already defined one**.

```
@Bean
@ConditionalOnMissingBean
public ObjectMapper objectMapper() {
   return new ObjectMapper();
}
```

So if you define your own ObjectMapper bean, Boot won't interfere.

This is why **customization is so easy in Boot** - you just define your own bean, and the default one backs off.

109. What is @ConditionalOnProperty in Spring Boot?

Answer

This activates beans only when a specific property is set in application.properties (or YAML, env vars, etc.).

```
@Bean
@ConditionalOnProperty(
   name = "feature.toggles.new-dashboard",
   havingValue = "true"
)
public DashboardService newDashboard() {
   return new DashboardService();
}
```

If feature.toggles.new-dashboard=true, the bean loads. Otherwise, it's skipped.

This is commonly used for **feature toggles** in enterprise apps.

Summary of Conditional Annotations

1. @ConditionalOnClass → Load only if a class exists.
2. @ConditionalOnMissingBean → Load only if user hasn't defined it.
3. @ConditionalOnProperty → Load only if a property is set.
4. (Bonus) @ConditionalOnWebApplication, @ConditionalOnResource, etc.

110. How does Spring Boot start up internally?

When you run SpringApplication.run(App.class, args) → this kicks off a multi-phase process:

1. **Bootstrap Phase**
    - SpringApplication is created.
    - Determines the **application type**: Servlet-based (Tomcat/Jetty/Undertow), Reactive (WebFlux/Netty), or non-web CLI.
    - Loads **Bootstrap listeners** (logging system, environment prep, etc.).
2. **Environment Preparation**
    - Reads properties from multiple sources:
        - application.properties / application.yml
        - System environment variables
        - Command-line args (--server.port=9001)
    - Merges them into a single **Environment** object.
3. **ApplicationContext Creation**
    - Depending on app type:
        - AnnotationConfigServletWebServerApplicationContext (MVC apps)
        - ReactiveWebServerApplicationContext (WebFlux apps)
        - AnnotationConfigApplicationContext (non-web).
4. **Bean Definition Phase**
    - @SpringBootApplication triggers:
        - @SpringBootConfiguration → marks it as a config class.
        - @EnableAutoConfiguration → loads all auto-configurations using spring.factories / AutoConfiguration.imports.
        - @ComponentScan → scans packages for beans (@Service, @Controller, etc.).
5. **Bean Creation & Dependency Injection**
    - Beans are instantiated in dependency order.
    - @Conditional... annotations decide which beans to load.
    - @ConfigurationProperties classes are bound from environment values.

- AOP proxies are applied for @Transactional, @Async, @Cacheable, etc.

6. **ApplicationContext Refresh**
   - Lifecycle callbacks are triggered:
     - BeanPostProcessors
     - ApplicationListeners
     - CommandLineRunners / ApplicationRunners
7. **Embedded Server Startup**
   - If web app, Tomcat/Jetty/Netty is started on server.port.
   - DispatcherServlet (for MVC) or HttpHandler (for WebFlux) registered.
8. **Ready State**
   - Boot logs "Started Application in 2.345 seconds (JVM running for 3.123)".
   - Now it starts serving HTTP requests.

111. What is the role of ApplicationContext in Spring Boot?

The **ApplicationContext** is the heart of Spring:

- It's a **bean factory + runtime environment**.
- Manages lifecycle of beans (creation, wiring, destruction).
- Provides features beyond BeanFactory:
    - Event publishing (ApplicationEventPublisher)
    - Resource loading (ResourceLoader)
    - Internationalization (message sources)
    - AOP proxy handling

In Boot, the ApplicationContext is enriched with:

- Auto-configuration beans
- Profiles (dev, prod)
- Embedded server integration

112. What are the different ApplicationContext types in Spring Boot?

1. **AnnotationConfigApplicationContext** → non-web CLI apps
2. **AnnotationConfigServletWebServerApplicationContext** → web (MVC, REST APIs)
3. **ReactiveWebServerApplicationContext** → reactive (WebFlux apps)

Spring Boot picks one **automatically** depending on classpath (spring-webmvc vs spring-webflux).

Interview Tip:

When asked "what happens when you run a Spring Boot app," walk them through this pipeline in 3–4 steps:

Environment setup → Bean scanning & auto-config → Context refresh → Server startup.

113. What is the Bean Lifecycle in Spring?

When Spring creates and manages beans, each bean goes through a well-defined lifecycle inside the ApplicationContext.

High-level phases:

1. **Instantiation** → Bean object is created (via new).
2. **Populate Properties** → Dependencies injected (@Autowired, @Value, constructor/setter DI).
3. **BeanPostProcessors (before init)** → Hooks like @AutowiredAnnotationBeanPostProcessor run.
4. **Initialization** → Custom init logic runs (@PostConstruct, afterPropertiesSet(), initMethod).
5. **BeanPostProcessors (after init)** → Proxies applied (AOP, @Transactional, @Async).
6. **Ready for use** → Bean is available in context.
7. **Destruction** → On shutdown, cleanup methods (@PreDestroy, destroy(), destroyMethod).

114. How do you add custom initialization logic in a Spring Bean?

Spring provides multiple ways to run code after a bean is fully created but before it is used:

1. Using @PostConstruct

```
@Component
public class CacheService {

   private Map<String, String> cache;

   @PostConstruct
   public void init() {
      cache = new HashMap<>();
      System.out.println("Cache initialized");
   }
}
```

- Runs automatically after dependency injection is done.
- Best for simple initialization.

2. Implementing InitializingBean

```
@Component
public class DatabaseConnection implements InitializingBean {
   @Override
   public void afterPropertiesSet() {
      System.out.println("Validating DB connection...");
   }
}
```

- Gives you a callback after all properties are set.
- Useful when you must validate configurations.

3. Using @Bean(initMethod = "methodName")

```
@Configuration
public class AppConfig {
  @Bean(initMethod = "init")
  public PaymentService paymentService() {
    return new PaymentService();
  }
}
class PaymentService {
  public void init() {
    System.out.println("PaymentService initialized");
  }
}
```

- Best for third-party beans where you can't annotate the class.

115. How do you handle bean destruction in Spring?

Similar to initialization, Spring gives hooks for cleanup:

1. Using @PreDestroy

```
@Component
public class FileService {
   @PreDestroy
   public void cleanup() {
      System.out.println("Closing file resources...");
   }
}
```

2. Implementing DisposableBean

```
@Component
public class ConnectionPool implements DisposableBean {
   @Override
   public void destroy() {
      System.out.println("Shutting down connection pool...");
   }
}
```

3. Using @Bean(destroyMethod = "methodName")

```
@Bean(destroyMethod = "shutdown")
public ThreadPoolTaskExecutor executor() {
   return new ThreadPoolTaskExecutor();
}
```

116. What is a BeanPostProcessor and why is it important?

A BeanPostProcessor allows custom logic **before and after bean initialization**.
Spring internally uses it for:

- Applying @Autowired injection
- Wrapping beans with AOP proxies (@Transactional, @Async)
- Validating beans

Example - logging bean initialization:

```
@Component
public class CustomBeanPostProcessor implements BeanPostProcessor {

  @Override
  public Object postProcessBeforeInitialization(Object bean, String beanName) {
    System.out.println("Before Init: " + beanName);
    return bean;
  }

  @Override
  public Object postProcessAfterInitialization(Object bean, String beanName) {
    System.out.println("After Init: " + beanName);
    return bean;
  }
}
```

Interview-ready summary:

- @PostConstruct → best for simple init logic.
- InitializingBean → more explicit, framework-driven.
- @PreDestroy & DisposableBean → for cleanup.
- BeanPostProcessor → powerful hook for cross-cutting concerns (used internally by Spring AOP, DI, validation).

117. What are Bean Scopes in Spring?

A **bean scope** defines the lifecycle and visibility of a bean within the Spring container. In Spring Boot, the default scope is **singleton** - one bean instance per ApplicationContext.

Spring supports several scopes:

1. Singleton (default)

- Only one instance of the bean is created per container.
- All requests for the bean return the same instance.
- Best for **stateless services** like @Service classes.

```
@Component
public class SingletonService { }
```

Every @Autowired injection will point to the same SingletonService instance.

2. Prototype

- A new instance is created every time the bean is requested.
- Best for **stateful, short-lived objects** (e.g., DTO builders, command objects).

```
@Component
@Scope("prototype")
public class PrototypeService { }
```

Each @Autowired or context.getBean() call gives a fresh object.

Gotcha: if you inject a prototype bean into a singleton bean, Spring injects only one instance (at startup). To solve this, use ObjectFactory<PrototypeBean> or Provider<PrototypeBean>.

3. Request (Web Applications only)

- One bean instance per HTTP request.
- Useful for storing **request-scoped data** (e.g., user input).

```
@Component
@Scope("request")
public class RequestScopedBean {
```

```
    private final String id = UUID.randomUUID().toString();
    public String getId() { return id; }
}
```

If two HTTP requests hit the server, each gets a different bean instance.

4. Session (Web Applications only)

- One bean instance per HTTP session.
- Useful for storing **user session data**.

```
@Component
@Scope("session")
public class SessionScopedBean {
    private String userPreference;
}
```

Different users will have different instances, but the same user session reuses the same instance.

5. Application (Web Applications only)

- One bean instance per ServletContext (shared across all sessions).
- Rarely used, but useful for global application-wide data.

6. WebSocket

- One bean instance per WebSocket connection.
- Useful when dealing with real-time apps (e.g., chat, notifications).

```
@Component
@Scope("websocket")
public class WebSocketScopedBean { }
```

118. Practical Example: Singleton vs Prototype

Imagine a ticket booking system:

- **Singleton**: PaymentService - stateless, used by all bookings.
- **Prototype**: Ticket - each booking generates a new Ticket object.

```
@Component
@Scope("singleton")
class PaymentService { }

@Component
@Scope("prototype")
class Ticket {
   private final String id = UUID.randomUUID().toString();
   public String getId() { return id; }
}
```

119. Advanced: Mixing Scopes (Prototype inside Singleton)

Problem: injecting prototype into singleton doesn't work as expected - only one instance is created.

Solution: use **Provider** or **lookup method injection**.

```
@Component
class BookingService {
   @Autowired
   private Provider<Ticket> ticketProvider;

   public Ticket createTicket() {
      return ticketProvider.get(); // Always new instance
   }
}
```

Now, every call to createTicket() returns a new Ticket.

Interview-ready summary:

- Default scope is singleton - best for stateless services.
- Use prototype when you need new instances each time.
- Request/session scopes work only in web contexts.
- Beware of injecting prototype into singleton → must use Provider or ObjectFactory.

120. What is Dependency Injection in Spring?

Dependency Injection means that objects don't create their dependencies directly (new), but instead, the Spring container **provides (injects)** them.

- Without DI: classes are tightly coupled → harder to test, maintain, or replace.
- With DI: classes declare what they need (constructor params, setters, fields), and Spring resolves and provides the actual beans.

Analogy: Think of a food delivery app. Instead of the delivery boy (Service) cooking the food (Dependency), the restaurant (Spring Container) provides it. The delivery boy just says what dish he needs.

121. Types of Dependency Injection in Spring

1. Constructor Injection (Recommended Best Practice)

- Dependencies are passed via constructor.
- Enforces immutability (dependencies can't be changed later).
- Ensures required dependencies are always available.

```
@Service
public class OrderService {
  private final PaymentService paymentService;

  @Autowired  // Not mandatory in Spring 4.3+
  public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
  }
}
```

Pros:

- Immutable → no accidental changes.
- Easy to test → you can pass mocks/fakes.
- Makes dependencies explicit (interviewers love this).

2. Setter Injection

- Dependencies are set via setter methods.
- Useful for **optional dependencies** or when values may change.

```
@Service
public class OrderService {
  private DiscountService discountService;

@Autowired
  public void setDiscountService(DiscountService discountService) {
    this.discountService = discountService;
  }
}
```

Pros:

- Flexible, supports optional dependencies.

Cons:

- Dependency might not be set before usage (risk of NullPointerException).

3. Field Injection (Not Recommended)

- Dependencies are injected directly into fields using @Autowired.

```
@Service
public class OrderService {
  @Autowired
  private PaymentService paymentService;
}
```

Pros:

- Short, less boilerplate.

Cons:

- Harder to test (can't construct manually without reflection).
- Breaks immutability → dependencies can be changed.
- Tightly couples class to Spring (no pure POJO).

122. How Spring Performs Injection Internally

- When the application starts, Spring **scans packages** for beans (@Component, @Service, @Repository, @Controller).
- Spring creates beans and resolves dependencies via the **ApplicationContext**.
- If a class has a constructor with dependencies, Spring looks up matching beans from the context and injects them.
- If multiple beans of the same type exist, Spring resolves ambiguity using:
    - @Qualifier
    - @Primary
    - Bean name

123. Real-World Example: Payment Gateway in E-commerce

```
interface PaymentGateway {
   boolean processPayment(String user, double amount);
}

@Service
class RazorpayGateway implements PaymentGateway {
   public boolean processPayment(String user, double amount) {
      return true; // mock
   }
}

@Service
class OrderService {
   private final PaymentGateway gateway;

   public OrderService(PaymentGateway gateway) {
      this.gateway = gateway;
   }

   public String placeOrder(String user, double amount) {
      if (!gateway.processPayment(user, amount)) throw new
RuntimeException("Payment failed");
      return "Order placed for " + user;
   }
}
```

Here:

- PaymentGateway is an interface.
- Spring injects the correct implementation (RazorpayGateway).
- The OrderService is testable and flexible (can swap Razorpay with PayPal).

124. Best Practices to Mention in Interviews

1. **Prefer constructor injection** for required dependencies (cleaner, immutable, testable).
2. **Use setter injection** for truly optional dependencies.
3. **Avoid field injection** in production code - good for prototypes only.
4. **Use interfaces for dependencies** - improves flexibility.
5. **Resolve ambiguity** with @Qualifier or @Primary.
6. Keep beans **stateless** as much as possible.

Interview Tip: If they ask, “Which injection type do you prefer?” - answer **constructor injection** and back it up with points on immutability, testability, and clarity.

125. What happens when Spring finds multiple beans of the same type?

When you use @Autowired and Spring finds **more than one candidate bean**, it throws a **NoUniqueBeanDefinitionException**, because it doesn't know which one to inject.

To solve this, Spring gives us three strategies:

1. @Primary – Default Preference

Mark one bean as the primary candidate. Spring will inject this unless another one is explicitly requested.

```
public interface PaymentGateway {
   boolean processPayment(String user, double amount);
}

@Service
@Primary
class RazorpayGateway implements PaymentGateway {
   public boolean processPayment(String user, double amount) {
      return true;
   }
}

@Service
class PaypalGateway implements PaymentGateway {
   public boolean processPayment(String user, double amount) {
      return true;
   }
}

@Service
class OrderService {
   private final PaymentGateway gateway;

   @Autowired
   public OrderService(PaymentGateway gateway) {
      this.gateway = gateway; // Razorpay gets injected (because of @Primary)
   }
}
```

Use case: when you want a sensible default implementation.

2. @Qualifier – Explicit Selection

Use @Qualifier when you need to specify **which bean to inject**.

```
@Service
class OrderService {
   private final PaymentGateway gateway;

   @Autowired
   public OrderService(@Qualifier("paypalGateway") PaymentGateway gateway) {
      this.gateway = gateway; // Injects PayPal explicitly
   }
}
```

Use case: when you have multiple beans and want fine-grained control in different services.

3. Injecting a List/Map of Beans

Spring can inject **all beans of a type** into a collection.

```
@Service
class OrderService {
   private final List<PaymentGateway> gateways;

   @Autowired
   public OrderService(List<PaymentGateway> gateways) {
      this.gateways = gateways; // Both Razorpay and PayPal beans injected
   }

   public void processAll(String user, double amount) {
      gateways.forEach(g -> g.processPayment(user, amount));
   }
}
```

Or as a **Map** (with bean names as keys):

```
@Autowired
private Map<String, PaymentGateway> gatewayMap;
```

Use case: plugin architectures, strategy patterns (e.g., payment gateways, shipping providers).

4. @Profile – Environment-Specific Beans

You can define beans active only in certain environments.

```
@Service
@Profile("dev")
```

```
class MockPaymentGateway implements PaymentGateway {
   public boolean processPayment(String user, double amount) {
      System.out.println("Mock payment processed");
      return true;
   }
}

@Service
@Profile("prod")
class RazorpayGateway implements PaymentGateway {
   public boolean processPayment(String user, double amount) {
      // real gateway call
      return true;
   }
}
```

When you run with spring.profiles.active=prod, only the Razorpay bean is active.

5. Custom Annotations (Advanced)

You can create your own qualifiers to avoid repeating @Qualifier("beanName") everywhere.

```
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Qualifier
public @interface Razorpay {}

@Service
@Razorpay
class RazorpayGateway implements PaymentGateway {
   public boolean processPayment(String user, double amount) {
      return true;
   }
}

@Service
class OrderService {
   private final PaymentGateway gateway;

   @Autowired
   public OrderService(@Razorpay PaymentGateway gateway) {
      this.gateway = gateway; // Clean & type-safe injection
   }
}
```

Interview-Ready Points

1. If multiple beans exist → Spring fails with NoUniqueBeanDefinitionException.
2. Solutions:
    - @Primary → set a default bean.
    - @Qualifier → explicitly choose one.
    - Inject List/Map when you need all.
    - @Profile → environment-specific beans.
    - Custom qualifiers for cleaner code.
3. Best practice: **use constructor injection with @Qualifier for clarity**.

126. What are the different bean scopes in Spring, and how do they work?

Spring manages beans in the IoC container, and **scope** defines the lifecycle and visibility of a bean. By default, all beans are singletons, but Spring supports multiple scopes.

1. Singleton Scope (default)

- Only **one instance per Spring IoC container**.
- Any injection of that bean returns the same object.

```
@Service
@Scope("singleton") // default
class PaymentService {
  public void pay() {
    System.out.println("Processing payment...");
  }
}
@Autowired
private PaymentService service1;

@Autowired
private PaymentService service2;

// service1 == service2 → true
```

Use case: stateless services like repositories, service layer classes

2. Prototype Scope

- A **new instance is created every time** it's requested.
- Spring only creates the bean; you manage its lifecycle.

```
@Component
@Scope("prototype")
class Invoice {
  private final UUID id = UUID.randomUUID();
  public UUID getId() { return id; }
}
@Autowired
private Invoice invoice1;

@Autowired
private Invoice invoice2;
```

```
// invoice1 == invoice2 → false
```

Use case: objects with short lifespans, e.g., generating invoices, documents, or temporary calculations.

3. Request Scope (web apps only)

- A **new bean instance is created per HTTP request**.
- Scoped to a single HTTP request lifecycle.

```
@Component
@Scope("request")
class RequestTracker {
   private final long timestamp = System.currentTimeMillis();
   public long getTimestamp() { return timestamp; }
}
```

Each request to the server gets a fresh RequestTracker.

Use case: request metadata, request-level caching in REST APIs.

4. Session Scope (web apps only)

- One bean per **HTTP session**.
- Shared across multiple requests within the same session.

```
@Component
@Scope("session")
class UserSession {
   private String username;
   // getters and setters
}
```

Use case: storing logged-in user details across multiple requests.

5. Application Scope

- A bean instance is tied to the **ServletContext** lifecycle.
- One bean shared across the whole application context.

```
@Component
@Scope("application")
class AppStatistics {
```

```
    private AtomicInteger visitorCount = new AtomicInteger(0);
    public int increment() { return visitorCount.incrementAndGet(); }
}
```

Use case: application-wide counters, statistics, or shared resources.

6. WebSocket Scope

- Available in Spring for **WebSocket communication**.
- Each WebSocket session gets its own bean instance.

```
@Component
@Scope("websocket")
class ChatSession {
    private String sessionId = UUID.randomUUID().toString();
}
```

Use case: per-user WebSocket chat session data.

Real-World Analogy

Think of **scope like coffee cups at a café**:

- **Singleton** → one shared coffee pot for everyone.
- **Prototype** → each customer gets a fresh coffee.
- **Request** → one cup per customer request.
- **Session** → one pot reserved for the same customer for their entire stay.
- **Application** → one pot shared for the entire café.

Interview-Ready Points

1. **Default scope is singleton**.
2. **Prototype beans are not managed after creation** (Spring won't call destroy).
3. **Request/session/application scopes require a web-aware Spring container**.
4. If you inject a **prototype bean into a singleton**, you usually need a @Lookup or ObjectProvider to ensure a fresh instance each time.

127. How do you inject Prototype beans into Singleton beans in Spring?

The Problem

- By default, **singleton beans are created once at container startup**.
- If a **singleton depends on a prototype bean**, Spring resolves that dependency **at injection time only** (startup).
- This means the singleton will always reuse the same prototype instance, **defeating the purpose of prototype scope**.

Example of the Problem

```
@Component
@Scope("prototype")
class Report {
   private final UUID id = UUID.randomUUID();
   public UUID getId() { return id; }
}

@Service
class ReportService {
   @Autowired
   private Report report;

   public void generate() {
      System.out.println("Report ID: " + report.getId());
   }
}
@Autowired
private ReportService service;

service.generate();
service.generate();
```

Both calls print the same Report ID - because the singleton ReportService got one Report at startup.

Solutions

1. Using @Lookup method injection

Spring overrides the method at runtime to always fetch a new bean.

```
@Service
class ReportService {
```

```
    public void generate() {
        Report report = createReport();
        System.out.println("Report ID: " + report.getId());
    }

    @Lookup
    protected Report createReport() {
        // Spring overrides this method at runtime
        return null;
    }
}
```

Each call to createReport() gets a new prototype instance.

2. Using ObjectFactory or ObjectProvider

These act as bean factories that fetch fresh instances when called.

```
@Service
class ReportService {
    @Autowired
    private ObjectProvider<Report> reportProvider;

    public void generate() {
        Report report = reportProvider.getObject();
        System.out.println("Report ID: " + report.getId());
    }
}
```

Every getObject() gives a new Report. This is often preferred because it's explicit and avoids runtime subclassing like @Lookup.

3. Using ApplicationContext directly (not best practice)

```
@Service
class ReportService {
    @Autowired
    private ApplicationContext context;

    public void generate() {
        Report report = context.getBean(Report.class);
        System.out.println("Report ID: " + report.getId());
    }
}
```

This works but couples your business code to Spring APIs. Usually avoided except for infrastructure beans.

Real-World Analogy

Think of it like this:

- A **singleton** is a permanent employee at a company.
- A **prototype** is a contractor hired for each task.
- If the permanent employee hires one contractor at the start and keeps using him forever, you lose flexibility.
- Instead, he should **request a fresh contractor for each task** - that's exactly what @Lookup or ObjectProvider helps with.

Interview-Ready Answer

1. Spring resolves dependencies at injection time, so injecting a prototype into a singleton normally gives only one instance.
2. To fix this, use:
    - @Lookup for method injection (Spring overrides it).
    - ObjectProvider or ObjectFactory for on-demand retrieval.
3. Best practice: prefer ObjectProvider for clarity and testability.
4. Avoid directly using ApplicationContext unless absolutely necessary.

128. Explain the Bean Lifecycle in Spring

Big Picture

Every Spring bean goes through a well-defined lifecycle, managed entirely by the IoC container.
From creation → dependency injection → initialization → use → destruction.

1. Instantiation

Spring creates a bean instance (via constructor or factory method).

```
@Component
class Employee {
   public Employee() {
      System.out.println("Step 1: Constructor called");
   }
}
```

2. Dependency Injection

Spring injects all dependencies (via constructor, setters, or field injection).

```
@Component
class Department {
   public Department() { System.out.println("Department created"); }
}

@Component
class Employee {
   @Autowired
   private Department dept;
}
```

3. Aware Interfaces (Optional)

If a bean implements special “aware” interfaces, Spring provides container-related info:

- BeanNameAware → bean gets its name.
- ApplicationContextAware → gets reference to the context.

```
@Component
class Employee implements ApplicationContextAware {
   public void setApplicationContext(ApplicationContext ctx) {
      System.out.println("Step 3: ApplicationContext injected");
```

```
    }
}
```

4. Custom Initialization

You can run custom logic right after dependencies are injected:

- Using **@PostConstruct** (preferred):

```
@Component
class Employee {
    @PostConstruct
    public void init() {
        System.out.println("Step 4: PostConstruct - init method called");
    }
}
```

- Using **InitializingBean**:

```
@Component
class Employee implements InitializingBean {
    @Override
    public void afterPropertiesSet() {
        System.out.println("Step 4: afterPropertiesSet() called");
    }
}
```

- Using **initMethod in XML/Java Config**:

```
@Bean(initMethod = "customInit")
public Employee employee() { return new Employee(); }

public void customInit() { System.out.println("Custom init"); }
```

5. Ready for Use

Now the bean is fully initialized and available for use inside the app.

6. Destruction (Cleanup)

When the container shuts down, destruction callbacks are triggered:

- Using **@PreDestroy**:

```
@PreDestroy
public void cleanup() {
  System.out.println("Step 6: PreDestroy - cleanup called");
}
```

- Using **DisposableBean**:

```
@Component
class Employee implements DisposableBean {
  @Override
  public void destroy() {
    System.out.println("Step 6: destroy() called");
  }
}
```

- Using **destroyMethod**:

```
@Bean(destroyMethod = "customDestroy")
public Employee employee() { return new Employee(); }

public void customDestroy() { System.out.println("Custom destroy"); }
```

Lifecycle Order (Simplified)

1. Constructor (instantiation)
2. Dependency injection
3. *Aware interfaces (if implemented)
4. @PostConstruct / afterPropertiesSet() / custom init methods
5. Bean ready for use
6. @PreDestroy / destroy() / custom destroy methods

Real-World Analogy

Think of a **restaurant worker**:

1. He's hired (instantiation).
2. He's trained with tools (dependencies injected).
3. He's told where he fits in the org (aware interfaces).
4. He finishes onboarding tasks (init methods).
5. He works shifts (bean usage).
6. He hands over responsibilities and exits (destroy methods).

Interview-Ready Answer

The Spring container fully manages bean lifecycle.

- After instantiation and dependency injection, it allows initialization callbacks (@PostConstruct, InitializingBean, custom init).
- During shutdown, destruction callbacks (@PreDestroy, DisposableBean, destroyMethod) are triggered.
- For container metadata, you can implement Aware interfaces.

This makes beans highly configurable and lets developers hook into creation and cleanup phases without manual resource management.

129. What is a BeanPostProcessor in Spring, and how does it affect the Bean Lifecycle?

Core Idea

A **BeanPostProcessor** is like a plugin hook into the Spring container.
It lets you run custom logic **before and after** any bean's initialization methods are called.
This is how many Spring features (like @Autowired, @Transactional, @Async) are actually implemented under the hood.

Lifecycle with BeanPostProcessor

Normal lifecycle order (simplified):

1. Constructor (instantiation)
2. Dependency injection
3. **BeanPostProcessor: postProcessBeforeInitialization()**
4. @PostConstruct / afterPropertiesSet() / init-method
5. **BeanPostProcessor: postProcessAfterInitialization()**
6. Bean ready for use
7. @PreDestroy / destroy methods

Example

Custom BeanPostProcessor:

```
@Component
public class CustomBeanPostProcessor implements BeanPostProcessor {

  @Override
  public Object postProcessBeforeInitialization(Object bean, String beanName) {
    System.out.println("Before Init: " + beanName);
    return bean; // return the same or modified bean
  }

  @Override
  public Object postProcessAfterInitialization(Object bean, String beanName) {
    System.out.println("After Init: " + beanName);
    return bean; // return a proxy or enhanced bean if needed
  }
}
```

```
Target bean:

@Component
public class Employee {
   @PostConstruct
   public void init() {
      System.out.println("Employee @PostConstruct called");
   }
}
```

Output during startup:

Before Init: employee
Employee @PostConstruct called
After Init: employee

Real-World Usage

1. **Spring's built-in features use it heavily**
    - AutowiredAnnotationBeanPostProcessor injects @Autowired fields.
    - CommonAnnotationBeanPostProcessor handles @PostConstruct and @PreDestroy.
    - AnnotationAwareAspectJAutoProxyCreator creates proxies for @Transactional and AOP.
2. **Custom use cases**
    - Adding logging for all beans.
    - Modifying bean properties at runtime.
    - Wrapping beans with dynamic proxies (like decorators).

Analogy

Think of a **quality inspector in a factory**.

- Before a product leaves the line (before init), the inspector can tweak it.
- After finishing touches (after init), the inspector might wrap it with extra packaging.

  The product (bean) is still functional, but now it may have enhancements added during inspection.

Interview-Ready Answer

A **BeanPostProcessor** allows custom modification of new bean instances before and after initialization callbacks.

- postProcessBeforeInitialization runs just before @PostConstruct or afterPropertiesSet.
- postProcessAfterInitialization runs after init - often used to wrap beans with proxies (like for AOP).

Most Spring magic (dependency injection, AOP, lifecycle annotations) is implemented using BeanPostProcessors, making them one of the most powerful extension points in the container.

130. What is the difference between BeanFactoryPostProcessor and BeanPostProcessor?

1. When They Run

- **BeanFactoryPostProcessor**

  Runs *before* any bean instances are created.

  It works on **bean definitions** (metadata) inside the BeanFactory - for example, modifying property values or adding new definitions.

- **BeanPostProcessor**

  Runs *after* beans are created and dependencies are injected, but around initialization callbacks.

  It works on **actual bean instances**.

2. Purpose

- **BeanFactoryPostProcessor** → modify configuration before beans are instantiated.

  Example: change a property value in the BeanDefinition so that every future bean instance reflects that change.

- **BeanPostProcessor** → customize or wrap beans after creation.

  Example: apply proxies for @Transactional or inject additional behavior.

3. Example Code

BeanFactoryPostProcessor Example

```
@Component
public class CustomBeanFactoryPostProcessor implements
BeanFactoryPostProcessor {

   @Override
   public void postProcessBeanFactory(ConfigurableListableBeanFactory
beanFactory) {
      BeanDefinition bd = beanFactory.getBeanDefinition("employee");
      bd.getPropertyValues().add("name", "Modified by BeanFactoryPostProcessor");
   }
}
```

BeanPostProcessor Example

```
@Component
public class CustomBeanPostProcessor implements BeanPostProcessor {

  @Override
  public Object postProcessBeforeInitialization(Object bean, String beanName) {
    if (bean instanceof Employee) {
      System.out.println("Before init of Employee");
    }
    return bean;
  }

  @Override
  public Object postProcessAfterInitialization(Object bean, String beanName) {
    if (bean instanceof Employee) {
      System.out.println("After init of Employee");
    }
    return bean;
  }
}
```

4. Analogy

- **BeanFactoryPostProcessor** → like editing the **blueprint** of a house before construction starts.
- **BeanPostProcessor** → like inspecting the **actual built house** and adding smart devices or alarms after construction.

5. Interview-Ready Answer

The key difference is **what they operate on and when they run**:

- BeanFactoryPostProcessor modifies **bean definitions** before beans are created.
- BeanPostProcessor modifies **bean instances** after they're created but before they're fully ready.

Spring uses PropertySourcesPlaceholderConfigurer (a BeanFactoryPostProcessor) to resolve ${...} placeholders from application.properties, while features like @Autowired, @Transactional, and @Async rely on BeanPostProcessors.

131. What is SmartInitializingSingleton in Spring and when do you use it?

1. Context

Normally, Spring beans are lazily initialized until they're requested (unless marked as eager). Even with eager singletons, some beans (like proxies or event listeners) may not be "fully ready" until the container has finished creating all other singletons.

That's where **SmartInitializingSingleton** comes in. It provides a callback that Spring calls **after all singleton beans are created and initialized** in the context.

2. How It Works

- It's an interface with a single method:
- void afterSingletonsInstantiated();
- Spring invokes it **once per bean** that implements this interface, but only **after all other singletons are fully created**.

3. Example Code

```
@Component
public class CacheWarmupService implements SmartInitializingSingleton {

   @Autowired
   private ProductRepository productRepository;

   private final Map<Integer, Product> cache = new HashMap<>();

   @Override
   public void afterSingletonsInstantiated() {
      // Warm up cache after all singletons are initialized
      productRepository.findAll().forEach(p -> cache.put(p.getId(), p));
      System.out.println("Cache initialized with " + cache.size() + " products");
   }

   public Product getFromCache(int id) {
      return cache.get(id);
   }
}
```

Here:

- CacheWarmupService waits until the whole context is ready.

- Only then does it load all products into memory.
- This ensures no dependency issues (since ProductRepository is guaranteed to exist).

4. Analogy

Imagine you're setting up a new office:

- **BeanPostProcessor** is like setting up desks and furniture for each employee when they arrive.
- **SmartInitializingSingleton** is like saying: "Okay, everyone's here, now let's start the team briefing."

It runs **once, after the whole team (all beans) is present**.

5. Interview-Ready Points

- Use SmartInitializingSingleton for **final setup tasks** that depend on the availability of all beans.
- Common use cases:
    - Pre-loading caches.
    - Scheduling background tasks only after context initialization.
    - Registering beans in an external registry.
- It's **container-wide**, unlike InitializingBean or @PostConstruct which are bean-specific.

132. What is the difference between CommandLineRunner and ApplicationRunner in Spring Boot?

1. Purpose

Both CommandLineRunner and ApplicationRunner are special Spring Boot interfaces that allow you to **execute code after the Spring application context is fully initialized** (i.e., all beans are ready).

They are most often used for **bootstrapping logic** like:

- Preloading data into DB.
- Running startup validations.
- Initializing caches or calling external services.

2. Key Difference

- **CommandLineRunner**: gives you the raw String[] args from the command line.
- **ApplicationRunner**: gives you a more structured ApplicationArguments object (with parsed options and non-option arguments).

3. Example Code

CommandLineRunner

```
@Component
public class MyCommandLineRunner implements CommandLineRunner {
   @Override
   public void run(String... args) {
      System.out.println("CommandLineRunner args:");
      for (String arg : args) {
         System.out.println(arg);
      }
   }
}
```

Run the app:

java -jar app.jar --server.port=9090 myCustomArg

Output:

CommandLineRunner args:

--server.port=9090
myCustomArg

ApplicationRunner

```
@Component
public class MyApplicationRunner implements ApplicationRunner {
   @Override
   public void run(ApplicationArguments args) {
      System.out.println("Option args: " + args.getOptionNames());
      System.out.println("Non-option args: " + args.getNonOptionArgs());
   }
}
```

Run the app:

java -jar app.jar --server.port=9090 myCustomArg

Output:

Option args: [server.port]
Non-option args: [myCustomArg]

4. Analogy

Think of them as **startup scripts**:

- CommandLineRunner → You get the raw input line as it is.
- ApplicationRunner → You get a parsed, more **developer-friendly API** to access arguments.

5. Interview-Ready Points

- Both run **after Spring context initialization** but before the application is considered fully started.
- If you need **just arguments**, use CommandLineRunner.
- If you need **structured argument parsing**, use ApplicationRunner.
- You can control their execution order by implementing the Ordered interface or using @Order.

133. What is Auto-Configuration in Spring Boot, and how does it work?

1. Concept

Auto-Configuration in Spring Boot means that **beans are automatically configured** based on what's available on the classpath and existing Spring beans.
It's what makes Spring Boot so convenient - you add a dependency, and the required beans are ready without manual XML or @Bean definitions.

2. How it Works

- Uses @EnableAutoConfiguration (part of @SpringBootApplication).
- Scans META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports for available configuration classes.
- Conditions (via @ConditionalOnClass, @ConditionalOnProperty, etc.) decide if a bean should be created.

3. Example

Suppose you add spring-boot-starter-data-jpa dependency.

Boot auto-configures:

- A DataSource
- An EntityManagerFactory
- A TransactionManager

Without you writing any config.

4. Simple Custom Auto-Configuration

```
@Configuration
@ConditionalOnClass(DataSource.class) // activate only if DataSource exists
public class MyAutoConfiguration {

   @Bean
   @ConditionalOnMissingBean
   public MyService myService() {
      return new MyService();
   }
}
```

Register in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports:

com.example.config.MyAutoConfiguration

Now whenever this jar is on the classpath, Boot auto-configures MyService.

5. Interview-Ready Points

- **Key Annotations**:
  - @ConditionalOnClass → create bean only if class is available.
  - @ConditionalOnMissingBean → don't override developer's custom beans.
  - @ConditionalOnProperty → enable/disable bean based on config.
- **Best Practice**: Auto-configs should never force themselves - always respect user's overrides.

134. What is a Custom Spring Boot Starter, and why create one?

1. Concept

A **starter** is just a dependency that bundles:

- A curated set of libraries.
- Auto-configuration classes.

You create custom starters when multiple projects in your company need the same **reusable feature set** (logging, monitoring, security policies).

2. Steps to Create

1. **Create a library module** (e.g., company-logging-starter).
2. Add required dependencies (e.g., Logback, Micrometer).
3. Add auto-config class with beans.
4. Register it in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports.

3. Example

```
@Configuration
public class LoggingAutoConfig {
   @Bean
   public LoggerService loggerService() {
      return new LoggerService();
   }
}
```

Register:

com.company.LoggingAutoConfig

Now, in any project, just add:

```
<dependency>
 <groupId>com.company</groupId>
 <artifactId>company-logging-starter</artifactId>
</dependency>
```

And it's ready - no config needed.

4. Why Create One?

- Standardize logging, monitoring, security, or metrics across all microservices.
- Reduce boilerplate.
- Enforce org-wide policies automatically.

135. What are Conditional Beans in Spring Boot?

1. Concept

Conditional beans are beans that are created **only if certain conditions are true**.
Spring Boot uses this heavily for auto-config.

2. Key Annotations

- @ConditionalOnClass → only load bean if a class is on classpath.
- @ConditionalOnMissingBean → only load bean if no other bean of that type exists.
- @ConditionalOnProperty → enable/disable bean with a config property.
- @ConditionalOnResource → check if a resource (file, etc.) exists.
- @ConditionalOnExpression → use SpEL for conditions.

3. Example

```
@Configuration
public class PaymentConfig {

  @Bean
  @ConditionalOnProperty(name = "payment.gateway", havingValue = "razorpay")
  public PaymentService razorpayService() {
    return new RazorpayService();
  }

  @Bean
  @ConditionalOnProperty(name = "payment.gateway", havingValue = "stripe")
  public PaymentService stripeService() {
    return new StripeService();
  }
}
```

Set in application.properties:

payment.gateway=razorpay

Result: Only RazorpayService bean is created.

136. What are Meta-Annotations in Spring?

1. Concept

Meta-annotations are **annotations on other annotations**.
Spring uses them to create higher-level annotations by combining existing ones.

2. Example – @RestController

@Target(ElementType.TYPE)

@Retention(RetentionPolicy.RUNTIME)

@Controller

@ResponseBody

public @interface RestController { }

- @RestController itself is meta-annotated with @Controller + @ResponseBody.
- Developers don't need to repeat both.

3. Custom Meta-Annotation

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Controller
@RequestMapping("/api")
public @interface ApiController { }

Usage:

@ApiController
public class OrderController {
  @GetMapping("/orders")
  public String getOrders() { return "All orders"; }
}
```

Now every class annotated with @ApiController behaves like @Controller with /api prefix.

4. Interview-Ready Insights

- **Reduces boilerplate** → combine multiple annotations.
- **Improves consistency** → enforce org-wide conventions (e.g., all APIs under /api).

- Widely used in frameworks → @SpringBootApplication is itself a meta-annotation (@Configuration + @EnableAutoConfiguration + @ComponentScan).

137. Advanced Spring Profiles – How do they work in real projects?

1. Concept

Profiles allow multiple environment configs (dev, test, prod, staging).
Spring Boot merges configs at runtime depending on active profiles.

2. Activation Order

- Default: application.properties
- Profile-specific: application-dev.yml, application-prod.yml
- Active profile → spring.profiles.active=dev

3. Example

```
# application-dev.yml
server:
  port: 8081
logging.level.root: DEBUG

# application-prod.yml
server:
  port: 8080
logging.level.root: WARN

Run app:
java -jar app.jar --spring.profiles.active=prod
```

4. Profile-Specific Beans

```
@Configuration
@Profile("dev")
public class DevConfig {
   @Bean
   public String message() {
      return "Running in DEV";
   }
}
```

```
@Configuration
@Profile("prod")
public class ProdConfig {
   @Bean
   public String message() {
      return "Running in PROD";
   }
}
```

5. Best Practices

- **Use profiles for infra differences** → DB URLs, logging, queues.
- **Don’t abuse profiles for business logic.**
- For enterprise apps → use **Spring Cloud Config** or **Vault** for secure config management.

138. Advanced Configuration Properties Binding

1. Why?

Instead of sprinkling @Value("${...}") everywhere, bind related configs into POJOs.
This keeps config **type-safe, validated, and organized**.

2. Example

```
app:
 payment:
  gateway: razorpay
  timeout: 30
@Component
@ConfigurationProperties(prefix = "app.payment")
@Validated
public class PaymentProperties {
  @NotBlank
  private String gateway;
  @Min(1)
  private int timeout;

  // getters and setters
}
```

Usage:

```
@Service
public class PaymentService {
  private final PaymentProperties props;

  public PaymentService(PaymentProperties props) {
    this.props = props;
  }

  public void pay() {
    System.out.println("Gateway: " + props.getGateway());
  }
}
```

3. Features

- **Type-Safety** → converts YAML/Properties to Java types.
- **Validation** → @NotBlank, @Min works.
- **Grouping** → one class holds all payment config.

4. Interview-Ready Points

- Preferred over @Value in large apps.
- Works well with Spring Cloud Config for distributed configs.
- Can be combined with profiles → different configs per env.

139. Observability in Spring Boot (Logs, Metrics, Tracing)

1. Concept

Observability = **Logs + Metrics + Traces**.
Spring Boot integrates all three out of the box via **Micrometer + Actuator + Sleuth**.

2. Logging

- Default: **Logback**.
- Configurable in application.properties:

```
logging.level.com.example=DEBUG
logging.file.name=app.log
```

3. Metrics (via Micrometer)

- Default metrics: JVM, HTTP requests, DB connection pool.
- Exposed at /actuator/metrics.
- Can export to **Prometheus, Datadog, CloudWatch, Grafana**.

Example custom metric:

```
@Component
public class OrderMetrics {
  private final Counter orderCounter;

  public OrderMetrics(MeterRegistry registry) {
    this.orderCounter = Counter.builder("orders_placed").register(registry);
  }

  public void recordOrder() { orderCounter.increment(); }
}
```

4. Distributed Tracing

- **Spring Cloud Sleuth** adds traceId + spanId in logs.
- Works with Zipkin or Jaeger.

  Log output:

```
2025-09-29 10:12:45 [orderservice,traceId=12ab,spanId=45cd] Order placed
```

In interviews: Stress that **traceId correlation** is critical for debugging microservices.

140. Security Best Practices in Spring Boot

1. Password Management

- Always store **hashed passwords** (BCrypt via PasswordEncoder).

```
@Bean
public PasswordEncoder encoder() {
  return new BCryptPasswordEncoder();
}
```

2. Secrets & Configs

- Don't hardcode secrets in application.yml.
- Use **Vault, AWS Secrets Manager, or Spring Cloud Config + Encryption**.

3. CSRF

- Enabled by default for session-based apps.
- Disable for stateless APIs with JWT:

http.csrf().disable();

4. HTTPS + CORS

- Enable TLS (server.ssl.key-store).
- Restrict CORS origins in prod.

5. OWASP Protection

- Validate inputs.
- Prevent SQL Injection (@Param in queries).
- Use **Spring Security filters** against XSS, CSRF.

In interviews: Say you've applied **BCrypt, HTTPS, JWT, centralized secrets**, and **method-level security (@PreAuthorize)**.

141. Configuration for Cloud-Native Deployments

1. Graceful Shutdown

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

2. Health Checks for Kubernetes

```
management.endpoint.health.probes.enabled=true
```

Now /actuator/health/liveness and /actuator/health/readiness are available.

3. Containerization (Docker)

```
FROM openjdk:17-jdk-slim
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

4. Kubernetes Deployment (snippet)

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
```

5. Externalized Config with ConfigMaps & Secrets

- Keep environment-specific configs outside the app.
- Bind with @ConfigurationProperties.

In interviews: Mention that you've deployed Spring Boot apps with **Docker + K8s**, used **graceful shutdown**, and exposed **health endpoints for probes**.

142. Service Discovery with Eureka

1. Why?

In microservices, services scale dynamically, so hardcoding IPs/ports is not practical. **Eureka** acts as a registry where services register themselves and discover others.

2. Setup

Eureka Server

```
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServer {
   public static void main(String[] args) {
      SpringApplication.run(DiscoveryServer.class, args);
   }
}
```

application.yml

```
server:
 port: 8761
eureka:
 client:
   register-with-eureka: false
   fetch-registry: false
```

Client service

```
@SpringBootApplication
@EnableEurekaClient
public class OrderServiceApp { }
```

application.yml

```
spring:
 application:
   name: orderservice
eureka:
 client:
   service-url:
     defaultZone: http://localhost:8761/eureka/
```

Now, instead of calling http://localhost:8081, you call http://ORDERSERVICE.

143. API Gateway with Spring Cloud Gateway

1. Why?

- Acts as a **single entry point** for all microservices.
- Handles cross-cutting concerns: logging, authentication, rate limiting.

2. Example Config (application.yml)

```
spring:
 cloud:
  gateway:
   routes:
    - id: orderservice
      uri: lb://ORDERSERVICE
      predicates:
       - Path=/orders/**
    - id: paymentservice
      uri: lb://PAYMENTSERVICE
      predicates:
       - Path=/payments/**
```

Here, requests to /orders/** go to **ORDERSERVICE** discovered via Eureka.

3. Filters (rate limiting, auth, logging)

```
filters:
 - name: RequestRateLimiter
  args:
   redis-rate-limiter.replenishRate: 10
   redis-rate-limiter.burstCapacity: 20
```

Ensures only 10 requests/sec are allowed per client.

144. Resilience Patterns (Circuit Breaker, Retry, Fallback)

1. Why?

In microservices, one slow service can cascade failures. **Resilience4j** helps with:

- Circuit Breaker
- Retry
- Rate Limiting
- Bulkhead (isolation)

2. Circuit Breaker Example

```
@Service
public class OrderService {

   @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
   public String placeOrder() {
      return restTemplate.getForObject("http://PAYMENTSERVICE/pay", String.class);
   }

   public String paymentFallback(Exception ex) {
      return "Payment service unavailable, try later.";
   }
}
```

3. Retry Example

```
@Retry(name = "paymentService", fallbackMethod = "paymentFallback")
public String callPayment() {
   return restTemplate.getForObject("http://PAYMENTSERVICE/pay", String.class);
}
```

This retries failed calls automatically before falling back.

4. Bulkhead Example

```
@Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
public String callPayment() {
   return restTemplate.getForObject("http://PAYMENTSERVICE/pay", String.class);
}
```

Limits number of concurrent calls to prevent thread exhaustion.

145. Spring Cloud Config (Centralized Configuration)

1. Why?

In microservices, each service has its own application.yml. Managing configs across 10–20 services is painful.
Spring Cloud Config centralizes this → all configs live in one place (e.g., Git repo).

2. Setup – Config Server

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApp {
   public static void main(String[] args) {
      SpringApplication.run(ConfigServerApp.class, args);
   }
}
```

application.yml (Config Server)

```
server:
 port: 8888
spring:
 cloud:
  config:
   server:
    git:
     uri: https://github.com/my-org/config-repo
```

3. Client Service (e.g., Orderservice)

bootstrap.yml

```
spring:
 application:
  name: orderservice
 cloud:
  config:
   uri: http://localhost:8888
```

On startup, orderservice fetches config from orderservice-dev.yml in Git.

4. Why important in interviews?

- Avoids config drift between services.
- Enables dynamic refresh with @RefreshScope.
- Used in production with Vault for secrets management.

146. Distributed Tracing with Sleuth and Zipkin

1. Why?

In microservices, a single request flows through multiple services. Debugging latency/issues without tracing is impossible.

2. Sleuth Setup

Add dependency:

```
<dependency>
 <groupId>org.springframework.cloud</groupId>
 <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
```

This automatically adds **traceId** and **spanId** to logs.

Example log output:

2025-09-29 14:15:23 [orderservice, traceId=4a2f1b, spanId=9c2d7b] Placing order
2025-09-29 14:15:24 [paymentservice, traceId=4a2f1b, spanId=3a7b1c] Processing payment

3. Zipkin Integration

Add Zipkin dependency:

```
<dependency>
 <groupId>org.springframework.cloud</groupId>
 <artifactId>spring-cloud-starter-zipkin</artifactId>
</dependency>
```

Start Zipkin server (docker run -d -p 9411:9411 openzipkin/zipkin).
Now trace logs flow into Zipkin UI.

4. Why important?

- Identifies bottlenecks across services.
- Helps in debugging latency.
- Essential for production monitoring.

147. Messaging with Kafka and RabbitMQ

1. Why?

Services should not always call each other synchronously (REST). Instead, use **event-driven messaging** for decoupling and handling spikes in load.

2. Kafka Example

Producer Service

```
@Service
public class OrderProducer {
   @Autowired
   private KafkaTemplate<String, String> kafkaTemplate;

   public void sendOrder(String order) {
      kafkaTemplate.send("orders", order);
   }
}
```

Consumer Service

```
@Service
public class OrderConsumer {
   @KafkaListener(topics = "orders", groupId = "order_group")
   public void consume(String order) {
      System.out.println("Received order: " + order);
   }
}
```

3. RabbitMQ Example

Producer

```
@Service
public class RabbitProducer {
   @Autowired
   private AmqpTemplate template;

   public void send(String msg) {
      template.convertAndSend("orderExchange", "orderKey", msg);
   }
}
```

Consumer

```
@Service
public class RabbitConsumer {
   @RabbitListener(queues = "orderQueue")
   public void consume(String msg) {
      System.out.println("Received: " + msg);
   }
}
```

4. Why interviewers love this?

- Shows you understand async communication.
- Demonstrates knowledge of Kafka vs RabbitMQ tradeoffs.
- Event-driven systems are the backbone of modern large-scale apps.

148. Spring Cloud Gateway – Filters and API Gateway

1. Why?

In microservices, clients don't call services directly → they go via an API Gateway. Spring Cloud Gateway is the **preferred choice** over Zuul because it's **reactive, fast, and simple**.

2. Config Example

application.yml

```
spring:
 cloud:
  gateway:
   routes:
    - id: orderservice
      uri: lb://ORDERSERVICE
      predicates:
       - Path=/orders/**
      filters:
       - AddRequestHeader=X-Request-ID, #{UUID.randomUUID()}
       - RewritePath=/orders/(?<segment>.*), /api/v1/orders/${segment}
```

Features:

- **Predicates** → Match request paths, headers, methods.
- **Filters** → Modify requests/responses (auth, logging, rate limiting).
- **Load-balanced URIs** → Works with Eureka for service discovery.

3. Interview Pointers

- Replaces Zuul (non-blocking, built on Project Reactor).
- Can add custom filters for **auth, tracing, response transformation**.
- Works as **entry point** in a microservice architecture

149. Centralized Logging with ELK / EFK

1. Why?

In production, you can't SSH into containers to check logs. You need a **centralized logging solution**.

2. ELK/EFK Stack

- **Elasticsearch** → storage + search.
- **Logstash / Fluentd** → log shipping.
- **Kibana** → visualization.

Spring Boot → logs JSON → shipped via Logstash/Fluentd → stored in Elasticsearch → visualized in Kibana.

3. Spring Boot JSON Logs

application.properties

logging.pattern.console={"timestamp":"%d{yyyy-MM-dd HH:mm:ss}", "level":"%p", "trace":"%X{traceId}", "span":"%X{spanId}", "logger":"%c", "message":"%m"}%n

Includes **traceId** and **spanId** (from Sleuth) so logs across microservices can be correlated.

4. Interview Pointers

- Logs should go to **stdout/stderr** (not local disk) → containers forward them.
- Always include **traceId** for distributed tracing.
- ELK/EFK is industry standard.

150. Metrics & Monitoring with Micrometer

1. Why?

Logs tell *what happened*, metrics tell *how the system is performing*.
Spring Boot integrates **Micrometer** to collect metrics → export to Prometheus, Grafana, Datadog.

2. Setup Example

application.properties

management.endpoints.web.exposure.include=health,metrics,prometheus

Metrics exposed at: /actuator/prometheus

3. Custom Metrics

```
@Component
public class OrderMetrics {
   private final Counter orderCounter;

   public OrderMetrics(MeterRegistry registry) {
      this.orderCounter = Counter.builder("orders_placed")
                          .description("Total orders placed")
                          .register(registry);
   }

   public void recordOrder() {
      orderCounter.increment();
   }
}
```

You'll see orders_placed_total in Prometheus.

4. Interview Pointers

- Default metrics: JVM, memory, HTTP latency, DB pool.
- Custom metrics → for business KPIs (orders placed, payments failed).
- Works seamlessly with Grafana dashboards.

151. Kubernetes Integration (Readiness & Liveness)

1. Why?

When deploying Spring Boot in **Kubernetes**, you need health checks for scaling and auto-restarts.

2. Enable Probes

application.properties

management.endpoint.health.probes.enabled=true
management.endpoints.web.exposure.include=health

Endpoints:

- /actuator/health/liveness → Is app running?
- /actuator/health/readiness → Is app ready to take traffic?

3. Kubernetes Deployment Example

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 5
```

4. Interview Pointers

- Ensures **zero-downtime deployments**.
- Prevents traffic from hitting unhealthy pods.
- Liveness probe restarts pods that are stuck.

Spring Boot Security – Advanced Topics

152. Difference Between Authentication and Authorization

Answer

Authentication = *Who you are*. Authorization = *What you can do*.

- **Authentication**: Verifying credentials (username/password, JWT, OAuth2 token).
- **Authorization**: Checking roles/permissions before allowing actions.

Example in Spring Boot:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
  @Bean
  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
     .authorizeHttpRequests(auth -> auth
       .requestMatchers("/admin/**").hasRole("ADMIN")
       .requestMatchers("/user/**").hasAnyRole("USER","ADMIN")
       .anyRequest().permitAll()
     )
     .httpBasic();
    return http.build();
  }
}
```

/admin/** is only accessible if the user has ADMIN role.

153. JWT Best Practices in Spring Security

Answer

JWTs (JSON Web Tokens) are commonly used for stateless authentication in microservices.

Flow:

1. User logs in → Server validates credentials.
2. Server generates JWT (signed using secret/private key).
3. Client stores JWT in memory or cookies.
4. Each request includes Authorization: Bearer <token>.
5. Server validates token signature & claims before allowing access.

Implementation (Custom Filter):

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
  @Override
  protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
FilterChain chain)
      throws IOException, ServletException {
    String header = req.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")) {
      String token = header.substring(7);
      // validate token...
      UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken("user", null,
          List.of(new SimpleGrantedAuthority("ROLE_USER")));
      SecurityContextHolder.getContext().setAuthentication(auth);
    }
    chain.doFilter(req, res);
  }
}
```

Best Practices:

- Keep JWTs short-lived (e.g., 15 mins) + use refresh tokens.
- Store JWT secrets securely (Vault, AWS KMS).
- Don’t store JWT in localStorage (XSS risk). Prefer **HTTP-only cookies**.
- Include roles & permissions in claims for quick checks.

154. OAuth2 and OpenID Connect in Spring Security

Answer
OAuth2 → Authorization framework (grant access without sharing credentials).
OpenID Connect (OIDC) → Identity layer on top of OAuth2 (provides user profile info).

Use Case: Login with Google/GitHub in a Spring Boot app.

application.yml

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: your-client-id
            client-secret: your-client-secret
            scope: email,profile
```

Now hitting a secured endpoint redirects the user to Google → after successful login, Spring Security handles tokens & user details automatically.

Interview Pointers:

- OAuth2 is used for **authorization**, OIDC adds **authentication (who the user is)**.
- In microservices, you often run **Resource Servers** (validate tokens) + a **Central Authorization Server** (Keycloak, Okta).
- Common grant types: Authorization Code (for web apps), Client Credentials (service-to-service).

155. Method-Level Security with Spring

Answer

Beyond URL-based restrictions, Spring allows **fine-grained method-level security**.

```
@EnableGlobalMethodSecurity(prePostEnabled = true)
@Configuration
class SecurityConfig {}

@Service
class AccountService {

  @PreAuthorize("hasRole('ADMIN')")
  public void closeAccount(Long id) {
    // only admins can close accounts
  }

  @PreAuthorize("#userId == authentication.principal.username")
  public Account viewAccount(Long userId) {
    // users can only view their own account
    return new Account();
  }
}
```

Uses **Spring Expression Language (SpEL)** to write rules.

Interview Pointers:

- Good for enforcing **business rules** (e.g., “user can only update their own profile”).
- Use with care → too much complexity in SpEL makes code hard to maintain.

Spring Boot Security – Advanced Topics (Part 2)

156. CSRF Protection in Spring Security

Answer

- **CSRF (Cross-Site Request Forgery)** → attacker tricks a logged-in user into sending unintended requests (e.g., money transfer).
- **Spring Security defaults**:
  - CSRF **enabled** for stateful apps (session + cookies).
  - CSRF **disabled** automatically for stateless APIs (JWT/OAuth2).

Example (Disable for APIs):

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
   http
    .csrf(csrf -> csrf.disable())  // disable for stateless REST APIs
    .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
    .httpBasic();
   return http.build();
}
```

Interview note:

- For **session-based apps** → keep CSRF ON.
- For **REST APIs with JWTs** → disable CSRF, since tokens already protect requests.

157. Session Management in Spring Security

Answer

Spring Security provides controls over **how user sessions are created & managed**.

```
http
  .sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS) // For JWT
    .maximumSessions(1) // prevent multiple logins
    .expiredUrl("/session-expired.html") // redirect if expired
  );
```

Modes:

- ALWAYS → always create a session.
- IF_REQUIRED → create only when needed.
- NEVER → never create, but use existing.
- STATELESS → no session at all (JWT/OAuth2).

Interview note: In **microservices** → always prefer **STATELESS** + JWT.

158. Securing APIs at the API Gateway (Spring Cloud Gateway)

Answer

In microservices, securing every service individually is messy. Instead, apply **security at the gateway**.

```
spring:
 cloud:
  gateway:
   routes:
    - id: orderservice
      uri: lb://ORDERSERVICE
      predicates:
       - Path=/orders/**
      filters:
       - name: JwtAuthFilter
```

- Gateway checks **JWT / OAuth2 tokens**.
- If valid → forwards request to service.
- If invalid → reject at gateway (services don't care about auth).

Interview note: This is a **best practice in microservices**. Reduces duplicate security configs.

159. Keycloak / Okta Integration with Spring Boot

Answer

Instead of managing security yourself, integrate with **Identity Providers (IdPs)** like **Keycloak, Okta, Auth0**.

```
spring:
 security:
  oauth2:
   resourceserver:
    jwt:
     issuer-uri: http://localhost:8080/realms/myrealm
```

- Keycloak issues JWT tokens.
- Spring Boot validates them automatically using issuer URI.
- No custom filter needed → Spring Security checks roles from token claims.

Example claim in JWT:

```
{
 "sub": "user123",
 "preferred_username": "rajat",
 "realm_access": {
  "roles": ["USER", "ADMIN"]
 }
}
```

Interview note:

- External IdPs simplify scaling.
- Centralized SSO & RBAC (Role Based Access Control).

160. Distributed Security Patterns in Microservices

Answer
Security in microservices = multiple challenges:

1. **JWT Everywhere** → each service validates JWT (stateless).
2. **API Gateway Security** → validate token once at gateway.
3. **Token Propagation** → downstream services forward the original JWT.
    - Example: Order Service → Payment Service (use Feign client + token relay).
4. **Zero Trust** → even internal services authenticate each other.
    - Use **mTLS (mutual TLS)** or service mesh (Istio/Linkerd).

Real-world production pattern:

- API Gateway validates user JWT.
- Service-to-service communication secured via **service accounts / mTLS**.
- Sensitive data requests require **scopes/roles in JWT**.

Spring Boot Security – Best Practices for Production

161. Password Hashing & User Authentication

Answer

- Never store plain text passwords.
- Always use **BCrypt** (built into Spring Security).
- BCrypt automatically salts + hashes passwords → prevents rainbow table attacks.

Example:

```
@Bean
public PasswordEncoder passwordEncoder() {
   return new BCryptPasswordEncoder();
}
```

When saving user:

```
user.setPassword(passwordEncoder.encode(rawPassword));
userRepository.save(user);
```

Interview note: **BCrypt** is the de facto choice. Don't use MD5/SHA256 for passwords.

162. Securing Application Secrets

Answer

Avoid hardcoding secrets (DB password, API keys) in application.properties.

Better options:

1. **Environment variables**
2. **Spring Cloud Config + Vault** (HashiCorp Vault, AWS Secrets Manager)
3. **Kubernetes Secrets**

Example with Vault:

```
spring:
  cloud:
    vault:
      uri: http://127.0.0.1:8200
      authentication: TOKEN
      token: s.XXXXXX
      kv:
        enabled: true
        application-name: myapp
```

Interview note:
In production, **rotate secrets regularly** and integrate with Vault/KMS.

163. Rate Limiting & DDOS Protection

Answer

Spring Boot apps in production are often exposed to APIs → must handle abuse.

- Use **Spring Cloud Gateway + Redis RateLimiter** for API protection.
- Example config:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/orders/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10   # 10 req/sec
                redis-rate-limiter.burstCapacity: 20   # spike buffer
```

Interview note: mention **token bucket algorithm** and protecting services behind API Gateway.

164. Secure HTTP Headers

Answer

Spring Security provides HttpHeadersWriter for common **OWASP-recommended headers**.

```
http
 .headers(headers -> headers
    .contentSecurityPolicy("script-src 'self'")
    .xssProtection(xss -> xss.block(true))
    .frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)
 );
```

Important headers:

- **CSP (Content-Security-Policy)** → blocks malicious scripts.
- **X-XSS-Protection** → prevents reflected XSS.
- **X-Frame-Options** → stops clickjacking.
- **Strict-Transport-Security** → enforces HTTPS.

165. Auditing & Security Logging

Answer

For compliance (e.g., GDPR, PCI DSS), you must log **security-sensitive events**:

- User logins, failures, and lockouts.
- Role/permission changes.
- Access to sensitive APIs.

Example with Spring Boot Auditing:

```
@EntityListeners(AuditingEntityListener.class)
@Entity
public class Order {
   @CreatedBy
   private String createdBy;

   @LastModifiedBy
   private String modifiedBy;
}
```

Combine with **Spring Security's SecurityContextHolder** to automatically log the current user.

In production → export logs to **ELK/EFK stack** or **SIEM tools (Splunk, Datadog)**.

166. Security Testing & Vulnerability Scans

Answer

- Use **Spring Security Test** for mocking users/roles in unit tests.

```
@Test
@WithMockUser(username = "admin", roles = {"ADMIN"})
void testAdminAccess() throws Exception {
  mockMvc.perform(get("/admin/dashboard"))
      .andExpect(status().isOk());
}
```

- Run **OWASP Dependency Check** to detect vulnerable libraries.
- Use tools like **SonarQube** + **Checkmarx** + **OWASP ZAP** in CI/CD.

Interview note: mention **shift-left security** – testing early in the pipeline.

Enterprise-Grade Spring Boot Security

167. Mutual TLS (mTLS) for Service-to-Service Security

Answer

- Normal TLS → client verifies server's certificate.
- **mTLS** → both client and server verify each other's certificates.
- Prevents impersonation → only trusted services can talk.

Spring Boot Config (application.yml):

```
server:
 ssl:
  key-store: classpath:server.p12
  key-store-password: changeit
  trust-store: classpath:truststore.jks
  trust-store-password: changeit
  client-auth: need
```

In microservices:

- Client must present a certificate signed by CA.
- Server verifies it using truststore.

Interview Tip: Mention mTLS is **mandatory in Banking APIs (PCI DSS)** and often enforced at API Gateway or Service Mesh level.

168. Zero Trust Architecture with Spring Boot

Answer

Zero Trust = “Never trust, always verify.”
Every request must be authenticated & authorized, even inside the private network.

How to achieve in Spring Boot:

1. **Short-lived tokens (JWT, OAuth2)** → no long-lived sessions.
2. **mTLS** between microservices.
3. **Fine-grained role-based and attribute-based access control**.
4. **Continuous verification** → validate user/device every request.

Example: Using Spring Security’s method-level security

```
@PreAuthorize("hasRole('ADMIN') and #account.owner == authentication.name")
public Account getAccount(String accountId, UserAccount account) { ... }
```

Common in **Fintech + Healthcare** apps where regulations (GDPR, HIPAA) require strict access control.

169. SAML Authentication in Spring Boot

Answer

- **SAML (Security Assertion Markup Language)** → XML-based protocol for Single Sign-On (SSO).
- Used in **enterprise SSO (Okta, Azure AD, PingIdentity)**.
- Spring Security supports it via spring-security-saml2-service-provider.

Example Config (application.yml):

```
spring:
  security:
    saml2:
      relyingparty:
        registration:
          okta:
            identityprovider:
              entity-id: https://dev-xxxx.okta.com/app/xxxx/sso/saml/metadata
              sso-url: https://dev-xxxx.okta.com/app/xxxx/sso/saml
              verification:
                credentials:
                  - certificate-location: "classpath:okta-cert.pem"
```

Flow:

1. User clicks “Login with Okta”.
2. Redirects to IdP (Okta).
3. IdP authenticates → sends **SAML Assertion** back.
4. Spring Security validates assertion → user authenticated.

Interview Tip: SAML is common in **legacy enterprise**, OAuth2/OIDC in **modern apps**.

170. API Gateway Security (Spring Cloud Gateway / Kong / NGINX)

Answer

In microservices, never expose services directly. Use an **API Gateway**.

Responsibilities:

1. Authentication/Authorization (JWT, OAuth2).
2. Rate Limiting (Redis RateLimiter).
3. IP Whitelisting / Geo Blocking.
4. Centralized Logging & Tracing.

Spring Cloud Gateway Example:

```
spring:
 cloud:
  gateway:
   routes:
    - id: payment-service
      uri: lb://PAYMENT-SERVICE
      predicates:
       - Path=/payments/**
      filters:
       - name: TokenRelay
```

API Gateway ensures **zero-trust enforcement** at entry point.

171. Service Mesh Security (Istio / Envoy with Spring Boot)

Answer

- A **service mesh** adds a **sidecar proxy (Envoy)** next to each Spring Boot service.
- Proxies handle:

  ✓ mTLS automatically between services

  ✓ Retry & Circuit breaking

  ✓ Policy enforcement

  ✓ Distributed tracing

With Istio:

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
 name: default
spec:
 mtls:
  mode: STRICT
```

This forces **all traffic inside the cluster to use mTLS**.

Interview Tip: Service Mesh = **"security + observability + reliability baked in"** without changing app code.

172. Data Encryption in Spring Boot (At-Rest & In-Transit)

Answer

- **At Rest**: Encrypt DB fields using JPA converters.

```
@Converter
public class EncryptConverter implements AttributeConverter<String, String>
{
  private static final String SECRET = "aes-key";

  public String convertToDatabaseColumn(String attribute) {
    return AES.encrypt(attribute, SECRET);
  }

  public String convertToEntityAttribute(String dbData) {
    return AES.decrypt(dbData, SECRET);
  }
}
```

- **In Transit**: Always enforce HTTPS using Strict-Transport-Security.

Bonus: In interviews, mention **FPE (Format Preserving Encryption)** for credit card numbers.

173. What does SOC 2 mean for a Spring Boot service, practically?

Answer

SOC 2 is about proving you consistently operate secure, available, confidential systems. For a Spring Boot team, it translates into repeatable controls and evidence:

1. Access control & least privilege
   • Enforce SSO/OIDC (Keycloak/Okta) and role-based access in code (@PreAuthorize).
   • Lock prod access behind break-glass procedures and audited approvals.
2. Change management
   • Every deploy is via CI/CD with code review + artifact signing (SLSA/Provenance).
   • Actuator endpoints restricted; config changes tracked (Spring Cloud Config + Git history).
3. Logging, monitoring, and incident response
   • Structured JSON logs with traceId/spanId; shipped to SIEM (ELK/Splunk/Datadog).
   • Alerts on auth failures, unusual rate spikes, and privileged actions.
4. Data protection
   • TLS everywhere; database encryption at rest; secrets in Vault/KMS.
   • PII masking in logs; field-level encryption for sensitive columns.
5. Evidence automation
   • Keep “evidence bots” in CI to export audit artifacts: policy JSON, pipeline runs, access reviews.
   • Periodic access recertification (report produced from IdP + service RBAC).

Mini checklist you can cite in interviews: SSO enforced, least privilege, encrypted transport+at rest, SIEM with alerts, immutable logs, ticketed changes, periodic access reviews, and runbooks for incidents.

174. How do you implement audit trails and make logs tamper-evident?

Answer

You need both application-level auditing and immutable storage.

App-level audit (who did what, when, to which resource)

```
@EntityListeners(AuditingEntityListener.class)
@Entity
public class Order {
  @CreatedDate  private Instant createdAt;
  @CreatedBy    private String createdBy;
  @LastModifiedDate private Instant updatedAt;
  @LastModifiedBy   private String updatedBy;
}
```

Enable auditing with a AuditorAware<String> that reads the username from SecurityContextHolder.

Emit security events (login, role change, money movement) as structured logs:

```
log.info("AUDIT event=ROLE_CHANGE actor={} targetUser={} newRole={}",
      actor, targetUser, role);
```

Make it tamper-evident/immutable

- Ship logs off-box immediately (Fluentd/Vector).
- Write to WORM/S3 Object Lock or an append-only index.
- Add hash-chaining if you need cryptographic integrity:

```
class ChainLogger {
  private String lastHash = "GENESIS";
  void audit(String json) {
    String h = sha256(lastHash + json);
    log.info("AUDIT chainedHash={} prevHash={} payload={}", h, lastHash, json);
    lastHash = h;
  }
}
```

Store chain tips externally (e.g., separate bucket) for verification.

175. What's your key rotation strategy (JWT, DB encryption, TLS)?

Answer

Treat keys as versioned, short-lived, and discoverable.

JWT signing keys

• Use JWKs with kid so services can verify against a rotating set. IdP (Keycloak/Okta) publishes /.well-known/jwks.json.

• Configure Spring Resource Server to auto-refresh:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.myco.com/realms/prod
```

Rotation playbook: publish new key → start signing with it → wait for caches → retire old key.

Database/field encryption
• Wrap with KMS (AWS KMS/Azure Key Vault/GCP KMS) using a DEK/KEK model:
– KEK in KMS; DEKs encrypt columns/files; rotate KEK routinely, re-encrypt DEKs (fast).
• For JPA, use a converter that fetches DEK from Vault and caches briefly.

TLS certs

• Automate via ACME/Let's Encrypt or your PKI.

• In Kubernetes, cert-manager rotates; apps reload via sidecar or server reload hooks.

Secrets rotation

• Store in Vault/Secrets Manager; enable rotation lambdas/jobs.

• Spring reads via spring-cloud-starter-vault or aws-secretsmanager-jdbc.

• Design for hot reload: @ConfigurationProperties beans with refresh scope where safe.

176. How do you integrate Spring services with IAM systems (AWS IAM, Keycloak/Okta) cleanly?

Answer

Inbound (human/user auth) → OIDC/OAuth2; outbound (service-to-service) → workload identity.

Inbound with OIDC (Keycloak/Okta)

```
spring:
 security:
  oauth2:
   resourceserver:
    jwt:
     issuer-uri: https://auth.myco.com/realms/prod
```

Map realm/client roles to authorities with a JwtAuthenticationConverter, then gate methods:

```
@PreAuthorize("hasAuthority('ROLE_PAYMENTS_ADMIN')")
public void refund(...) { ... }
```

Outbound to cloud APIs without static keys

• AWS: use IAM Roles for Service Accounts (IRSA) in EKS; the pod gets a projected token, the SDK exchanges it for temporary creds-no secrets in app.

• GCP/Azure: Workload Identity / Managed Identities similarly.

Service-to-service auth

• Prefer mTLS + JWT token relay at Gateway.

• Propagate end-user principal when necessary (Authorization: Bearer); otherwise use service credentials with narrow scopes (client credentials grant).

Periodic access review

• Export mappings (IdP groups → app roles) and reconcile with what the app actually uses. Automate a report.

177. How do you handle secret hygiene end-to-end (creation → usage → rotation → revocation)?

Answer

Policy + automation:

1. Creation
   • Generate in KMS/Vault; never by humans; minimum scope and TTL.
2. Distribution
   • Mounted or fetched at runtime (Vault agent injector / Secrets Store CSI).
   • Apps read via providers (Vault, SM) instead of env vars when possible.
3. Usage
   • Central config binding (@ConfigurationProperties) with fail-closed defaults.
   • No logging of secrets; redact with log filters.
4. Rotation
   • Coordinate: set overlap window; old+new accepted; health checks verify new secret before cutover.
   • Canary first; then roll all pods.
5. Revocation & incident response
   • Immediate revoke in Vault/KMS; roll workloads; invalidate tokens at IdP; rotate dependent keys; run retrospective searches to confirm no leakage.

178. What does “provable” security look like in CI/CD for Spring Boot?

Answer

Defense-in-depth gates you can point to:

• SBOM + dependency scanning (OWASP Dependency-Check, Snyk) fail the build on CVEs.

• Static analysis (Sonar/CodeQL) for injection/XSS/crypto misuse.

• Container scan (Trivy/Grype) and minimal base images (distroless).

• Sign images (Sigstore/cosign) and verify in admission controller (Kyverno/Gatekeeper).

• Runtime policies (Falco) + network policies (K8s) restrict egress/ingress.

• Post-deploy smoke + security tests (ZAP/Burp) in staging.

• Evidence archiving: pipeline logs, signatures, policy decisions attached to the release ticket.

179. What is Hibernate, and why is it used in Java development?

Answer:

Hibernate is an **Object Relational Mapping (ORM) framework** for Java that helps map Java classes (objects) to relational database tables. Instead of manually writing SQL for every CRUD operation, developers can work directly with objects, and Hibernate generates the necessary SQL behind the scenes.

Think of it as a **translator**: you talk in Java objects, Hibernate translates it into SQL queries and sends them to the database.

Key benefits:

1. Eliminates boilerplate JDBC code (no need to manage connections, result sets manually).
2. Database independence (same code works with MySQL, PostgreSQL, Oracle, etc.).
3. Features like lazy loading, caching, batch processing, and transaction management.

Example: Saving an employee without writing SQL

```
@Entity
@Table(name = "employee")
public class Employee {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;
   private String name;
   private String department;
}
SessionFactory factory = new Configuration().configure().buildSessionFactory();
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Employee emp = new Employee();
emp.setName("Alice");
emp.setDepartment("Finance");

session.save(emp);  // Hibernate generates INSERT query internally
tx.commit();
session.close();
```

This inserts a row in the employee table without you writing an SQL INSERT.

180. What is the difference between JPA and Hibernate?

Answer:

JPA (Java Persistence API) is just a **specification**, while Hibernate is one of its **implementations**.

- **JPA** defines how ORM should work (interfaces, annotations, standard APIs).
- **Hibernate** is a concrete ORM framework that implements JPA and adds extra features.

Analogy: JPA is like a **set of rules for playing cricket**, while Hibernate is one of the **teams that follows those rules but also adds its own style**.

Key differences:

1. JPA provides EntityManager API; Hibernate provides Session API (and also implements EntityManager).
2. JPA is limited to the standard features; Hibernate adds extras like @Cascade, FetchProfiles, Envers (auditing).
3. If you code to JPA interfaces, you can switch providers (Hibernate, EclipseLink, OpenJPA).

Example (JPA way vs Hibernate way):

- JPA:

entityManager.persist(employee);

- Hibernate:

session.save(employee);

In practice: Most teams use **Spring Data JPA (JPA on top) but Hibernate under the hood**.

181. Explain Hibernate architecture and its main components.

Answer:
Hibernate architecture has **layers** that connect your Java code to the database:

1. **Configuration**
    - Reads hibernate.cfg.xml or properties.
    - Contains DB connection details, dialect, mappings.
2. **SessionFactory**
    - Heavyweight, thread-safe object created once per application.
    - Provides Session objects.
3. **Session**
    - Lightweight, not thread-safe, represents a unit of work.
    - Manages persistent objects and first-level cache.
4. **Transaction**
    - Manages database transactions (commit, rollback).
5. **Query API**
    - For HQL, Criteria, or native SQL queries.
6. **Entities (POJOs)**
    - Your Java classes annotated with @Entity, @Table, etc.

Flow in practice:

1. Load configuration → create SessionFactory.
2. Get a Session → begin transaction.
3. Perform CRUD operations using entities.
4. Commit or rollback.
5. Close session.

Real-world analogy:

- SessionFactory = Bank Headquarters (created once).
- Session = Branch where you do transactions.
- Transaction = Your withdrawal/deposit action.
- Entities = Customers' accounts.

182. What is @Entity in Hibernate, and how is it used?

Answer:
@Entity is the most fundamental annotation in JPA/Hibernate. It marks a Java class as a persistent entity, meaning Hibernate will map it to a database table.

Rules:

1. The class must have a no-arg constructor.
2. Must have a primary key (@Id).
3. Should not be final (Hibernate uses proxies).

Example:

```
import jakarta.persistence.*;

@Entity
public class Product {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   private String name;
   private double price;

   // getters, setters
}
```

Here, Hibernate creates a product table with columns id, name, and price.

Real-world analogy: Declaring a class as @Entity is like registering a citizen in a government database - now it's tracked and can be queried, updated, or deleted.

183. What is the difference between @Entity and @Table in Hibernate?

Answer:

- @Entity: declares the class as a persistent entity. Without this, Hibernate ignores the class.
- @Table: customizes table mapping (name, schema, unique constraints).

If @Table is omitted, Hibernate uses the class name as the table name by default.

Example:

```
@Entity
@Table(name = "products", schema = "inventory",
    uniqueConstraints = @UniqueConstraint(columnNames = {"sku"}))
public class Product {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;

  @Column(nullable = false)
  private String sku;

  private String name;
}
```

This maps the class Product to the table inventory.products with a unique constraint on sku.

In practice:

- Use @Entity always.
- Use @Table only when you need customization.

184. What is the purpose of @Id and @GeneratedValue in Hibernate?

Answer:

Every entity must have a **primary key**, and @Id marks that field.
@GeneratedValue tells Hibernate how to generate values for that primary key.

Strategies:

1. GenerationType.IDENTITY → uses DB auto-increment (MySQL).
2. GenerationType.SEQUENCE → uses a database sequence (Oracle, PostgreSQL).
3. GenerationType.TABLE → uses a special table to generate IDs (less common now).
4. GenerationType.AUTO → Hibernate picks the best strategy based on DB.

Example:

```
@Entity
public class Order {
   @Id
   @GeneratedValue(strategy = GenerationType.SEQUENCE)
   private Long orderId;

   private String customer;
}
```

Here, orderId will be auto-generated using a DB sequence.

Why this matters: It ensures unique IDs without manual effort, especially useful in multi-threaded, high-traffic applications like **e-commerce order systems**.

185. What is the purpose of @Column in Hibernate?

Answer:

@Column customizes how a field is mapped to a database column. If omitted, Hibernate maps the field name to a column with the same name by default.

Key attributes:

- name → custom column name.
- nullable → whether the column allows nulls.
- unique → whether the column enforces uniqueness.
- length → column length (for strings).
- columnDefinition → override column type with raw SQL.

Example:

```
@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   @Column(name = "user_name", nullable = false, length = 100)
   private String username;

   @Column(unique = true)
   private String email;
}
```

This maps username to user_name (VARCHAR(100), NOT NULL) and makes email unique.

Best practice: only use @Column when customization is needed; otherwise, let Hibernate defaults work.

186. What is the use of @Transient in Hibernate?

Answer:

@Transient marks a field as **not persistent**. Hibernate ignores it completely (not mapped to DB).

When to use:

- For temporary/calculated values.
- For caching non-persistent state.
- For fields used in business logic but not stored.

Example:

```
@Entity
public class Product {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   private double price;
   private double tax;

   @Transient
   private double finalPrice;

   public double getFinalPrice() {
      return price + tax;
   }
}
```

Here, finalPrice is never stored in DB; it's computed at runtime.

Common mistake: Forgetting @Transient on helper fields can cause schema mismatches or errors.

187. What is @Enumerated in Hibernate, and when is it used?

Answer:
@Enumerated maps Java enums to DB columns. Without it, Hibernate doesn't know how to persist enums.

Options:

- EnumType.ORDINAL → stores the enum index (0,1,2,...).
- EnumType.STRING → stores the enum name ("ACTIVE", "INACTIVE").

Example:

```
public enum Status {
   ACTIVE, INACTIVE, SUSPENDED
}

@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   @Enumerated(EnumType.STRING)
   private Status status;
}
```

If status = Status.ACTIVE, DB will store "ACTIVE".

Best practice: **Always use EnumType.STRING** to avoid issues if enum order changes in future.

188. What is @Temporal in Hibernate, and when is it used?

Answer:

@Temporal specifies how Java date/time types (java.util.Date, java.util.Calendar) should be mapped to SQL.

Options:

- TemporalType.DATE → stores only the date (yyyy-MM-dd).
- TemporalType.TIME → stores only the time (HH:mm:ss).
- TemporalType.TIMESTAMP → stores both date and time.

Example:

```
@Entity
public class Event {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   @Temporal(TemporalType.DATE)
   private Date eventDate;

   @Temporal(TemporalType.TIMESTAMP)
   private Date createdAt;
}
```

Best practice: For modern apps, prefer java.time API (LocalDate, LocalDateTime) introduced in JPA 2.2, which doesn't need @Temporal.

189. What is @Lob in Hibernate, and when is it used?

Answer:

@Lob (Large Object) is used to store large data types:

- CLOB (Character Large Object) → for text (e.g., String, char[]).
- BLOB (Binary Large Object) → for binary (e.g., byte[], images, files).

Hibernate automatically decides based on the Java type.

Example (CLOB):

```
@Entity
public class Article {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   @Lob
   private String content;  // stored as CLOB
}

Example (BLOB):

@Entity
public class Document {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   @Lob
   private byte[] fileData;  // stored as BLOB
}
```

Best practice: Use @Lob for large, rarely queried fields. Keep frequently accessed fields outside to avoid performance issues.

190. What are @Embedded and @Embeddable in Hibernate, and how are they used?

Answer:

They allow you to reuse a class as a value type (not a separate entity/table).

- @Embeddable → marks the reusable class.
- @Embedded → marks the field inside the entity.

Example:

```
@Embeddable
public class Address {
   private String street;
   private String city;
   private String zipcode;
}

@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   private String name;

   @Embedded
   private Address address;
}
```

DB schema → all Address fields (street, city, zipcode) are stored in User table itself.

Benefits:

- Avoids code duplication.
- Keeps models clean by grouping related fields.
- Useful for common objects like Address, AuditInfo (createdAt, updatedAt).

191. What is @ElementCollection in Hibernate, and when should you use it?

Answer:

@ElementCollection is used when you want to map a collection of simple types (String, Integer, etc.) or embeddable objects without creating a separate entity. Hibernate creates a separate table to store the collection.

Example with simple types:

```
@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   private String name;

   @ElementCollection
   private List<String> phoneNumbers;
}
```

This creates a user_phone_numbers table internally with user_id and phone_number.

Example with embeddables:

```
@Embeddable
public class Address {
   private String city;
   private String zipcode;
}

@Entity
public class Customer {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   @ElementCollection
   private List<Address> addresses;
}
```

Best use case: when related data has no independent lifecycle (e.g., tags, phone numbers, addresses).

192. How do you define composite keys using @IdClass vs @EmbeddedId in Hibernate?

Answer:

Hibernate supports composite primary keys in two ways:

1. Using @IdClass

- Create a separate class with key fields.
- Reference them in the entity with @Id.

```
class OrderId implements Serializable {
  private int orderId;
  private int productId;
}

@Entity
@IdClass(OrderId.class)
public class Order {
  @Id private int orderId;
  @Id private int productId;
  private int quantity;
}
```

2. Using @EmbeddedId

- Create an embeddable class.
- Use it directly as a field.

```
@Embeddable
class OrderKey implements Serializable {
  private int orderId;
  private int productId;
}

@Entity
public class Order {
  @EmbeddedId
  private OrderKey id;
  private int quantity;
}
```

Differences:

- @IdClass → looks like multiple @Id fields, simpler but less encapsulated.
- @EmbeddedId → cleaner, more object-oriented (recommended).

193. What is the role of @MappedSuperclass in inheritance mapping?

Answer:

@MappedSuperclass defines a base class with common fields, but it's **not an entity itself**.

- Its fields are mapped into child entities' tables.
- No separate table is created.
- Cannot be queried directly.

Example:

```
@MappedSuperclass
public abstract class BaseEntity {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private LocalDateTime createdAt;
   private LocalDateTime updatedAt;
}

@Entity
public class Employee extends BaseEntity {
   private String name;
}

@Entity
public class Department extends BaseEntity {
   private String departmentName;
}
```

☞ Both Employee and Department tables will have id, createdAt, updatedAt columns without duplicating code.

Best use case: Common audit fields (id, createdAt, updatedAt, createdBy).

194. What are the different inheritance strategies in Hibernate, and when should you use them?

Answer:

Hibernate supports mapping Java inheritance to database tables using three main strategies:

1. **Single Table Strategy (InheritanceType.SINGLE_TABLE)**

- All classes in the hierarchy are stored in **one table**.
- A discriminator column identifies the subclass type.
- Fast queries but lots of NULL columns.

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "type")
public class Employee {
  @Id private int id;
  private String name;
}

@Entity
@DiscriminatorValue("MGR")
public class Manager extends Employee {
  private String department;
}
```

2. **Joined Strategy (InheritanceType.JOINED)**

- Parent class fields in one table, child-specific fields in another.
- Normalized schema, no redundant NULLs.
- But queries require joins (slower).

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Employee { ... }

@Entity
public class Manager extends Employee {
  private String department;
}
```

3. **Table per Class Strategy (InheritanceType.TABLE_PER_CLASS)**

- Each class has its own table with all fields (including inherited).
- No joins needed, but **redundant columns** across tables.

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Employee { ... }
```

Rule of Thumb:

- Use **SINGLE_TABLE** for small/simple hierarchies (fastest).
- Use **JOINED** for normalized schema with many subclasses.
- Avoid **TABLE_PER_CLASS** in most real projects (duplication).

195. What are the entity lifecycle states in Hibernate?

Answer:

Entities in Hibernate go through **four lifecycle states**:

1. **Transient**

- Newly created object (not associated with Hibernate Session).
- Not stored in DB.

Employee e = new Employee(); // transient

2. **Persistent (Managed)**

- Associated with a Session and tracked by persistence context.
- Any changes are automatically synchronized to DB on flush/commit.

session.save(e); // persistent

3. **Detached**

- Was persistent, but session is closed or entity is evicted.
- Changes are NOT tracked unless merged back.

session.evict(e); // detached

4. **Removed**

- Entity marked for deletion.
- Actual delete happens on flush/commit.

session.delete(e); // removed

Interview Tip: Always explain with an example of Session → commit → detach.

196. What are Hibernate event listeners, and how are they used?

Answer:

Event listeners in Hibernate let you intercept entity lifecycle events (insert, update, delete, load).
They're often used for **auditing, logging, or enforcing business rules**.

Example: Logging entity creation

```
public class EmployeeListener {
   @PrePersist
   public void beforeSave(Employee e) {
      System.out.println("Saving employee: " + e.getName());
   }

   @PostPersist
   public void afterSave(Employee e) {
      System.out.println("Employee saved with ID: " + e.getId());
   }
}

@Entity
@EntityListeners(EmployeeListener.class)
public class Employee {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;
   private String name;
}
```

Common callbacks:

- @PrePersist, @PostPersist
- @PreUpdate, @PostUpdate
- @PreRemove, @PostRemove
- @PostLoad

Real-world usage: Auditing (createdAt, updatedAt), soft deletes, validation.

197. What are the different types of caching in Hibernate, and how do they improve performance?

Answer:

Hibernate reduces database hits using **multiple caching layers**:

1. **First-Level Cache (Session Cache)**

- Mandatory, enabled by default.
- Scoped to a Hibernate Session.
- Same entity fetched multiple times in one session comes from cache, not DB.

```
Session session = factory.openSession();
Employee e1 = session.get(Employee.class, 1); // DB hit
Employee e2 = session.get(Employee.class, 1); // Cache hit
```

2. **Second-Level Cache (SessionFactory Cache)**

- Optional, shared across sessions.
- Requires a cache provider (Ehcache, Redis, Infinispan).
- Good for reference data like product catalog or lookup tables.

```
<property name="hibernate.cache.use_second_level_cache" value="true"/>
<property name="hibernate.cache.region.factory_class"
value="org.hibernate.cache.ehcache.EhCacheRegionFactory"/>
```

3. **Query Cache**

- Works with 2nd-level cache.
- Caches query results (IDs of entities).
- Entities themselves must still be cached in 2nd-level cache.

```
Query q = session.createQuery("FROM Employee e WHERE e.department = :d");
q.setCacheable(true);
```

Interview Tip:

- First-level is per-session (always ON).
- Second-level is across sessions (configurable).
- Query cache stores results, but you must combine it with 2nd-level cache.

198. What is batch fetching in Hibernate, and how does it help?

Answer:
Batch fetching reduces the **N+1 problem** when loading collections or related entities. Instead of fetching one child at a time, Hibernate fetches them in **batches**.

Example:

```
@Entity
public class Department {
   @OneToMany(mappedBy = "department")
   @BatchSize(size = 10)
   private List<Employee> employees;
}
```

If you load 50 departments, Hibernate will fetch employees in batches of 10 departments per query instead of firing 50 queries.

Real-world analogy: Instead of calling your database 50 times to fetch employees of each department, you call it only 5 times (10 departments per batch).

Benefits:

- Reduces DB round-trips.
- Works well with large datasets.
- Good balance between JOIN FETCH (too heavy sometimes) and individual queries.

199. What is dirty checking in Hibernate, and how does it work?

Answer:
Dirty checking means Hibernate automatically detects changes to persistent entities and synchronizes them with the database at flush/commit time.

Example:

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Employee e = session.get(Employee.class, 1);
e.setName("Updated Name"); // No update yet

tx.commit(); // Hibernate issues UPDATE automatically
```

How it works internally:

- Hibernate keeps a **snapshot** of entity state when it is loaded.
- At flush/commit, it compares the current state with the snapshot.
- If changes are detected → UPDATE query is fired.

Benefits:

- Developers don't need to explicitly call update().
- Reduces boilerplate.

Pitfalls:

- Can cause unexpected UPDATEs if you modify entities unintentionally.
- Use @DynamicUpdate to avoid updating unchanged columns.

200. What are the different flush modes in Hibernate, and how do they impact performance?

Answer:

The **flush mode** determines when Hibernate synchronizes in-memory changes with the database.

Types of flush modes:

1. **AUTO (default)**

- Flush happens automatically before query execution or transaction commit.
- Most commonly used.

session.setFlushMode(FlushMode.AUTO);

2. **COMMIT**

- Flush happens only at transaction commit.
- Reduces unnecessary flushes but may return stale data if queries are executed mid-transaction.

session.setFlushMode(FlushMode.COMMIT);

3. **MANUAL (or NEVER)**

- Flush happens only when explicitly called (session.flush()).
- Best for batch operations where you want fine-grained control.

session.setFlushMode(FlushMode.MANUAL);

session.flush(); // explicitly flush

Best Practices:

- Use **AUTO** for most apps.
- Use **COMMIT** for read-heavy apps where intermediate queries don’t need fresh data.
- Use **MANUAL** for batch inserts/updates to optimize performance.

201. How does JOIN FETCH solve the N+1 problem compared to normal fetch joins?

Answer:

- The **N+1 problem**: 1 query for parent entities + N queries for child collections.
- Solution → **JOIN FETCH** loads parent and child in **one query**.

Example without JOIN FETCH:

```
List<Department> departments = session.createQuery("FROM Department").list();
for (Department d : departments) {
   System.out.println(d.getEmployees()); // N additional queries
}

With JOIN FETCH:

List<Department> departments = session.createQuery(
   "SELECT d FROM Department d JOIN FETCH d.employees"
).list();
```

Advantages:

- Single SQL query.
- Prevents performance bottlenecks.

Pitfalls:

- JOIN FETCH may produce **Cartesian products** if multiple collections are fetched.
- Use cautiously with pagination, as it can mess up results.

Best practice:

- For one collection → JOIN FETCH.
- For multiple collections → consider **Entity Graphs** or **batch fetching** instead.

202. What is the difference between Hibernate Interceptors and Event Listeners?

Answer:

Both allow you to hook into Hibernate's lifecycle, but they differ in **scope and use cases**.

1. **Interceptor**

- Single point to intercept entity lifecycle events.
- Implement org.hibernate.Interceptor interface.
- Works globally (applies to all entities in a Session).
- Example use: auditing, logging SQL, enforcing soft deletes.

```
public class AuditInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id, Object[] state,
                String[] propertyNames, Type[] types) {
    System.out.println("Entity saved: " + entity);
return false;
  }
}
```

2. **Event Listener**

- Fine-grained, event-specific.
- Register listeners for specific events (pre-insert, post-update, etc.).
- More flexible and scalable than interceptors.
- Example use: validating entities before insert, publishing domain events.

```
public class CustomSaveListener implements PreInsertEventListener {
  @Override
  public boolean onPreInsert(PreInsertEvent event) {
    System.out.println("Pre-insert: " + event.getEntity());
    return false;
  }
}
```

Key Differences:

- **Interceptor** → Broad, centralized, simpler (like a “catch-all”).
- **Event Listener** → Granular, event-driven, preferred for large systems.

203. How can you monitor Hibernate performance using Statistics API?

Answer:

Hibernate provides a **Statistics API** that gives insights into how sessions interact with the database. It tracks metrics like entity loads, cache hits, query execution counts, etc.

Enable statistics:

```
SessionFactory sessionFactory = new
Configuration().configure().buildSessionFactory();
sessionFactory.getStatistics().setStatisticsEnabled(true);

Statistics stats = sessionFactory.getStatistics();
System.out.println("Entity fetch count: " + stats.getEntityFetchCount());
System.out.println("Query execution count: " + stats.getQueryExecutionCount());
```

Key metrics you get:

- **Entity statistics** → loads, inserts, updates, deletes.
- **Query statistics** → execution count, execution time.
- **Cache statistics** → hits, misses, puts for 2nd-level cache.

Real-world usage:

- In production, expose these metrics via **Actuator/Prometheus**.
- Detect N+1 problems by checking query execution counts.
- Identify hotspots (e.g., a query executing 10,000 times).

204. What is Hibernate Envers, and how does it support auditing?

Answer:
Hibernate **Envers** is a module that provides **automatic auditing/versioning** of entity changes. Instead of manually keeping audit tables, Envers manages it for you.

Setup:

- Add hibernate-envers dependency.
- Annotate entity with @Audited.

Example:

```
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue
   private int id;
   private String name;
   private String department;
}
```

Envers automatically creates audit tables (e.g., Employee_AUD) with version info.

Querying revisions:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
Employee oldVersion = reader.find(Employee.class, 1, 2); // id=1, revision=2
```

Benefits:

- Full change history without writing manual triggers.
- Useful for compliance (banking, healthcare, government systems).
- Supports rollback to previous versions if needed.

205. How do you optimize fetch strategies in real-world Hibernate systems?

Answer:
Optimizing fetch strategies is crucial to avoid performance bottlenecks like **N+1 queries** and **huge joins**.

Common strategies:

1. **Lazy by default**

```
@OneToMany(mappedBy = "dept", fetch = FetchType.LAZY)
private List<Employee> employees;
```

- Load only when accessed.

2. **JOIN FETCH when always needed**

```
SELECT d FROM Department d JOIN FETCH d.employees
```

- Avoids N+1, but use carefully with pagination.

3. **Batch fetching**

```
@BatchSize(size = 20)
private List<Employee> employees;
```

- Loads collections in batches instead of one by one.

4. **Entity Graphs**

```
EntityGraph<?> graph = em.getEntityGraph("Employee.detail");
Map<String, Object> hints = Map.of("jakarta.persistence.fetchgraph", graph);
Employee emp = em.find(Employee.class, 1, hints);
```

- Dynamic fetch plans per use case.

5. **DTO Projections**

```
@Query("SELECT new com.dto.EmployeeDTO(e.id, e.name) FROM Employee e")
List<EmployeeDTO> findEmployeeDTOs();
```

- Load only required fields instead of entire entity.

Best practices:

- Default to **LAZY** → prevents over-fetching.
- Use **JOIN FETCH/Entity Graphs** when a specific use case demands it.
- Use **DTO projections** for API responses.
- Monitor queries with Hibernate Statistics to detect inefficiencies.

206. How does Hibernate manage caching in detail, and what strategies should be used?

Answer:
Hibernate uses **two levels of caching** to reduce database hits:

1. **First-Level Cache (Session Cache)**

- Enabled by default.
- Scope: one Hibernate Session.
- Example:

```
Employee e1 = session.get(Employee.class, 1); // Hits DB
Employee e2 = session.get(Employee.class, 1); // From cache
```

- Useful for avoiding duplicate queries within a transaction.

2. **Second-Level Cache (SessionFactory Cache)**

- Optional, shared across sessions.
- Backed by providers: Ehcache, Infinispan, Redis.
- Config:

  hibernate.cache.use_second_level_cache=true
  hibernate.cache.region.factory_class=org.hibernate.cache.ehcache.EhCacheRegionFactory

- Entities must be annotated:

```
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Employee { ... }
```

3. **Query Cache**

- Stores results of queries (with parameters).
- Must be explicitly enabled.

```
hibernate.cache.use_query_cache=true
List<Employee> employees = session.createQuery("FROM Employee")
```

```
        .setCacheable(true)
        .list();
```

Real-world caching strategy:

- **First-level cache** → always there, nothing to configure.
- **Second-level cache** → use for reference/read-mostly data (catalog, locations).

  Avoid for volatile transactional data.
- **Query cache** → only for repeated queries with stable data.

207. What concurrency and locking strategies does Hibernate support?

Answer:
Concurrency happens when multiple transactions access/modify the same data.
Hibernate supports:

1. **Optimistic Locking** (default, non-blocking)

- Uses a @Version field.
- Example:

```
@Entity
public class Employee {
  @Id @GeneratedValue private int id;
  @Version private int version;
}
```

- If two transactions try updating the same row, one fails with OptimisticLockException.
- Best for systems where conflicts are rare (CRM, HR apps).

2. **Pessimistic Locking** (explicit DB locks)

- Prevents conflicts by locking rows until transaction ends.
- Example:

```
Employee emp = session.get(Employee.class, 1,
LockMode.PESSIMISTIC_WRITE);
```

- Blocks other transactions from reading/writing.
- Best for high-contention systems like banking fund transfers.

3. **Isolation Levels (via DB/JDBC)**

- READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE.
- Controlled by DB, Hibernate respects the configured level.

Best practices:

- Default to **optimistic locking** for scalability.
- Use **pessimistic locks** only when conflicts are likely and data consistency is critical (banking, ticket booking).
- Always handle lock exceptions gracefully (retry or fail fast).

208. How does Hibernate integrate with JTA for distributed transactions?

Answer:

In enterprise systems, a transaction might span multiple resources (DB, message broker, etc.). Hibernate integrates with **JTA (Java Transaction API)** to manage these distributed transactions.

Setup:

- JTA manages transactions across resources (e.g., DB + JMS).
- Hibernate delegates transaction handling to JTA provider (like Atomikos, Bitronix, or application servers).

Example in Spring Boot (with JTA):

```
spring.jta.enabled=true
spring.datasource.xa.data-source-class-
name=com.mysql.cj.jdbc.MysqlXADataSource

Code sample:

@Service
public class OrderService {
   @Transactional
   public void placeOrder(Order order) {
      orderRepo.save(order); // Hibernate → DB
      jmsTemplate.convertAndSend("orders", order); // JMS
   }
}
```

Here, both DB insert and JMS publish happen in **one JTA transaction**. If one fails, both roll back.

Use cases:

- E-commerce → save order + send payment event.
- Banking → debit one account + credit another + log transaction.

209. What are Projections in Hibernate, and how are they used for optimized queries?

Answer:

Projections allow you to **fetch only selected columns** instead of entire entities. This improves performance and reduces memory usage.

1. **Hibernate Criteria API with Projection**

```
Criteria criteria = session.createCriteria(Employee.class);
criteria.setProjection(Projections.property("name"));
List<String> names = criteria.list();
```

This fetches only employee names (not full objects).

2. **HQL / JPQL with projection**

```
List<String> names = session.createQuery(
  "SELECT e.name FROM Employee e", String.class).list();
```

3. **Multiple fields → Tuple/List**

```
List<Object[]> result = session.createQuery(
   "SELECT e.name, e.salary FROM Employee e").list();
for(Object[] row : result){
   System.out.println(row[0] + " " + row[1]);
}
```

When to use:

- Large entities with many fields but API only needs a subset.
- Reports and dashboards (e.g., only employee names + salaries).

210. How do you map DTOs (Data Transfer Objects) in Hibernate queries?

Answer:

Fetching partial data into DTOs avoids exposing entities directly and improves performance.

1. **HQL with DTO Constructor Expression**

```
public class EmployeeDTO {
  private String name;
  private double salary;
  public EmployeeDTO(String name, double salary) {
    this.name = name; this.salary = salary;
  }
}

List<EmployeeDTO> employees = session.createQuery(
  "SELECT new com.example.EmployeeDTO(e.name, e.salary) FROM
Employee e",
  EmployeeDTO.class).list();
```

This directly maps query results into DTO objects.

2. **Criteria API with DTO mapping**

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<EmployeeDTO> cq = cb.createQuery(EmployeeDTO.class);
Root<Employee> root = cq.from(Employee.class);
cq.select(cb.construct(EmployeeDTO.class, root.get("name"),
root.get("salary")));
List<EmployeeDTO> list = entityManager.createQuery(cq).getResultList();
```

3. **Native SQL → DTO with @SqlResultSetMapping**

```
@SqlResultSetMapping(
  name = "EmployeeDTOMapping",
  classes = @ConstructorResult(
    targetClass = EmployeeDTO.class,
    columns = {
      @ColumnResult(name = "name"),
      @ColumnResult(name = "salary")
```

```
        }
      )
    )
    Then use:
    List<EmployeeDTO> list = entityManager
      .createNativeQuery("SELECT name, salary FROM employee",
    "EmployeeDTOMapping")
      .getResultList();
```

Use DTOs when:

- Building REST APIs (never expose entity directly).
- Avoiding lazy loading issues.
- Improving performance (fetch only required fields).

211. How do you use the Criteria API for dynamic queries in Hibernate?

Answer:

Criteria API is a **type-safe, programmatic way** of building queries dynamically. Useful when conditions vary at runtime.

Example: search employees dynamically by name and salary:

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> root = cq.from(Employee.class);

List<Predicate> predicates = new ArrayList<>();

if(name != null){
   predicates.add(cb.equal(root.get("name"), name));
}
if(minSalary != null){
   predicates.add(cb.greaterThan(root.get("salary"), minSalary));
}

cq.select(root).where(predicates.toArray(new Predicate[0]));

List<Employee> results = entityManager.createQuery(cq).getResultList();
```

Benefits:

- **Type safety** → compiler checks attributes.
- **Dynamic conditions** → build filters on the fly.
- **Reusability** → can combine predicates across services.

Real-world example:
E-commerce search (filter by category, price range, rating, stock). Criteria API dynamically builds the WHERE clause.

212. How is pagination handled in Hibernate, and what are the best practices?

Answer:

Pagination helps manage large datasets by fetching results in **chunks** instead of loading everything into memory.

1. **HQL with setFirstResult + setMaxResults**

```
Query<Employee> query = session.createQuery("FROM Employee e ORDER
BY e.id", Employee.class);
query.setFirstResult(0);   // starting row
query.setMaxResults(10);   // page size
List<Employee> employees = query.list();
```

2. **Criteria API**

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
cq.from(Employee.class);

TypedQuery<Employee> query = entityManager.createQuery(cq);
query.setFirstResult(20); // offset
query.setMaxResults(10);  // limit
List<Employee> results = query.getResultList();
```

3. **Spring Data JPA Pageable**

```
Page<Employee> employees = repo.findAll(PageRequest.of(0, 10,
Sort.by("id")));
```

Best Practices:

- Always **use ORDER BY** for consistent paging.
- Use **indexed columns** to improve performance.
- For large datasets, prefer **keyset pagination** (WHERE id > lastSeenId) instead of OFFSET to avoid slow scans.

213. How do you use native SQL queries in Hibernate?

Answer:
Hibernate supports **native SQL** when HQL/JPQL isn't sufficient.

1. **Simple Native Query**

```
List<Object[]> results = session.createNativeQuery(
  "SELECT id, name FROM employee").list();
```

2. **Mapping results to entity**

```
List<Employee> employees = session.createNativeQuery(
  "SELECT * FROM employee", Employee.class).list();
```

3. **Mapping results to DTO**

```
@SqlResultSetMapping(
  name = "EmployeeDTOMapping",
  classes = @ConstructorResult(
    targetClass = EmployeeDTO.class,
    columns = {
      @ColumnResult(name = "name"),
      @ColumnResult(name = "salary")
    }
  )
)
```

Usage:

```
List<EmployeeDTO> list = entityManager
  .createNativeQuery("SELECT name, salary FROM employee",
"EmployeeDTOMapping")
  .getResultList();
```

When to use:

- Complex queries involving DB-specific functions.
- Bulk operations (updates, deletes).
- Performance tuning (e.g., DB-specific optimizations).

214. What is Multi-Tenancy in Hibernate, and how is it implemented?

Answer:
Multi-Tenancy means a single application serves **multiple customers (tenants)** with **isolated data**. Hibernate supports this natively.

Strategies:

1. **Database per Tenant** → each tenant has its own DB.
2. **Schema per Tenant** → same DB, separate schemas.
3. **Discriminator Column** → one DB, one schema, tenant_id column in each table.

Configuration Example:

```
@Configuration
public class HibernateConfig {

   @Bean
   public MultiTenantConnectionProvider multiTenantConnectionProvider() {
      return new DataSourceMultiTenantConnectionProviderImpl();
   }

   @Bean
   public CurrentTenantIdentifierResolver tenantResolver() {
      return new CurrentTenantIdentifierResolverImpl();
   }
}
```

Resolver Example:

```
public class CurrentTenantIdentifierResolverImpl implements
CurrentTenantIdentifierResolver {
   @Override
   public String resolveCurrentTenantIdentifier() {
      return TenantContext.getTenantId(); // e.g., from HTTP header
   }
   @Override
   public boolean validateExistingCurrentSessions() {
      return true;
   }
}
```

Best Practices:

- Use **schema-per-tenant** for medium-sized apps (balance of isolation + maintenance).
- Use **discriminator column** only for small-scale apps (least secure).
- Always **encrypt tenant identifiers** when passing via headers.

215. What are Event Listeners in Hibernate, and how are they used?

Answer:
Hibernate provides **event listeners** to intercept lifecycle events such as entity insert, update, delete, or load. They allow custom logic to run automatically during persistence operations.

Common Events:

- PreInsertEvent, PostInsertEvent
- PreUpdateEvent, PostUpdateEvent
- PreDeleteEvent, PostDeleteEvent
- LoadEvent

Example – Logging entity insert:

```
public class InsertEventListenerImpl implements PreInsertEventListener {
  @Override
  public boolean onPreInsert(PreInsertEvent event) {
    System.out.println("Entity about to be inserted: " + event.getEntity());
    return false; // return true to veto the insert
  }
}
```

Register Listener:

```
Configuration config = new Configuration();
config.getEventListeners().setPreInsertEventListeners(new PreInsertEventListener[] {
  new InsertEventListenerImpl()
});
```

Use Cases:

- Auditing (tracking changes).
- Validations before persistence.
- Soft deletes instead of hard deletes.

216. What are Interceptors in Hibernate, and how do they differ from Event Listeners?

Answer:
Interceptors are hooks into Hibernate's session lifecycle. They allow modification of SQL statements or entities at runtime.

Key Difference:

- **Event Listeners** → fine-grained, specific events (insert, update).
- **Interceptors** → broader control over session operations (e.g., SQL manipulation).

Example – Auditing with Interceptor:

```
public class AuditInterceptor extends EmptyInterceptor {
   @Override
   public boolean onSave(Object entity, Serializable id, Object[] state,
                         String[] propertyNames, Type[] types) {
      if (entity instanceof Auditable) {
         int index = Arrays.asList(propertyNames).indexOf("createdAt");
         state[index] = LocalDateTime.now();
         return true;
      }
      return false;
   }
}
```

Attach to Session:

```
Session session = sessionFactory.withOptions()
   .interceptor(new AuditInterceptor())
   .openSession();
```

Use Cases:

- Custom auditing.
- Multitenancy (inject tenant_id dynamically).
- SQL logging or modification.

217. What is StatelessSession in Hibernate, and when should it be used?

Answer:

A StatelessSession is a **lightweight alternative to Session** that does **not maintain a first-level cache or persistence context**.

Characteristics:

- No automatic dirty checking.
- No cascades.
- No lazy loading.
- Faster for bulk operations.

Example – Bulk Insert with StatelessSession:

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee e = new Employee("Emp" + i);
   session.insert(e);
}

tx.commit();
session.close();
```

When to Use:

- **ETL jobs** (import/export of large data).
- **Batch processing** (mass inserts/updates).
- Scenarios where caching and entity lifecycle management are unnecessary.

Not suitable for:

- Regular OLTP applications (needs entity lifecycle + relationships).

218. What is Hibernate Envers, and how is it used for auditing?

Answer:

Hibernate Envers is a module that provides **auditing** by automatically keeping track of entity changes. It maintains a history of modifications so you can query what changed, when, and by whom.

How it works:

- Creates an audit table for each audited entity.
- Inserts a new row in the audit table whenever the entity is inserted, updated, or deleted.
- Provides APIs to query historical states.

Setup:

1. Add dependency:

```
<dependency>
 <groupId>org.hibernate.orm</groupId>
 <artifactId>hibernate-envers</artifactId>
</dependency>
```

2. Annotate entity:

```
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String name;
   private String department;
}
```

3. Query history:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
List<Number> revisions = reader.getRevisions(Employee.class, 1L);

for (Number rev : revisions) {
   Employee emp = reader.find(Employee.class, 1L, rev);
   System.out.println("Revision " + rev + ": " + emp.getName());
}
```

Use Cases:

- Financial/banking systems (track who updated amounts).
- Healthcare (patient record history).
- Debugging accidental changes.

219. What are the different Second-Level Cache providers supported by Hibernate?

Answer:

The second-level cache stores entities/queries **across sessions** to reduce database hits. Hibernate doesn't provide caching itself but integrates with providers.

Common Providers:

1. **Ehcache** – Lightweight, easy to use.
2. **Infinispan** – Distributed, suitable for clusters.
3. **Hazelcast** – In-memory data grid with clustering support.
4. **Redis** – Popular for distributed caching.
5. **OSCache** – Older, not widely used now.

Configuration Example (Ehcache):

```
<property name="hibernate.cache.use_second_level_cache" value="true"/>
<property name="hibernate.cache.region.factory_class"
      value="org.hibernate.cache.ehcache.EhCacheRegionFactory"/>
```

Entity-level cache usage:

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product {
   @Id
   private int id;
   private String name;
}
```

When to use:

- Read-heavy applications (catalogs, reference data).
- Applications with repeated lookups of the same entities.

Avoid for highly volatile data (e.g., stock prices, rapidly changing orders).

220. What are Batch Fetching strategies in Hibernate?

Answer:

Batch fetching optimizes **lazy loading of collections and associations** by fetching them in groups instead of one query per entity.

Problem Example:

If you fetch 50 employees and each has a department, Hibernate may run 51 queries (N+1 problem).

Solution – Batch Fetching:

```
@Entity
public class Employee {
   @ManyToOne
   @BatchSize(size = 10)
   private Department department;
}
```

How it works:

- Instead of firing 1 query per department, Hibernate loads departments in batches of 10.
- So for 50 employees, only 5 queries will be run.

Global configuration option:

```
<property name="hibernate.default_batch_fetch_size" value="20"/>
```

Best for:

- Lazy-loaded associations accessed in bulk.
- E-commerce (loading products with categories).

Be careful: setting batch size too high may lead to large queries and memory pressure.

221. What is Multi-Tenancy in Hibernate, and what are its strategies?

Answer:

Multi-tenancy allows a single application instance to serve multiple tenants (clients/organizations), while keeping their data isolated. Hibernate supports multiple strategies:

1. DATABASE strategy

- Each tenant has its **own database**.
- Connection provider switches DBs based on tenant ID.
- Strongest isolation but highest cost.

2. SCHEMA strategy

- All tenants share one database but have **separate schemas**.
- Less overhead than multiple DBs, but still isolated.

3. DISCRIMINATOR strategy

- All tenants share the same DB and schema.
- A **tenant_id column** is added to each table.
- Cheapest but least secure (risk of data leaks).

Example Configuration (Schema-based):

```
SessionFactory factory = new Configuration()
   .setProperty("hibernate.multiTenancy", "SCHEMA")
   .setProperty("hibernate.tenant_identifier_resolver",
"com.example.MyTenantResolver")
   .buildSessionFactory();
```

Use Cases:

- SaaS apps (e.g., CRM, HR platforms).
- Discriminator for startups (low cost), Schema/DB for enterprises.

222. How does Hibernate Query Cache work, and how is it different from Second-Level Cache?

Answer:

Hibernate provides two caching layers:

1. **Second-Level Cache** – stores entities and collections.
    - Example: caching Employee entity by ID.
    - Works across sessions.
2. **Query Cache** – caches **result sets** of queries.
    - Example: caching SELECT e FROM Employee e WHERE dept='IT'.
    - Stores **IDs of entities**, not actual data. Entities are still retrieved from the second-level cache or DB.

Setup:

```
<property name="hibernate.cache.use_second_level_cache" value="true"/>
<property name="hibernate.cache.use_query_cache" value="true"/>
```

Code Usage:

```
Query query = session.createQuery("FROM Employee e WHERE e.department =
:dept");
query.setParameter("dept", "IT");
query.setCacheable(true);
List<Employee> employees = query.list();
```

Query Cache is useful for frequently repeated queries with the same parameters.
But it needs **second-level cache** for entities, otherwise Hibernate still hits the DB.

223. What are Natural IDs in Hibernate, and when should you use them?

Answer

A **Natural ID** is a **business key** that uniquely identifies an entity, instead of a surrogate key (like auto-increment ID).

- Example: Passport number, Email, Social Security Number.
- Hibernate provides APIs to fetch entities using natural IDs efficiently.

Example:

```
@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NaturalId
   @Column(unique = true, nullable = false)
   private String email;

   private String name;
}
```

Querying with Natural ID:

```
User user = session.byNaturalId(User.class)
            .using("email", "john@example.com")
            .load();
```

Advantages:

- Makes sense for **real-world unique identifiers**.
- Hibernate optimizes natural ID lookups with caching.

Limitations:

- Natural IDs must be **immutable** (e.g., email shouldn't change frequently).
- If they change, caching and consistency issues arise.

224. What are Interceptors in Hibernate, and how are they used?

Answer:

Hibernate **Interceptors** allow developers to hook into the lifecycle of entities and sessions without modifying business logic. They act as listeners for CRUD operations.

Examples of when to use Interceptors:

- Auditing (track createdBy, updatedBy).
- Logging SQL operations.
- Multi-tenancy checks.

Example – Audit Interceptor:

```
public class AuditInterceptor extends EmptyInterceptor {
   @Override
   public boolean onFlushDirty(Object entity, Serializable id,
                               Object[] currentState, Object[] previousState,
                               String[] propertyNames, Type[] types) {
      if (entity instanceof AuditableEntity) {
         for (int i = 0; i < propertyNames.length; i++) {
            if ("lastUpdatedAt".equals(propertyNames[i])) {
               currentState[i] = LocalDateTime.now();
               return true;
            }
         }
      }
      return false;
   }
}
```

Attaching Interceptor to Session:

```
Session session = sessionFactory
      .withOptions()
      .interceptor(new AuditInterceptor())
      .openSession();
```

Useful for centralized auditing without polluting entity classes.

225. What is StatelessSession in Hibernate, and when should you use it?

Answer:

A **StatelessSession** is a lightweight Hibernate session that:

- Does not maintain a **first-level cache**.
- Skips automatic dirty checking.
- Bypasses cascades and lifecycle events.

It directly maps objects to SQL, making it **very efficient** for bulk operations.

Example – Bulk Insert with StatelessSession:

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee emp = new Employee("Emp" + i, "IT");
   session.insert(emp);
}

tx.commit();
session.close();
```

When to use:

- Batch inserts/updates (e.g., importing millions of records).
- ETL jobs, data migration tools.
- Cases where you don't need entity lifecycle management.

Avoid for business transactions that need cascades, associations, or caching.

226. How can you optimize batch inserts and updates in Hibernate?

Answer:

Hibernate sends one SQL per entity by default, which is slow for bulk operations. Batch processing optimizes this.

Techniques:

1. **Set batch size in config:**

```
hibernate.jdbc.batch_size=30
hibernate.order_inserts=true
hibernate.order_updates=true
```

2. **Flush and clear session periodically:**

```
for (int i = 0; i < employees.size(); i++) {
  session.save(employees.get(i));
  if (i % 30 == 0) {
    session.flush();
    session.clear();
  }
}
```

3. **Use StatelessSession** for massive bulk ops (skips caching overhead).
4. **JDBC batch update with Spring Data JPA:**

```
@Modifying
@Query("UPDATE Employee e SET e.department = :dept WHERE e.id IN :ids")
void bulkUpdateDept(@Param("dept") String dept, @Param("ids") List<Long> ids);
```

Best Practices:

- Always set hibernate.jdbc.batch_size.
- Use batching + connection pool tuning.
- Disable auto-flush when doing huge inserts.

227. What are Event Listeners in Hibernate, and how do they differ from Interceptors?

Answer:

Hibernate **Event Listeners** are callback hooks tied to specific lifecycle events such as pre-insert, post-update, or load. They are more **fine-grained** than interceptors.

Difference from Interceptors:

- **Interceptors** → Broad, session-wide hooks (e.g., auditing, logging).
- **Event Listeners** → Targeted, entity-level lifecycle events.

Example – PreInsert Event Listener:

```
public class AuditEventListener implements PreInsertEventListener {
   @Override
   public boolean onPreInsert(PreInsertEvent event) {
      if (event.getEntity() instanceof AuditableEntity auditable) {
         auditable.setCreatedAt(LocalDateTime.now());
      }
      return false; // return true to veto insert
   }
}
```

Registering the Listener:

```
Configuration config = new Configuration();
config.getEventListeners().setPreInsertEventListeners(
      new PreInsertEventListener[]{ new AuditEventListener() });
```

Use case: Enforcing audit fields, validation, soft deletes.

228. What are Hibernate's second-level cache providers, and how do you choose one?

Answer:

The **second-level cache (L2 cache)** is shared across sessions. Hibernate doesn't implement caching itself but integrates with providers.

Common providers:

1. **Ehcache** – Simple, widely used, works for small/medium apps.
2. **Infinispan** – Clustered, distributed caching for large-scale apps.
3. **Hazelcast** – In-memory data grid, good for cloud/microservices.
4. **Redis** – Popular for distributed caching, fast key-value store.

Configuration Example (Ehcache):

hibernate.cache.use_second_level_cache=true

hibernate.cache.region.factory_class=org.hibernate.cache.ehcache.EhCacheRegionFactory

When to choose what:

- **Ehcache** → Standalone apps, small-scale.
- **Redis** → Cloud-native, microservices.
- **Infinispan/Hazelcast** → Enterprise, distributed clusters.

Use L2 cache only for **read-heavy, rarely-changing data** (product catalog, lookup tables). Avoid for transactional data (e.g., bank balances).

229. What are Fetch Joins in Hibernate, and how do they improve performance?

Answer:

A **fetch join** in JPQL allows fetching associated entities in a single query, avoiding the N+1 select problem.

Example – Without Fetch Join:

```
List<Department> depts = session.createQuery("FROM Department").list();
for (Department d : depts) {
   System.out.println(d.getEmployees()); // triggers N extra queries
}
```

With Fetch Join:

```
List<Department> depts = session.createQuery(
   "SELECT d FROM Department d JOIN FETCH d.employees"
).list();
```

Now Hibernate fetches departments and employees in **one query**.

Best Practices:

- Use fetch joins to avoid N+1 queries.
- But avoid fetching multiple large collections in a single query → can cause Cartesian explosion.
- Combine with pagination carefully.

230. What is the Criteria API in Hibernate, and when should it be used?

Answer:

The **Criteria API** is a programmatic way to build queries instead of writing HQL/JPQL strings. It is **type-safe** and useful when queries need to be built **dynamically at runtime**.

Example – Find employees by department:

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Employee> query = cb.createQuery(Employee.class);
Root<Employee> root = query.from(Employee.class);

query.select(root).where(cb.equal(root.get("department"), "IT"));

List<Employee> employees = session.createQuery(query).getResultList();
```

When to use:

- When query conditions depend on user input.
- When building reusable, modular query logic.
- Avoid for simple queries → JPQL is shorter and easier to read.

231. How does Hibernate support Native SQL queries, and what are the trade-offs?

Answer:
Hibernate lets you run **raw SQL** when JPQL/Criteria isn't enough (e.g., vendor-specific functions, complex joins).

Example – Native SQL Query:

```
List<Object[]> results = session.createNativeQuery(
   "SELECT id, name FROM employee WHERE department = :dept"
).setParameter("dept", "IT")
 .getResultList();
```

Mapping to Entity:

```
List<Employee> employees = session.createNativeQuery(
   "SELECT * FROM employee WHERE department = :dept", Employee.class
).setParameter("dept", "IT")
 .getResultList();
```

Trade-offs:

- ✅ Flexibility, full SQL power.
- ❌ Loses database independence (tied to vendor-specific SQL).
- ❌ Harder to maintain than JPQL/Criteria.

Best practice → Use JPQL/Criteria for portability, fall back to native SQL for performance tuning or DB-specific features.

232. What is Query Caching in Hibernate, and how does it work?

Answer:

Query caching in Hibernate stores the **result set of queries** so that repeated queries return data without hitting the database.

How it works:

- Hibernate maintains **first-level cache (per session)** and **second-level cache (shared)**.
- Query cache is **optional** and works alongside L2 cache.

Enable Query Cache:

hibernate.cache.use_query_cache=true
hibernate.cache.region.factory_class=org.hibernate.cache.ehcache.EhCacheRegionFactory

Using query cache:

```
List<Employee> employees = session.createQuery(
   "FROM Employee e WHERE e.department = :dept"
).setParameter("dept", "IT")
 .setCacheable(true)
 .getResultList();
```

Considerations:

- Query cache only caches IDs of results, not full entities.
- Entities must also be cached in the **second-level cache**.
- Best for **read-mostly queries** (like product catalog, reference data).

233. How can you enable and manage SQL logging in Hibernate?

Answer:

Hibernate allows developers to log all SQL queries generated, which is very useful for debugging and query optimization.

Enable SQL logging in application.properties or hibernate.cfg.xml:

```
# Show SQL queries
hibernate.show_sql=true

# Format SQL for readability
hibernate.format_sql=true

# Show parameter values
hibernate.type_contributors=org.hibernate.type.BasicType
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql=TRACE
```

Example log output:

```
Hibernate: select employee0_.id as id1_0_, employee0_.name as name2_0_ from
employee employee0_ where employee0_.department=?
TRACE: binding parameter [1] as [VARCHAR] - [IT]
```

Best Practices:

- Use only in **dev/test environments** (can slow performance in production).
- Combine with **profilers (VisualVM, JProfiler)** for performance tuning.
- For production, prefer **query statistics via Hibernate's metrics API**.

234. What is StatelessSession in Hibernate, and when should it be used?

Answer:

A **StatelessSession** is a lightweight alternative to the regular Hibernate Session.

Key Characteristics:

- No **first-level cache** (entities are not tracked).
- No **dirty checking** → Hibernate doesn't detect changes automatically.
- Suitable for **bulk operations** (massive inserts, updates, deletes).

Example:

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee e = new Employee("User" + i, "IT");
   session.insert(e);
}

tx.commit();
session.close();
```

When to use:

- Large **batch insert/update jobs** (ETL pipelines, data migration).
- Avoid in business apps where entity state tracking and caching are required.

235. What is Batch Fetching in Hibernate, and how does it improve performance?

Answer:

Batch Fetching reduces the **N+1 query problem** by fetching multiple related entities in **batches** instead of one by one.

Enable Batch Fetching:

```
@Entity
public class Department {
   @OneToMany(mappedBy = "department")
   @BatchSize(size = 10)   // Fetch 10 employees per batch
   private List<Employee> employees;
}
```

Hibernate configuration:

```
hibernate.default_batch_fetch_size=20
```

How it works:

- If you query 50 employees and each has a department, instead of 50 separate selects, Hibernate groups them into batches of 20.

Example SQL with batch size = 20:

```
SELECT * FROM department WHERE id IN (1,2,...,20);
```

Advantages:

- Reduces round trips to DB.
- Efficient for collections and associations.
- Especially useful when you know you'll load many child entities.

236. What is the Event System in Hibernate, and how is it used?

Answer:

Hibernate has an **event system** that lets developers intercept entity lifecycle events (insert, update, delete, load, flush). This is useful for **auditing, logging, or custom logic**.

Types of Events:

- **Pre/Post Insert** – Before or after an entity is inserted.
- **Pre/Post Update** – Before or after an update.
- **Pre/Post Delete** – Before or after deletion.
- **Pre/Post Load** – Before or after an entity is loaded.

Example – Audit entity before insert:

```
public class AuditListener {
   @PrePersist
   public void beforeInsert(Object entity) {
      if (entity instanceof Auditable) {
         ((Auditable) entity).setCreatedAt(LocalDateTime.now());
      }
   }

   @PreUpdate
   public void beforeUpdate(Object entity) {
      if (entity instanceof Auditable) {
         ((Auditable) entity).setUpdatedAt(LocalDateTime.now());
      }
   }
}
```

Usage:

```
@EntityListeners(AuditListener.class)
public class Employee implements Auditable {
   private LocalDateTime createdAt;
   private LocalDateTime updatedAt;
}
```

When to use:

- Auto-updating audit fields (createdAt, updatedAt).
- Enforcing business rules (e.g., prevent delete of active users).

- Logging DB changes.

237. What are Interceptors in Hibernate, and how do they differ from Listeners?

Answer:

Interceptors are a **programmatic way** to hook into Hibernate's lifecycle, while listeners are **annotation-driven**.

Key Points:

- Interceptors are global (applied to a session or session factory).
- Can modify SQL, log queries, or apply global rules.
- More flexible than listeners but require more code.

Example – Logging interceptor:

```
public class LoggingInterceptor extends EmptyInterceptor {
   @Override
   public String onPrepareStatement(String sql) {
      System.out.println("SQL Executed: " + sql);
      return super.onPrepareStatement(sql);
   }
}
```

Usage:

```
Session session = sessionFactory
      .withOptions()
      .interceptor(new LoggingInterceptor())
      .openSession();
```

Difference from Listeners:

- **Listeners** → entity-level, declarative (@PrePersist, @PostUpdate).
- **Interceptors** → session-level, programmatic, can modify SQL.

238. What is Multi-Tenancy in Hibernate, and what are its strategies?

Answer:

Multi-tenancy means supporting multiple clients (tenants) in the same application with **data isolation**. Hibernate supports this natively.

Strategies:

1. **Database per tenant**
    - Each tenant has its own DB.
    - Highest isolation, harder to manage.
2. **Schema per tenant**
    - Same DB, different schemas per tenant.
    - Balance between isolation and manageability.
3. **Discriminator-based (Table per tenant)**
    - Same table, but each row has a tenant_id column.
    - Easy to manage, but weaker isolation.

Example – Configuring multi-tenancy:

```
@Configuration
public class HibernateConfig {

  @Bean
  public LocalContainerEntityManagerFactoryBean entityManagerFactory() {
    Map<String, Object> props = new HashMap<>();
    props.put(Environment.MULTI_TENANT, MultiTenancyStrategy.SCHEMA);
    props.put(Environment.MULTI_TENANT_CONNECTION_PROVIDER,
connectionProvider());
    props.put(Environment.MULTI_TENANT_IDENTIFIER_RESOLVER,
tenantResolver());
    // ...
  }
}
```

TenantResolver example:

```
public class CustomTenantResolver implements CurrentTenantIdentifierResolver {
  @Override
  public String resolveCurrentTenantIdentifier() {
    return TenantContext.getCurrentTenant();
  }
```

```
    @Override
    public boolean validateExistingCurrentSessions() {
        return true;
    }
}
```

When to use:

- SaaS apps (multi-client architecture).
- Banking apps (separate tenants for each branch).
- E-commerce platforms with multiple vendors.

239. What is Hibernate Envers, and how is it used for auditing?

Answer:

Hibernate **Envers** is a module that provides **automatic auditing/versioning** of entity changes. It keeps historical records of entity states, enabling rollback, comparison, and audit trails.

How it works:

- When you annotate an entity with @Audited, Hibernate automatically creates audit tables (<entity>_AUD).
- Each insert/update/delete is logged with a revision number and timestamp.

Example – Auditing Employee entity:

```
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;
   private String name;
   private String department;
}
```

Audit Table (Employee_AUD):

```
id | name  | department | REV | REVTYPE
1  | John  | IT         | 101 | ADD
1  | John  | HR         | 102 | MOD
```

Querying audit history:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
List<Number> revisions = reader.getRevisions(Employee.class, 1);

for (Number rev : revisions) {
   Employee e = reader.find(Employee.class, 1, rev);
   System.out.println("Revision " + rev + ": " + e.getDepartment());
}
```

Use cases:

- Tracking changes for compliance (finance, healthcare).

- Restoring deleted/updated records.
- Auditing user activities.

240. How can you implement Soft Deletes in Hibernate?

Answer:

Soft deletes mean marking a record as deleted instead of physically removing it. Hibernate supports this via annotations.

Approaches:

1. **@SQLDelete + @Where**

```
@Entity
@SQLDelete(sql = "UPDATE employee SET deleted = true WHERE id = ?")
@Where(clause = "deleted = false")
public class Employee {
  @Id
  private int id;
  private String name;
  private String department;
  private boolean deleted = false;
}
```

- On delete(), Hibernate updates deleted=true.
- Queries automatically exclude soft-deleted records due to @Where.

2. **Filter-based**

```
@FilterDef(name="deletedFilter", defaultCondition="deleted=false")
@Filter(name="deletedFilter")
@Entity
public class Employee {
  @Id private int id;
  private String name;
  private boolean deleted;
}
Enable filter in session:
session.enableFilter("deletedFilter");
```

Benefits:

- Preserves history.
- Easy to restore records.

- Safer than hard deletes in critical systems.

241. What are Hibernate Filters, and when should they be used?

Answer:

Hibernate Filters allow you to **dynamically filter entity data** at runtime using conditions applied to queries.

Example – Tenant-specific filter:

```
@FilterDef(name="tenantFilter", parameters=@ParamDef(name="tenantId",
type="int"))
@Filter(name="tenantFilter", condition="tenant_id = :tenantId")
@Entity
public class Employee {
   @Id private int id;
   private String name;
   private int tenant_id;
}
```

Usage:

```
Session session = sessionFactory.openSession();
session.enableFilter("tenantFilter").setParameter("tenantId", 101);

List<Employee> employees = session.createQuery("from Employee").list();
```

When to use Filters:

- Multi-tenancy (filter by tenant_id).
- Soft deletes (deleted=false).
- Role-based data restrictions.
- Temporal filters (records valid within a date range).

242. What are Hibernate Statistics, and how can they be enabled?

Answer:

Hibernate provides a **Statistics API** to monitor session-level and global-level metrics (queries executed, cache hits, entity loads, etc.). This is crucial for **performance tuning**.

How to enable:

```
hibernate.generate_statistics=true
```

Usage:

```
SessionFactory sessionFactory = ...;
Statistics stats = sessionFactory.getStatistics();

System.out.println("Entity fetch count: " + stats.getEntityFetchCount());
System.out.println("Query execution count: " + stats.getQueryExecutionCount());
System.out.println("Second-level cache hits: " +
stats.getSecondLevelCacheHitCount());
```

Benefits:

- Identify N+1 select issues.
- Monitor cache effectiveness.
- Debug slow queries.

Use case: In a large e-commerce app, statistics can show whether product details are being repeatedly queried instead of coming from the cache.

243. What are some Hibernate Caching Best Practices?

Answer:

Hibernate caching improves performance but must be used wisely.

Best Practices:

1. **Enable First-Level Cache (default).**
    - Always present per Session.
    - No config needed.
2. **Use Second-Level Cache for Read-Mostly Data.**
    - Good for reference data (products, locations).
    - Avoid for rapidly changing transactional data.
3. **Choose the Right Cache Provider:**
    - Ehcache, Hazelcast, Infinispan, Redis.
4. **Use Query Cache selectively.**
    - Only enable for frequently reused queries.
5. **Invalidate/Update Cache properly.**
    - Use @Cache(usage = CacheConcurrencyStrategy.READ_WRITE) for consistent updates.
6. **Batch fetch + cache = win.**
    - Reduces queries and leverages cache efficiency.

Example:

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_ONLY)
public class Country {
   @Id
   private int id;
   private String name;
}
```

244. What is Batch Fetching in Hibernate, and how does it improve performance?

Answer:

Batch fetching reduces **multiple SELECTs** by fetching entities/collections in batches.

Example – N+1 problem:

Fetching 50 employees → Hibernate fires 1 query for employees + 50 queries for departments.

Batch fetching solution:

```
@Entity
public class Employee {
   @ManyToOne
   @BatchSize(size = 10)
   private Department department;
}
```

Now, Hibernate fetches departments in batches of 10 → total 6 queries instead of 51.

Global config:

hibernate.default_batch_fetch_size=20

When to use:

- Parent-child relationships (orders → items).
- Avoids N+1 problem.
- Works well with lazy loading.

245. What is Hibernate StatelessSession, and when should it be used?

Answer:

StatelessSession is a lightweight version of Session in Hibernate. Unlike the regular Session, it does **not maintain a first-level cache** or persistence context.

Key traits:

- No dirty checking.
- No first-level cache.
- No lifecycle callbacks (@PreUpdate, etc.).
- Direct JDBC-like behavior with ORM benefits.

Usage:

```
SessionFactory factory = ...;
StatelessSession stateless = factory.openStatelessSession();
Transaction tx = stateless.beginTransaction();

Employee emp = new Employee();
emp.setName("John");
stateless.insert(emp);  // behaves like JDBC insert

tx.commit();
stateless.close();
```

When to use:

- **Batch operations** (bulk inserts/updates/deletes).
- ETL jobs (migrating millions of rows).
- Scenarios where performance matters more than entity lifecycle.

Real-world example: Loading **10 million bank transactions** into a reporting system without needing persistence context overhead.

246. What are Fetch Joins in Hibernate, and why are they useful?

Answer:
Fetch join is a way to fetch an entity **along with its associations** in a single query. It is a solution to the **N+1 query problem**.

Example:

```
// Without fetch join (may trigger N+1 queries)
List<Employee> employees = em.createQuery("SELECT e FROM Employee e",
Employee.class).getResultList();

// With fetch join (single query)
List<Employee> employees = em.createQuery(
   "SELECT e FROM Employee e JOIN FETCH e.department",
Employee.class).getResultList();
```

Why useful:

- Reduces number of SQL queries.
- Efficient when related data is always needed.
- Works well with lazy relationships.

Caution:

- Too many fetch joins can create **Cartesian product explosions**.
- Fetch joins cannot be paginated properly in some databases.

Best practice: Use fetch joins only for relationships you always need in a given query (like Order → Items for order details page).

247. What are the Entity Lifecycle States in Hibernate?

Answer:
Hibernate manages entities in different **lifecycle states**:

1. **Transient:**
    - Entity created using new.
    - Not associated with a session.
    - No DB record.
    - Example: Employee e = new Employee();.
2. **Persistent:**
    - Associated with an open session + persistence context.
    - Tracked for changes → dirty checking.
    - Example: session.save(e);.
3. **Detached:**
    - Entity was persistent but session is closed.
    - Not tracked by Hibernate anymore.
    - Example: After session.close(), entity is detached.
4. **Removed:**
    - Entity is marked for deletion.
    - Will be deleted once transaction commits.
    - Example: session.delete(e);.

Lifecycle example:

```
Employee emp = new Employee("John"); // Transient
session.save(emp);              // Persistent
session.close();              // Detached
session.delete(emp);            // Removed (if reattached)
```

Analogy: Think of an Employee record like a job application:

- Draft (transient) → Submitted (persistent) → Archived (detached) → Shredded (removed).

248. What is Dirty Checking in Hibernate, and how does it work?

Answer:
Dirty checking is the process by which Hibernate automatically **detects changes** made to persistent entities and **updates the database** at flush/commit time.

How it works:

- Hibernate keeps a **snapshot** of entity state when it is first loaded.
- Before flushing, it compares current state with snapshot.
- If differences exist → Hibernate issues UPDATE SQL.

Example:

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Employee emp = session.get(Employee.class, 1); // Persistent
emp.setName("Updated Name");              // Change tracked

tx.commit();  // Hibernate automatically executes UPDATE
session.close();
```

Key points:

- No need to explicitly call session.update().
- Works only for **persistent** entities.
- Can be disabled with dynamic-update to reduce unnecessary SQL.

Interview tip: Mention that dirty checking **reduces boilerplate** but can create **extra queries** if not carefully used (e.g., updating unchanged fields).

249. What is Automatic Schema Generation in Hibernate, and what are its strategies?

Answer:
Hibernate can automatically create, validate, or update the database schema based on entity mappings. Controlled by the property:

hibernate.hbm2ddl.auto=update

Strategies:

1. **create** → Drops and creates schema every time app starts.
2. **create-drop** → Creates schema, drops it when session factory closes.
3. **update** → Updates schema without dropping data.
4. **validate** → Validates schema against mappings (throws error if mismatch).
5. **none** → Disables schema generation.

Example (application.properties in Spring Boot):

spring.jpa.hibernate.ddl-auto=update

Best practices:

- Use update or validate in development.
- Never use create or create-drop in production (risk of data loss).
- For production: manage schema via **Flyway** or **Liquibase** migrations.

250. What are Hibernate Proxies, and how do they support Lazy Loading?

Answer:
Hibernate uses **proxies** (subclass objects created with libraries like CGLIB or ByteBuddy) to implement **lazy loading**.

How it works:

- When a lazy association is fetched, Hibernate returns a **proxy** instead of the actual object.
- The proxy only loads data when a method is invoked.

Example:

```
Employee emp = session.get(Employee.class, 1);
Department dept = emp.getDepartment(); // dept is a proxy
System.out.println(dept.getName());    // Triggers SQL query
```

Advantages:

- Improves performance by delaying queries until data is needed.
- Reduces unnecessary DB calls.

Common pitfalls:

- **LazyInitializationException** → occurs if proxy is accessed outside of session/transaction.
- Solution:
    - Use JOIN FETCH in queries.
    - Use DTOs or entity graphs.
    - Keep transaction open until proxy initialization.

Analogy: A proxy is like a **Netflix movie thumbnail**. You only stream the movie when you actually click play.

251. What is Query Caching in Hibernate, and how does it work?

Answer:
Query caching in Hibernate allows caching the **results of queries** (not just entities) for reuse. It works on top of the **second-level cache**.

Steps to enable:

1. Enable second-level and query cache:

```
hibernate.cache.use_second_level_cache=true
hibernate.cache.use_query_cache=true
```

2. Mark query as cacheable:

```
List<Employee> employees = session.createQuery("FROM Employee e")
  .setCacheable(true)
  .list();
```

Key points:

- Entities must also be cacheable.
- Useful for **frequently executed, read-heavy queries**.
- Not suitable for highly dynamic or write-heavy data.

Example use case: Dashboard queries that fetch the same data repeatedly (e.g., “top-selling products”).

252. How can you execute Native SQL Queries in Hibernate?

Answer:
Sometimes, JPQL/Criteria is not enough. Hibernate allows **native SQL queries** for DB-specific optimizations.

Syntax:

```
List<Object[]> results = session.createNativeQuery(
   "SELECT id, name FROM employee WHERE department = :dept")
   .setParameter("dept", "IT")
   .list();
```

Entity mapping example:

```
List<Employee> employees = session.createNativeQuery(
   "SELECT * FROM employee WHERE department = :dept", Employee.class)
   .setParameter("dept", "IT")
   .list();
```

Best practices:

- Use native queries for **performance-critical** or **vendor-specific SQL**.
- Prefer JPQL for portability.
- Always parameterize queries to prevent **SQL injection**.

253. How do you integrate Hibernate with Spring Data JPA?

Answer:
Spring Data JPA internally uses Hibernate as the default JPA provider. Integration is seamless with just dependencies and configs.

Setup (Gradle):

```
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
runtimeOnly 'mysql:mysql-connector-java'
```

Entity example:

```
@Entity
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;
 private String name;
}
```

Repository:

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
   List<Employee> findByName(String name);
}
```

Service Layer:

```
@Service
public class EmployeeService {
 @Autowired
 private EmployeeRepository repo;

 public List<Employee> getByName(String name) {
  return repo.findByName(name);
 }
}
```

Advantages:

- No need to manage SessionFactory or EntityManager.
- CRUD, pagination, and sorting out of the box.
- Still allows custom JPQL, Criteria, or native queries if needed.

254. How does Hibernate handle connection pooling?

Answer:

Hibernate itself doesn't manage database connections directly; it relies on a **connection pool** for efficiency. Without pooling, each DB request would require opening/closing a new connection, which is expensive.

Options for connection pooling in Hibernate:

1. **Built-in (not recommended):** Hibernate had a simple internal pool, but it's not production-grade.
2. **Third-party pools (recommended):**
    - HikariCP (default in Spring Boot)
    - C3P0
    - DBCP2

Example with HikariCP (application.properties in Spring Boot):

```
spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=pass
spring.datasource.hikari.maximum-pool-size=10
```

How it works:

- When Hibernate needs a connection, it borrows from the pool.
- After use, it returns the connection to the pool instead of closing it.

Real-world impact:
Connection pooling greatly reduces latency in **high-traffic apps** like e-commerce checkouts or banking systems where every millisecond counts.

255. What is Multi-Tenancy in Hibernate, and how is it implemented?

Answer:

Multi-tenancy means supporting **multiple customers (tenants)** in a single application while keeping their data isolated. Hibernate provides multiple strategies:

1. **DATABASE strategy:** Each tenant has its own database.
2. **SCHEMA strategy:** Shared DB, separate schemas per tenant.
3. **DISCRIMINATOR strategy:** Shared DB + shared schema, rows distinguished by a tenant column.

Example configuration (Schema strategy):

```
hibernate.multiTenancy=SCHEMA
hibernate.tenant_identifier_resolver=myTenantResolver
```

Tenant Resolver Example:

```
public class MyTenantResolver implements CurrentTenantIdentifierResolver {
 @Override
 public String resolveCurrentTenantIdentifier() {
  return TenantContext.getCurrentTenant(); // custom logic
 }

 @Override
 public boolean validateExistingCurrentSessions() {
  return true;
 }
}
```

Use case: SaaS platforms (like CRM systems) where multiple clients use the same app but need strict data isolation.

256. What are Hibernate Interceptors, and when should they be used?

Answer:

Interceptors in Hibernate allow developers to **intercept entity lifecycle events** (save, update, delete, load) and inject custom logic.

Example: Logging whenever an entity is saved.

```
public class LoggingInterceptor extends EmptyInterceptor {
 @Override
 public boolean onSave(Object entity, Serializable id,
                Object[] state, String[] propertyNames, Type[] types) {
  System.out.println("Entity saved: " + entity.getClass().getSimpleName());
  return super.onSave(entity, id, state, propertyNames, types);
 }
}
```

Enabling Interceptor:

```
Session session = sessionFactory
            .withOptions()
            .interceptor(new LoggingInterceptor())
            .openSession();
```

When to use:

- Auditing changes (createdBy, modifiedBy).
- Logging or debugging entity operations.
- Implementing soft deletes (marking isDeleted=true instead of physically deleting).

Caution: Avoid heavy logic in interceptors as it may impact performance.

257. What are Hibernate Event Listeners, and how do they differ from Interceptors?

Answer:

Hibernate **Event Listeners** provide a more fine-grained way to hook into entity lifecycle events compared to interceptors.

Key difference:

- **Interceptors:** General-purpose, catch-all hooks (one class handles many events).
- **Event Listeners:** Target specific events (like save, update, delete) with dedicated listener interfaces.

Example – PreInsert listener:

```
public class AuditListener implements PreInsertEventListener {
 @Override
 public boolean onPreInsert(PreInsertEvent event) {
  Object entity = event.getEntity();
  if (entity instanceof Auditable) {
   ((Auditable) entity).setCreatedAt(LocalDateTime.now());
  }
  return false; // return true to veto insertion
 }
}
```

Registration in configuration:

```
configuration.getEventListeners()
        .setPreInsertEventListeners(new PreInsertEventListener[]{ new AuditListener()
});
```

When to use:

- Auditing (timestamps, createdBy, modifiedBy).
- Custom validation before insert/update.
- Business logic enforcement at persistence level.

258. What is the difference between Hibernate Session and JPA EntityManager?

Answer:
Both manage persistence contexts but differ in scope and API.

Feature	Hibernate Session	JPA EntityManager
Origin	Hibernate-specific	JPA standard (vendor-agnostic)
Features	Rich APIs (batch, scroll, filters)	Portable, standardized
Flush/Clear	Direct control with flush(), clear()	Available but less granular
Query API	HQL, Criteria, native SQL, filters	JPQL, Criteria, native SQL
Portability	Hibernate-only	Works with any JPA provider (EclipseLink, OpenJPA)

Example – Hibernate Session:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
Employee e = session.get(Employee.class, 1);
tx.commit();
session.close();
```

Example – JPA EntityManager:

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Employee e = em.find(Employee.class, 1);
em.getTransaction().commit();
em.close();
```

Best practice:

- Use **EntityManager** for standard, portable apps.
- Use **Session** when you need advanced Hibernate-specific features.

259. What is Hibernate Validator, and how is it used for Bean Validation?

Answer:

Hibernate Validator is the **reference implementation** of the Bean Validation API (Jakarta Validation, JSR 380). It ensures entities and DTOs meet constraints before persistence or API usage.

Common annotations:

- @NotNull, @Size, @Email, @Pattern, @Min, @Max, @Past, @Future.

Example – Entity validation:

```
@Entity
public class User {
 @Id
 @GeneratedValue
 private int id;

 @NotBlank(message = "Name is required")
 private String name;

 @Email(message = "Invalid email format")
 private String email;

 @Min(18)
 private int age;
}
```

Triggering validation in Spring Boot (auto-enabled):

```
@PostMapping("/users")
public ResponseEntity<User> createUser(@Valid @RequestBody User user) {
   return ResponseEntity.ok(user);
}
```

If validation fails: Spring automatically returns a **400 Bad Request** with validation error messages.

Use cases:

- Validate request DTOs in REST APIs.
- Enforce data integrity at persistence level.

- Custom validators for domain-specific rules (e.g., PAN number, employee code).

260. What are the different fetching strategies in Hibernate (Lazy, Eager, Subselect)?

Answer:

Fetching strategy decides **how and when associated entities are loaded**.

1. Lazy Fetching (default for collections)

- Data is fetched **only when accessed**.
- Uses proxies to defer DB calls.
- Prevents unnecessary queries.

```
@OneToMany(mappedBy = "employee", fetch = FetchType.LAZY)
private List<Project> projects;
```

Risk: LazyInitializationException if accessed outside transaction.

2. Eager Fetching

- Loads associated entities **immediately** with the parent.
- May cause **performance issues** (N+1 queries or big joins).

```
@OneToMany(mappedBy = "employee", fetch = FetchType.EAGER)
private List<Project> projects;
```

3. Subselect Fetching (Hibernate-specific)

- Loads child collections for **all parents in a single query** (instead of one query per parent).
- Reduces N+1 problem.

```
@OneToMany(mappedBy = "department")
@Fetch(FetchMode.SUBSELECT)
private List<Employee> employees;
```

Best practice:

- Default to **Lazy**.
- Use **Eager/Subselect** only when you know you always need related data.

261. What are Named Queries in Hibernate, and why use them?

Answer:

Named Queries are **predefined static queries** that are declared at entity level.

Advantages:

- Better readability and reuse.
- Precompiled at startup → fewer runtime errors.
- Centralized query management.

Example – JPQL Named Query:

```
@Entity
@NamedQuery(
   name = "Employee.findByDept",
   query = "SELECT e FROM Employee e WHERE e.department = :dept"
)
public class Employee { ... }
```

Usage:

```
List<Employee> emps = em.createNamedQuery("Employee.findByDept",
Employee.class)
                .setParameter("dept", "IT")
                .getResultList();
```

Native Named Query:

```
@NamedNativeQuery(
   name = "Employee.findAll",
   query = "SELECT * FROM employee",
   resultClass = Employee.class
)
```

262. What's the difference between Hibernate Criteria API, JPQL, and Native SQL?

Answer:

Feature	Criteria API	JPQL	Native SQL
Style	Type-safe, programmatic	String-based, object-oriented SQL	Raw SQL
Portability	Portable	Portable	DB-specific
Readability	Verbose	Concise	Familiar for SQL users
Use case	Dynamic queries (filters, search)	Static, common queries	Performance-tuned, vendor-specific

Example – JPQL:

```
@Query("SELECT e FROM Employee e WHERE e.name = :name")
List<Employee> findByName(@Param("name") String name);
```

Example – Criteria API:

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> root = cq.from(Employee.class);
cq.select(root).where(cb.equal(root.get("name"), "John"));
List<Employee> results = em.createQuery(cq).getResultList();
```

Example – Native SQL:

```
@Query(value = "SELECT * FROM employee WHERE name = ?1", nativeQuery = true)
List<Employee> findByNameNative(String name);
```

Rule of thumb:

- **JPQL** → for simple, readable queries.
- **Criteria API** → for **dynamic queries**.
- **Native SQL** → for **performance-critical DB-specific logic**.

263. How does Hibernate handle concurrency control (Optimistic vs Pessimistic Locking)?

Answer:

When multiple transactions update the same entity, Hibernate provides two strategies:

1. Optimistic Locking (default, lightweight)

- Uses a **version column** (@Version).
- Each update checks version → if mismatch → OptimisticLockException.
- Assumes conflicts are **rare**.

```
@Entity
public class Employee {
 @Id
 private int id;

 @Version
 private int version;  // auto-incremented
}
```

Flow:

- Tx A loads Employee(v1).
- Tx B also loads Employee(v1).
- Tx A commits → version=2.
- Tx B tries to commit → sees v1 but DB has v2 → fails.

Use case: web apps, microservices, distributed systems.

2. Pessimistic Locking (heavy, DB-level locks)

- Explicitly locks row in DB until transaction completes.
- Prevents other transactions from reading/updating.
- Uses LockModeType.PESSIMISTIC_WRITE.

```
Employee emp = em.find(Employee.class, 1,
LockModeType.PESSIMISTIC_WRITE);
```

Use case: banking systems, critical transactions.

Comparison:

- Optimistic → better performance, lower contention.
- Pessimistic → safer but can cause **deadlocks** and **blocking**.

264. What is the difference between Hibernate Interceptors and Event Listeners?

Answer:

Both allow custom logic during entity lifecycle, but they differ:

Interceptors

- Implement Interceptor interface.
- Provide **global hooks** for session-wide events (onSave, onDelete, onFlush).
- Best for cross-cutting concerns (audit logging, soft delete).

```
public class AuditInterceptor extends EmptyInterceptor {
 @Override
 public boolean onSave(Object entity, Serializable id, Object[] state, String[]
propertyNames, Type[] types) {
  System.out.println("Saving: " + entity);
  return true;
 }
}
```

Event Listeners

- More fine-grained, specific lifecycle events.
- Registered per-event type (e.g., @PrePersist, @PostUpdate).

```
@Entity
@EntityListeners(AuditListener.class)
public class Employee { ... }

public class AuditListener {
 @PrePersist
 public void beforeSave(Object entity) {
  System.out.println("Entity being saved: " + entity);
 }
}
```

Key difference:

- **Interceptor** → broad, session-wide.
- **Event Listener** → precise, entity lifecycle.

265. What are Hibernate Second-Level Cache providers, and how do they differ?

Answer:

Second-level cache stores entities/queries **beyond a session**, shared across transactions. Hibernate supports multiple providers:

1. Ehcache (most popular, simple)

- In-memory + disk persistence.
- Easy to configure.
- Good for small to medium apps.

```
<property name="hibernate.cache.region.factory_class">
 org.hibernate.cache.ehcache.EhCacheRegionFactory
</property>
```

2. Infinispan (clustered, enterprise-grade)

- Distributed, scalable cache.
- Supports replication across nodes.
- Best for microservices / cloud-native.

3. Redis (external in-memory cache)

- Fast, network-based cache.
- Shared across multiple apps.
- Useful when multiple services access same DB.

4. Hazelcast

- In-memory grid.
- Peer-to-peer, supports distributed computing.
- Good for real-time apps (IoT, fintech).

When to use what:

- **Ehcache** → standalone apps.
- **Redis/Infinispan** → microservices, distributed systems.
- **Hazelcast** → large-scale, compute + caching.

266. What is the difference between Batch Fetching and Fetch Joins in Hibernate?

Answer:

Both optimize queries but solve different problems:

Batch Fetching

- Hibernate groups multiple lazy fetches into batches.
- Instead of firing N queries, it fires in chunks.
- Configurable via @BatchSize or global property.

```
@Entity
public class Employee {
 @OneToMany(mappedBy = "employee", fetch = FetchType.LAZY)
 @BatchSize(size = 10)
 private List<Project> projects;
}
```

Flow:

- If 100 employees → Hibernate loads projects in **10 batches** instead of 100 queries.

Good for reducing N+1 queries when looping through collections.

Fetch Joins

- Uses JPQL JOIN FETCH to retrieve parent + child in a single query.

```
List<Employee> emps = em.createQuery(
 "SELECT e FROM Employee e JOIN FETCH e.projects",
Employee.class).getResultList();
```

Flow:

- Employee + Projects are fetched in **one query**.

Good when you always need child entities.

Key Difference:

- **Batch Fetching** → reduces query count by batching.
- **Fetch Joins** → forces eager fetch in a single query.
- Best practice → Use **fetch joins for small data**, **batch fetching for large data sets**.

267. What is the difference between Hibernate Query Cache and Second-Level Cache?

Answer:

Second-Level Cache

- Caches **entities & collections**.
- Example: Employee(id=1) stored in cache.
- Accessing the same entity again skips DB.

Query Cache

- Caches **query results** (list of IDs).
- Needs **Second-Level Cache enabled** to be effective.
- Example: FROM Employee WHERE dept='HR' → stores IDs [1,3,5]. Entities still fetched from L2 cache.

```
        Query query = session.createQuery("FROM Employee WHERE dept=:dept");
        query.setCacheable(true);
        query.setParameter("dept", "HR");
    List<Employee> result = query.list();
```

Comparison:

- **L2 Cache** → stores actual entity data.
- **Query Cache** → stores query results (depends on L2 cache to hydrate entities).

Use L2 cache for frequently accessed entities.
Use Query cache sparingly (cache invalidation can be tricky).

268. What are the pros and cons of using Native SQL in Hibernate?

Answer:
Hibernate supports writing **native SQL queries** for flexibility.

```
Query q = session.createNativeQuery("SELECT * FROM employee WHERE dept =
:dept", Employee.class);
q.setParameter("dept", "IT");
List<Employee> emps = q.getResultList();
```

Pros:

1. **Leverage DB features** → window functions, JSON, full-text search.
2. **Performance tuning** → optimized queries beyond JPQL.
3. **Legacy DB support** → when schema doesn't match entities.

Cons:

1. **DB-specific** → reduces portability.
2. **No automatic caching** → entities bypass some optimizations.
3. **Harder maintenance** → complex SQL inside code.

Best practice:

- Use **JPQL** for portability.
- Use **Native SQL** only for performance-critical or DB-specific queries.

269. What are Named Queries in Hibernate, and how do they differ from dynamic queries?

Answer:

Named Queries

- Predefined, reusable queries declared at the entity or XML level.
- Validated at startup (catch errors early).
- Stored in Hibernate's query cache.

```
@Entity
@NamedQueries({
 @NamedQuery(
  name = "Employee.findByDept",
  query = "SELECT e FROM Employee e WHERE e.department = :dept"
 )
})
public class Employee { ... }
```

Usage:

```
List<Employee> emps = em.createNamedQuery("Employee.findByDept",
Employee.class)
        .setParameter("dept", "HR")
        .getResultList();
```

Dynamic Queries

- Written at runtime with JPQL, Criteria API, or Native SQL.
- More flexible, but prone to runtime errors.

```
List<Employee> emps = em.createQuery(
 "SELECT e FROM Employee e WHERE e.department = :dept",
Employee.class)
 .setParameter("dept", "HR")
 .getResultList();
```

Key Differences:

- **Named Queries** → precompiled, reusable, better performance.
- **Dynamic Queries** → flexible, runtime creation, slower on first execution.

Best practice: Use Named Queries for frequent/standard queries; Dynamic for ad-hoc queries.

270. What is Hibernate StatelessSession, and when should it be used?

Answer:

- StatelessSession is a lightweight version of Session in Hibernate.
- It does **not** maintain a **first-level cache** or track entity states.
- Good for **bulk operations** where you don't need change tracking.

StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();

```
for (int i = 0; i < 1000; i++) {
  Employee emp = new Employee("Emp" + i, "IT");
  session.insert(emp);
}

tx.commit();
session.close();
```

When to Use:

- Large batch inserts/updates/deletes.
- Read-only operations where caching isn't required.
- ETL jobs, data migration, or analytics.

Limitations:

- No automatic dirty checking.
- No lifecycle callbacks (@PrePersist, @PostUpdate).
- No first-level cache → every query hits DB.

Best for performance-critical, stateless bulk jobs.

271. What are Hibernate Proxies, and what is LazyInitializationException?

Answer:

Hibernate Proxies

- Hibernate uses proxies (subclasses) to implement **lazy loading**.
- When you fetch an entity with lazy associations, Hibernate gives you a proxy object instead of the actual entity until accessed.

Employee emp = session.get(Employee.class, 1);
Department dept = emp.getDepartment(); // may be a proxy object

LazyInitializationException

- Occurs when a lazy-loaded proxy is accessed **outside an active session/transaction**.

Example:

```
Session session = factory.openSession();
Employee emp = session.get(Employee.class, 1);
session.close();
```

System.out.println(emp.getDepartment().getName()); // ❌ LazyInitializationException

How to Avoid:

1. **Open Session in View (OSIV)** → keep session open for web request (not always recommended).
2. **JOIN FETCH** → load associations eagerly in query.
3. **DTO Pattern** → fetch required fields in custom queries.
4. **Initialize Before Closing Session** → use Hibernate.initialize() or access inside transaction.

Best practice: Avoid OSIV in large apps. Prefer DTO-based fetching or JOIN FETCH.

272. What are Hibernate Event Listeners and Interceptors, and how do they differ?

Answer:

Hibernate allows you to hook into its lifecycle using **Event Listeners** and **Interceptors**.

Event Listeners

- Fine-grained hooks into **entity lifecycle events**.
- Examples: pre-insert, post-update, pre-delete, load.
- Declared programmatically or via annotations.

```
public class AuditEventListener implements PreInsertEventListener {
  @Override
  public boolean onPreInsert(PreInsertEvent event) {
    Object entity = event.getEntity();
    if (entity instanceof AuditableEntity ae) {
      ae.setCreatedAt(LocalDateTime.now());
    }
    return false; // continue insert
  }
}
```

Register:

```
configuration.getEventListeners().setPreInsertEventListeners(
  new PreInsertEventListener[] { new AuditEventListener() });
```

Interceptors

- Work at a **session level**.
- Provide **generic callbacks** (not tied to specific entity events).
- Example: logging SQL, tracking dirty entities.

```
public class LoggingInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id,
                        Object[] state, String[] propertyNames, Type[] types) {
    System.out.println("Saving entity: " + entity.getClass().getName());
    return false;
  }
}
```

Key Differences:

- **Event Listeners** → entity-specific, lifecycle hooks.
- **Interceptors** → session-wide, broader control.

Use **listeners** for domain-driven auditing, and **interceptors** for session-wide behaviors like logging or multi-tenant support.

273. What are Multi-Tenancy Strategies in Hibernate?

Answer:

Multi-tenancy means supporting multiple clients (tenants) with isolation in the same application. Hibernate supports:

1. **DATABASE Strategy** → Each tenant has a separate database.
    - Pros: Strongest isolation.
    - Cons: Expensive to maintain many DBs.
2. **SCHEMA Strategy** → Same database, different schemas per tenant.
    - Pros: Isolation + less overhead than separate DBs.
    - Cons: Schema management is tricky.
3. **DISCRIMINATOR (Shared Schema)** → Same table, a tenant_id column distinguishes tenants.
    - Pros: Simple, fewer resources.
    - Cons: Weakest isolation, needs strong row-level security.

Example – Shared Schema with Tenant Identifier:

```
public class TenantIdentifierResolver implements CurrentTenantIdentifierResolver {
  @Override
  public String resolveCurrentTenantIdentifier() {
    return TenantContext.getCurrentTenant(); // custom thread-local
  }

  @Override
  public boolean validateExistingCurrentSessions() {
    return true;
  }
}

Configure in Hibernate:

hibernate.multiTenancy=SCHEMA
hibernate.tenant_identifier_resolver=TenantIdentifierResolver
```

Best practice:

- **Small SaaS startups** → discriminator strategy.
- **Enterprise SaaS** → schema or DB strategy (better isolation).

274. How does Hibernate handle Batch Processing with StatelessSession vs Session?

Answer:

Session (Stateful)

- Maintains **first-level cache**.
- Tracks entities → automatic dirty checking.
- Needs explicit flush() & clear() in batches to avoid memory leaks.

```
for (int i = 0; i < 1000; i++) {
   session.save(new Employee("Emp" + i));
   if (i % 50 == 0) {
      session.flush();
      session.clear();
   }
}
```

StatelessSession

- No first-level cache, no dirty checking.
- Each operation directly hits DB.
- Faster for **pure insert/update** workloads.

```
StatelessSession statelessSession = sessionFactory.openStatelessSession();
Transaction tx = statelessSession.beginTransaction();

for (int i = 0; i < 1000; i++) {
   statelessSession.insert(new Employee("Emp" + i));
}

tx.commit();
statelessSession.close();
```

Key Differences:

- **Session** → use when you need entity lifecycle, caching, or associations.
- **StatelessSession** → use for **ETL, data migration, or bulk jobs** with massive inserts/updates.

Rule of Thumb:

- Regular apps → Session with batch settings.
- High-performance bulk ops → StatelessSession.

275. What are Hibernate Fetch Plans, and how do they help with performance optimization?

Answer:

A **fetch plan** defines how Hibernate retrieves associated entities. Without a plan, you either over-fetch (performance hit) or under-fetch (lazy loading exceptions).

Fetch Types in Hibernate

1. **Lazy (default for collections)** → fetch data only when accessed.
2. **Eager** → fetch data immediately along with parent.

Optimizing with Fetch Plans

1. **JOIN FETCH (JPQL / HQL)** – single query for parent + child.

```
String hql = "SELECT d FROM Department d JOIN FETCH d.employees";
List<Department> result = session.createQuery(hql, Department.class).list();
```

2. **Entity Graphs (JPA way)** – define fetch plan dynamically.

```
EntityGraph<Employee> graph = em.createEntityGraph(Employee.class);
graph.addAttributeNodes("department");
Map<String, Object> hints = Map.of("jakarta.persistence.fetchgraph", graph);
Employee emp = em.find(Employee.class, 1, hints);
```

3. **Batch Fetching** – reduces N+1 queries by fetching associations in batches.

```
@OneToMany(mappedBy = "department")
@BatchSize(size = 20)
private List<Employee> employees;
```

4. **Subselect Fetching** – fetch children in one extra query for all parents.

Best Practice:

- Default → Lazy loading.

- For dashboards/lists → JOIN FETCH.
- For profile/details → Entity Graphs.
- For bulk → Batch or Subselect.

This tuning avoids the **N+1 query problem** and reduces DB round-trips.

276. What Second-Level Cache Providers can be used in Hibernate, and how do they differ?

Answer:

Second-level cache (L2 cache) stores entities/collections across sessions to reduce DB queries. Hibernate supports multiple providers:

1. **Ehcache**
    - Oldest, widely used.
    - Supports in-memory + disk storage.
    - Best for monolithic apps.
2. **Infinispan**
    - Distributed cache.
    - Suitable for clustered environments.
3. **Redis** (via Hibernate-Redis)
    - External, in-memory store.
    - Great for microservices & cloud-native apps.
4. **Hazelcast**
    - Distributed, supports near caching + cluster discovery.
    - Popular in financial services.

Example Config – Ehcache

```
hibernate.cache.use_second_level_cache=true
hibernate.cache.region.factory_class=org.hibernate.cache.ehcache.EhCacheRegionFactory
```

When to Use

- **Ehcache** → small-medium apps.
- **Redis/Hazelcast** → scalable microservices.
- **Infinispan** → large enterprise clusters.

Rule: Cache **reference data** (products, categories, configs), not frequently changing transactional data.

277. How does Hibernate integrate with Spring Data JPA?

Answer:

Spring Data JPA **sits on top of Hibernate** and hides most boilerplate.

Integration Flow

- Entities → still mapped with Hibernate annotations (@Entity).
- Hibernate → acts as JPA provider under the hood.
- Spring Data → exposes repositories (JpaRepository) for CRUD.

Example

```
@Entity
public class Employee {
   @Id
   @GeneratedValue
   private Long id;
   private String name;
}

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
   List<Employee> findByName(String name);
}
```

Spring Data JPA internally uses **EntityManager** (Hibernate session underneath) to execute queries.

Benefits of Integration

- No manual SessionFactory or Transaction management.
- Query derivation → auto-generates queries by method names.
- Still allows custom Hibernate features (@BatchSize, native queries).
- Integrates with Spring Boot seamlessly.

In practice:

- Use **Spring Data JPA** for productivity.
- Use direct Hibernate APIs only when you need advanced features (e.g., StatelessSession, native cache tuning).

278. What are Hibernate Performance Tuning Best Practices?

Answer:

In real-world apps, Hibernate performance issues usually come from excessive queries, poor caching, or unoptimized mappings.

Best Practices

1. **Always prefer Lazy Loading by default**
    - Fetch associations only when needed.
    - Override with JOIN FETCH or EntityGraph for specific use cases.
2. **Avoid N+1 queries**
    - Use JOIN FETCH, @BatchSize, or subselect fetching.
3. **Use DTO Projections for Read-heavy APIs**
    - Fetch only required fields instead of entire entities.
4. @Query("SELECT new com.app.dto.EmployeeDTO(e.id, e.name) FROM Employee e")
5. List<EmployeeDTO> findAllEmployeeNames();
6. **Enable Second-Level Cache carefully**
    - Cache reference data (products, locations), not transactional data.
7. **Batch Inserts/Updates**
    - Set hibernate.jdbc.batch_size=20.
    - Flush & clear session in loops to avoid memory bloat.
8. **Indexing**
    - Ensure proper DB indexes on join columns and frequently queried fields.
9. **Use StatelessSession for Bulk Operations**
    - Skips caching overhead for massive imports/updates.

Rule of thumb: Profile queries with Hibernate's show_sql or tools like p6spy + analyze slow queries in DB.

279. How is Hibernate used in Microservices, and what challenges arise?

Answer:

Hibernate is common in monoliths, but in **microservices** architecture, each service usually owns its own database.

Integration in Microservices

- Each service has its **own Hibernate + JPA setup**.
- Transactions are **local**, not distributed.
- For inter-service communication → use REST, gRPC, or messaging (Kafka).

Challenges

1. **Distributed Transactions (2PC issue)**
    - Microservices avoid XA transactions.
    - Solution → Saga pattern or Outbox pattern.
2. **Schema Evolution**
    - Each service evolves its schema independently.
    - Use Flyway/Liquibase with Hibernate to manage DB migrations.
3. **Caching Consistency**
    - L2 caches don't sync across services.
    - Solution → use distributed caches (Redis/Hazelcast).
4. **N+1 Queries in APIs**
    - Can cause performance bottlenecks in high-traffic microservices.
    - Solution → optimize with projections, batch fetching.

In practice: Hibernate works fine inside each microservice, but **don't share databases across services**. Stick to "Database per Service" principle.

280. What is Hibernate Reactive, and how does it differ from classic Hibernate?

Answer:

Hibernate Reactive is a non-blocking, asynchronous ORM built on top of **Vert.x and R2DBC**. It's designed for reactive applications.

Difference from Classic Hibernate

1. **Classic Hibernate**
    - JDBC-based → blocking I/O.
    - Good for synchronous, thread-per-request apps.
2. **Hibernate Reactive**
    - Uses **R2DBC (Reactive Relational Database Connectivity)**.
    - Non-blocking I/O, event-driven.
    - Works with reactive stacks (Spring WebFlux, Quarkus).

Example (Reactive Hibernate with Mutiny API – Quarkus style)

```
sessionFactory.withSession(session ->
   session.find(Employee.class, 1L)
        .onItem().invoke(emp -> System.out.println(emp.getName()))
);

Instead of returning Employee, it returns a reactive type (Uni<Employee> or
CompletionStage<Employee>).
```

Use Cases

- High-concurrency apps (chat systems, trading platforms, IoT).
- APIs where scalability > blocking simplicity.

Classic Hibernate remains default for most enterprise apps, but **Hibernate Reactive** is growing in event-driven, microservice-heavy environments.

281. What is Hibernate Envers, and how is it used for Auditing?

Answer:

Hibernate Envers is a module that provides **auditing/versioning** of entity changes. It automatically records history whenever an entity is **inserted, updated, or deleted**.

How It Works

- Envers creates an **audit table** for each audited entity.
- Example: for Employee, an Employee_AUD table is created.
- Every change is logged as a new revision with metadata (timestamp, revision id, user).

Example Setup

```
<dependency>
 <groupId>org.hibernate</groupId>
 <artifactId>hibernate-envers</artifactId>
</dependency>
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue
   private Long id;
   private String name;
   private String department;
}
```

Now, Hibernate automatically creates Employee_AUD.

Fetching Audit Data

```
AuditReader reader = AuditReaderFactory.get(entityManager);
Employee oldEmp = reader.find(Employee.class, 1L, 2);
// entity at revision 2
```

Use Cases

- Compliance (banking, healthcare).
- Debugging production issues.
- Tracking who changed what and when.

282. What are Hibernate Interceptors and Event Listeners?

Answer:

Hibernate provides hooks to intercept or listen to entity lifecycle events.

Interceptors

- Implement Interceptor or extend EmptyInterceptor.
- Methods like onSave, onDelete, onFlushDirty.
- Used for **auditing, logging, soft deletes, multi-tenancy**.

```
public class AuditInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id, Object[] state,
                        String[] propertyNames, Type[] types) {
    System.out.println("Saving entity: " + entity);
    return false;
}
  }
```

Event Listeners

- More fine-grained control than interceptors.
- Register listeners for specific events (PreInsertEventListener, PostLoadEventListener, etc.).

```
public class EmployeeInsertListener implements PreInsertEventListener {
  @Override
  public boolean onPreInsert(PreInsertEvent event) {
    System.out.println("Before insert: " + event.getEntity());
    return false;
  }
}
```

Difference

- **Interceptor**: one per SessionFactory, broad and centralized.
- **Event Listener**: specific to certain events, more fine-tuned.

283. How can Hibernate integrate with Legacy Databases?

Answer:

Legacy databases often don't follow modern naming conventions or normalized design. Hibernate can still work with them using mappings and customization.

Challenges

1. **Non-standard table/column names**
    - Example: EMP_TBL instead of employee.
    - Fix with @Table(name="EMP_TBL"), @Column(name="EMP_NM").
2. **No primary keys**
    - Hibernate requires identifiers.
    - Use **composite keys** (@EmbeddedId, @IdClass) or surrogate keys.
3. **Stored Procedures**
    - Legacy systems may rely heavily on procedures.
    - Use @NamedStoredProcedureQuery.
4. **Denormalized structures**
    - Sometimes a single table stores multiple entity types.
    - Use inheritance strategies (SINGLE_TABLE).

Example Mapping for Legacy Table

```
@Entity
@Table(name = "EMP_TBL")
public class Employee {
   @Id
   @Column(name = "EMP_ID")
   private Long id;

   @Column(name = "EMP_NM")
   private String name;
}
```

Best Practices

- Use **reverse engineering tools** (Hibernate Tools, JHipster) to generate mappings.
- Start with **read-only** entities if schema evolution is risky.
- Gradually refactor by introducing DTOs and services.

284. How can Hibernate be used with Database Migration Tools like Flyway or Liquibase?

Answer:

Hibernate manages ORM mappings, but **schema evolution** in production requires versioned migrations. Tools like **Flyway** and **Liquibase** complement Hibernate.

Why Needed

- Hibernate's hbm2ddl.auto=update works only for dev/test.
- In production, DB changes must be **tracked, versioned, and reversible**.

Flyway Example

- Place SQL files in db/migration:

```
V1__create_employee_table.sql
V2__add_salary_column.sql
```

- Config in application.properties:

```
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
```

When the app starts, Flyway runs pending migrations **before Hibernate initializes**.

Liquibase Example

- XML or YAML-based migrations:

```
<changeSet id="1" author="rajat">
  <createTable tableName="employee">
    <column name="id" type="BIGINT" autoIncrement="true"
primaryKey="true"/>
    <column name="name" type="VARCHAR(100)"/>
  </createTable>
</changeSet>
```

Liquibase integrates with Spring Boot via spring-boot-starter-liquibase.

Best Practice

- Use Hibernate for **runtime ORM**.
- Use Flyway/Liquibase for **schema management and evolution**.

285. How does Hibernate support Multi-Tenancy?

Answer:

Multi-tenancy means one application serving multiple customers (tenants), each with isolated data. Hibernate supports multiple strategies:

1. **DATABASE** – separate DB per tenant.
2. **SCHEMA** – same DB, different schemas.
3. **DISCRIMINATOR** – single DB + schema, tenant identified by column.

Example (Schema-based multi-tenancy)

```
public class SchemaMultiTenantConnectionProvider implements
MultiTenantConnectionProvider {
  @Override
  public Connection getConnection(String tenantIdentifier) throws SQLException {
    Connection connection = dataSource.getConnection();
    connection.createStatement().execute("SET SCHEMA '" + tenantIdentifier + "'");
    return connection;
  }
}
```

Use Cases

- SaaS products (e.g., cab booking system where each city is a tenant).
- Banking (each branch as tenant).

Best Practices

- **Database** isolation → strongest but expensive.
- **Schema** → balanced.
- **Discriminator** → cheapest but least secure.

286. How can you implement Soft Deletes in Hibernate?

Answer:

Soft delete means **marking a record as deleted** instead of actually removing it. Useful for audit/history.

Using @SQLDelete and @Where

```
@Entity
@SQLDelete(sql = "UPDATE employee SET deleted = true WHERE id=?")
@Where(clause = "deleted=false")
public class Employee {
   @Id
   private Long id;

   private String name;

   private boolean deleted = false;
}
```

- On delete(), Hibernate runs UPDATE employee SET deleted=true.
- With @Where, all queries automatically exclude soft-deleted rows.

Advantages

- Prevents accidental data loss.
- Allows restoring records.

Caveats

- Constraints (unique indexes) may still include deleted records.
- Must handle reporting queries carefully.

287. How can Hibernate interact with Stored Procedures?

Answer:

Hibernate supports **stored procedure calls** using both **JPA's @NamedStoredProcedureQuery** and native SQL.

Example with @NamedStoredProcedureQuery

```
@Entity
@NamedStoredProcedureQuery(
   name = "getEmployeesByDept",
   procedureName = "get_employees_by_dept",
   parameters = {
      @StoredProcedureParameter(mode = ParameterMode.IN, name = "dept", type =
String.class),
      @StoredProcedureParameter(mode = ParameterMode.REF_CURSOR, name =
"emp_cursor", type = void.class)
   }
)
public class Employee { ... }
```

Usage:

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("getEmployeesByDept");
query.setParameter("dept", "IT");
List<Employee> employees = query.getResultList();
```

Native SQL Example

```
Session session = entityManager.unwrap(Session.class);
ProcedureCall call = session.createStoredProcedureCall("get_employees_by_dept",
Employee.class);
call.registerParameter("dept", String.class, ParameterMode.IN).bindValue("HR");
List<Employee> result = call.getResultList();
```

Use Cases:

- Complex DB logic (payroll, financial transactions).
- Performance optimizations where DB-side execution is faster.

288. How can Hibernate map DTO projections instead of Entities?

Answer:

Sometimes you don't need full entities-just specific fields for APIs or reports. Hibernate supports **DTO projections** using JPQL or Criteria.

JPQL Constructor Expression

```
public class EmployeeDTO {
   private String name;
   private String dept;
   public EmployeeDTO(String name, String dept) {
      this.name = name; this.dept = dept;
   }
}
```

Query:

```
List<EmployeeDTO> result = em.createQuery(
   "SELECT new com.example.EmployeeDTO(e.name, e.department) FROM
Employee e",
   EmployeeDTO.class
).getResultList();
```

Native Query + ResultTransformer (Hibernate-specific)

```
List<EmployeeDTO> dtos = session.createNativeQuery(
   "SELECT name, department FROM employee"
)
.addScalar("name", StringType.INSTANCE)
.addScalar("department", StringType.INSTANCE)
.setResultTransformer(Transformers.aliasToBean(EmployeeDTO.class))
.list();
```

Advantages:

- Avoids unnecessary data loading.
- Reduces memory usage.
- Prevents LazyInitializationException when only limited fields are needed.

289. What is the difference between Batch Fetching and JOIN FETCH in Hibernate?

Answer:

Both are strategies to solve the **N+1 Select problem**, but they work differently.

JOIN FETCH

- Loads parent + children in a single SQL using a join.

```
List<Department> depts = em.createQuery(
  "SELECT d FROM Department d JOIN FETCH d.employees",
Department.class
).getResultList();
```

- Good when you **always need the children**.
- But can cause **Cartesian product explosion** if multiple collections are joined.

Batch Fetching (@BatchSize)

- Hibernate loads children in batches (not one-by-one).

```
@Entity
public class Department {
  @OneToMany(mappedBy = "department")
  @BatchSize(size = 10)
  private List<Employee> employees;
}
```

If 50 departments are loaded with size=10, Hibernate fires **5 queries** instead of 51.

Comparison

Strategy	Pros	Cons
JOIN FETCH	Single query, simpler	Risk of huge joins, duplicate parent rows
Batch Fetching	Flexible, reduces N+1	Still multiple queries, tuning required

Rule of Thumb:

- Use JOIN FETCH when data is always needed.
- Use @BatchSize for large datasets with optional child loading.

290. What are Natural IDs in Hibernate, and how are they used?

Answer:

A **Natural ID** is a domain-specific identifier that uniquely identifies an entity (like email, SSN, username) instead of the surrogate primary key (id). Hibernate provides APIs to query by natural IDs efficiently.

Example:

```
@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NaturalId
   @Column(unique = true, nullable = false)
   private String email;
}
```

Fetching by Natural ID

```
Session session = sessionFactory.openSession();
User user = session.byNaturalId(User.class)
            .using("email", "john@example.com")
            .load();
```

Advantages:

- Faster lookups (Hibernate can cache natural IDs separately).
- Business-friendly keys (like email) rather than surrogate IDs.

291. What is Hibernate Envers, and how does it handle Auditing?

Answer:

Hibernate Envers is a module that provides **auditing/versioning** of entities. It automatically tracks changes and stores historical versions of data.

Setup:

```
<dependency>
 <groupId>org.hibernate</groupId>
 <artifactId>hibernate-envers</artifactId>
</dependency>
```

Annotating an Entity:

```
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String name;
   private String department;
}
```

Envers automatically creates an **audit table** (employee_AUD) with revision history.

Fetching Historical Versions:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
Employee oldVersion = reader.find(Employee.class, 1L, 2); // revision 2
```

Use Cases:

- Regulatory compliance (banking, healthcare).
- Debugging historical changes.
- Rollback to previous states.

292. What are different Caching Strategies in Hibernate's Second-Level Cache?

Answer:

Hibernate's **Second-Level Cache (2LC)** reduces database calls by caching entities, collections, and queries. Different **cache concurrency strategies** exist depending on consistency needs.

Common Strategies:

1. **READ_ONLY**
    - For static reference data (e.g., country list).
    - No updates allowed.
2. **NONSTRICT_READ_WRITE**
    - Allows occasional stale reads.
    - Good for rarely-updated data (e.g., product catalog).
3. **READ_WRITE**
    - Guarantees strong consistency using **soft locks**.
    - Slightly slower, but safer for frequently updated entities.
4. **TRANSACTIONAL**
    - Requires JTA & cache providers supporting XA transactions.
    - Ensures full ACID compliance.

Example Config (EhCache):

```
<property name="hibernate.cache.use_second_level_cache" value="true"/>
<property name="hibernate.cache.region.factory_class"
      value="org.hibernate.cache.ehcache.EhCacheRegionFactory"/>

Entity-level:

@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product { ... }
```

Interview Tip:

- Always mention: *first-level cache = mandatory, per session*; *second-level cache = optional, shared across sessions*.

293. What is Multi-Tenancy in Hibernate, and what are its strategies?

Answer:

Multi-Tenancy means running a single application for multiple tenants (customers), while isolating their data. Hibernate provides multiple strategies:

Strategies:

1. **DATABASE** → Each tenant has its own database.
    - Strong isolation, but resource-heavy.
2. **SCHEMA** → All tenants share a database, but each has its own schema.
    - Balanced approach.
3. **DISCRIMINATOR** → A single schema, with a tenant_id column to differentiate rows.
    - Lightweight, but weakest isolation.

Example Config (Schema strategy):

```
hibernate.multiTenancy=SCHEMA
hibernate.tenant_identifier_resolver=myTenantResolver
hibernate.multi_tenant_connection_provider=myConnectionProvider
```

Tenant Resolver:

```
public class TenantResolver implements CurrentTenantIdentifierResolver {
  @Override
  public String resolveCurrentTenantIdentifier() {
    return TenantContext.getCurrentTenant(); // e.g., from request
  }
  @Override
  public boolean validateExistingCurrentSessions() {
    return true;
  }
}
```

Real-world Use Case: SaaS platforms like multi-organization ERP/CRM systems.

294. What are Interceptors and Event Listeners in Hibernate?

Answer:

Hibernate allows **custom hooks** into its lifecycle events:

Interceptors

- Work at a **session-wide level**.
- Override methods like onSave, onFlushDirty, onDelete.

```
public class AuditInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id,
                Object[] state, String[] propertyNames, Type[] types) {
    System.out.println("Entity saved: " + entity);
    return true;
  }
}
```

Attach:

```
Session session = sessionFactory
    .withOptions()
    .interceptor(new AuditInterceptor())
    .openSession();
```

Event Listeners

- More fine-grained, tied to specific Hibernate events.
- Example: PreInsertEventListener, PostUpdateEventListener.

```
public class MyInsertListener implements PreInsertEventListener {
  @Override
  public boolean onPreInsert(PreInsertEvent event) {
    System.out.println("Before insert: " + event.getEntity());
    return false;
  }
}
```

When to use?

- Auditing
- Soft deletes

- Enforcing business rules at persistence layer

295. What is the difference between using Hibernate with JPA vs Native Hibernate APIs?

Answer:

Hibernate can be used in two modes:

JPA (Java Persistence API)

- Standardized API.
- Portable across JPA providers (Hibernate, EclipseLink, OpenJPA).
- Uses EntityManager & JPA annotations.
- Example:

```
@PersistenceContext
private EntityManager em;

public Employee find(int id) {
   return em.find(Employee.class, id);
}
```

Native Hibernate API

- Provides advanced features not in JPA.
- Uses Session, Criteria, Interceptor, StatelessSession, etc.
- Example:

```
Session session = sessionFactory.openSession();
Employee emp = session.get(Employee.class, 1);
```

Key Differences

Feature	JPA	Hibernate Native
Portability	High	Low (vendor-specific)
API	EntityManager	Session, Criteria
Features	Standardized	Extended (e.g., NaturalId, FetchProfiles)
When to Use	Multi-vendor, portability	Need Hibernate-specific power

Best Practice:

- Use **JPA** for general apps (portable, standard).
- Use **Hibernate-native** when you need extra features.

296. What is StatelessSession in Hibernate, and when should you use it?

Answer:

A **StatelessSession** is a lightweight version of a Hibernate Session that does not maintain a first-level cache or track entity states.

Characteristics:

- No automatic dirty checking.
- No caching (neither first-level nor second-level).
- Entities are not associated with persistence context.
- Directly issues SQL to DB.

Example:

```
StatelessSession statelessSession = sessionFactory.openStatelessSession();
Transaction tx = statelessSession.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee emp = new Employee("Emp-" + i, "IT");
   statelessSession.insert(emp);
}

tx.commit();
statelessSession.close();
```

Use Cases:

- Bulk inserts, updates, or deletes.
- ETL (extract-transform-load) jobs.
- Data migration scripts.

Analogy: Normal Session is like a smart assistant keeping notes of everything you do. StatelessSession is like a courier-it just delivers stuff without remembering.

297. What are Query Hints in Hibernate, and how are they used?

Answer:

Query hints are **directives given to the query engine** to modify how queries are executed. They are often vendor-specific but Hibernate allows them via JPQL or native queries.

Example with JPA-style hints:

```
TypedQuery<Employee> query =
   em.createQuery("SELECT e FROM Employee e", Employee.class);

query.setHint("org.hibernate.readOnly", true);  // Read-only optimization
query.setHint("javax.persistence.query.timeout", 5000); // Timeout in ms
```

Common Hibernate Query Hints:

- org.hibernate.cacheable → Enable query cache
- org.hibernate.readOnly → Entities won't be tracked/updated
- javax.persistence.query.timeout → Timeout for long-running queries

Example:

```
List<Employee> employees = em.createQuery(
   "SELECT e FROM Employee e WHERE e.department = :dept", Employee.class)
   .setParameter("dept", "IT")
   .setHint("org.hibernate.cacheable", true)
   .getResultList();
```

When useful?

- Performance tuning (caching, read-only queries).
- Controlling timeout on expensive queries.

298. What are Hibernate Performance Tuning Best Practices?

Answer:

Hibernate is powerful but can cause performance issues if misused. Some best practices:

1. **Lazy Loading by Default**
    - Keep associations LAZY and fetch explicitly when needed.
2. **Batch Fetching**
    - Use @BatchSize or query batching to avoid N+1 selects.
3. **JOIN FETCH / EntityGraphs**
    - Fetch relationships in a single query when you know you need them.
4. **Second-Level Cache**
    - Cache reference data (like product catalog, lookup tables). Avoid caching transactional data.
5. **StatelessSession for Bulk Operations**
    - Use it for ETL or large batch jobs.
6. **Pagination**
    - Use setFirstResult and setMaxResults to handle large result sets.
7. **Avoid Overusing CascadeType.ALL**
    - It can cause accidental deletes/updates.
8. **Use Projections/DTOs**
    - Fetch only required fields instead of entire entities when showing lists.

Example DTO Query:

```
List<EmployeeDTO> result = em.createQuery(
  "SELECT new com.example.EmployeeDTO(e.id, e.name) FROM Employee e",
EmployeeDTO.class)
  .getResultList();
```

Real-world analogy: Don't carry the entire warehouse when you just need one product.

299. What is Hibernate Envers, and how does it provide auditing?

Answer:

Hibernate Envers is a module that provides **auditing/versioning of entity changes**. It keeps track of insert, update, and delete operations automatically in a separate audit table.

How it Works:

- For every entity annotated with @Audited, Envers creates an audit table (e.g., Employee_AUD).
- Each change (insert/update/delete) adds a new row in the audit table.
- You can query history of an entity using AuditReader.

Example:

```
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue
   private int id;
   private String name;
   private String department;
}
```

Audit query:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
Employee oldVersion = reader.find(Employee.class, 1, 2); // version 2
```

Use Cases:

- Regulatory compliance (banking, healthcare).
- Audit logs of user activity.
- Rollback support to previous states.

300. What are Natural IDs in Hibernate, and why are they used?

Answer:

A **Natural ID** is a business key that uniquely identifies an entity apart from its primary key. Example: an email in User, or an ISBN in Book.

Hibernate provides special support via @NaturalId.

Example:

```
@Entity
public class User {
   @Id
   @GeneratedValue
   private Long id;

   @NaturalId
   @Column(unique = true, nullable = false)
   private String email;

   private String name;
}

Querying by Natural ID:

User user = session.byNaturalId(User.class)
            .using("email", "rajat@example.com")
            .load();
```

Advantages:

- Faster lookups with caching.
- Cleaner code when business identifiers are used often.
- Avoids exposing surrogate DB IDs externally.

Note: Natural IDs must be **immutable** (never change after creation).

301. What are Projections in Hibernate, and when are they used?

Answer:

Projections allow you to **fetch only selected columns** instead of the entire entity. This improves performance when you don't need full objects.

Example using Criteria API:

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Object[]> query = cb.createQuery(Object[].class);
Root<Employee> root = query.from(Employee.class);

query.multiselect(root.get("id"), root.get("name"));
List<Object[]> result = session.createQuery(query).getResultList();

for (Object[] row : result) {
   System.out.println("ID: " + row[0] + ", Name: " + row[1]);
}
```

Example using JPQL DTO projection:

```
List<EmployeeDTO> result = em.createQuery(
  "SELECT new com.example.EmployeeDTO(e.id, e.name) FROM Employee e",
  EmployeeDTO.class
).getResultList();
```

When to Use:

- API responses (don't return entire entity).
- Reporting queries.
- Large tables with heavy relationships (reduce memory).

Analogy: Instead of carrying the entire book, just photocopy the 2 pages you need.

302. What are Hibernate Event Listeners, and how are they used?

Answer:

Hibernate provides **event listeners** to intercept entity lifecycle events (insert, update, delete, load, flush, etc.). This is useful for adding custom logic like auditing, logging, or validation.

Common Events:

- pre-insert, post-insert
- pre-update, post-update
- pre-delete, post-delete
- load

Example:

```
public class AuditEventListener implements PreInsertEventListener,
PreUpdateEventListener {

   @Override
   public boolean onPreInsert(PreInsertEvent event) {
      System.out.println("Before insert: " + event.getEntity());
      return false; // false → continue operation
   }

   @Override
   public boolean onPreUpdate(PreUpdateEvent event) {
      System.out.println("Before update: " + event.getEntity());
      return false;
   }
}
```

Registering the listener:

configuration.getEventListeners().setPreInsertEventListeners(new PreInsertEventListener[]{new AuditEventListener()});

Use Cases:

- Auditing changes.
- Auto-populating metadata (createdBy, updatedAt).
- Custom validation before persisting.

303. What is the Hibernate Statistics API, and why is it useful?

Answer:

Hibernate provides a **Statistics API** for monitoring performance and analyzing ORM behavior. It helps detect **N+1 issues, cache hits/misses, query execution counts, and entity loading details**.

Enabling Statistics:

hibernate.generate_statistics=true

Example:

```
Statistics stats = sessionFactory.getStatistics();
System.out.println("Entity fetch count: " + stats.getEntityFetchCount());
System.out.println("Query execution count: " + stats.getQueryExecutionCount());
System.out.println("Second-level cache hit count: " +
stats.getSecondLevelCacheHitCount());
```

Benefits:

- Identifying performance bottlenecks.
- Measuring effectiveness of caching.
- Debugging query inefficiencies (N+1 select).
- Production monitoring when integrated with Actuator/Prometheus.

304. What is Hibernate Validation, and how does it integrate with JPA?

Answer:

Hibernate Validator is the **reference implementation of Bean Validation (JSR 380)**. It provides **declarative constraints** on entity fields for validation.

Example:

```
@Entity
public class User {
   @Id
   @GeneratedValue
   private Long id;

   @NotBlank(message = "Name is required")
   private String name;

   @Email(message = "Invalid email")
   private String email;

   @Min(18)
   private int age;
}

When persisting:

User user = new User("", "invalidEmail", 15);
entityManager.persist(user); // Throws ConstraintViolationException
```

Integration in Spring Boot:

- Use @Valid in controller methods.

```
@PostMapping("/users")
public ResponseEntity<User> createUser(@Valid @RequestBody User user) {
   return ResponseEntity.ok(user);
}
```

Advantages:

- Centralized validation rules.
- Reduces boilerplate checks.
- Works seamlessly with REST APIs (auto 400 Bad Request for violations).

305. How can you use Native SQL queries in Hibernate, and when should you use them?

Answer:

Hibernate supports **Native SQL queries** when JPQL or Criteria API can't handle complex cases, such as vendor-specific features (e.g., Postgres JSONB, Oracle hints).

Example – Native SQL Query:

```
List<Object[]> results = entityManager
   .createNativeQuery("SELECT id, name FROM employee WHERE department = ?")
   .setParameter(1, "IT")
   .getResultList();

for (Object[] row : results) {
   System.out.println("ID: " + row[0] + ", Name: " + row[1]);
}
```

Mapping results directly to an entity:

```
List<Employee> employees = entityManager
   .createNativeQuery("SELECT * FROM employee", Employee.class)
   .getResultList();
```

When to use:

- Performance tuning with DB-specific SQL.
- Complex queries not supported by JPQL (window functions, CTEs, JSON operators).
- Bulk operations.

Best practice: Prefer JPQL for portability, use native queries only when necessary.

306. What are the differences between Hibernate Criteria API and JPQL?

Answer:

Feature	JPQL	Criteria API
Style	String-based queries	Programmatic, type-safe
Readability	Easier for simple queries	Verbose, but structured
Flexibility	Static queries	Great for dynamic queries
Compile-time safety	No (errors at runtime)	Yes (fields checked at compile time)

Example with JPQL:

```
@Query("SELECT e FROM Employee e WHERE e.department = :dept")
List<Employee> findByDepartment(@Param("dept") String dept);
```

Example with Criteria API:

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> root = cq.from(Employee.class);
cq.select(root).where(cb.equal(root.get("department"), "IT"));
List<Employee> employees = entityManager.createQuery(cq).getResultList();
```

When to use:

- JPQL → static, simple queries.
- Criteria API → dynamic search filters (e.g., multi-criteria search in an e-commerce site).

307. What are Hibernate performance tuning best practices?

Answer:

Hibernate can cause **N+1 issues, excessive joins, and memory bloat** if not tuned properly. Some key practices:

1. **Use Lazy Loading by Default**

```
@OneToMany(fetch = FetchType.LAZY)
private List<Order> orders;
```

2. **Batch Fetching / Fetch Joins**

```
@Query("SELECT d FROM Department d JOIN FETCH d.employees")
List<Department> findWithEmployees();
```

3. **Enable Second-Level Cache (Ehcache, Redis, Infinispan)**

```
hibernate.cache.use_second_level_cache=true
hibernate.cache.region.factory_class=org.hibernate.cache.ehcache.EhCache
RegionFactory
```

4. **Use DTO projections** (instead of fetching full entities).

```
@Query("SELECT new com.example.EmployeeDTO(e.name, e.department)
FROM Employee e")
```

```
List<EmployeeDTO> findEmployeeSummaries();
```

5. **Batch inserts/updates** with hibernate.jdbc.batch_size=20.
6. **Use StatelessSession** for bulk operations.
7. **Monitor with Hibernate Statistics API** to detect N+1 queries.

Real-World Analogy:

Think of Hibernate tuning like running a **restaurant kitchen**:

- Lazy fetch → cook meals only when ordered.
- Batch fetch → prepare multiple orders together.
- Cache → keep frequently ordered dishes pre-prepared.

308. What is the difference between Session and StatelessSession in Hibernate?

Session:

- The standard interface in Hibernate used for CRUD, queries, and transaction management.
- Maintains **first-level cache** (persistence context).
- Tracks entity states (transient, persistent, detached).
- Supports cascading, lazy loading, and dirty checking.

StatelessSession:

- A lightweight version of Session designed for **bulk operations**.
- No first-level cache, no persistence context.
- Entities are not automatically tracked → you must explicitly update/insert.
- Faster for batch inserts/updates, but less safe (no dirty checking).

Example:

```
// Using normal Session (tracks entities)
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
Employee e = session.get(Employee.class, 1); // cached in session
e.setName("Updated Name"); // auto-detected change
tx.commit();
session.close();

// Using StatelessSession (no tracking)
StatelessSession stateless = sessionFactory.openStatelessSession();
Transaction tx2 = stateless.beginTransaction();
Employee e2 = (Employee) stateless.get(Employee.class, 1);
e2.setName("Updated Again");
stateless.update(e2);  // must explicitly call update
tx2.commit();
stateless.close();
```

Best Practice:

- Use Session for normal apps (CRUD, APIs).

- Use StatelessSession for **ETL, data migration, bulk imports**, where you don’t need caching or dirty checking.

309. What is the difference between Interceptors and Event Listeners in Hibernate?

Interceptors:

- Implement Interceptor interface.
- Used to intercept session-level operations (entity load, save, delete).
- Simple to add but limited flexibility.
- Example: Audit createdBy/updatedBy automatically.

Event Listeners:

- More powerful, fine-grained approach.
- Hibernate triggers events like pre-insert, post-update, etc.
- You can attach listeners to handle these lifecycle events.
- Useful for cross-cutting concerns like auditing, logging, enforcing rules.

Example – Interceptor:

```
public class AuditInterceptor extends EmptyInterceptor {
   @Override
   public boolean onSave(Object entity, Serializable id, Object[] state, String[]
propertyNames, Type[] types) {
      if (entity instanceof Auditable) {
         int index = Arrays.asList(propertyNames).indexOf("createdAt");
         state[index] = LocalDateTime.now();
         return true;
      }
      return false;
   }
}
```

Example – +Event Listener:

```
public class AuditListener {
   @PrePersist
   public void beforeInsert(Object entity) {
      if (entity instanceof Auditable) {
         ((Auditable) entity).setCreatedAt(LocalDateTime.now());
      }
   }
}
```

When to use:

- Interceptor → quick, session-wide custom logic.
- Event Listeners → structured, entity-lifecycle-specific hooks (preferred for enterprise).

310. How does Hibernate support Multi-Tenancy?

Multi-tenancy = supporting multiple customers (tenants) with **isolation** but shared application code.

Hibernate provides **3 strategies**:

1. **DATABASE (Separate Databases per tenant)**
    - Each tenant has its own database.
    - Strongest isolation, hardest to manage (many DBs).
2. **SCHEMA (Separate schemas per tenant in the same DB)**
    - One database, multiple schemas.
    - Balance between isolation and manageability.
3. **DISCRIMINATOR (Shared DB + shared schema, tenant column distinguishes data)**
    - Cheapest option, lowest isolation.
    - Each table includes a tenant_id column.

Configuration Example (SCHEMA strategy):

```
hibernate.multiTenancy=SCHEMA
hibernate.tenant_identifier_resolver=myTenantResolver
hibernate.multi_tenant_connection_provider=myConnectionProvider
```

Tenant Resolver Example:

```
public class MyTenantResolver implements CurrentTenantIdentifierResolver {
  @Override
  public String resolveCurrentTenantIdentifier() {
    return TenantContext.getCurrentTenant(); // e.g. tenant from JWT/session
  }
  @Override
  public boolean validateExistingCurrentSessions() {
    return true;
  }
}
```

Best Practices:

- **DATABASE** → enterprise SaaS with strict compliance.
- **SCHEMA** → mid-size SaaS apps.

- **DISCRIMINATOR** → simple apps where cost > isolation.

311. What is Hibernate Envers, and how is it used for auditing?

Hibernate Envers is a module that lets you track **historical changes** of entities without writing custom audit logic. It automatically maintains revision tables so you can query old versions of data.

How it works:

- For each audited entity, Envers creates an audit table (<entity>_AUD).
- Whenever you insert, update, or delete, Envers inserts a record into the audit table.
- You can later query historical states.

Setup:

```
<dependency>
 <groupId>org.hibernate</groupId>
 <artifactId>hibernate-envers</artifactId>
</dependency>
```

Usage:

```
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue
   private Long id;
   private String name;
   private String department;
}
```

Query past revisions:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
Employee old = reader.find(Employee.class, 1L, 2); // revision 2
```

Use case:

Think of a **banking system** – when account details change, auditors want to see *who changed what and when*. Envers provides this without custom triggers or audit tables.

312. What is the difference between UserType and AttributeConverter in Hibernate?

Both allow mapping custom Java types to database columns, but they differ in scope and usage.

UserType (Hibernate-specific):

- Gives full control over how a property is persisted and retrieved.
- You implement methods like nullSafeGet, nullSafeSet, etc.
- Powerful but verbose.
- Example: Storing a JSON object into a column.

AttributeConverter (JPA standard):

- Simpler and portable across providers.
- Annotate a class with @Converter and implement convertToDatabaseColumn and convertToEntityAttribute.
- Best for simple transformations.

Example – AttributeConverter:

```
@Converter(autoApply = true)
public class BooleanToIntConverter implements AttributeConverter<Boolean,
Integer> {
   public Integer convertToDatabaseColumn(Boolean value) {
      return (value != null && value) ? 1 : 0;
   }
   public Boolean convertToEntityAttribute(Integer value) {
      return value != null && value == 1;
   }
}
```

Example – UserType (advanced JSON mapping):

```
public class JsonUserType implements UserType {
   @Override
   public Object nullSafeGet(ResultSet rs, String[] names,
SharedSessionContractImplementor session, Object owner)
      throws SQLException {
      String json = rs.getString(names[0]);
      return new Gson().fromJson(json, Map.class);
   }
```

```
    @Override
    public void nullSafeSet(PreparedStatement st, Object value, int index,
SharedSessionContractImplementor session)
        throws SQLException {
        st.setString(index, new Gson().toJson(value));
    }
}
```

Rule of Thumb:

- Use AttributeConverter for simple conversions (booleans, enums, encrypted strings).
- Use UserType for complex custom types (JSON, proprietary DB types).

313. What are some best practices for using Hibernate with Spring Boot?

1. **Keep Entities Lightweight**
    - Entities should represent DB tables only.
    - Avoid putting business logic inside entities.
    - Use DTOs for API responses.
2. **Use Lazy Loading by Default**
    - Prevent unnecessary data fetching.
    - Explicitly fetch only what you need with JOIN FETCH or projections.
3. **Avoid N+1 Selects**
    - Use EntityGraph, @BatchSize, or JOIN FETCH.
4. **Transaction Management**
    - Annotate service layer methods with @Transactional.
    - Avoid using transactions directly in controllers.
5. **Connection Pooling**
    - Use HikariCP (default in Spring Boot) for efficient DB connections.
6. **Pagination for Large Data**
    - Always use Pageable or setMaxResults() for queries returning large datasets.
7. **Second-Level Cache Wisely**
    - Use only for reference/read-mostly data (like products, lookup tables).
    - Avoid caching highly dynamic transactional data.
8. **Schema Management**
    - Use spring.jpa.hibernate.ddl-auto=validate in production.
    - Let Flyway/Liquibase handle schema evolution.

Example Service Layer:

```
@Service
public class EmployeeService {
  private final EmployeeRepository repo;

  public EmployeeService(EmployeeRepository repo) {
    this.repo = repo;
```

```
  }

  @Transactional
  public Employee updateDepartment(Long id, String dept) {
    Employee e = repo.findById(id).orElseThrow();
    e.setDepartment(dept);
    return e;
  }
}
```

314. How does Hibernate integrate with NoSQL databases (MongoDB, Cassandra)?

Hibernate traditionally works with relational databases, but Hibernate OGM (Object/Grid Mapper) extends it to NoSQL stores.

Key idea:

You still use JPA annotations (@Entity, @Id, etc.), but Hibernate translates them into NoSQL operations instead of SQL.

Supported NoSQL backends (via Hibernate OGM):

- MongoDB
- Cassandra
- CouchDB
- Neo4j (graph DB)

Example – MongoDB with Hibernate OGM:

```
@Entity
public class Product {
   @Id
   private String id;
   private String name;
   private double price;
}
```

Configuration (persistence.xml):

```
<property name="hibernate.ogm.datastore.provider" value="mongodb"/>
<property name="hibernate.ogm.datastore.database" value="shopDB"/>
<property name="hibernate.ogm.datastore.host" value="127.0.0.1"/>
```

Usage:

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Product p = new Product();
p.setId("p1");
p.setName("Laptop");
p.setPrice(75000);
em.persist(p);
em.getTransaction().commit();
```

When useful:

For **event logs**, **caching layers**, or **catalog data** that doesn't need strong SQL joins. But for OLTP banking/e-commerce transactions, stick to relational DB + Hibernate ORM.

315. What are Projections in Hibernate, and why are they useful?

Projections allow fetching **only specific fields** instead of whole entities. This improves performance when you don't need the full object.

Example – Entity:

```
@Entity
public class Employee {
   @Id
   private Long id;
   private String name;
   private String department;
   private double salary;
}
```

Without Projection:

```
List<Employee> employees = session.createQuery("FROM Employee").list();
// Fetches all fields, even if you only need name
```

With Projection (JPQL):

```
List<String> names = session.createQuery(
   "SELECT e.name FROM Employee e", String.class).list();
```

DTO Projection:

```
List<EmployeeDTO> result = session.createQuery(
   "SELECT new com.example.EmployeeDTO(e.name, e.department) FROM
Employee e", EmployeeDTO.class).list();
```

Criteria API Projection:

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Object[]> query = cb.createQuery(Object[].class);
Root<Employee> root = query.from(Employee.class);
query.multiselect(root.get("name"), root.get("salary"));
List<Object[]> data = em.createQuery(query).getResultList();
```

Why useful:

- Saves memory and bandwidth.
- Avoids lazy-loading issues.
- Common in **reporting dashboards** (e.g., only showing employee names and salaries, not entire entity graphs).

316. How does Hibernate integrate with QueryDSL for dynamic queries?

QueryDSL is a framework that allows **type-safe queries** in Java. Instead of writing string-based JPQL, you write queries using generated Q-classes.

Setup:

- Add QueryDSL JPA dependency.
- Annotation processor generates classes like QEmployee.

Entity:

```
@Entity
public class Employee {
   @Id
   private Long id;
   private String name;
   private String department;
}
```

Generated class: QEmployee with fields like QEmployee.employee.name.

QueryDSL Example:

```
JPAQueryFactory queryFactory = new JPAQueryFactory(entityManager);
QEmployee emp = QEmployee.employee;

List<Employee> result = queryFactory
   .selectFrom(emp)
   .where(emp.department.eq("IT"))
   .fetch();
```

Advantages:

- Compile-time safety (no more string typos in JPQL).
- Cleaner dynamic queries (especially with multiple filters).
- Works well in large applications where user-defined filters are common (like search pages in e-commerce).

317. How does Hibernate support multitenancy (useful in SaaS apps)?

In SaaS platforms, different customers (tenants) need data isolation. Hibernate supports multitenancy so you can serve multiple tenants with a single app.

Strategies supported by Hibernate:

1. **DATABASE** → separate DB for each tenant.
    - Each tenant gets its own DB instance.
    - Strongest isolation, but costly.
2. **SCHEMA** → same DB, separate schemas per tenant.
    - Middle ground: cheaper than multiple DBs, still isolated.
3. **DISCRIMINATOR** → single table, tenant column distinguishes data.
    - Cheapest, but least isolation (logical partition).

Configuration Example (Schema strategy):

```
hibernate.multiTenancy=SCHEMA
hibernate.tenant_identifier_resolver=myTenantResolver
hibernate.multi_tenant_connection_provider=myConnectionProvider
```

TenantResolver Example:

```
public class MyTenantResolver implements CurrentTenantIdentifierResolver {
  @Override
  public String resolveCurrentTenantIdentifier() {
    return TenantContext.getCurrentTenant(); // e.g., "tenant1"
  }

  @Override
  public boolean validateExistingCurrentSessions() {
    return true;
  }
}
```

Use case:

An HR SaaS app where each company should only see their employees. Hibernate routes queries to the correct schema/DB based on tenant id.

318. How does Hibernate integrate with Stored Procedures?

Even though Hibernate handles most SQL generation, sometimes you need stored procedures for performance or legacy DB integration.

Entity Example:

```
@Entity
@NamedStoredProcedureQuery(
   name = "getEmployeesByDept",
   procedureName = "GET_EMPLOYEES_BY_DEPT",
   parameters = {
      @StoredProcedureParameter(mode = ParameterMode.IN, name = "dept", type =
String.class)
   }
)
public class Employee {
   @Id
   private Long id;
   private String name;
}
```

Usage:

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("getEmployeesByDept");
query.setParameter("dept", "IT");
List<Employee> employees = query.getResultList();
```

When useful:

- Legacy DBs where business logic lives in procedures.
- Performance-sensitive operations (bulk updates, reports).
- Banks and government systems often rely heavily on stored procedures.

319. What are Event Listeners and Interceptors in Hibernate?

Both allow you to hook into Hibernate's entity lifecycle, but at different levels.

Event Listeners:

- Triggered on specific events (insert, update, delete, load).
- Example: automatically updating createdAt or updatedAt.

```
public class AuditListener {
  @PrePersist
  public void beforeInsert(Object entity) {
    if (entity instanceof Auditable auditable) {
      auditable.setCreatedAt(LocalDateTime.now());
    }
  }
}
```

Attach with @EntityListeners(AuditListener.class).

Interceptors:

- Lower-level, intercept all operations globally.
- Implement Interceptor interface.

```
public class MyInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id,
      Object[] state, String[] propertyNames, Type[] types) {
    System.out.println("Entity saved: " + entity);
    return false;
  }
}
```

Enable:

```
SessionFactory sessionFactory = new Configuration()
  .setInterceptor(new MyInterceptor())
  .buildSessionFactory();
```

Difference:

- Listeners → Entity-specific lifecycle hooks.

- Interceptors → Global, cross-cutting concerns (like auditing/logging).

Real-world use:

- Audit logs (who created/modified an entity).
- Soft deletes (mark as deleted instead of physical removal).
- Multi-tenant validations.

320. How does Hibernate optimize performance with Dirty Checking?

Answer:

Hibernate automatically detects changes in managed entities (those attached to a persistence context) and generates SQL UPDATE statements only for modified fields.

- This is called **dirty checking**.
- Hibernate compares the in-memory state of the entity with the snapshot taken when it was loaded.

Example:

```
Employee emp = session.get(Employee.class, 1);
emp.setName("Updated Name"); // Hibernate marks this entity dirty
// No explicit update() call
tx.commit(); // Hibernate issues UPDATE automatically
```

Key Notes:

- Reduces boilerplate (no manual updates).
- Overhead for large entities (many fields).
- Best practice: keep entities lean, avoid unnecessary setters, and prefer DTOs for read-only views.

321. What are DTO Projections in Hibernate, and why are they used?

Answer:

DTO (Data Transfer Object) projections fetch only the required fields instead of entire entities, improving performance.

Example – JPQL Projection:

```
@Query("SELECT new com.example.EmployeeDTO(e.id, e.name) FROM Employee e")
List<EmployeeDTO> findEmployeeSummaries();
```

Example – Native Projection:

```
@Query(value = "SELECT id, name FROM employee", nativeQuery = true)
List<Object[]> findEmployeeSummary();
```

Why Use DTO Projections?

- Faster queries (avoid unnecessary joins and lazy initialization).
- Lower memory usage.
- Best for read-heavy APIs (dashboards, reports).

322. How does Hibernate support Auditing with Envers?

Answer:

Hibernate **Envers** is a module that tracks and stores changes to entities for audit/history purposes.

- Automatically creates audit tables (employee_AUD).
- Records INSERT, UPDATE, DELETE operations.
- Allows querying historical data.

Setup:

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-envers</artifactId>
</dependency>
```

Usage:

```
@Entity
@Audited
public class Employee {
  @Id
  private int id;
  private String name;
}
```

Querying history:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
List<Number> revisions = reader.getRevisions(Employee.class, 1);
Employee old = reader.find(Employee.class, 1, revisions.get(0));
```

Why Important in Interviews:

- Required in banking/finance/healthcare (compliance).
- Helps restore deleted/mistakenly updated data.

323. How does Hibernate handle JSON column mapping in modern databases?

Answer:

Many modern databases (PostgreSQL, MySQL 5.7+, Oracle 21c) support **JSON columns**. Hibernate allows mapping JSON data into entity fields using custom types.

Approaches:

1. **Hibernate Types Library (recommended):**

```
@Entity
@TypeDef(name = "json", typeClass = JsonType.class)
public class Employee {
 @Id
 private Long id;

 @Type(type = "json")
 @Column(columnDefinition = "jsonb")
 private Map<String, Object> metadata;
}
```

2. **Custom AttributeConverter:**

```
@Converter(autoApply = true)
public class JsonConverter implements AttributeConverter<Map<String,
Object>, String> {
 private final ObjectMapper mapper = new ObjectMapper();

 @Override
 public String convertToDatabaseColumn(Map<String, Object> attribute)
throws PersistenceException {
  try { return mapper.writeValueAsString(attribute); }
  catch (Exception e) { throw new PersistenceException("Error", e); }
 }

 @Override
 public Map<String, Object> convertToEntityAttribute(String dbData) {
  try { return mapper.readValue(dbData, new TypeReference<>() {}); }
  catch (Exception e) { throw new PersistenceException("Error", e); }
 }
}
```

Why Useful?

- Store flexible, schema-less data inside relational DB.
- Mix structured + semi-structured data.
- Common in microservices (storing dynamic configs, preferences).

324. How does Hibernate handle streaming & pagination for large datasets?

Answer:

When dealing with **millions of rows**, loading everything into memory will cause OOM. Hibernate provides **streaming** and **pagination** solutions.

Pagination:

```
Query<Employee> query = session.createQuery("FROM Employee", Employee.class);
query.setFirstResult(0);   // offset
query.setMaxResults(50);   // limit
List<Employee> employees = query.list();
```

Streaming:

```
try (Stream<Employee> stream = session.createQuery("FROM Employee",
Employee.class).stream()) {
   stream.forEach(emp -> process(emp));
}
```

Best Practices:

- Use pagination for APIs (UI tables, dashboards).
- Use streaming for ETL/batch jobs.
- Always combine with stateless sessions or session.clear() to prevent memory leaks.

325. How do you test Hibernate-based apps effectively?

Answer:

Hibernate apps can be tested using **in-memory databases** and **Testcontainers**.

1. Using H2 In-Memory DB:

```
<dependency>
 <groupId>com.h2database</groupId>
 <artifactId>h2</artifactId>
 <scope>test</scope>
</dependency>
@DataJpaTest
class EmployeeRepositoryTest {
 @Autowired
 private EmployeeRepository repo;

 @Test
 void testSaveEmployee() {
  Employee e = new Employee();
  e.setName("John");
  repo.save(e);

  assertTrue(repo.findByName("John").isPresent());
 }
}
```

2. Using Testcontainers (real DB in Docker):

```
@Testcontainers
public class EmployeeRepositoryIT {
 @Container
 static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:15");

 @Autowired
 private EmployeeRepository repo;

 @Test
 void testInsert() {
  repo.save(new Employee("Alice"));
  assertEquals(1, repo.findAll().size());
 }
}
```

Why Important?

- In-memory DB → fast unit tests.
- Testcontainers → realistic integration tests.
- Interview takeaway: Show awareness of trade-offs (fast vs realistic).

326. How does Hibernate support multi-tenancy?

Answer:
Multi-tenancy means a single application serves multiple tenants (customers), each with isolated data. Hibernate supports this in multiple ways:

Strategies:

1. **Separate Database per Tenant**
    - Each tenant has its own DB.
    - Hibernate switches DataSource dynamically.
    - Secure, but resource-heavy.
2. **Separate Schema per Tenant**
    - Same DB, different schemas (tenant1.orders, tenant2.orders).
    - Hibernate switches schema per request.
3. **Discriminator Column (Single Table)**
    - One schema, one set of tables, but every row has a tenant_id.
    - Cheapest but less isolation.

Example using schema-based multi-tenancy:

```
@Configuration
public class MultiTenantConfig {

 @Bean
 public MultiTenantConnectionProvider multiTenantConnectionProvider(DataSource
dataSource) {
  return new SchemaMultiTenantConnectionProvider(dataSource);
 }

 @Bean
 public CurrentTenantIdentifierResolver tenantResolver() {
  return new CurrentTenantIdentifierResolver() {
   @Override
   public String resolveCurrentTenantIdentifier() {
    return TenantContext.getTenantId(); // e.g., from request header
   }
   @Override
   public boolean validateExistingCurrentSessions() { return true; }
  };
 }
}
```

Use case: SaaS platforms (like Shopify or Freshworks) where each customer has isolated data.

327. How does Hibernate integrate with database versioning tools (Flyway/Liquibase)?

Answer:

Hibernate auto-generates schema (via hibernate.hbm2ddl.auto), but that's not production-safe. Instead, teams use **Flyway or Liquibase** for database migrations.

Why?

- Versioned migration scripts ensure DB is consistent across dev, test, prod.
- Hibernate focuses on ORM, Flyway/Liquibase handle schema evolution.

Example with Flyway:

```
application.properties

spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration

Migration script: V1__create_employee.sql

CREATE TABLE employee (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(100),
  department VARCHAR(50)
);
```

Integration with Hibernate:

- Hibernate generates entities → you write migration scripts accordingly.
- Flyway ensures schema changes are applied before Hibernate runs.

Best Practice:

- Turn off hibernate.hbm2ddl.auto in production.
- Always track schema changes in versioned scripts.

328. How is Hibernate used in microservices architecture?

Answer:

Hibernate works differently in **monoliths vs microservices**.

Challenges in microservices:

1. **Each microservice has its own DB (DB-per-service pattern).**
    - No shared schema.
    - Avoids cross-service joins.
2. **Event-driven consistency** (instead of distributed transactions).
    - Services publish events to Kafka/RabbitMQ when DB changes.
    - Other services update their own DBs.

Example:

- OrderService DB: orders
- PaymentService DB: payments
- Hibernate entities map only to local DB tables.
- For cross-service data (e.g., Order + Payment), use events, not Hibernate joins.

Code Example – Order Service:

```
@Entity
public class Order {
 @Id @GeneratedValue
 private Long id;
 private String product;
 private String status;
}
```

Best Practices in Microservices:

- Keep Hibernate mappings **bounded to one service**.
- Use **DTOs + REST/Events** for cross-service data.
- Use Hibernate **stateless sessions** for batch/event handling.

329. What are the key performance pitfalls in Hibernate and how do you avoid them?

Answer:

Hibernate makes life easy, but careless use leads to performance issues. The common pitfalls are:

1. **N+1 Select Problem**
    - Cause: Lazy loading without batch fetching.
    - Fix: Use JOIN FETCH, @BatchSize, or entity graphs.
2. **Unbounded Fetching (Eager Loading)**
    - Cause: Fetching huge collections eagerly when not needed.
    - Fix: Prefer FetchType.LAZY, fetch associations only when necessary.
3. **Overusing session.save() in loops**
    - Cause: Flushing after every insert.
    - Fix: Use batching (hibernate.jdbc.batch_size), flush/clear periodically.
4. **No Second-Level Cache**
    - Cause: Repeated queries for frequently accessed data.
    - Fix: Enable second-level cache for reference data.

330. How do you tune Hibernate batch processing?

Answer:

When inserting/updating thousands of rows, Hibernate must be tuned for batching.

Steps:

1. Configure batch size:

```
hibernate.jdbc.batch_size=50
hibernate.order_inserts=true
hibernate.order_updates=true
```

2. Use session flushing:

```
for (int i = 0; i < employees.size(); i++) {
  session.save(employees.get(i));
  if (i % 50 == 0) {
    session.flush();
  session.clear();
  }
}
```

3.

3. Use StatelessSession for massive imports (skips caches).

Analogy:

Instead of sending thousands of letters one by one to the post office, you bundle them in packets of 50.

331. How do you analyze and optimize slow Hibernate queries?

Answer:
Hibernate sometimes generates inefficient SQL. You need to profile and optimize.

1. **Enable SQL logging**

```
hibernate.show_sql=true
hibernate.format_sql=true
hibernate.generate_statistics=true
```

2. **Use tools:**
   - Hibernate Statistics API (sessionFactory.getStatistics())
   - Database query logs (EXPLAIN plans).
   - Profiling tools like VisualVM, YourKit, or JProfiler.
3. **Optimization techniques:**
   - Add indexes on frequently queried columns.
   - Use projections (DTOs) instead of fetching full entities.
   - Use pagination (setFirstResult, setMaxResults) for large result sets.
   - Avoid cartesian product joins by carefully using JOIN FETCH.

Example – DTO Projection:

```
Query<EmployeeDTO> query = session.createQuery(
  "SELECT new com.example.EmployeeDTO(e.id, e.name) FROM Employee e",
  EmployeeDTO.class
);
List<EmployeeDTO> result = query.getResultList();
```

This avoids fetching unnecessary columns or associations.

332. How do you use caching effectively in Hibernate?

Answer:
Caching can drastically reduce DB hits.

1. **First-Level Cache (Session)**
    - Default, per-session.
    - Always active.
    - Avoids duplicate queries inside one transaction.
2. **Second-Level Cache (SessionFactory)**
    - Shared across sessions.
    - Needs provider: Ehcache, Redis, Hazelcast.
    - Best for reference data (products, categories).

hibernate.cache.use_second_level_cache=true

hibernate.cache.region.factory_class=org.hibernate.cache.ehcache.EhCacheRegionFactory

3. **Query Cache**
    - Caches results of queries.
    - Must be enabled explicitly.

hibernate.cache.use_query_cache=true

Best Practices:

- Cache static/reference data (roles, countries).
- Don’t cache transactional data (bank balances, orders).
- Always combine caching with eviction policies.

333. What is the role of Fetch Profiles, Entity Graphs, and DTO projections in performance tuning?

Answer:
These are advanced fetch tuning tools:

1. **Fetch Profiles (Hibernate-specific)**
    - Predefined fetch strategy, activated dynamically.
    - Useful when same entity needs different fetch plans in different use cases.
2. **Entity Graphs (JPA-standard)**
    - Control lazy/eager associations at runtime.
    - Keeps mappings clean while still flexible.
3. **DTO Projections**
    - Avoids fetching entire entities when only partial fields are needed.
    - Helps reduce memory usage and query cost.

Rule of Thumb:

- Use entity graphs for flexible fetch tuning.
- Use DTO projections for read-heavy queries.
- Use fetch profiles when Hibernate-specific optimization is fine.

334. What is Hibernate multi-tenancy, and how is it implemented?

Answer

Multi-tenancy means running the same application for multiple customers (tenants), where each tenant's data is isolated. Hibernate supports this out of the box.

Types of multi-tenancy:

1. **DATABASE** – each tenant has its own database.
2. **SCHEMA** – all tenants share a DB, but each has its own schema.
3. **DISCRIMINATOR** – all tenants share the same tables, but rows are separated by a tenant_id column.

Example – schema-based multi-tenancy:

```
@Configuration
public class HibernateConfig {

  @Bean
  public MultiTenantConnectionProvider multiTenantConnectionProvider() {
    return new SchemaMultiTenantConnectionProvider();
  }

  @Bean
  public CurrentTenantIdentifierResolver tenantIdentifierResolver() {
    return new TenantIdentifierResolver();
  }
}
```

Use case:
Think of SaaS products like Shopify or Zoho CRM. Each business is a tenant, but they all run on the same Spring Boot + Hibernate app. Depending on business needs:

- Small SaaS → Discriminator.
- Medium scale → Schema.
- Enterprise → Separate databases.

335. How can you call stored procedures using Hibernate?

Answer

Sometimes, businesses use stored procedures for performance-critical tasks (like monthly billing or inventory reconciliation). Hibernate provides ways to integrate with them.

Using JPA:

```
@NamedStoredProcedureQuery(
   name = "calculate_bonus",
   procedureName = "sp_calculate_bonus",
   parameters = {
      @StoredProcedureParameter(mode = ParameterMode.IN, name = "emp_id", type
= Integer.class),
      @StoredProcedureParameter(mode = ParameterMode.OUT, name = "bonus",
type = Double.class)
   }
)
@Entity
public class Employee {
   @Id
   private int id;
}
```

Calling it:

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("calculate_bonus");
query.setParameter("emp_id", 101);
query.execute();
Double bonus = (Double) query.getOutputParameterValue("bonus");
```

Real-world: banking apps often keep critical logic in stored procedures for regulatory compliance, and Hibernate lets you leverage that.

336. What are caching concurrency strategies in Hibernate second-level cache?

Answer

When using second-level cache, you must decide how concurrent reads/writes are handled. Hibernate provides several strategies:

1. **READ_ONLY** – best for static data (countries, product categories). No updates allowed.
2. **NONSTRICT_READ_WRITE** – cache may be slightly stale. Good for reference data updated infrequently.
3. **READ_WRITE** – uses soft locks to ensure consistency. Balanced, but slower.
4. **TRANSACTIONAL** – requires JTA, guarantees full consistency. Suitable for critical financial apps.

Example config with Ehcache:

```
<cache name="com.app.Employee"
    maxEntriesLocalHeap="1000"
    timeToLiveSeconds="600"
    eternal="false"
    statistics="true"/>
```

Best practices:

- Use READ_ONLY for master data.
- Use READ_WRITE for mutable entities.
- Never blindly enable L2 cache for transactional entities like orders or payments.

337. What is the role of Interceptors and Event Listeners in Hibernate?

Answer

Interceptors and event listeners let you hook into Hibernate's lifecycle.

- **Interceptor** → global hooks for operations (onSave, onDelete, onFlushDirty).
- **Event Listener** → specific hooks (PreInsert, PostUpdate, etc.).

Example – auditing entity updates:

```
public class AuditInterceptor extends EmptyInterceptor {
  @Override
  public boolean onFlushDirty(Object entity, Serializable id,
                              Object[] currentState, Object[] previousState,
                              String[] propertyNames, Type[] types) {
    if (entity instanceof Auditable) {
      ((Auditable) entity).setUpdatedAt(LocalDateTime.now());
      return true;
    }
    return false;
  }
}
```

Use case:

- Audit trails in finance.
- Soft deletes (mark deleted flag instead of deleting).
- Multi-tenant logic (adding tenant_id automatically).

338. How do you tune Hibernate for high-performance systems?

Answer

Tuning involves a combination of Hibernate settings and DB-level optimization.

Key strategies:

1. **Connection Pooling** – use HikariCP instead of default.
2. **Batching** – enable hibernate.jdbc.batch_size=30.
3. **FetchSize** – configure for streaming large datasets.
4. **Second-level caching** – use wisely with Ehcache/Redis.
5. **DTO Projections** – fetch only required columns for APIs.
6. **Query Plan Cache** – tune hibernate.query.plan_cache_max_size.

Real-world:

In an e-commerce app, fetching 1,000 orders with customer + product details can be optimized by:

- Using JOIN FETCH for required data.
- Using DTO projection for summary views.
- Caching reference data like product categories.

339. How does Hibernate integrate with Spring Boot in real-world projects?

Answer

Spring Boot auto-configures Hibernate as the default JPA provider. Integration is seamless with spring-boot-starter-data-jpa.

1. Define application.yml:

```
spring:
 datasource:
  url: jdbc:mysql://localhost:3306/shopdb
  username: root
  password: pass
 jpa:
  hibernate:
   ddl-auto: update
 show-sql: true
```

2. Create entities + repositories:

```
@Entity
public class Product {
 @Id @GeneratedValue
 private Long id;
 private String name;
}
@Repository
public interface ProductRepository extends JpaRepository<Product,
Long> {}
```

3. Use in services:

```
@Service
public class ProductService {
  private final ProductRepository repo;
  public ProductService(ProductRepository repo) { this.repo = repo; }
}
```

Why it matters:

- Spring handles transactions (@Transactional).
- Boot manages EntityManagerFactory.
- Devs focus on business logic, not plumbing.

340. How does Hibernate handle auditing, and what is Envers?

Answer

Auditing means keeping track of who changed what and when. Instead of writing triggers or manual audit tables, Hibernate Envers provides this out of the box.

How it works:

- Envers creates special audit tables (e.g., Employee_AUD).
- On insert, update, or delete, Envers writes entries automatically.
- Each entry includes revision info (revision number, timestamp, etc.).

Setup:

Add dependency:

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-envers</artifactId>
</dependency>
```

Enable auditing:

```
@Entity
@Audited
public class Employee {
  @Id
  private Long id;
  private String name;
  private String department;
}
```

Fetching historical data:

```
AuditReader reader = AuditReaderFactory.get(em);
Employee old = reader.find(Employee.class, 1L, 2); // revision 2
```

Use case:

- Banking systems (track who updated an account balance).
- Healthcare (keep medical record changes).
- Compliance-driven industries.

341. What are Hibernate Interceptor vs EventListener differences?

Answer

Both allow extending Hibernate behavior, but scope differs.

- **Interceptor** → Global, attached at Session/SessionFactory level. Good for cross-cutting concerns like audit timestamps.
- **EventListener** → Fine-grained, tied to specific events like PreInsert, PostLoad, etc.

Example – EventListener:

```
public class AuditListener {
  @PrePersist
  public void beforeInsert(Object entity) {
    if (entity instanceof Auditable) {
      ((Auditable) entity).setCreatedAt(LocalDateTime.now());
    }
  }
}
```

Difference in practice:

- Interceptors are like a “big umbrella hook” → good for broad changes.
- EventListeners are surgical → useful when you want to customize specific entity events only.

342. How do you use Hibernate with Flyway or Liquibase for schema evolution?

Answer

Hibernate can auto-generate schemas (ddl-auto=update), but that's risky in production. For real projects, schema migrations are handled by Flyway or Liquibase.

Example with Flyway:

1. Add dependency:

```
<dependency>
 <groupId>org.flywaydb</groupId>
 <artifactId>flyway-core</artifactId>
</dependency>
```

2. Create migration file V1__create_employee.sql:

```
CREATE TABLE employee (
 id BIGINT AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(100),
 department VARCHAR(50)
);
```

3. On app startup, Flyway runs migrations in order.

Benefit:

- Hibernate manages entities.
- Flyway/Liquibase ensures schema is controlled, versioned, and auditable.

In interviews, mention:

"ddl-auto is good for dev/test, but in production, we always pair Hibernate with a migration tool for reliability."

343. What are StatelessSession and when would you use it?

Answer

A StatelessSession in Hibernate is a lightweight session that doesn't keep a first-level cache or perform dirty checking.

Key properties:

- No persistence context.
- No automatic flushing.
- No entity lifecycle events.

Example:

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();
for (int i = 0; i < 1000; i++) {
   Employee e = new Employee("User" + i, "Dept");
   session.insert(e);
}
tx.commit();
session.close();
```

Use cases:

- Bulk inserts, updates, or deletes.
- ETL jobs (e.g., importing millions of records).
- Data migration tools.

But don't use it for business logic where entity lifecycle management (cascade, lazy loading, etc.) is needed.

344. How does Hibernate support reactive programming?

Answer

Traditional Hibernate is blocking. Hibernate Reactive is a new initiative built on **Vert.x and Mutiny** that supports non-blocking database operations.

Key differences:

- Uses Uni and Multi (reactive types) instead of direct objects.
- Works with PostgreSQL, MySQL, DB2, and CockroachDB drivers.
- Runs without JDBC (since JDBC is blocking).

Example with Hibernate Reactive:

```
Mutiny.SessionFactory factory = ...;
factory.withSession(session ->
   session.createQuery("from Employee", Employee.class)
         .getResultList()
         .invoke(list -> list.forEach(System.out::println))
);
```

Use case:

- Reactive microservices where scalability and non-blocking I/O is critical (chat apps, streaming platforms).

345. What are database connection pooling best practices with Hibernate?

Answer

Hibernate itself doesn't manage connections well. It delegates to connection pools. The best choice in modern apps is **HikariCP**.

Example config (application.yml):

```
spring:
 datasource:
  hikari:
   maximum-pool-size: 20
   minimum-idle: 5
   idle-timeout: 30000
   connection-timeout: 30000
   max-lifetime: 1800000
```

Best practices:

- Size pool based on CPU cores and DB connections allowed.
- Use monitoring (Micrometer, Actuator) to track pool usage.
- Always close connections properly in non-Spring apps.

Real-world:
In a fintech system, wrong pool size can choke the DB under high load. Optimized pooling reduces latency dramatically.

346. How do you integrate Hibernate with event-driven systems like Kafka?

Answer

Hibernate itself is DB-focused, but in event-driven systems, you often need to publish events when DB changes happen.

Two patterns are common:

1. **Transactional Outbox Pattern**
    - Instead of publishing directly to Kafka inside the transaction (risky because DB commit and Kafka publish may go out of sync), you write events into an outbox table within the same DB transaction.
    - A separate service or Debezium (CDC) picks events from the outbox and pushes them to Kafka.

Example Outbox entity:

```
@Entity
public class OutboxEvent {
 @Id
 @GeneratedValue
 private Long id;

 private String aggregateType;
 private String payload;

 private LocalDateTime createdAt;
}
```

Workflow:

- Save Order + OutboxEvent in the same transaction.
- Kafka connector (Debezium) streams OutboxEvent rows to Kafka.

This guarantees **exactly-once** delivery between DB and Kafka.

2. **Post-Commit Listener**
    - Use @PostPersist or EventListener to send to Kafka.
    - Simpler but risk of inconsistency if Kafka fails after DB commit.

Real-world:
Most fintech and e-commerce systems use **Transactional Outbox** because it's resilient to failure.

347. How does Hibernate handle multi-threading, and what are common pitfalls?

Answer

A **Session in Hibernate is not thread-safe**. One session should only ever be used by one thread.

Common pitfalls:

1. Sharing the same Session across multiple threads → leads to LazyInitializationException or corrupted state.
2. Long-lived sessions with concurrent modifications → dirty checking issues.

Best practices:

- One Session per request (Spring does this with OpenEntityManagerInViewFilter).
- Use @Transactional boundaries properly.
- For background tasks, always create a fresh session.

If you need multi-threaded processing (like batch jobs), use StatelessSession or partition work across multiple sessions.

Analogy:
Think of a Hibernate Session like a “shopping cart” tied to one person. If multiple people use the same cart, items get mixed up and checkout fails.

348. What are the differences between Fetch Joins, Subselects, and Projections in Hibernate?

Answer

When optimizing queries, Hibernate gives multiple ways:

1. **Fetch Join**
    - Uses JOIN FETCH in JPQL to load associations in a single query.
2. SELECT d FROM Department d JOIN FETCH d.employees
    - Pros: avoids N+1 queries.
    - Cons: Can cause Cartesian explosion if not careful.
3. **Subselect Fetching**
    - Hibernate loads parent entities first, then uses one subselect to fetch all children.
4. SELECT e.* FROM employee e WHERE e.department_id IN (..parentIds..)
    - Useful when loading many parents + children at once.
5. **Projections (DTO Queries)**
    - Instead of fetching whole entities, fetch only required fields.
6. SELECT new com.example.EmployeeDTO(e.id, e.name)
7. FROM Employee e
    - Best for read-only APIs where full entity lifecycle isn't needed.

Rule of thumb:

- Fetch join → small data sets with necessary children.
- Subselect → when fetching many parent entities with children.
- Projections → performance-critical read APIs.

Real-world example:
In an e-commerce system, for the "product listing" page, you'd use projections (id, name, price) instead of fetching full Product entities with all associations.

349. What are best practices for using Hibernate in microservices?

Answer

1. **Keep schema per service**
    - Each microservice owns its DB schema. Don't share entities across services.
2. **DTOs instead of Entities in APIs**
    - Entities are internal; APIs should expose DTOs. Prevents leaking lazy-loaded fields or unintentional updates.
3. **Avoid Open Session in View in Microservices**
    - In monoliths, it helps lazy-loading outside services. In microservices, it causes performance traps. Always fetch needed data in service layer.
4. **Use Connection Pooling (HikariCP)**
    - Essential for scaling microservices with high traffic.
5. **Combine with Caching**
    - Use second-level cache (e.g., Redis) for reference data (products, categories).
6. **Events > Distributed Transactions**
    - Don't use Hibernate across multiple microservices with 2PC. Use Kafka/outbox pattern instead.

350. How do you debug Hibernate performance issues in production?

Answer

1. **Enable SQL logging**
    - hibernate.show_sql=true or use P6Spy for SQL logging.
2. **Check N+1 problem**
    - Look for excessive queries when loading lists. Solve with fetch joins or DTO projections.
3. **Profile with tools**
    - VisualVM, YourKit, or Flight Recorder for thread dumps and memory.
4. **Database monitoring**
    - Use slow query logs and APM tools (Datadog, New Relic, Grafana).
5. **Actuator Metrics** (Spring Boot + Micrometer)
    - Monitor connection pool usage, query counts, latency.

Real-world example:
In a payments system, a slow API was traced to an N+1 query where each transaction fetched its customer separately. Fixed with a join fetch, response time dropped from 2s → 200ms.

351. What are Hibernate Interceptors, and how are they different from Event Listeners?

Answer

Hibernate Interceptors allow you to hook into the entity lifecycle and perform custom logic. They sit between Hibernate and the database operations.

- **Interceptors**
  - Implement Interceptor interface or extend EmptyInterceptor.
  - Work at the Session level.
  - Can intercept operations like onSave, onFlushDirty, onDelete, onLoad.
  - Example: Audit logging every time an entity is updated.

```
public class AuditInterceptor extends EmptyInterceptor {
   @Override
   public boolean onFlushDirty(Object entity, Serializable id,
                  Object[] currentState, Object[] previousState,
                  String[] propertyNames, Type[] types) {
      System.out.println("Entity updated: " +
entity.getClass().getSimpleName());
 return false;
   }
}
```

- **Event Listeners**
  - Hibernate has a richer event system (PostInsertEventListener, PreUpdateEventListener).
  - They are more granular and tied to Hibernate's event model.
  - Example: Trigger business logic when an Order is persisted.

Key difference:

- Interceptors = broader, Session-level hooks.
- Event Listeners = fine-grained, event-specific callbacks.

352. What is the difference between StatelessSession and Session in Hibernate?

Answer

- **Session**
  - Default Hibernate session.
  - Maintains a first-level cache (Persistence Context).
  - Supports dirty checking, cascading, and lazy loading.
- **StatelessSession**
  - Lightweight session without cache.
  - No dirty checking or cascading.
  - Best for **bulk inserts/updates**.

Example: Batch insert using StatelessSession:

```
StatelessSession statelessSession = sessionFactory.openStatelessSession();
Transaction tx = statelessSession.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee emp = new Employee("User" + i);
   statelessSession.insert(emp);
}

tx.commit();
statelessSession.close();
```

Use case:

- Session → normal web requests where caching and lazy loading help.
- StatelessSession → ETL jobs, data migration, or imports where performance matters more than cache.

353. How does Hibernate support stored procedures and functions?

Answer

Hibernate can call stored procedures in two ways:

1. **Native SQL with createNativeQuery**

```
Query query = session.createNativeQuery("{CALL
getEmployeesByDept(:dept)}", Employee.class);
query.setParameter("dept", "IT");
List<Employee> employees = query.getResultList();
```

2. **Using @NamedStoredProcedureQuery**

```
@Entity
@NamedStoredProcedureQuery(
  name = "getEmployeesByDept",
  procedureName = "getEmployeesByDept",
  parameters = {
    @StoredProcedureParameter(mode = ParameterMode.IN, name = "dept",
type = String.class)
  },
  resultClasses = Employee.class
)
public class Employee { ... }
Call it via:
StoredProcedureQuery spQuery = session
    .createNamedStoredProcedureQuery("getEmployeesByDept")
    .setParameter("dept", "HR");
List<Employee> result = spQuery.getResultList();
```

Use cases:

- Legacy databases where business logic is in stored procs.
- Performance-heavy operations (batch salary calculations in payroll systems).

354. What are different caching concurrency strategies in Hibernate's second-level cache?

Answer

Hibernate second-level cache supports different concurrency strategies depending on use case:

1. **READ_ONLY**
    - For immutable data (lookup tables like countries, currencies).
    - Fastest since no updates are allowed.
2. **NONSTRICT_READ_WRITE**
    - For rarely updated data.
    - Allows stale data in some cases.
    - Example: Product catalog in e-commerce.
3. **READ_WRITE**
    - Ensures strong consistency with soft locks.
    - Higher overhead.
    - Example: User profiles.
4. **TRANSACTIONAL**
    - Works only with JTA transactions.
    - Guarantees full ACID consistency.
    - Rarely used, more enterprise-focused.

Configuration example (Ehcache):

```
<cache name="com.example.Employee"
    maxEntriesLocalHeap="1000"
    eternal="false"
    timeToIdleSeconds="300"
    timeToLiveSeconds="600"
    memoryStoreEvictionPolicy="LRU"/>
```

Real-world analogy:

- READ_ONLY = dictionary book (never changes).
- READ_WRITE = shared Google Doc with edit locks.

355. What is Hibernate multi-tenancy, and what strategies does it support?

Answer

Multi-tenancy in Hibernate allows multiple tenants (clients) to share the same application while keeping their data isolated.

Hibernate supports **4 strategies**:

1. **DATABASE**
    - Each tenant has its own physical database.
    - Pros: Strong isolation, easy backup.
    - Cons: Higher infra cost.
2. **SCHEMA**
    - Each tenant has a separate schema in the same database.
    - Example: tenant1.users, tenant2.users.
3. **DISCRIMINATOR (Shared Schema)**
    - A single schema, but tenant data is distinguished by a tenant column.
    - Example: users table with column tenant_id.
    - Best for SaaS with many small tenants.
4. **SEPARATE SESSION FACTORIES**
    - Each tenant has its own SessionFactory.
    - Flexible but consumes more memory.

Example (Discriminator strategy):

```
@Entity
@FilterDef(name = "tenantFilter", parameters = @ParamDef(name = "tenantId", type
= "string"))
@Filter(name = "tenantFilter", condition = "tenant_id = :tenantId")
public class Employee {
   @Id
   private Long id;
   private String name;
   private String tenantId;
}
```

Before queries:

```
session.enableFilter("tenantFilter").setParameter("tenantId", "tenant1");
```

In interviews:

- DATABASE/SCHEMA for **enterprise clients**.
- DISCRIMINATOR for **low-cost SaaS platforms**.

356. How does Hibernate handle schema generation and validation?

Answer

Hibernate can automatically create, update, validate, or drop database schemas based on entity mappings.

Controlled by property:

spring.jpa.hibernate.ddl-auto=update

Modes:

- **validate** → validates schema vs entity mappings.
- **update** → updates schema if needed.
- **create** → drops and recreates schema at startup.
- **create-drop** → drops schema at shutdown.
- **none** → do nothing.

Example use cases:

- create-drop → unit tests (fresh DB each run).
- update → local dev environment.
- validate → production (catch mismatches).

Pitfalls:

- Avoid update or create-drop in production (risk of data loss).
- Always manage schema with migration tools (Flyway, Liquibase).

357. How do you implement auditing/versioning with Hibernate Envers?

Answer

Hibernate Envers provides automatic auditing of entity changes (create, update, delete).

Steps:

1. Add Envers dependency.
2. Annotate entity with @Audited.

```
@Entity
@Audited
public class Employee {
  @Id
  @GeneratedValue
  private Long id;
  private String name;
}
```

Hibernate creates an audit table:

- employee → main table.
- employee_AUD → audit table with revision info.

Query audit history:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
Employee oldVersion = reader.find(Employee.class, 1L, 2); // version 2
```

Benefits:

- Tracks “who changed what and when.”
- Useful in banking, healthcare, and compliance-heavy domains.

358. How can you optimize queries in Hibernate?

Answer

Key strategies:

1. **Use projections (select specific fields)**
2. Query q = session.createQuery("SELECT e.name FROM Employee e");
3. **Batch fetching**
    - @BatchSize(size = 20)
    - Reduces N+1 problems.
4. **JOIN FETCH**
    - Fetch related entities in one query.
5. **Second-level cache + query cache**
    - Store frequent queries in cache.
6. **StatelessSession for bulk ops**
    - No cache overhead.
7. **DTO Projections (Constructor expressions)**
8. Query q = session.createQuery(
9. "SELECT new com.example.EmployeeDTO(e.id, e.name) FROM Employee e");

Rule of thumb:

- Load **only what you need**.
- Avoid fetching entire object graphs.
- Profile queries with Hibernate's SQL logs.

359. What are Hibernate filters, and how are they different from WHERE clauses?

Answer

Hibernate filters allow dynamic query conditions applied transparently at the session level.

Define filter:

```
@Entity
@FilterDef(name = "activeFilter", parameters = @ParamDef(name = "isActive", type = "boolean"))
@Filter(name = "activeFilter", condition = "is_active = :isActive")
public class Employee {
   @Id
   private Long id;
   private String name;
   private boolean isActive;
}
```

Enable filter at runtime:

```
session.enableFilter("activeFilter").setParameter("isActive", true);
List<Employee> employees = session.createQuery("from Employee").list();
```

Difference from WHERE clause:

- WHERE is **hardcoded** in query.
- Filter is **dynamic** and applied globally across all queries.
- Great for **multi-tenancy, soft deletes, security restrictions**.

360. How does Hibernate handle concurrency control?

Answer

Hibernate uses two main strategies:

1. **Optimistic Locking** (default)
    - Uses a @Version column.
    - Prevents lost updates.
    - Good when conflicts are rare.
2. **Pessimistic Locking**
    - Locks the row at DB level (SELECT FOR UPDATE).
    - Prevents conflicts but reduces concurrency.
    - Example: Booking system seat reservation.

Usage:

```
Employee emp = entityManager.find(Employee.class, 1L,
LockModeType.PESSIMISTIC_WRITE);
```

Interview tip:

- Optimistic → default for web apps.
- Pessimistic → financial apps (bank transfers, inventory systems).

361. What is the Hibernate Query Cache, and how does it work?

Answer

Hibernate's query cache stores the results of queries so repeated executions don't hit the database.

- It is **optional** and works on top of the **second-level cache**.
- Stores only **identifiers** of entities, not the entity data itself.
- Requires enabling both query cache and second-level cache.

Configuration:

```
hibernate.cache.use_query_cache=true
hibernate.cache.use_second_level_cache=true
```

Usage:

```
Query query = session.createQuery("FROM Employee WHERE department = :dept");
query.setParameter("dept", "IT");
query.setCacheable(true);
List<Employee> employees = query.list();
```

Best for queries that:

- Don't change often (read-heavy).
- Return the same result sets frequently (e.g., product catalog, lookup tables).

362. What are Natural IDs in Hibernate, and when should you use them?

Answer

A **natural ID** is a domain-specific unique identifier that comes from business logic (e.g., email, SSN, ISBN), unlike surrogate IDs (auto-generated primary keys).

Hibernate supports @NaturalId:

```
@Entity
public class User {
   @Id
   @GeneratedValue
   private Long id;

   @NaturalId
   private String email;
}

Fetching with natural ID:

User user = session.byNaturalId(User.class)
            .using("email", "john@example.com")
            .load();
```

Advantages:

- Expressive queries (no need for primary key).
- Hibernate automatically caches natural IDs in the second-level cache.

Use cases:

- Login via email.
- Library systems with ISBN.
- HR systems with employee code.

363. What is bytecode enhancement in Hibernate, and why is it used?

Answer

Bytecode enhancement modifies entity classes at build-time or runtime to add extra behavior.

Hibernate uses it for:

1. **Lazy loading of attributes** – fetch individual fields lazily.
2. **Dirty checking optimization** – track which fields actually changed.
3. **Bidirectional association management** – automatically synchronize both sides.

Enable with build-time enhancement plugin:

```
<plugin>
 <groupId>org.hibernate.orm.tooling</groupId>
 <artifactId>hibernate-enhance-maven-plugin</artifactId>
 <version>6.4.0.Final</version>
 <configuration>
  <enableLazyInitialization>true</enableLazyInitialization>
  <enableDirtyTracking>true</enableDirtyTracking>
  <enableAssociationManagement>true</enableAssociationManagement>
 </configuration>
</plugin>
```

Without enhancement, Hibernate relies on proxies (CGLIB/Javassist). With enhancement, entities behave more natively and perform better.

364. How does Hibernate handle database-specific features like stored procedures?

Answer

Hibernate allows calling stored procedures using:

1. **JPA @NamedStoredProcedureQuery**

```
@NamedStoredProcedureQuery(
 name = "getEmployeeByDept",
 procedureName = "get_employee_by_dept",
 resultClasses = Employee.class,
 parameters = {
  @StoredProcedureParameter(mode = ParameterMode.IN, name = "dept",
type = String.class)
 }
)
@Entity
public class Employee { ... }
```

Usage:

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("getEmployeeByDept");
query.setParameter("dept", "IT");
List<Employee> employees = query.getResultList();
```

2. **Session createStoredProcedureCall** (Hibernate API)

```
StoredProcedureQuery query =
session.createStoredProcedureCall("get_employee_by_dept",
Employee.class);
query.registerStoredProcedureParameter("dept", String.class,
ParameterMode.IN);
query.setParameter("dept", "HR");
List<Employee> employees = query.getResultList();
```

Use cases:

- Heavy DB logic (banking interest calculation, payroll).
- Performance-critical queries already optimized in DB.

365. What are Hibernate Event Listeners and Interceptors?

Answer

Hibernate provides hooks to intercept entity lifecycle events:

1. **Event Listeners** (more fine-grained, for specific events)

```
public class AuditListener {
  @PrePersist
  public void beforeInsert(Object entity) {
    System.out.println("Inserting: " + entity);
  }
}
```

Attach:

```
@EntityListeners(AuditListener.class)
@Entity
public class Employee { ... }
```

2. **Interceptors** (global, applied to all entities)

```
public class AuditInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id, Object[] state, String[]
propertyNames, Type[] types) {
    System.out.println("Entity saved: " + entity);
return false;
  }
}
```

Attach:

```
SessionFactory sf = new Configuration().setInterceptor(new
AuditInterceptor()).buildSessionFactory();
```

Use Event Listeners for entity-specific hooks, Interceptors for global logging, auditing, or soft deletes.

366. How do you integrate Hibernate with Spring Boot?

Answer

Spring Boot provides out-of-the-box Hibernate integration via **Spring Data JPA**.

Steps:

1. Add dependency:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

2. Configure datasource:

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=pass
spring.jpa.hibernate.ddl-auto=update
```

3. Define repository:

```
public interface EmployeeRepository extends
JpaRepository<Employee, Long> {}
```

4. Use in service:

```
@Service
public class EmployeeService {
   private final EmployeeRepository repo;
   public EmployeeService(EmployeeRepository repo) { this.repo = repo; }

   public List<Employee> findAll() { return repo.findAll(); }
```

```
}
```

Best practices:

- Use JpaRepository instead of raw SessionFactory.
- Keep Hibernate configs in application.yml.
- Use DTOs, not entities, in API responses.

367. How do you handle batch processing efficiently in Hibernate?

Answer

Batch processing is crucial when inserting or updating thousands of records (e.g., payroll generation, e-commerce order imports).

Best practices:

1. **Set batch size**

```
hibernate.jdbc.batch_size=30
```

2. **Flush and clear periodically**

```
for (int i = 0; i < employees.size(); i++) {
  session.save(employees.get(i));
  if (i % 30 == 0) {
    session.flush();
    session.clear();
  }
}
```

3. **Use StatelessSession for huge imports**

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();
for (Employee emp : employees) {
  session.insert(emp);
}
```

```
        tx.commit();
        session.close();
```

StatelessSession skips first-level cache, making it faster for raw inserts/updates.

4. **Disable second-level cache for batch jobs** (avoids polluting cache with temporary objects).

368. What are the differences between Hibernate's first-level cache, second-level cache, and query cache?

Answer

1. **First-Level Cache (Session Cache)**

- Always enabled, tied to Hibernate Session.
- Caches entities within a transaction.
- Example:

```
Employee e1 = session.get(Employee.class, 1); // DB hit
Employee e2 = session.get(Employee.class, 1); // From cache
```

2. **Second-Level Cache (SessionFactory Cache)**

- Shared across sessions.
- Requires explicit configuration (Ehcache, Redis, Infinispan).
- Good for read-heavy apps.

3. **Query Cache**

- Stores query result sets (only identifiers).
- Works with second-level cache.
- Useful for frequently repeated queries (e.g., product catalog).

In practice:

- Use first-level (always on).
- Enable second-level for reference data (roles, lookup tables).
- Use query cache carefully (avoid for dynamic queries).

369. How does Hibernate handle concurrency control?

Answer

Concurrency control ensures data consistency when multiple transactions access the same entity.

Hibernate strategies:

1. **Optimistic Locking (default)**

- Uses @Version field.
- Assumes conflicts are rare.
- On conflict → throws OptimisticLockException.

@Version

private int version;

2. **Pessimistic Locking**

- Uses database locks (SELECT ... FOR UPDATE).
- Prevents others from modifying until lock is released.

Employee emp = session.get(Employee.class, 1, LockOptions.UPGRADE);

3. **Isolation Levels (DB-level)**

- READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE.
- Set via connection pool or Spring transaction config.

Real-world:

- Banking system updates → pessimistic lock.
- Profile updates (rare conflicts) → optimistic lock.

370. What is a Detached entity in Hibernate, and how do you manage it?

Answer

A **detached entity** is one that was once managed by a session but is no longer associated (session closed or transaction ended).

Example:

```
Session session1 = factory.openSession();
Employee emp = session1.get(Employee.class, 1);
session1.close(); // emp becomes detached
```

Ways to reattach:

1. **update()** – assumes entity is unchanged in DB.

session.update(emp);

2. **merge()** – copies state of detached object into a managed one.

Employee mergedEmp = (Employee) session.merge(emp);

3. **saveOrUpdate()** – decides between insert and update.

Interview Tip:

- update() → throws exception if entity already exists in session.
- merge() → safer for detached entities.

371. What is the difference between lazy loading proxies and bytecode enhancement in Hibernate?

Answer

1. **Lazy Proxies (default)**

- Hibernate creates proxy objects for associations.
- When accessed, DB query is triggered.
- Example:

Employee emp = session.get(Employee.class, 1);

Department dept = emp.getDepartment(); // Proxy until accessed

2. **Bytecode Enhancement**

- Entities are enhanced at build-time or runtime to support lazy loading of **individual attributes** (not just associations).
- More efficient because it doesn't require proxies.
- Requires Maven/Gradle enhancement plugin.

In practice:

- Proxies are simpler, widely used.
- Enhancement is for advanced optimization (large entities, fine-grained lazy loading).

372. How do you prevent the N+1 query problem in Hibernate?

Answer

The N+1 problem happens when:

- 1 query fetches N entities.
- Each entity triggers another query for its associations.

Example:

```
List<Department> depts = session.createQuery("FROM Department").list();
for (Department d : depts) {
   System.out.println(d.getEmployees()); // N extra queries
}
```

Fixes:

1. **JOIN FETCH**

```
List<Department> depts = session.createQuery(
 "SELECT d FROM Department d JOIN FETCH d.employees",
Department.class).list();
```

2.

2. **Entity Graphs (@NamedEntityGraph)** – define fetch plans.
3. **Batch fetching (@BatchSize)**

```
@OneToMany
@BatchSize(size = 10)
private List<Employee> employees;
```

In real-world:

- JOIN FETCH → when always needed.
- Batch fetching → when conditionally needed.

373. What is multi-tenancy in Hibernate, and how is it implemented?

Answer

Multi-tenancy means running a single application serving multiple clients (tenants), each with its own data isolation. Hibernate supports different multi-tenancy strategies:

1. **Separate Database per Tenant**

- Each tenant has its own database.
- Hibernate switches DB connections dynamically.

2. **Separate Schema per Tenant**

- Same DB, different schemas for tenants.
- Hibernate changes schema at runtime.

3. **Shared Schema (Discriminator Column)**

- One schema, tenant column distinguishes records.

Implementation:

```
@Configuration
public class MultiTenantConfig {

  @Bean
  public MultiTenantConnectionProvider connectionProvider() {
    return new DataSourceBasedMultiTenantConnectionProviderImpl();
  }

  @Bean
  public CurrentTenantIdentifierResolver tenantResolver() {
    return () -> TenantContext.getTenantId();
  }
```

Example: In a SaaS e-commerce platform, each seller (tenant) has separate data.

374. How does Hibernate handle auditing, and what is Envers?

Answer

Hibernate Envers is a module that automatically maintains **audit history** of entities. It creates audit tables (e.g., employee_aud) to track inserts, updates, deletes.

Usage:

```
@Entity
@Audited
public class Employee {
   @Id
   @GeneratedValue
   private int id;
   private String name;
}
```

Hibernate generates audit logs automatically.
To fetch revisions:

```
AuditReader reader = AuditReaderFactory.get(entityManager);
Employee old = reader.find(Employee.class, 1, 2); // revision 2
```

Useful for banking, healthcare, and legal apps where **audit trails** are mandatory.

375. What are Hibernate interceptors and event listeners?

Answer

Hibernate provides hooks to intercept persistence operations.

1. **Interceptors** – callback interface.

```
public class AuditInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id, Object[] state, String[]
propertyNames, Type[] types) {
    System.out.println("Saving entity: " + entity);
    return true;
  }
}
```

Attach interceptor:

```
Session session = sessionFactory.withOptions().interceptor(new
AuditInterceptor()).openSession();
```

2. **Event Listeners** – fine-grained event system.

```
public class CustomEventListener implements PostInsertEventListener {
  @Override
  public void onPostInsert(PostInsertEvent event) {
    System.out.println("Inserted: " + event.getEntity());
  }
}
```

Use cases:

- Auditing changes.
- Logging SQL operations.
- Soft deletes.

376. What are caching concurrency strategies in Hibernate?

Answer
When using **second-level cache**, Hibernate provides strategies:

1. **Read-Only**

- For reference data (e.g., country codes).
- No updates allowed.

2. **Nonstrict-Read-Write**

- Cached data may be stale temporarily.
- Useful for less critical data.

3. **Read-Write**

- Uses timestamps to maintain consistency.
- Safe for frequently read but occasionally updated data.

4. **Transactional**

- Works with JTA transactions.
- Ensures strict consistency.

Example config:

<class-cache class="Employee" usage="read-write"/>

Best practice:

- Use **read-only** for lookups.
- Use **read-write** for critical entities with moderate updates.

377. What is Hibernate's Statistics API, and how is it used for monitoring?

Answer

Hibernate provides Statistics API to monitor performance (query count, cache hit ratio, entity load time).

Enable in config:

hibernate.generate_statistics=true

Access in code:

```
Statistics stats = sessionFactory.getStatistics();
System.out.println("Query count: " + stats.getQueryExecutionCount());
System.out.println("Cache hit count: " + stats.getSecondLevelCacheHitCount());
```

Helps identify:

- N+1 query issues.
- Cache hit/miss ratio.
- Slow queries.

In production, integrate with **Actuator + Micrometer** for real-time metrics.

378. How do you handle deadlocks in Hibernate?

Answer

Deadlocks occur when multiple transactions wait for each other's locks. Hibernate relies on DB for deadlock detection, but developers can minimize risk:

1. **Keep transactions short** – avoid holding locks too long.
2. **Consistent ordering** – always update tables/entities in the same order across transactions.
3. **Optimistic Locking** – reduce contention with @Version.
4. **Pessimistic Locking with timeout**

```
Employee emp = session.get(Employee.class, 1,
LockOptions.UPGRADE.setTimeOut(5000));
```

Example: In an e-commerce checkout, one thread updates Order, another updates Payment. To prevent deadlock, always update Order before Payment in both flows.

379. What are native SQL queries in Hibernate, and when should you use them?

Answer

Sometimes JPQL or Criteria API isn't enough. Hibernate allows **native SQL** queries.

Example:

```
List<Object[]> employees = session.createNativeQuery(
   "SELECT id, name FROM employee WHERE department_id = :dept")
   .setParameter("dept", 10)
   .list();

Mapping to entities:

@SqlResultSetMapping(
  name="EmployeeMapping",
  entities=@EntityResult(entityClass=Employee.class)
)
```

Use cases:

- Performance tuning (complex joins, DB-specific features).
- Stored procedures.
- Legacy DB integration.

Best practice: prefer JPQL for portability, use native SQL for edge cases.

380. How do you call stored procedures in Hibernate?

Answer

Hibernate supports stored procedures via JPA's @NamedStoredProcedureQuery or through EntityManager.createStoredProcedureQuery.

Using annotation:

```
@Entity
@NamedStoredProcedureQuery(
   name = "getEmployeesByDept",
   procedureName = "get_employees_by_dept",
   parameters = {
      @StoredProcedureParameter(mode = ParameterMode.IN, name = "deptId", type
= Integer.class)
   },
   resultClasses = Employee.class
)
public class Employee { ... }
```

Execution:

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("getEmployeesByDept");
query.setParameter("deptId", 10);
List<Employee> employees = query.getResultList();
```

Use case: Banking apps often use stored procedures for complex financial calculations.

381. How does Hibernate handle schema evolution (DDL auto-generation)?

Answer

Hibernate can generate or update DB schema automatically based on entity mappings. Controlled via hibernate.hbm2ddl.auto:

- create → drops and creates schema each startup.
- update → updates schema without dropping data.
- validate → checks schema matches entities, throws error if mismatch.
- none → no schema changes.

Example (application.properties):

spring.jpa.hibernate.ddl-auto=update

Best practice:

- Use validate in production (avoid accidental schema changes).
- Use update or create in dev/testing environments.

382. What is StatelessSession in Hibernate, and when would you use it?

Answer

StatelessSession is a lightweight session without a first-level cache or persistence context.

Example:

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee emp = new Employee("Emp" + i);
   session.insert(emp);
}

tx.commit();
session.close();
```

Differences from normal Session:

- No L1 cache → each operation hits DB directly.
- No dirty checking → must explicitly update.
- No cascading.

Best for **batch inserts/updates**, ETL jobs, or migration scripts where caching would waste memory.

383. How do you handle database-specific features with Hibernate?

Answer

Hibernate abstracts SQL, but sometimes you need DB-specific behavior.

1. **Dialect** → Hibernate uses Dialect classes (MySQLDialect, PostgreSQLDialect, etc.) to generate SQL. You can extend these for custom behavior.
2. **Custom SQL Functions**

```
MetadataBuilder builder = new
MetadataSources().getMetadataBuilder();
builder.applySqlFunction("group_concat", new
StandardSQLFunction("group_concat", StandardBasicTypes.STRING));
```

3. **Native Queries** → directly write DB-specific SQL.
4. **Stored Procedures** → for vendor-optimized logic.

Example: MySQL's GROUP_CONCAT or Postgres' JSONB features often require custom functions.

384. What is the difference between dynamic-update and dynamic-insert in Hibernate?

Answer

Both are Hibernate optimizations to generate SQL dynamically.

- @DynamicUpdate → updates only modified columns.

```
@Entity
@DynamicUpdate
public class Employee { ... }

If only name changes, Hibernate runs:

UPDATE employee SET name=? WHERE id=?;

instead of updating all columns.

    • @DynamicInsert → inserts only non-null columns.

@Entity
@DynamicInsert
public class Employee { ... }
```

If department is null, insert excludes it.

Reduces SQL overhead and improves performance, especially with wide tables.

385. How does Hibernate support reactive programming?

Answer
Hibernate introduced **Hibernate Reactive** (based on Mutiny + Vert.x) for non-blocking DB access.

- Normal Hibernate: synchronous JDBC.
- Hibernate Reactive: asynchronous, event-driven.

Example (Panache style in Quarkus):

```
Uni<Employee> emp = session.find(Employee.class, 1);
emp.onItem().invoke(e -> System.out.println("Employee: " + e.getName()));
```

Use cases:

- High-concurrency apps (chat systems, streaming services).
- When you want non-blocking I/O with RDBMS.

386. How do you implement soft deletes in Hibernate?

Answer
Instead of deleting rows physically, mark them as deleted.

Approaches:

1. **@SQLDelete**

```
@Entity
@SQLDelete(sql = "UPDATE employee SET deleted = true WHERE id =
?")
@Where(clause = "deleted = false")
public class Employee {
  @Id
  private int id;
  private String name;
  private boolean deleted = false;
}
```

2. **Event Listeners** – intercept delete events and replace with update.

Used in **banking and audit systems** where data must never be physically removed.

387. How does Hibernate support multi-tenancy, and what strategies are available?

Answer

Multi-tenancy = one app serving multiple customers (tenants), while isolating their data. Hibernate supports it via:

1. **Separate Database Strategy**
    - Each tenant has its own DB.
    - MultiTenantConnectionProvider selects DB per tenant.
2. **Separate Schema Strategy**
    - Same DB, different schemas.
    - Hibernate switches schema dynamically.
3. **Discriminator Strategy**
    - Same DB + schema, but tables have a tenant_id column.
    - Hibernate filters queries using tenant_id.

Example setup:

```
@Configuration
public class MultiTenantConfig {
 @Bean
 public MultiTenantConnectionProvider multiTenantConnectionProvider() {
  return new MyMultiTenantConnectionProvider();
 }

 @Bean
 public CurrentTenantIdentifierResolver tenantResolver() {
  return new MyTenantIdentifierResolver();
 }
}
```

Use case: SaaS apps (e.g., CRM software) where multiple companies share the same app but data is isolated.

388. How do you tune Hibernate for high-performance applications?

Answer

Key tuning practices:

1. **Batching**
    - hibernate.jdbc.batch_size=20
    - Use StatelessSession for bulk operations.
2. **Caching**
    - First-level (session) → default.
    - Second-level (EhCache, Redis).
    - Query cache for read-heavy queries.
3. **Fetch Strategies**
    - Prefer LAZY loading by default.
    - Use JOIN FETCH or @EntityGraph when needed.
4. **Connection Pool**
    - Use HikariCP (default in Spring Boot).
    - Tune max/min pool size.
5. **SQL Optimization**
    - Use projections instead of fetching full entities.
    - Use indexes in DB.

Example: For an e-commerce order service, use LAZY for orderItems, JOIN FETCH for reporting queries, and cache for product catalog.

389. What is database sharding, and how can Hibernate work with it?

Answer

Sharding = splitting data across multiple DB instances (shards) for scalability.

Hibernate strategies:

1. **Application-level routing**
    - App decides which shard to query based on shard key (e.g., userId % 4).
    - Multiple SessionFactories, one per shard.
2. **Sharding Frameworks**
    - Hibernate Shards (legacy).
    - External tools (e.g., Vitess, Citus, YugabyteDB).

Example routing:

```
SessionFactory shard1 = ...;
SessionFactory shard2 = ...;

int shardKey = userId % 2;
Session session = (shardKey == 0 ? shard1 : shard2).openSession();
```

Use case: Large-scale apps like LinkedIn or Amazon, where a single DB cannot handle billions of rows.

390. How does Hibernate integrate with Redis for caching?

Answer

Hibernate's 2nd-level cache can be backed by Redis to reduce DB load.

Steps:

1. Add dependency:

```
<dependency>
 <groupId>com.github.johnpoth</groupId>
 <artifactId>hibernate-redis</artifactId>
 <version>2.0.4</version>
</dependency>
```

2. Config in application.properties:

```
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=org.hibern
ate.cache.redis.RedisRegionFactory
spring.jpa.properties.hibernate.cache.redis.expiry=120
```

3. Annotate entity:

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product { ... }
```

Redis provides distributed caching, which is critical in microservices running across multiple nodes.

391. How does Hibernate manage concurrency in distributed systems?

Answer

Concurrency control ensures consistent data in multi-user environments.

Hibernate offers:

1. **Optimistic Locking** (@Version)
    - Uses version column.
    - Fails fast if conflict occurs.
2. **Pessimistic Locking**
    - Uses DB-level locks.
    - LockMode.PESSIMISTIC_WRITE.

Example:

```
Employee emp = session.get(Employee.class, 1, LockMode.PESSIMISTIC_WRITE);
```

3. **Clustered Environments**
    - Combine Hibernate with distributed caches (Redis, Infinispan).
    - Use messaging/event systems to propagate entity changes across nodes.

Example: A stock trading app prevents two brokers from selling the same stock simultaneously by applying optimistic/pessimistic locking.

392. What is query plan cache in Hibernate, and how does it work?

Answer

Query plan cache stores parsed SQL query plans so Hibernate doesn't re-parse JPQL/HQL every time.

Config:

```
hibernate.query.plan_cache_max_size=2048
hibernate.query.plan_parameter_metadata_max_size=128
```

Benefits:

- Faster query execution (skip query parsing overhead).
- Useful for high-traffic apps with many repeated queries.

Example: If an order service runs the same findByCustomerId query 10,000 times per minute, Hibernate caches the query plan instead of regenerating SQL.

393. How can Hibernate be integrated with GraphQL APIs?

Answer

GraphQL APIs often need flexible queries and nested entity fetching, which maps well to Hibernate's ORM capabilities.

Ways to integrate:

1. **Spring Boot + Hibernate + GraphQL (SPQR or GraphQL Java Tools):**
    - Define GraphQL schema (.graphqls or annotations).
    - Map schema resolvers to Hibernate repositories.
2. **N+1 Problem Handling:**
    - GraphQL can trigger many queries for nested fields.
    - Use **EntityGraph**, **JOIN FETCH**, or **DataLoader** to batch queries.

Example:

```
@Component
public class EmployeeGraphQLResolver implements GraphQLQueryResolver {
 @Autowired
 private EmployeeRepository repo;

 public List<Employee> getEmployees() {
  return repo.findAll();
 }
}
```

Useful for APIs that allow clients to request custom entity subsets (e.g., Netflix GraphQL APIs).

394. How do you implement query timeouts in Hibernate?

Answer

Hibernate allows setting timeouts at query or transaction level to prevent long-running queries.

1. **Query-level timeout:**

```
Query q = session.createQuery("FROM Employee");
q.setTimeout(5); // 5 seconds
```

2. **Transaction-level timeout:**

```
Transaction tx = session.beginTransaction();
tx.setTimeout(10); // 10 seconds
```

3. **Global configuration:**

```
hibernate.jdbc.timeout=30
```

Prevents database locks or slow queries from hanging the system. Essential in financial or trading systems.

395. What is Hibernate Envers, and how is auditing implemented?

Answer

Hibernate Envers = auditing framework that tracks historical changes of entities.

Setup:

1. **Add dependency:**

```
<dependency>
 <groupId>org.hibernate</groupId>
 <artifactId>hibernate-envers</artifactId>
</dependency>
```

2. **Annotate entity:**

```
@Entity
@Audited
public class Employee {
 @Id
 private int id;
 private String name;
}
```

3. **Query audit history:**

```
AuditReader reader = AuditReaderFactory.get(entityManager);
List<Number> revisions = reader.getRevisions(Employee.class, 1);
Employee oldEmp = reader.find(Employee.class, 1, revisions.get(0));
```

Example: In a banking system, Envers can track who modified account balances and when.

396. How can Hibernate be tuned for write-heavy applications?

Answer

For apps with heavy inserts/updates (e.g., logging, IoT systems):

1. **Batch inserts/updates:**
    - hibernate.jdbc.batch_size=50
    - Flush & clear in loops.
2. **StatelessSession:**
    - Faster because it skips caching and dirty checking.
3. **Disable second-level cache:**
    - Cache adds overhead for frequent writes.
4. **Use database bulk operations:**
    - session.createQuery("DELETE FROM Log").executeUpdate();

Example: IoT sensor data ingestion - millions of records/minute.

397. What is entity lifecycle in Hibernate?

Answer

Hibernate entities go through these **states**:

1. **Transient** – New object, not associated with session or DB.
2. **Persistent** – Attached to session, managed by Hibernate.
3. **Detached** – Was persistent, session closed, still exists in memory.
4. **Removed** – Marked for deletion.

Example:

```
Employee emp = new Employee("John"); // Transient
session.save(emp);              // Persistent
session.close();              // Detached
session.delete(emp);            // Removed
```

Understanding lifecycle helps optimize performance and avoid issues like LazyInitializationException.

398. How do you prevent LazyInitializationException in Hibernate?

Answer

Occurs when accessing a LAZY field outside of a session.

Solutions:

1. **Open Session in View (OSIV):** Keep session open till request ends.
2. **DTO Projection:** Fetch needed fields only.
3. **JOIN FETCH / EntityGraph:** Force eager fetch for specific queries.
4. **Initialize manually:**

Hibernate.initialize(emp.getProjects());

Best practice: use **DTOs or projections**, not OSIV (bad for scalability).

399. How does Hibernate handle immutable entities?

Answer

Hibernate supports **read-only/immutable entities** for performance.

1. Mark entity as immutable:

```
@Entity
@Immutable
public class Country { ... }
```

2. Mark session as read-only:

session.setDefaultReadOnly(true);

Benefits:

- No dirty checking.
- Faster reads.
- Useful for reference data (countries, currencies).

Example: A currency table in a payment system (USD, INR, EUR) never changes.

400. What are Natural IDs in Hibernate, and how are they different from primary keys?

Answer

A **natural ID** is a business key (like email, SSN, or PAN number) that uniquely identifies an entity apart from the primary key.

- **Primary Key (PK):**
  - Surrogate, usually auto-generated (@Id @GeneratedValue).
  - Internal to DB, not exposed to users.
- **Natural ID:**
  - User-facing unique attribute (e.g., email in User).
  - Declared with @NaturalId.
  - Faster lookups via caching.

Example:

```
@Entity
public class User {
 @Id
 @GeneratedValue
 private Long id;

 @NaturalId
 @Column(unique = true)
 private String email;
}
```

Lookup:

```
User user = session.byNaturalId(User.class)
            .using("email", "abc@example.com")
            .load();
```

PK is technical, Natural ID is business-driven. Both can co-exist.

401. What are Hibernate Event Listeners, and when would you use them?

Answer

Hibernate provides **event system hooks** to intercept entity lifecycle events.

Types of events:

- pre-insert, post-insert
- pre-update, post-update
- pre-delete, post-delete
- load, save

Use cases:

- Auto-populating audit fields.
- Soft deletes (mark deleted instead of actual delete).
- Logging entity changes.

Example:

```
public class AuditListener {
  @PrePersist
  public void beforeInsert(Object entity) {
    System.out.println("Inserting: " + entity);
  }
}
```

Annotation:

```
@EntityListeners(AuditListener.class)
@Entity
public class Employee { ... }
```

Great for cross-cutting concerns (auditing, logging, soft delete).

402. How does Hibernate handle schema validation and auto-generation?

Answer

Hibernate can auto-create or validate database schema using hbm2ddl.auto.

Options:

- validate → Validate schema against entity mappings.
- update → Update schema without dropping data.
- create → Drops + recreates schema on startup.
- create-drop → Drops schema at shutdown.
- none → Do nothing.

Example in application.properties:

```
spring.jpa.hibernate.ddl-auto=update
```

In **production**, use validate.

Use create or update only in **development/testing**.

403. What are Hibernate Interceptors, and how do they differ from Listeners?

Answer

- **Interceptors:**
  - Implement Interceptor interface.
  - Intercept operations at the **session level**.
  - Used for logging SQL, auditing, multi-tenancy.
- **Listeners:**
  - Annotated methods triggered at **entity lifecycle events**.
  - Bound to entity, not session.

Example Interceptor:

```
public class AuditInterceptor extends EmptyInterceptor {
 @Override
 public boolean onSave(Object entity, Serializable id, Object[] state,
                String[] propertyNames, Type[] types) {
  System.out.println("Entity saved: " + entity);
  return super.onSave(entity, id, state, propertyNames, types);
 }
}

Register:

Session session = sessionFactory
            .withOptions()
            .interceptor(new AuditInterceptor())
            .openSession();
```

Listeners = entity lifecycle; **Interceptors = session/global operations**.

404. How does Hibernate support multi-tenancy?

Answer

Multi-tenancy = multiple clients (tenants) sharing one app but isolated data.

Hibernate supports 3 strategies:

1. **DATABASE** → Each tenant has separate DB.
2. **SCHEMA** → Same DB, separate schemas.
3. **DISCRIMINATOR** → Same table, tenant column filters rows.

Setup:

```
hibernate.multiTenancy=DATABASE
hibernate.tenant_identifier_resolver=myTenantResolver
hibernate.multi_tenant_connection_provider=myConnectionProvider
```

Example:

```
public class MyTenantResolver implements CurrentTenantIdentifierResolver {
  public String resolveCurrentTenantIdentifier() {
    return TenantContext.getTenantId(); // dynamic per request
  }
}
```

Used in SaaS products (e.g., Shopify, Salesforce).

405. How can Hibernate be optimized for read-heavy applications?

Answer

For reporting/analytics systems:

1. **Second-level cache** – Use Ehcache, Redis, or Infinispan.
2. **Query cache** – Cache frequently executed queries.
3. **Stateless sessions** – For pure read-only bulk queries.
4. **DTO projections** – Avoid fetching full entities when not required.

```
List<UserDTO> users = session.createQuery(
 "SELECT new com.app.UserDTO(u.id, u.name) FROM User u")
 .list();
```

5. **Batch fetching** – Prefetch associations using @BatchSize.
6. **Read-only transactions** – @Transactional(readOnly = true) disables dirty checking.

Example: A stock trading dashboard where data is queried millions of times but rarely updated.

406. What are Hibernate cache concurrency strategies, and when should you use each?

Hibernate second-level cache supports different concurrency strategies that define how cached entities are updated in multi-transaction environments:

1. **READ_ONLY**
    - Best for reference/lookup data (e.g., country codes, product categories).
    - Data never changes, so cache stays consistent.
2. **NONSTRICT_READ_WRITE**
    - Suitable for rarely modified data.
    - Cache may be temporarily inconsistent with DB.
    - Example: product catalog where updates are infrequent.
3. **READ_WRITE**
    - Ensures strong consistency using a “soft lock” mechanism.
    - Safe for frequently updated data but slower due to synchronization overhead.
4. **TRANSACTIONAL**
    - Requires JTA and supports distributed transactions.
    - Used in highly transactional systems (e.g., banking).

Example config (EhCache):

<class-cache class="com.example.Employee" usage="read-write"/>

407. How can you prevent LazyInitializationException in Hibernate?

This exception occurs when a lazy association is accessed **outside the persistence context** (e.g., after session is closed).

Ways to handle:

1. **Open Session in View (OSIV)**
    - Keep the session open until the view is rendered.
    - Easy but risky (may fetch too much data).
2. **Explicit fetching with JOIN FETCH**
    - Define queries that eagerly fetch required associations.

```
@Query("SELECT d FROM Department d JOIN FETCH
d.employees WHERE d.id = :id")
Department findByIdWithEmployees(@Param("id") int id);
```

    -
3. **Entity Graphs**
    - Define custom fetch plans.
4. **DTO Projection**
    - Map results directly to DTOs in repository queries.

Best practice: prefer **DTO projections or JOIN FETCH** instead of OSIV for APIs.

408. What is Hibernate Envers, and how does it support auditing?

Hibernate **Envers** is a module that provides automatic **audit/history tracking** of entity changes.

- Each entity modification (insert, update, delete) is stored in a separate audit table (<entity>_AUD).
- Tracks *who changed what and when*.

Example:

```
@Entity
@Audited
public class Employee {
  @Id
  private int id;
  private String name;
  private String department;
}
```

Hibernate creates a table Employee_AUD with revision info.

Use cases:

- Compliance (finance, healthcare).
- Debugging and rollback (see old versions).
- Change tracking in enterprise systems.

409. What is the difference between Session and StatelessSession in Hibernate?

1. **Session**
    - Full-featured Hibernate persistence context.
    - Supports **first-level cache**, dirty checking, lazy loading, and lifecycle management.
    - Example:
    - Session session = sessionFactory.openSession();
    - session.beginTransaction();
    - Employee emp = session.get(Employee.class, 1); // cached in session
    - session.getTransaction().commit();
    - session.close();
2. **StatelessSession**
    - Lightweight version, skips caching and lifecycle management.
    - No first-level cache, no automatic dirty checking.
    - Best for **batch operations** (bulk insert/update/delete).
    - Example:
    - StatelessSession sSession = sessionFactory.openStatelessSession();
    - Transaction tx = sSession.beginTransaction();
    - for (int i = 0; i < 1000; i++) {
    - Employee emp = new Employee("Emp " + i);
    - sSession.insert(emp);
    - }
    - tx.commit();
    - sSession.close();

Use Session for normal business workflows, StatelessSession for **high-volume batch processing**.

410. How does Hibernate support event listeners, and why are they useful?

Hibernate provides an **event system** to intercept entity lifecycle events (insert, update, delete, load, flush, etc.).

- **Common use cases:** auditing, logging, validation, soft deletes.

Example – Pre-insert listener:

```
public class AuditListener implements PreInsertEventListener {
   public boolean onPreInsert(PreInsertEvent event) {
      Object entity = event.getEntity();
      if (entity instanceof Auditable) {
         ((Auditable) entity).setCreatedAt(LocalDateTime.now());
      }
      return false; // continue insert
   }
}

Registered via config:

<property name="hibernate.ejb.event.pre-insert"
value="com.example.AuditListener"/>
```

This helps add cross-cutting concerns like **timestamps, user tracking, logging** without modifying entity code.

411. What are Hibernate Projections, and how are they used?

Sometimes, you don't need the whole entity, just selected fields. Hibernate Projections allow **partial object retrieval**.

1. **Using Criteria API with Projections:**

```
Criteria criteria = session.createCriteria(Employee.class)
   .setProjection(Projections.property("name"));
List<String> names = criteria.list();
```

2. **JPQL constructor expressions:**

```
@Query("SELECT new com.example.dto.EmployeeDTO(e.id, e.name)
FROM Employee e")
List<EmployeeDTO> findEmployeeSummaries();
```

3. **Native SQL projection:**

```
@Query(value = "SELECT id, name FROM employee", nativeQuery = true)
List<Object[]> findEmployeeNames();
```

Benefits:

- Reduces data transfer from DB.
- Improves performance in **list APIs** where full entity isn't needed.
- Prevents LazyInitialization issues by selecting only required data.

412. What is the difference between native SQL, HQL, and Criteria API in Hibernate?

1. **Native SQL**
    - Direct SQL queries written in database dialect.
    - Full control, but not portable across DB vendors.
    - Example:
    - List<Object[]> result = session.createSQLQuery("SELECT * FROM employee").list();
2. **HQL (Hibernate Query Language)**
    - Object-oriented query language, works on entities not tables.
    - Portable across databases.
    - Example:
    - Query<Employee> query = session.createQuery("FROM Employee e WHERE e.name = :name", Employee.class);
    - query.setParameter("name", "John");
    - List<Employee> employees = query.list();
3. **Criteria API**
    - Programmatic and type-safe query building.
    - Useful for dynamic queries.
    - Example:
    - CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    - CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
    - Root<Employee> root = cq.from(Employee.class);
    - cq.select(root).where(cb.equal(root.get("name"), "John"));
    - List<Employee> employees = entityManager.createQuery(cq).getResultList();

In interviews, you can say:

- Use **HQL** for standard queries,
- **Criteria** for dynamic filters,
- **Native SQL** for DB-specific optimizations.

413. What is the difference between Session.save() and Session.persist() in Hibernate?

1. **save()**
    - Returns the generated identifier immediately.
    - Can be used **outside of a transaction** (though not recommended).
    - May trigger **INSERT immediately** depending on flush mode.
    - Example:
    - int id = (int) session.save(new Employee("John"));
2. **persist()**
    - Doesn't return identifier.
    - JPA-compliant method, must be used **inside a transaction**.
    - Guarantees that the entity is managed and inserted during flush/commit.
    - Example:
    - session.persist(new Employee("John"));

Best practice:

- Prefer persist() in **JPA-compliant apps**.
- Use save() in legacy Hibernate code when you need the ID quickly.

414. How does Hibernate handle composite primary keys?

Sometimes a table has a **composite key** (multiple columns). Hibernate supports this with @Embeddable or @IdClass.

1. **Using @Embeddable**

```
@Embeddable
public class OrderId implements Serializable {
  private int orderId;
  private int productId;
}
@Entity
public class Order {
  @EmbeddedId
  private OrderId id;
  private int quantity;
}
```

2. **Using @IdClass**

```
@IdClass(OrderId.class)
@Entity
public class Order {
  @Id private int orderId;
  @Id private int productId;
  private int quantity;
}
```

Key differences:

- @EmbeddedId: better for reusable composite keys, keeps them encapsulated.
- @IdClass: simpler but requires duplication of fields in both entity and IdClass.

415. What is the difference between Session.get() and Session.load() in Hibernate?

1. **get()**
    - Immediately hits the database and returns the actual object or null if not found.
    - Example:
    - Employee emp = session.get(Employee.class, 1);
    - if (emp == null) System.out.println("Employee not found");
2. **load()**
    - Returns a **proxy object** without hitting the DB immediately.
    - If you access a property, then Hibernate queries the DB (lazy).
    - Throws ObjectNotFoundException if no record exists.
    - Example:
    - Employee emp = session.load(Employee.class, 1);
    - System.out.println(emp.getName()); // triggers DB query

Best practice:

- Use get() when you need to check existence safely.
- Use load() when you are sure the entity exists and want lazy behavior.

416. What is dirty checking in Hibernate?

- Hibernate automatically detects changes in entity objects that are in a persistent state and updates them during flush() or commit().
- You don’t need to call update() explicitly for every modification.

Example:

```
Employee emp = session.get(Employee.class, 1);
emp.setName("Updated Name");  // Hibernate tracks this change
session.getTransaction().commit();  // issues UPDATE automatically
```

Benefits:

- Reduces boilerplate code.
- Ensures the DB stays in sync with entity state.
- Uses minimal SQL (only updates changed fields if configured with @DynamicUpdate).

417. What are cascading types in Hibernate, and when should you use them?

Cascading defines how operations on a parent entity affect child entities.

1. **CascadeType.PERSIST** → Saving a parent also saves children.
2. **CascadeType.MERGE** → Updating a parent also updates children.
3. **CascadeType.REMOVE** → Deleting a parent deletes children.
4. **CascadeType.REFRESH** → Refreshing a parent also refreshes children.
5. **CascadeType.DETACH** → Detaches both parent and children from persistence context.
6. **CascadeType.ALL** → Applies all operations above.

Example:

```
@Entity
public class Department {
   @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
   private List<Employee> employees;
}
```

Use cases:

- Cascade PERSIST for master-detail inserts (Order → Items).
- Cascade REMOVE for parent-child deletion (Department → Employees).
- Avoid overusing ALL blindly, as it may delete unintended data.

418. What are the different states of an object in Hibernate?

Hibernate entities can be in 4 main states during their lifecycle:

1. **Transient** → Newly created object, not associated with any session, no database record.

```
Employee emp = new Employee();
emp.setName("John");
// Not saved yet → transient
```

2. **Persistent** → Associated with a session, tracked by Hibernate. Changes are automatically synchronized with the database (dirty checking).

```
session.save(emp);  // now persistent
```

3. **Detached** → Once persistent, but now the session is closed or the object is explicitly detached. Hibernate no longer tracks it.

```
session.close();
// emp is now detached
```

4. **Removed** → Object is marked for deletion but not yet deleted until commit.

```
session.delete(emp);
```

Knowing these states is critical to understand caching, updates, and lazy loading.

419. What is the difference between merge() and update() in Hibernate?

1. **update()**
    - Reattaches a detached object to the session.
    - If the session already has another instance with the same ID, it throws a NonUniqueObjectException.
    - Example:
    - Employee emp = session1.get(Employee.class, 1);
    - session1.close(); // now detached
    - session2.update(emp); // reattached to session2
2. **merge()**
    - Copies the state of the detached object into a new persistent object inside the session.
    - Does not throw an exception if another instance with the same ID already exists.
    - Example:
    - Employee emp = session1.get(Employee.class, 1);
    - session1.close();
    - session2.merge(emp); // safer alternative to update

Rule of thumb:

- Use update() if you are sure no duplicate session instances exist.
- Use merge() when working with detached objects from web layers or APIs (safer).

420. How does Hibernate manage database transactions?

- Hibernate integrates with **JDBC transactions** and **JTA (Java Transaction API)** depending on the environment.
- Commonly, you start and commit transactions via Session.

Example with JDBC transaction (standalone app):

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Employee emp = new Employee("Alice");
session.save(emp);

tx.commit();  // commits to DB
session.close();
```

Example with Spring (declarative transaction):

```
@Service
public class EmployeeService {
   @Transactional
   public void saveEmployee(Employee emp) {
      repo.save(emp);  // Hibernate handles transaction behind the scenes
   }
}
```

Key points:

- Without transactions, Hibernate executes auto-commit after each SQL → inefficient.
- Transactions ensure **ACID** properties (atomicity, consistency, isolation, durability).
- Rollback occurs automatically if an exception is thrown.

421. What are the different fetching strategies in Hibernate?

Hibernate provides multiple strategies to fetch associated entities:

1. **Lazy Loading (default)**
    - Data is loaded only when accessed.
    - Example:
    - @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
    - private List<Employee> employees;
    - Pros: Saves memory and queries when associations are not needed.
    - Cons: Can cause LazyInitializationException if accessed outside a session.
2. **Eager Loading**
    - Data is fetched immediately with the parent entity.
    - Example:
    - @OneToMany(mappedBy = "department", fetch = FetchType.EAGER)
    - private List<Employee> employees;
    - Pros: No lazy init issues.
    - Cons: Can load huge amounts of data unnecessarily.
3. **Fetch Joins** (JPQL/HQL)
    - Fetch associations in one query.
    - Example:
    - SELECT d FROM Department d JOIN FETCH d.employees

Best practice: Default to **Lazy**, use **Join Fetch/Entity Graphs** for optimization.

422. What are composite keys in Hibernate, and how are they mapped?

Composite keys = primary keys made up of multiple columns.

Hibernate supports two approaches:

1. **@Embeddable with @EmbeddedId**

```
@Embeddable
public class OrderId implements Serializable {
   private int orderId;
   private int productId;
}

@Entity
public class Order {
   @EmbeddedId
   private OrderId id;
   private int quantity;
}
```

2. **@IdClass**

```
@IdClass(OrderId.class)
@Entity
public class Order {
   @Id
   private int orderId;
   @Id
   private int productId;
   private int quantity;
}
```

Difference:

- **@EmbeddedId** keeps composite key fields grouped in an object.
- **@IdClass** keeps them directly in the entity.

Use @EmbeddedId when keys logically belong together (like orderId + productId).

423. What is the difference between load() and get() in Hibernate?

1. **get()**
    - Immediately hits the database and returns the entity.
    - If not found → returns null.
    - Example:
    - Employee emp = session.get(Employee.class, 1);
2. **load()**
    - Returns a proxy object (lazy), database hit happens only when a property is accessed.
    - If not found → throws ObjectNotFoundException.
    - Example:
    - Employee emp = session.load(Employee.class, 1);

Rule of thumb:

- Use get() when you actually need the data.
- Use load() when you're sure the object exists and want performance benefits via proxies.

424. What are native SQL queries in Hibernate, and when should you use them?

Hibernate normally generates SQL from HQL/JPQL, but sometimes you need **database-specific SQL** for performance tuning or features not supported by HQL. That's where native SQL queries come in.

Example:

```
List<Object[]> employees = session
   .createNativeQuery("SELECT id, name FROM employee WHERE department =
:dept")
   .setParameter("dept", "IT")
   .getResultList();

for (Object[] row : employees) {
   System.out.println("ID: " + row[0] + ", Name: " + row[1]);
}
```

You can also map results directly to entities:

```
List<Employee> employees = session
   .createNativeQuery("SELECT * FROM employee WHERE department = :dept",
Employee.class)
   .setParameter("dept", "IT")
   .getResultList();
```

When to use:

425. What are the differences between save(), persist(), and merge() in Hibernate?

1. **save()**
    - Returns generated identifier immediately.
    - Works outside of a transaction (though not recommended).
    - Not strictly JPA-compliant.

    ```
    int id = (Integer) session.save(emp);
    ```

2. **persist()**
    - Does not return an ID.
    - Only makes the entity managed (saved during flush/commit).
    - JPA standard method.

    ```
    entityManager.persist(emp);
    ```

3. **merge()**
    - Updates a detached entity by copying its state into a managed one.
    - Returns a new managed instance.

    ```
    Employee managedEmp = (Employee) session.merge(detachedEmp);
    ```

Quick analogy (think of editing a Google Doc):

- **save()** = create a new doc immediately.
- **persist()** = mark the doc to be saved when you hit “commit”.
- **merge()** = upload an old copy, system merges changes into the latest version.

426. What is the role of SessionFactory in Hibernate?

- **SessionFactory** is a heavyweight, thread-safe object that creates and manages **Session** objects.
- Built once at application startup and reused.

Responsibilities:

1. Hold database connection details and ORM mappings.
2. Provide caching (second-level).
3. Create **Session** objects for CRUD operations.

Example:

```
SessionFactory factory = new Configuration()
   .configure("hibernate.cfg.xml")
   .buildSessionFactory();

Session session = factory.openSession();
```

Best practice:

- Create only one SessionFactory per database in the entire application.
- Use it as a singleton or manage via Spring.

427. What is the difference between openSession() and getCurrentSession() in Hibernate?

1. **openSession()**
    - Creates a brand-new Session each time you call it.
    - You need to explicitly close it after use.
    - Not tied to any transaction context.

```
        Session session = sessionFactory.openSession();
        session.beginTransaction();
        // work
        session.getTransaction().commit();
    session.close();
```

2. **getCurrentSession()**
    - Returns a session **bound to the current transaction** (thread-local).
    - Automatically closed when the transaction ends.
    - Requires hibernate.current_session_context_class=thread (or managed by Spring).

```
Session session = sessionFactory.getCurrentSession();
session.beginTransaction();
// work
session.getTransaction().commit(); // session auto-closes
```

In practice:

- Use getCurrentSession() in most enterprise apps (less boilerplate, works with Spring transactions).
- Use openSession() for short-lived, independent sessions.

428. What is lazy loading in Hibernate, and how does it work internally?

Lazy loading means Hibernate loads associated entities **only when accessed**, not at the time of parent fetch.

Example:

```
@Entity
public class Department {
 @Id
 private int id;

 @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
 private List<Employee> employees;
}
```

Usage:

```
Department dept = session.get(Department.class, 1);
System.out.println(dept.getName());  // employees not fetched yet
System.out.println(dept.getEmployees().size()); // triggers query for employees
```

Internally:

- Hibernate uses **proxies** (subclass objects) for lazy fields.
- When you call a getter, it fires a SQL query to load data.

Gotcha:

- If you access lazy fields **outside of an active session**, you'll hit LazyInitializationException.
- Fixes:
    - Use DTOs with fetch joins.
    - Keep session open (OpenSessionInView pattern).
    - Use JOIN FETCH when you know you'll need the data.

429. What is the difference between get() and load() in Hibernate?

1. **get()**
    - Immediately hits the database.
    - Returns the actual object or null if not found.

    Employee emp = session.get(Employee.class, 1);

2. **load()**
    - Returns a **proxy object** first (lazy initialization).
    - Only hits the DB when a property is accessed.
    - Throws ObjectNotFoundException if entity doesn't exist.

    Employee emp = session.load(Employee.class, 1);

When to use:

- **get()** → when you are not sure if the entity exists.
- **load()** → when you are sure it exists and want lazy loading for performance.

430. What is the difference between save(), persist(), and saveOrUpdate() in Hibernate?

1. **save()**
    - Inserts a new record immediately.
    - Returns the generated identifier.
    - Can be called outside of a transaction (not recommended).

    ```
    int id = (int) session.save(new Employee("John"));
    ```

2. **persist()**
    - JPA-standard method.
    - Doesn't return an ID.
    - Entity becomes managed, but the actual insert happens only when the transaction is committed or flushed.

    ```
    session.persist(new Employee("John"));
    ```

3. **saveOrUpdate()**
    - If the object is new → inserts it.
    - If it already exists (has an identifier) → updates it.

    ```
    session.saveOrUpdate(existingEmployee);
    ```

Rule of thumb:

- Use persist() in JPA-based apps.
- Use saveOrUpdate() when syncing detached objects.
- Avoid mixing them randomly.

431. What is dirty checking in Hibernate?

Dirty checking is Hibernate's mechanism to detect changes in managed entities and automatically update them in the database.

Example:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Employee emp = session.get(Employee.class, 1);
emp.setName("Updated Name");  // no explicit update call

tx.commit(); // Hibernate auto-generates UPDATE query
session.close();
```

How it works internally:

- Hibernate maintains a **snapshot** of the entity when it's loaded.
- On flush()/commit(), it compares the snapshot with the current state.
- If there's a difference → Hibernate issues an UPDATE.

Benefits:

- Less boilerplate (no need to explicitly call update).
- Keeps persistence context consistent with the DB.

432. What is the difference between merge() and update() in Hibernate?

1. **update()**
    - Reattaches a detached object to the current session.
    - If another session already has the same entity loaded, it throws NonUniqueObjectException.

    session.update(detachedEmployee);

2. **merge()**
    - Creates a copy of the detached object inside the current session.
    - Safe to use when you're not sure if the entity is already loaded in the session.

    Employee managed = (Employee) session.merge(detachedEmployee);

In interviews:

- **update()** is faster but risky (fails if duplicates exist in session).
- **merge()** is safer, as it avoids exceptions by copying state into a managed entity.

433. What is the difference between Session and SessionFactory in Hibernate?

1. **SessionFactory**
   - Heavyweight object created once per application.
   - Thread-safe, designed to be shared.
   - Responsible for creating Session instances.
   - Typically configured at startup (via hibernate.cfg.xml or persistence.xml).

```
SessionFactory factory = new
Configuration().configure().buildSessionFactory();
```

2. **Session**
   - Lightweight, non-thread-safe object.
   - Represents a single unit of work with the DB.
   - Holds the first-level cache and persistence context.
   - Opened and closed per transaction/request.

```
Session session = factory.openSession();
```

Analogy: SessionFactory is like a **connection pool manager**, while Session is like a **single DB connection for a user request**.

434. What are the different entity states in Hibernate?

Hibernate entities go through four lifecycle states:

1. **Transient** – newly created, not associated with any session.

   Employee emp = new Employee("John"); // transient

2. **Persistent** – associated with an open session, tracked for changes.

   session.save(emp); // persistent

3. **Detached** – was persistent earlier, but session is closed.

   session.close(); // emp is now detached

4. **Removed** – marked for deletion in the session, but row isn't deleted until commit.

   session.delete(emp); // removed state

Key to note: Hibernate auto-syncs persistent objects with the DB, but detached ones must be merged/updated.

435. What is the difference between get() and load() in Hibernate?

1. **get()**
    - Immediately hits the database.
    - Returns null if entity doesn't exist.
    - Safe for optional lookups.

    Employee emp = session.get(Employee.class, 1);

2. **load()**
    - Returns a proxy without hitting DB initially.
    - Only fetches when a property is accessed (lazy).
    - Throws ObjectNotFoundException if entity doesn't exist.

    Employee emp = session.load(Employee.class, 1);

In interviews:

- Use get() when you need the actual object immediately.
- Use load() when you want lazy loading for performance.

436. What are Hibernate proxies, and how do they work?

- Hibernate often uses **proxy objects** for lazy loading.
- A proxy is a subclass generated at runtime using CGLIB/Javassist that looks like your entity but doesn't actually have data until you access it.
- Example:

```
Employee emp = session.load(Employee.class, 1);
// At this point, emp is a proxy, DB not hit yet.
System.out.println(emp.getName()); // triggers DB query
```

- Proxies allow Hibernate to avoid unnecessary queries.
- Pitfall: If the session is closed before accessing the proxy, you'll get a LazyInitializationException.

Analogy: A proxy is like a "hollow placeholder" ticket you buy. Only when you present it at the counter, you actually get the real movie seat details.

437. What is the difference between first-level and second-level cache in Hibernate?

1. **First-Level Cache**
    - Built into Hibernate, mandatory.
    - Session-scoped (lives as long as the session).
    - Avoids multiple selects for the same entity in one session.

    Employee e1 = session.get(Employee.class, 1); // DB hit

    Employee e2 = session.get(Employee.class, 1); // cached in session

2. **Second-Level Cache**
    - Optional, configurable via providers (Ehcache, Redis, Infinispan).
    - SessionFactory-scoped, shared across sessions.
    - Useful for read-heavy data like product catalogs.

Quick summary:

- First-level = per session, always there.
- Second-level = across sessions, optional.

438. What is Hibernate's Query Cache, and how is it different from Second-Level Cache?

- Query Cache stores the results of HQL/SQL queries, not entities.
- Works in combination with the second-level cache, because query results still reference entities.
- Example:

```
<property name="hibernate.cache.use_query_cache" value="true"/>
Query query = session.createQuery("FROM Employee WHERE
department = :dept");
query.setParameter("dept", "IT");
query.setCacheable(true);
List<Employee> employees = query.list();
```

- Next time the same query is run with the same parameters, results come from the query cache.

Difference:

- Second-level cache = entities & collections.
- Query cache = result sets of queries.

439. What is the difference between session.get() and session.load() in Hibernate?

- **get()**
  - Immediately hits the database.
  - Returns the actual entity or null if not found.
  - Example:

```
Employee emp = session.get(Employee.class, 1);
if (emp == null) {
   System.out.println("No employee found");
}
```

- **load()**
  - Returns a **proxy object** without hitting the DB immediately.
  - DB is accessed only when a method on the proxy is called.
  - Throws ObjectNotFoundException if entity doesn't exist.
  - Example:

```
Employee emp = session.load(Employee.class, 1);
System.out.println(emp.getName()); // DB hit happens here
```

Rule of thumb:

- Use get() when you are unsure if the entity exists.
- Use load() when you are certain the entity exists and want lazy fetching.

440. What is the difference between save() and persist() in Hibernate?

- **save()**
  - Returns the generated identifier immediately.
  - Can be called outside of a transaction (not recommended).
  - Not strictly JPA compliant.
  - Example:

    ```
    int id = (int) session.save(new Employee("John"));
    ```

- **persist()**
  - Does not return the identifier.
  - Must be called within an active transaction.
  - JPA-compliant method.
  - Example:

    ```
    session.persist(new Employee("John"));
    ```

Difference in practice:

- save() = Hibernate-specific, returns ID.
- persist() = JPA standard, safer in portable code.

441. What is the difference between update(), merge(), and saveOrUpdate() in Hibernate?

- **update()**
  - Reattaches a **detached entity** to the session.
  - Throws exception if the entity with the same ID already exists in the session.

  session.update(emp); // emp must be detached

- **merge()**
  - Copies state of the detached entity into a **new persistent instance**.
  - Safer, doesn't throw exception if another instance with the same ID is in the session.

  Employee managed = (Employee) session.merge(emp);

- **saveOrUpdate()**
  - Decides automatically:
    - If ID is null → save()
    - If ID exists → update()

Analogy:

- update() → Bring the old guest back into the hotel room, but only if no one is inside.
- merge() → Copy the guest's luggage into a fresh room, even if one is already occupied.
- saveOrUpdate() → Receptionist decides whether it's a new or returning guest.

442. What is flush() in Hibernate, and when should it be used?

- **Definition**:

  flush() synchronizes the in-memory state of the persistence context with the database, but it does not commit the transaction.

- **When it happens automatically**:
  - Before a transaction commit.
  - Before executing a query (so that results are consistent).
- **Manual use**:

  You can force it when you want your changes reflected in the database immediately.

- **Example**:

```
Transaction tx = session.beginTransaction();
Employee emp = new Employee("John");
session.save(emp);

session.flush(); // INSERT executed now, but transaction not yet committed

tx.commit(); // commit finalizes the transaction
```

In practice, avoid unnecessary manual flush unless required (e.g., bulk operations).

443. What is clear() in Hibernate, and why is it used?

- **Definition**:

  clear() detaches all entities from the session, effectively clearing the first-level cache.

- **Use case**:
  - When processing large datasets to avoid memory bloat.
  - After batch inserts/updates, clear the session so Hibernate doesn't track thousands of entities in memory.
- **Example**:

```
for (int i = 0; i < 1000; i++) {
   Employee emp = new Employee("Emp" + i);
   session.save(emp);

   if (i % 50 == 0) {
      session.flush();
      session.clear(); // free memory
   }
}
```

Think of it as "resetting Hibernate's memory clipboard."

444. What is the difference between refresh() and clear() in Hibernate?

- **refresh()**
  - Reloads the entity from the database, overwriting any in-memory changes.
  - Used when the database might have been updated by another transaction.

```
Employee emp = session.get(Employee.class, 1);
emp.setName("Temp Name");

session.refresh(emp); // reloads from DB, discards "Temp Name"
```

- **clear()**
  - Removes all managed entities from the persistence context.
  - Doesn't reload, just detaches them.

Analogy:

- refresh() = "Update my copy of the file from the server."
- clear() = "Close all open files; I'll reopen later if needed."

445. What is the difference between get() and load() in Hibernate?

- **get()**
  - Immediately hits the database and returns the entity or null if not found.
  - Eager fetching.
  - Safe when you are not sure if the entity exists.

```
Employee emp = session.get(Employee.class, 1);
if (emp == null) {
   System.out.println("Employee not found");
}
```

- **load()**
  - Returns a proxy (lazy-loaded).
  - The actual DB hit happens only when a property is accessed.
  - Throws ObjectNotFoundException if entity does not exist.

```
Employee emp = session.load(Employee.class, 1);
System.out.println(emp.getName()); // DB hit here
```

Rule of thumb: use get() when entity existence is uncertain, load() when you're sure the record exists.

446. What is the difference between merge() and update() in Hibernate?

- **update()**
  - Reattaches a detached entity to the session.
  - Fails if another instance with the same identifier is already associated with the session (NonUniqueObjectException).
- **merge()**
  - Copies the state of a detached entity into a managed instance.
  - Works even if another instance with the same ID is already in the session.
- **Example**:

```
Employee emp = session.get(Employee.class, 1);
session.detach(emp);

emp.setName("Updated Name");

session.update(emp); // reattaches directly
// OR
session.merge(emp);  // safer, creates/uses a managed copy
```

In practice: prefer merge() in distributed systems where detached objects are common.

447. What is dirty checking in Hibernate, and how does it work?

- **Definition**:

  Dirty checking is Hibernate's mechanism to automatically detect changes in persistent objects and synchronize them with the database during flush/commit.

- **How it works**:
  - Hibernate keeps a snapshot of entity values when loaded.
  - At flush time, it compares the current state with the snapshot.
  - If different, it generates the required SQL (UPDATE, etc.).
- **Example**:

```
Employee emp = session.get(Employee.class, 1);
emp.setName("New Name"); // no SQL yet

session.getTransaction().commit(); // Hibernate issues UPDATE
```

Benefit: you don't need to call session.update() explicitly; Hibernate tracks changes for you.

448. What is the difference between transient, persistent, and detached states in Hibernate?

- **Transient**
  - The object is just created with new, not associated with any session, and no DB record exists.
  - Example

```
Employee emp = new Employee();
emp.setName("John"); // transient
```

- **Persistent**
  - The object is associated with a Hibernate session and tracked for dirty checking.
  - Example:

```
session.save(emp);  // now persistent
```

- **Detached**
  - The object was persistent once but the session is closed/cleared.
  - Hibernate doesn't track it anymore.
  - Example:

```
session.close();
emp.setName("Updated"); // detached, no DB sync
```

Lifecycle: Transient → Persistent → Detached.

449. What is the role of SessionFactory in Hibernate?

- **Definition**:

  A heavyweight object that creates Session instances. It's thread-safe and usually created once per application.

- **Responsibilities**:
  - Holds configuration and mapping metadata.
  - Provides Session objects for database interaction.
  - Manages second-level cache (if enabled).
- **Example**:

```
SessionFactory factory = new
Configuration().configure().buildSessionFactory();
Session session = factory.openSession();
```

Think of SessionFactory as a factory of database connections + metadata.

450. What are interceptors in Hibernate, and when would you use them?

- **Definition**:

  Interceptors let you hook into Hibernate's persistence life cycle to apply custom logic (like auditing, logging, soft deletes).

- **Types**:
  - **Session-level interceptors** → tied to one session.
  - **Global interceptors** → apply to all sessions.
- **Example (auditing)**:

```
public class AuditInterceptor extends EmptyInterceptor {
  @Override
  public boolean onSave(Object entity, Serializable id, Object[] state,
                String[] propertyNames, Type[] types) {
    if (entity instanceof Auditable) {
      for (int i = 0; i < propertyNames.length; i++) {
        if ("createdAt".equals(propertyNames[i])) {
          state[i] = LocalDateTime.now();
      }
      }
    }
    return true;
  }
}
```

- **Usage**:
  Attach when building session:

  Session session = factory.withOptions().interceptor(new AuditInterceptor()).openSession();

Interceptors are especially useful for cross-cutting concerns like auditing without polluting entity logic.

451. What are Hibernate proxies, and why are they used?

- **Definition**:

  Hibernate often doesn't load an entity fully right away. Instead, it returns a **proxy object** (a subclass created at runtime) that loads the entity lazily when you access it.

- **Why**:
  - Saves performance by delaying DB calls until data is actually needed.
  - Avoids unnecessary joins/queries.

- **Example**:

```
        Department dept = session.load(Department.class, 1);
        // At this point, Hibernate may return a proxy, not the real Department.
System.out.println(dept.getName()); // triggers SQL query
```

- **Caveat**:

  Accessing a proxy outside of a session can throw **LazyInitializationException** because Hibernate has no session to fetch the actual data.

452. What is the use of the @Immutable annotation in Hibernate?

- **Definition**:

  Marks an entity or collection as read-only. Hibernate won't generate SQL for updates or deletes on such entities.

- **Use cases**:
  - Reference data (e.g., list of countries, currencies).
  - Historical records that should never be changed.
- **Example**:

```
@Entity
@Immutable
public class Country {
   @Id
   private int id;
   private String name;
}
```

- **Effect**:

  If you try to modify a Country object, Hibernate ignores it and does not send update SQL.

453. What is Hibernate bytecode enhancement, and why is it used?

- **Definition**:

  Hibernate can enhance entity classes at build time or runtime using bytecode instrumentation. This adds extra code for features like:

  - Dirty checking without tracking snapshots.
  - Lazy loading of attributes.
  - Better performance in detecting changes.

- **How**:

  - With build tools like Maven/Gradle plugins.
  - Or using Hibernate's enhancer at runtime.

- **Example scenario**:

  Instead of Hibernate keeping an old copy of an entity to detect changes, bytecode enhancement can add setter tracking directly in the entity. This makes dirty checking more efficient.

- **When to use**:

  - Large enterprise systems where performance matters.
  - Entities with many fields, where dirty checking would be costly.

454. What is the purpose of hibernate.bytecode.use_reflection_optimizer property?

Answer:

- This property controls whether Hibernate should use its **bytecode reflection optimizer** for entity property access.
- Normally, Hibernate uses **Java Reflection** to get/set entity fields. Reflection is flexible but slower.
- With the reflection optimizer enabled, Hibernate generates bytecode-based accessors at runtime for faster field access.

Key Points:

- **Enabled by default** in most modern Hibernate versions.
- Gives **performance boost** in read/write operations on entities.
- If you're debugging issues with proxies or bytecode, you can disable it by:

hibernate.bytecode.use_reflection_optimizer=false

- Use case: High-performance apps with lots of entity load/save operations.

455. What is the role of the @Immutable annotation in Hibernate?

Answer:

- @Immutable marks an entity or collection as **read-only**.
- Once loaded, Hibernate will never update or delete such entities.
- Useful when working with **lookup tables, historical records, audit logs, or views**.

Example:

```
@Entity
@Immutable
public class Country {
   @Id
   private String code;
   private String name;
}
```

Behavior:

- Hibernate skips dirty-checking for immutable entities → improves performance.
- If you try to modify and persist an immutable entity, Hibernate **ignores** the update silently.

456. What is Hibernate Statistics API, and how is it useful?

Answer:

- The Hibernate **Statistics API** provides runtime metrics about entity operations, cache usage, and query performance.
- It helps **monitor and tune** Hibernate applications in production.

Example – Enable statistics:

hibernate.generate_statistics=true

Accessing Statistics:

```
SessionFactory sf = sessionFactory;
Statistics stats = sf.getStatistics();
System.out.println("Entities loaded: " + stats.getEntityLoadCount());
System.out.println("Second level cache hits: " +
stats.getSecondLevelCacheHitCount());
```

Metrics include:

- Entity load, fetch, update, insert, delete counts.
- Cache hit/miss counts.
- Query execution counts and average execution time.

Use cases:

- Detect **N+1 select issues**.
- Monitor **cache effectiveness**.
- Track **slow queries**.

457. What is Hibernate's Interceptor, and when would you use it?

Answer:

- An Interceptor in Hibernate lets you hook into the entity lifecycle and perform custom logic during persistence operations.
- Unlike entity callbacks (@PrePersist, @PostUpdate), interceptors apply at the **session level** and can affect multiple entities.

Example – logging interceptor:

```
public class LoggingInterceptor extends EmptyInterceptor {
   @Override
   public boolean onSave(Object entity, Serializable id, Object[] state, String[]
propertyNames, Type[] types) {
      System.out.println("Saving entity: " + entity.getClass().getSimpleName());
      return super.onSave(entity, id, state, propertyNames, types);
   }
}
```

Usage:

```
Session session = sessionFactory
      .withOptions()
      .interceptor(new LoggingInterceptor())
      .openSession();
```

Use cases:

- Audit logging (who updated what and when).
- Auto-populating fields (createdDate, modifiedBy).
- Enforcing security checks at persistence level.

458. What are Hibernate EventListeners, and how do they differ from Interceptors?

Answer:

- Hibernate EventListeners are a more fine-grained way of listening to entity lifecycle events compared to interceptors.
- Each persistence action (save, update, delete, load, flush, etc.) has its own event type.

Example – PreInsertEventListener:

```
public class AuditListener implements PreInsertEventListener {
   @Override
   public boolean onPreInsert(PreInsertEvent event) {
      Object entity = event.getEntity();
      System.out.println("Before inserting: " + entity.getClass().getSimpleName());
      return false; // false = continue insert
   }
}
```

Difference from Interceptor:

- **Interceptor**: session-wide, broader control, simpler to implement.
- **EventListener**: event-specific, more fine-grained, can handle many specific hooks.

Use cases:

- Automatic audit columns.
- Entity validation before insert/update.
- Triggering external systems (e.g., Kafka publish after insert).

459. What is Hibernate's StatelessSession, and when should you use it?

Answer:

- StatelessSession is a lightweight alternative to the regular Hibernate Session.
- It does not maintain a **first-level cache** or perform dirty-checking.

Characteristics:

- No persistence context → entities are not tracked.
- Each operation (insert, update, delete, get) hits the database directly.
- Faster for **batch processing** but lacks convenience of managed entities.

Example:

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();

for (Employee emp : employees) {
   session.insert(emp);
}

tx.commit();
session.close();
```

Use cases:

- Large-scale batch inserts/updates (e.g., ETL jobs).
- Read-only operations on large datasets.
- Situations where you don't need Hibernate's caching/dirty-checking overhead.

460. What is the difference between SessionFactory and EntityManagerFactory?

Answer:

- **SessionFactory** is Hibernate's native factory for creating Session objects.
- **EntityManagerFactory** is JPA's standard factory for creating EntityManager objects, which internally delegate to Hibernate's SessionFactory when Hibernate is the provider.

Key differences:

- SessionFactory → Hibernate-specific, exposes advanced features like StatelessSession, Interceptor, Criteria.
- EntityManagerFactory → JPA-standard, portable across providers (Hibernate, EclipseLink, etc.).

Analogy:

Think of EntityManagerFactory as a "standard power plug" and SessionFactory as the "actual power source." If you use the JPA API (EntityManagerFactory), you can swap Hibernate with another provider without changing code.

461. How does Hibernate support multi-tenancy?

Answer:

Hibernate supports multi-tenancy, meaning a single application can serve multiple tenants (customers), each with their own data.

Strategies:

1. **DATABASE** – each tenant has its own database.
2. **SCHEMA** – each tenant has its own schema in the same DB.
3. **DISCRIMINATOR** – single schema, but tenant data is separated by a tenant column.

Example configuration:

```
hibernate.multiTenancy=SCHEMA
hibernate.tenant_identifier_resolver=myTenantResolver
hibernate.multi_tenant_connection_provider=myConnectionProvider
```

Code – Tenant Identifier Resolver:

```
public class MyTenantResolver implements CurrentTenantIdentifierResolver {
  @Override
  public String resolveCurrentTenantIdentifier() {
    return TenantContext.getCurrentTenant(); // e.g., from ThreadLocal
  }
  @Override
  public boolean validateExistingCurrentSessions() {
    return true;
  }
}
```

Use cases:

- SaaS applications serving multiple customers.
- Banking, healthcare, or e-commerce platforms where each client has isolated data.

462. What is Hibernate bytecode enhancement, and why is it used?

Answer:
Hibernate can enhance entity classes at the bytecode level (compile-time or runtime) to add features without requiring you to write boilerplate.

Capabilities added via enhancement:

- **Lazy loading** of attributes without proxies.
- **Dirty checking** optimization (track field changes more efficiently).
- **Bidirectional association management**.

How it works:

- Done with **Hibernate Enhancer** plugin or JVM agent.
- Enhances compiled .class files by injecting extra code.

Example – build.gradle setup:

```
plugins {
  id "org.hibernate.orm" version "6.3.0.Final"
}

hibernate {
  enhance {
    enableLazyInitialization = true
    enableDirtyTracking = true
    enableAssociationManagement = true
  }
}
```

Benefit:

Instead of relying on runtime proxies (which can be fragile), enhancement makes entity behavior more consistent and efficient.

463. How does Hibernate handle dirty checking?

Answer:

Dirty checking means Hibernate automatically detects changes made to persistent entities and updates only the modified fields in the database when the transaction commits.

How it works:

- When you load an entity, Hibernate keeps a snapshot of its state in the persistence context.
- At flush time, Hibernate compares the current state with the snapshot.
- If differences are found, an UPDATE statement is generated only for changed fields.

Example:

```
Employee emp = session.get(Employee.class, 1);
emp.setSalary(60000); // Changed in memory
// No explicit session.update() needed
session.getTransaction().commit(); // Hibernate detects and updates salary
```

Advantages:

- Reduces boilerplate code (update not needed manually).
- Optimizes performance by updating only changed fields.

Note:

Dirty checking relies on the persistence context. If you detach the entity, Hibernate won't track changes anymore.

464. What is a proxy in Hibernate, and how does it support lazy loading?

Answer:

A **proxy** in Hibernate is a subclass generated at runtime using CGLIB or Javassist that stands in for an entity.

- When you call session.get(Employee.class, 1), Hibernate may return a proxy object instead of the actual entity.
- The proxy holds the identifier and only loads other fields when accessed.

Example:

```
Employee emp = session.load(Employee.class, 1); // returns proxy
System.out.println(emp.getId()); // doesn't hit DB
System.out.println(emp.getName()); // triggers DB query
```

Use case:

- Efficient lazy loading of large associations (like orders of a customer).

Caution:

- Accessing proxies outside an active session throws LazyInitializationException.
- That's why DTO mapping or JOIN FETCH is often used in APIs.

465. How does Hibernate handle optimistic vs. pessimistic locking?

Answer:
Both are concurrency control strategies:

1. **Optimistic Locking**
    - Assumes conflicts are rare.
    - Uses a @Version column to detect concurrent updates.
    - If two users update the same row, Hibernate throws OptimisticLockException when versions mismatch.
    - Best for read-heavy, low-conflict systems (e.g., customer profile updates).

    @Version

    private int version;

2. **Pessimistic Locking**
    - Locks rows in the database to prevent concurrent modifications.
    - Achieved with LockModeType.PESSIMISTIC_WRITE or SQL SELECT ... FOR UPDATE.
    - Ensures no one else can update until the lock is released.
    - Best for high-conflict scenarios (e.g., inventory stock deduction in e-commerce).

Example – pessimistic lock:

```
Employee emp = em.find(Employee.class, 1, LockModeType.PESSIMISTIC_WRITE);
emp.setSalary(70000);
```

Comparison:

- Optimistic = no blocking, safer for scalability.
- Pessimistic = blocking, safer for critical consistency.

466. What are dynamic inserts and updates in Hibernate?

Answer:
Dynamic inserts/updates ensure Hibernate only includes **non-null** or **modified** fields in the SQL statements, instead of all columns.

- **Dynamic Insert (@DynamicInsert)** → Hibernate generates INSERT statements only for non-null fields.
- **Dynamic Update (@DynamicUpdate)** → Hibernate generates UPDATE statements only for changed fields.

Example:

```
@Entity
@DynamicInsert
@DynamicUpdate
public class Employee {
   @Id
   @GeneratedValue
   private int id;
   private String name;
   private Double salary;
}
```

Behavior:

```
-- Without @DynamicInsert
INSERT INTO employee (id, name, salary) VALUES (1, 'John', NULL);

-- With @DynamicInsert
INSERT INTO employee (id, name) VALUES (1, 'John');
```

Advantages:

- Reduces unnecessary column updates.
- Improves performance for large tables with many nullable columns.

467. How does Hibernate handle natural IDs?

Answer:

A **natural ID** is a domain-specific unique identifier (like email, SSN, or username) instead of the surrogate primary key. Hibernate supports them via @NaturalId.

Example:

```
@Entity
public class User {
   @Id
   @GeneratedValue
   private int id;

   @NaturalId
   private String email;
}
```

Usage:

```
User user = session.byNaturalId(User.class)
            .using("email", "john@example.com")
            .load();
```

Advantages:

- Queries are more intuitive (find by email).
- Hibernate caches natural IDs for faster lookups.

Note:

- Still recommended to have a surrogate primary key (id), while natural IDs provide business-level uniqueness.

468. What is Hibernate's StatelessSession, and when should it be used?

Answer:

StatelessSession is a lightweight alternative to a normal Session that **does not maintain first-level cache or persistent context**.

Key properties:

- No dirty checking.
- No cascading operations.
- No automatic versioning.

Example – batch inserts:

```
StatelessSession session = sessionFactory.openStatelessSession();
Transaction tx = session.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee emp = new Employee("Emp" + i, 50000.0);
   session.insert(emp);
}

tx.commit();
session.close();
```

Use cases:

- Bulk operations (insert/update/delete) where caching is unnecessary.
- ETL processes, log ingestion, or importing millions of rows.

Trade-off:

- Faster, but you lose automatic change tracking and caching benefits.

469. How does Hibernate support immutable entities?

Answer:

Hibernate allows you to mark an entity as **immutable** using @Immutable. This means once the entity is persisted, Hibernate will not track or allow updates to it.

Example:

```
@Entity
@Immutable
public class Country {
   @Id
   private String code;
   private String name;
}
```

Behavior:

- INSERT is allowed when first saving.
- Any UPDATE or DELETE operations will be ignored.

Use cases:

- Reference/master data like countries, currencies, or status codes.
- Historical audit tables that should never change.

This reduces overhead because Hibernate skips dirty checking for immutable entities.

470. What are Hibernate's default naming strategies?

Answer:

When mapping entity classes to database tables, Hibernate uses **naming strategies** to decide how Java names map to SQL names.

1. **ImplicitNamingStrategy**
    - Defines how names are generated when no explicit name is given.
    - Example: class EmployeeAddress → table employee_address.
2. **PhysicalNamingStrategy**
    - Converts logical names into actual DB names (handles case, underscores, etc.).
    - Example: employeeAddress → employee_address.

Configuration:

```
spring.jpa.hibernate.naming.implicit-strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyJpaCompliantImpl
spring.jpa.hibernate.naming.physical-strategy=org.springframework.boot.orm.jpa.hibernate.SpringPhysicalNamingStrategy
```

Why it matters:

- Ensures consistent naming between Java and DB.
- Helps adapt to DB vendor conventions (uppercase in Oracle, snake_case in PostgreSQL).

471. What are transient, persistent, and detached states in Hibernate?

Answer:
Entities in Hibernate move through **three life-cycle states**:

1. **Transient**
    - Newly created object, not associated with any session, not stored in DB.
    - Example: new Employee("John").
2. **Persistent**
    - Object is associated with a Hibernate Session and DB row.
    - Any change is automatically tracked and synchronized during flush().
    - Example: session.save(emp);
3. **Detached**
    - Object was persistent, but the session is closed.
    - Changes are no longer tracked unless reattached with merge() or update().

Example flow:

```
Employee emp = new Employee("John");   // Transient
session.save(emp);                     // Persistent
session.close();
emp.setName("Johnny");                 // Detached
```

Analogy:

- Transient = drafted email not sent.
- Persistent = email saved in server (active draft).
- Detached = email downloaded offline; edits won't sync until re-uploaded.

472. How does Hibernate support polymorphic queries?

Answer:
Hibernate supports **polymorphic queries** through JPA inheritance. If you query a parent entity, Hibernate automatically includes all its subclasses.

Example:

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Payment {
   @Id
   private Long id;
   private double amount;
}

@Entity
public class CreditCardPayment extends Payment {
   private String cardNumber;
}

@Entity
public class UpiPayment extends Payment {
   private String upiId;
}
```

Query:

```
List<Payment> payments = session.createQuery("from Payment",
Payment.class).list();

Hibernate will fetch both CreditCardPayment and UpiPayment rows.
```

Use cases:

- E-commerce payment systems (multiple payment methods).
- HR systems (different types of employees: full-time, contract, intern).

473. How can you prevent SQL injection in Hibernate queries?

Answer:

SQL injection happens when user input is directly concatenated into queries. Hibernate helps prevent this with **parameter binding**.

Bad (vulnerable):

```
String hql = "FROM Employee e WHERE e.name = '" + userInput + "'";
session.createQuery(hql).list();
```

Safe (with parameters):

```
String hql = "FROM Employee e WHERE e.name = :name";
Query<Employee> query = session.createQuery(hql, Employee.class);
query.setParameter("name", userInput);
```

Other strategies:

1. Use setParameter() instead of string concatenation.
2. Prefer Criteria API (type-safe, avoids string queries).
3. Use Spring Data JPA's derived queries or @Query with parameters.

Real-world analogy: like using prepared statements in JDBC; Hibernate translates bound parameters into safe SQL.

474. What is the use of @Immutable annotation in Hibernate?

Answer:

@Immutable marks an entity or collection as **read-only**. Hibernate won't allow updates or deletions once persisted.

Example:

```
@Entity
@Immutable
public class Currency {
   @Id
   private String code;
   private String name;
}
```

Behavior:

- Insert works.
- Updates are ignored.
- No dirty checking overhead.

Use cases:

- Static data (currencies, country codes, product categories).
- Historical audit records.

This improves **performance** since Hibernate skips tracking changes for immutable entities.

475. How does Hibernate handle entity proxies with lazy loading?

Answer:

When you use FetchType.LAZY, Hibernate doesn't load the full entity immediately. Instead, it returns a **proxy object** (a subclass created dynamically using CGLIB or ByteBuddy).

- The proxy contains only the entity identifier.
- When you access a non-identifier field, Hibernate triggers a database query to fetch the data.

Example:

```
@Entity
public class Department {
   @Id
   private Long id;

   private String name;

   @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
   private List<Employee> employees;
}
```

Usage:

```
Department dept = session.get(Department.class, 1L);
// employees not yet loaded
List<Employee> list = dept.getEmployees(); // triggers a query
```

Caution: Accessing proxies outside an active session throws

LazyInitializationException.

Best practice: Use DTOs or JOIN FETCH if you need data immediately.

476. What is the difference between getSingleResult() and getResultList() in Hibernate/JPA?

Answer:
Both are used to execute queries, but their behavior differs:

- **getResultList()**
  - Returns a List of results.
  - Returns an empty list if no results.
- **getSingleResult()**
  - Returns exactly one result.
  - Throws NoResultException if no result.
  - Throws NonUniqueResultException if multiple results.

Example:

```
Query<Employee> q1 = session.createQuery("FROM Employee", Employee.class);
List<Employee> employees = q1.getResultList(); // can be empty

Query<Employee> q2 = session.createQuery("FROM Employee e WHERE e.id = :id",
Employee.class);
q2.setParameter("id", 1);
Employee emp = q2.getSingleResult(); // must be exactly one
```

Use cases:

- getResultList() → when fetching multiple rows (e.g., all employees in dept).
- getSingleResult() → when querying by primary key or unique fields.

477. How do you configure Hibernate to show generated SQL statements?

Answer:
Hibernate can log SQL queries to help in debugging and performance tuning.

Options:

1. **hibernate.show_sql**

```
spring.jpa.show-sql=true
Prints SQL to console (not formatted).
```

2. **hibernate.format_sql**

```
spring.jpa.properties.hibernate.format_sql=true
Formats SQL for readability.
```

3.
4. **hibernate.highlight_sql** (Hibernate 5.4+)

```
spring.jpa.properties.hibernate.highlight_sql=true
Highlights keywords for clarity
```

.

5. **Enable logging in logback/log4j2:**

```
<logger name="org.hibernate.SQL" level="DEBUG"/>
<logger name="org.hibernate.type.descriptor.sql.BasicBinder"
level="TRACE"/>
```

   - SQL logs the queries.
   - BasicBinder logs parameter values.

Real-world benefit: Debugging N+1 issues, verifying joins, and tuning performance.

478. What is the difference between StatelessSession and Session in Hibernate?

Answer:
Both are used to interact with the database, but they differ in caching and lifecycle management.

- **Session (Regular)**
  - Uses **first-level cache**.
  - Tracks entity states (new, persistent, detached, removed).
  - Supports dirty checking, cascading, and lazy loading.
  - More memory overhead but convenient for standard apps.
- **StatelessSession**
  - **Does not use first-level cache**.
  - No dirty checking or automatic synchronization.
  - No lifecycle events (@PrePersist, @PostLoad, etc.).
  - Best for **batch processing** (e.g., inserting 1M records).

Example:

```
StatelessSession stateless = sessionFactory.openStatelessSession();
Transaction tx = stateless.beginTransaction();

for (int i = 0; i < 1000; i++) {
   Employee e = new Employee("Emp " + i, "IT");
   stateless.insert(e);
}
tx.commit();
stateless.close();
```

Use Session for OLTP apps, StatelessSession for ETL/bulk inserts.

479. How do you configure optimistic vs. pessimistic locking in Hibernate?

Answer:

- **Optimistic Locking** (default, non-blocking):
  - Uses a @Version column.
  - Detects conflicts at commit time.
  - Best when conflicts are **rare**.

```
@Entity
public class Employee {
 @Id
 private Long id;

 @Version
 private int version;
}
```

- **Pessimistic Locking** (blocking):
  - Locks the row in the DB immediately to prevent concurrent updates.
  - Configured using LockModeType:

```
Employee emp = em.find(Employee.class, 1L,
LockModeType.PESSIMISTIC_WRITE);
```

Comparison:

- Optimistic → lightweight, no locks, fails on conflict.
- Pessimistic → heavy, locks row, avoids conflicts but reduces concurrency.

480. How does Hibernate handle associations when deleting entities?

Answer:
When deleting an entity, Hibernate must decide how to handle its related entities:

1. **CascadeType.REMOVE / CascadeType.ALL**
    - Deleting a parent also deletes children.

```
@OneToMany(mappedBy = "dept", cascade =
CascadeType.ALL, orphanRemoval = true)
private List<Employee> employees;
```

2. **Orphan Removal**
    - If a child is removed from a collection, Hibernate deletes it from DB.

orphanRemoval = true

3. **No Cascade**
    - If no cascade is set, trying to delete a parent with children leads to **ConstraintViolationException** unless children are removed manually.

Example:

```
Department dept = session.get(Department.class, 1L);
session.delete(dept); // employees also deleted if cascade + orphanRemoval

Best practice: Use orphanRemoval = true for parent-owned relationships (e.g., Department → Employees).
```

481. What is the difference between save(), persist(), and saveOrUpdate() in Hibernate?

Answer:

1. **save()**
    - Immediately returns the generated identifier (Serializable).
    - Can be called outside of a transaction (though not recommended).
    - Does not guarantee entity state synchronization with persistence context.
2. **persist()**
    - JPA standard method.
    - Does not return an ID (void method).
    - Entity is managed only within a transaction.
    - Throws exception if invoked outside of transaction.
3. **saveOrUpdate()**
    - If entity is **new** → behaves like save().
    - If entity is **detached** → updates existing row.
    - Useful for syncing detached objects.

Example:

```
Employee emp = new Employee("John", "IT");
session.save(emp);        // returns ID
session.persist(emp);     // no return, entity managed
session.saveOrUpdate(emp);  // saves or updates depending on state
```

482. How does Hibernate manage detached objects?

Answer:

- **Detached objects**: Entities once managed by a session but no longer attached (after session is closed).

  Hibernate provides ways to reattach them:

1. **update()** – Reattaches but fails if entity already exists in session.
2. **merge()** – Copies state of detached entity into a managed instance (safe, preferred).
3. **saveOrUpdate()** – Either inserts or updates depending on state.

Example:

```
Employee emp = session1.get(Employee.class, 1L);
session1.close();  // emp becomes detached

emp.setName("Updated Name");

Session session2 = sessionFactory.openSession();
session2.beginTransaction();
session2.merge(emp);   // safe reattachment
session2.getTransaction().commit();
```

Best practice: use merge() in web apps where detached objects are common.

483. What are Hibernate Natural IDs, and how are they used?

Answer:

- **Natural ID**: A unique, business-specific identifier (like email, SSN, username) instead of surrogate keys (auto-generated IDs).
- Hibernate provides @NaturalId for fast lookup and caching.

Example:

```
@Entity
public class User {
 @Id
 @GeneratedValue
 private Long id;

 @NaturalId
 @Column(unique = true, nullable = false)
 private String email;
}
```

Usage:

```
User user = session.byNaturalId(User.class)
            .using("email", "test@gmail.com")
            .load();
```

Benefits:

- Natural IDs are automatically cached by Hibernate.
- Faster lookups for unique business keys.
- Avoids writing custom queries for common identifiers.

484. What is the difference between merge() and update() in Hibernate?

Answer:

1. **update()**
    - Reattaches a detached entity to the current session.
    - Fails with NonUniqueObjectException if the same entity already exists in the session.
    - Doesn't return anything; works directly on the passed object.
2. **merge()**
    - Creates or retrieves a managed copy of the detached entity.
    - Safe: even if the entity already exists in the session, it merges the changes without errors.
    - Returns the managed instance (different object than the one passed).

Example:

```
Employee emp = session1.get(Employee.class, 1L);
session1.close(); // detached

emp.setName("Updated");

Session session2 = factory.openSession();
session2.beginTransaction();

// update() → works only if emp is not already loaded in session2
session2.update(emp);

// merge() → always safe
Employee merged = (Employee) session2.merge(emp);

Use merge() when unsure if the entity is already present in the session.
```

485. What is a Proxy Object in Hibernate?

Answer:

- A **proxy object** is a placeholder object created by Hibernate to implement **lazy loading**.
- It extends the entity class and loads actual data only when a method is accessed.

Example:

```
Employee emp = session.load(Employee.class, 1L);
// returns proxy, no SQL executed yet

String name = emp.getName();
// triggers SELECT query to fetch real data
```

Key points:

- Returned by load() (not get()).
- If accessed outside session, throws LazyInitializationException.
- Useful for performance by avoiding unnecessary DB hits.

486. What is the role of hibernate.hbm2ddl.auto property?

Answer:

This property controls **schema generation** in Hibernate.

Values:

- validate → Validates schema, does not modify.
- update → Updates schema (adds missing tables/columns).
- create → Drops & creates schema on startup.
- create-drop → Creates schema at startup, drops at shutdown.
- none → No schema management.

Example in application.properties:

```
spring.jpa.hibernate.ddl-auto=update
```

Best Practices:

- Use update in dev.
- Use validate in prod (never create-drop, as it deletes data).
- For production DB schema evolution, prefer Flyway or Liquibase.

487. What is the difference between session.get() and session.load() in Hibernate?

Answer:

1. **get()**
    - Immediately hits the database.
    - Returns the actual entity object.
    - If the row is not found, returns null.
    - Example:

```
Employee emp = session.get(Employee.class, 1L);
// Executes SELECT immediately
```

    -
2. **load()**
    - Returns a **proxy object** (lazy).
    - Database query is fired only when a method is accessed.
    - If the entity doesn't exist, throws ObjectNotFoundException when accessed.
    - Example:

```
Employee emp = session.load(Employee.class, 1L);
// No SQL yet
String name = emp.getName(); // SQL fires now
```

Use get() when you need the object right away.
Use load() when you may or may not need it, but want better performance.

488. What is the difference between save(), persist(), and saveOrUpdate() in Hibernate?

Answer:

1. **save()**
    - Inserts a new row.
    - Returns generated identifier (primary key).
    - Works outside transaction but may lead to issues.
2. **persist()**
    - JPA standard method.
    - Does not return ID.
    - Ensures the entity is only inserted when inside a transaction.
3. **saveOrUpdate()**
    - Decides whether to save or update.
    - If entity has no identifier → insert.
    - If entity already has identifier → update.

Example:

```
session.save(emp);        // insert only
session.persist(emp);     // insert only, no return ID
session.saveOrUpdate(emp);  // insert if new, update if existing
```

489. What is the difference between openSession() and getCurrentSession()?

Answer:

1. **openSession()**
    - Always creates a new Session.
    - Must close it manually.
    - No automatic transaction handling.
    - Example:

      Session session = factory.openSession();

2. **getCurrentSession()**
    - Returns a session bound to the current context (thread).
    - Automatically closed when the transaction ends.
    - Works well with Spring + Hibernate transaction management.
    - Example:

      Session session = factory.getCurrentSession();

In modern apps with Spring, getCurrentSession() is preferred because it integrates with transaction boundaries.

490. How does Hibernate handle database connection pooling?

Answer:

Hibernate itself doesn't manage database connections directly; instead, it integrates with connection pooling providers. A pool is essential for performance since creating/destroying DB connections per request is expensive.

- **Built-in options**:
    - Hibernate can use **c3p0**, **HikariCP**, **Proxool**, or any external DataSource.
    - Modern Spring Boot apps default to **HikariCP** (high-performance).

Example (hibernate.cfg.xml with c3p0):

```
<property name="hibernate.c3p0.min_size">5</property>
<property name="hibernate.c3p0.max_size">20</property>
<property name="hibernate.c3p0.timeout">300</property>
<property name="hibernate.c3p0.max_statements">50</property>
<property name="hibernate.c3p0.idle_test_period">3000</property>
```

Why important?

- Prevents resource exhaustion by limiting concurrent connections.
- Improves response time by reusing connections.
- Provides monitoring (idle connections, leaks).

491. What is the difference between SessionFactory and EntityManagerFactory?

Answer:
Both are factories for persistence contexts but differ by API standards:

- **SessionFactory (Hibernate-specific):**
  - Produces Session objects.
  - Provides Hibernate-specific features (filters, statistics).
  - Tightly coupled with Hibernate.
- **EntityManagerFactory (JPA standard):**
  - Produces EntityManager objects.
  - Part of JPA specification.
  - Works with any JPA provider (Hibernate, EclipseLink, OpenJPA).

Example:

```
// JPA style
EntityManagerFactory emf = Persistence.createEntityManagerFactory("my-pu");
EntityManager em = emf.createEntityManager();

// Hibernate style
SessionFactory sf = new Configuration().configure().buildSessionFactory();
Session session = sf.openSession();
```

In modern projects, we prefer **EntityManagerFactory** for portability but rely on **SessionFactory** for advanced Hibernate-only features.

492. How do you debug SQL queries generated by Hibernate?

Answer:
Sometimes Hibernate generates inefficient SQL (e.g., N+1). Debugging is crucial.

Ways to log SQL:

1. **Enable SQL logging in config:**

```
hibernate.show_sql=true
hibernate.format_sql=true
hibernate.use_sql_comments=true
```

2. **Spring Boot style:**

```
spring.jpa.show-sql=true
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql=TRACE
```

3. **Use tools:**
    - Hibernate Statistics API (sessionFactory.getStatistics()).
    - P6Spy or datasource-proxy for query interception.

Best practice: Enable logging in **dev/test only**, not in production, to avoid performance overhead and sensitive data leaks.

493. What is the difference between Session.get() and Session.byId().load() in Hibernate?

Answer:
Both are used to fetch entities by primary key, but their behavior differs:

- **session.get(Class, id)**
  - Immediately hits the database (or first-level cache).
  - Returns the actual entity or null if not found.
  - Eager fetching.
- **session.byId(Class).load(id)**
  - Returns a proxy (lazy-loaded reference).
  - Hits DB **only when a property is accessed**.
  - Throws ObjectNotFoundException if not present.

Example:

```
Employee emp1 = session.get(Employee.class, 1);  // Direct fetch
Employee emp2 = session.byId(Employee.class).load(2);  // Proxy
```

Use get() when you know you need full entity. Use load() for deferred fetching.

494. How does Hibernate handle composite primary keys?

Answer:
Some entities require **multiple fields** as primary key (e.g., OrderId = orderId + productId). Hibernate handles this via:

1. **@Embeddable + @EmbeddedId**

```
@Embeddable
public class OrderId implements Serializable {
   private int orderId;
private int productId;
}

@Entity
public class Order {
   @EmbeddedId
   private OrderId id;
   private int quantity;
}
```

2. **@IdClass** (alternate way, less preferred).

Best practice: Prefer **@EmbeddedId** because it keeps keys in a single embeddable object, making queries cleaner.

495. What is the purpose of hibernate.jdbc.batch_size property?

Answer:

This property controls the **batch size for JDBC operations** (INSERT/UPDATE/DELETE).

- Default = 0 (no batching).
- If set, Hibernate groups multiple statements into one batch → fewer DB round trips → faster performance.

Example (hibernate.cfg.xml):

```
<property name="hibernate.jdbc.batch_size">20</property>
```

Usage:

```
for (int i = 0; i < employees.size(); i++) {
   session.save(employees.get(i));
   if (i % 20 == 0) {
      session.flush();
      session.clear();
   }
}
```

Always tune batch size depending on DB + workload (commonly 20–50).
Combine with StatelessSession for very large bulk inserts.

496. What is the difference between merge() and update() in Hibernate?

Answer:

Both are used to reattach a **detached entity** to a Hibernate session, but they behave differently:

- **update(entity)**
  - Directly reattaches the detached entity to the session.
  - Fails with NonUniqueObjectException if another instance of the same entity is already in the session.
  - Updates DB immediately on flush.
- **merge(entity)**
  - Creates/returns a new **managed copy** of the detached entity.
  - Doesn't fail if the session already has another instance.
  - Safe to use in distributed systems where multiple detached copies exist.

Example:

```
Employee emp = session.get(Employee.class, 1);
session.close();

// Later
Session session2 = factory.openSession();
session2.update(emp); // Directly reattaches
// or
Employee mergedEmp = (Employee) session2.merge(emp); // Creates managed copy
```

Prefer **merge()** for safer reattachment.

497. What is the purpose of hibernate.show_sql and hibernate.format_sql?

Answer:
These are Hibernate debugging properties:

- **hibernate.show_sql=true** → Logs executed SQL statements to console.
- **hibernate.format_sql=true** → Formats SQL into readable multi-line queries.
- **hibernate.use_sql_comments=true** → Adds comments to queries for easier debugging.

Example (hibernate.cfg.xml):

```
<property name="hibernate.show_sql">true</property>
<property name="hibernate.format_sql">true</property>
<property name="hibernate.use_sql_comments">true</property>
```

In production, use proper logging frameworks (e.g., Log4j/SLF4J) instead of console.

498. What is the difference between optimistic and pessimistic locking in Hibernate?

Answer:

- **Optimistic Locking** (default with @Version)
  - Assumes conflicts are rare.
  - Uses a **version field** to detect conflicts.
  - If another transaction modifies data, Hibernate throws OptimisticLockException.
  - Lightweight, no DB locks.
- **Pessimistic Locking**
  - Locks rows in DB using SELECT ... FOR UPDATE.
  - Prevents other transactions from modifying until lock is released.
  - Useful in high-contention systems (banking, inventory).

Example:

```
// Optimistic
@Version
private int version;

// Pessimistic
Employee emp = session.get(Employee.class, 1, LockMode.PESSIMISTIC_WRITE);
```

Rule of thumb:

- Use **optimistic** for web apps & microservices (less contention).
- Use **pessimistic** when strong consistency is critical.

499. What is the difference between session.createQuery() and session.createSQLQuery() in Hibernate?

Answer:

- **createQuery()**
  - Uses **HQL (Hibernate Query Language)**.
  - Works with **entity names and properties**, not DB tables/columns.
  - Portable across databases.
  - Example:

```
Query<Employee> q = session.createQuery("FROM Employee
WHERE department = :dept", Employee.class);
q.setParameter("dept", "IT");
List<Employee> list = q.list();
```

- **createSQLQuery() (or createNativeQuery() in JPA)**
  - Executes **raw SQL** directly.
  - Useful for DB-specific queries or performance optimizations.
  - Returns Object arrays or entities (with addEntity() mapping).
  - Example:

```
Query q = session.createSQLQuery("SELECT * FROM employee
WHERE department = :dept")
        .addEntity(Employee.class);
q.setParameter("dept", "IT");
List<Employee> list = q.list();
```

Use createQuery() for portability, and createSQLQuery() only when SQL features not supported by HQL are required.

500. What are Named Queries in Hibernate, and what are their advantages?

Answer:
Named Queries are **predefined, reusable queries** defined with annotations or XML.

- **Declaration:**

```
@Entity
@NamedQuery(
 name = "Employee.findByDept",
 query = "FROM Employee e WHERE e.department = :dept"
)
public class Employee { ... }
```

- **Usage:**

```
Query<Employee> q =
session.createNamedQuery("Employee.findByDept", Employee.class);
q.setParameter("dept", "IT");
List<Employee> list = q.getResultList();
```

- **Advantages:**
  - Reusability (avoid repeating query strings).
  - Centralized management of queries.
  - Pre-compilation by Hibernate → faster execution.
  - Easier to tune and maintain.

501. What is the difference between session.save() and session.persist() in Hibernate?

Answer:

- **save(entity)**
  - Returns the generated identifier immediately.
  - Works outside of a transaction (but not recommended).
  - Hibernate-specific (not JPA standard).
  - Example:

    ```
    int id = (Integer) session.save(emp);
    ```

- **persist(entity)**
  - Does **not** return the identifier.
  - JPA-standard method.
  - Must be used inside a transaction.
  - Example:

    ```
    session.persist(emp); // No return value
    ```

Rule of thumb:

- Use **persist()** when writing portable JPA code.
- Use **save()** if you need the ID immediately and are okay with Hibernate-specific API.

502. What is the difference between session.merge() and session.update() in Hibernate?

Answer:

- **update(entity)**
  - Reattaches a **detached entity** to the current session.
  - Fails with NonUniqueObjectException if the session already has another instance with the same ID.
  - Example:

```
Employee emp = session.get(Employee.class, 1);
session.detach(emp);
emp.setName("Updated");
session.update(emp);  // reattaches
```

- **merge(entity)**
  - Creates a **new managed copy** of the detached entity.
  - Doesn't throw exception if another instance with the same ID is in session.
  - Example:

```
Employee emp = session.get(Employee.class, 1);
session.detach(emp);
emp.setName("Updated");
Employee managed = (Employee) session.merge(emp);
```

Use update() when you're sure no duplicate instance exists in session.
Use merge() for safer reattachment in complex apps.

503. What are Transient, Persistent, and Detached states in Hibernate?

Answer:

1. **Transient**
   - Object is created with new, not yet associated with a Hibernate session.
   - No DB record exists.
   - Example:

     Employee emp = new Employee(); // Transient

2. **Persistent**
   - Object is associated with an active session and tracked by Hibernate.
   - Changes are auto-synchronized with DB on flush/commit.
   - Example:

     session.save(emp); // Persistent

3. **Detached**
   - Object was persistent, but session is closed/cleared.
   - Not tracked by Hibernate anymore.
   - Example:

     session.close(); // emp becomes Detached

Knowing states is crucial for avoiding exceptions (LazyInitializationException, NonUniqueObjectException).

504. What is the difference between session.flush() and session.clear() in Hibernate?

Answer:

- **flush()**
  - Synchronizes in-memory session state with the database (executes pending SQL).
  - Does **not** close or detach objects.
  - Example:

    ```
    emp.setName("Updated");
    session.flush(); // UPDATE query sent
    ```

- **clear()**
  - Removes all entities from the persistence context (session cache).
  - Objects become **detached**.
  - Example:

    ```
    session.clear(); // Now emp is detached
    ```

Use flush() to push changes to DB without committing.
Use clear() to free memory in batch processing.

505. What is the difference between get() and load() in Hibernate?

Answer:

- **get(Class, id)**
  - Immediately hits the DB and returns the entity or null if not found.
  - Eager fetch.
  - Example:
  - Employee emp = session.get(Employee.class, 1);

    if (emp == null) System.out.println("Not found");

- **load(Class, id)**
  - Returns a **proxy** (lazy initialization).
  - Only hits DB when a property is accessed.
  - Throws ObjectNotFoundException if entity doesn't exist.
  - Example:
  - Employee emp = session.load(Employee.class, 1);

    System.out.println(emp.getName()); // triggers query

Use get() when you're not sure if entity exists.
Use load() when you're sure the entity exists and want lazy loading.

506. What are save(), persist(), and saveOrUpdate() in Hibernate?

Answer:

1. **save(entity)**
    - Immediately assigns an identifier (returns it).
    - Works outside transaction but not recommended.
    - Example:

      ```
      int id = (int) session.save(emp);
      ```

2. **persist(entity)**
    - Doesn't return ID.
    - Follows **JPA spec**, works only inside a transaction.
    - Example:

      ```
      session.persist(emp);
      ```

3. **saveOrUpdate(entity)**
    - Saves if entity is new, updates if it already exists (based on identifier).
    - Example:

      ```
      session.saveOrUpdate(emp);
      ```

Use persist() for JPA compliance.
Use saveOrUpdate() in Hibernate-only projects when unsure if object is new or existing.

507. What is the difference between openSession() and getCurrentSession() in Hibernate?

Answer:

- **openSession()**
  - Creates a new Session every time.
  - Must close it manually.
  - Example:

    Session session = factory.openSession();

- **getCurrentSession()**
  - Returns session bound to current context (e.g., thread).
  - Auto-closes at transaction commit.
  - Requires config:

    <property

    name="hibernate.current_session_context_class">thread</property>

Use getCurrentSession() in apps with declarative transaction management (Spring).
Use openSession() when you want manual session handling.

508. What is the difference between merge() and update() in Hibernate?

Answer:

- **update(entity)**
  - Reattaches a **detached entity** to the session.
  - Fails with NonUniqueObjectException if the same entity already exists in the session.
  - Example:

    session.update(emp);

- **merge(entity)**
  - Copies the state of the detached entity into a **managed entity**.
  - Always returns the managed instance, avoids NonUniqueObjectException.
  - Example:

    Employee managedEmp = (Employee) session.merge(emp);

Use update() when you're sure entity isn't already in session.
Use merge() when you're not sure or in distributed systems.

509. What is the difference between Hibernate's Session and JPA's EntityManager?

Answer:

- **Session (Hibernate-specific)**
  - Proprietary API with Hibernate-only methods (saveOrUpdate, load, createCriteria).
  - Richer API.
- **EntityManager (JPA standard)**
  - Standardized, portable across providers (EclipseLink, OpenJPA).
  - Methods: persist, merge, remove, find.

Example:

```
// JPA way
EntityManager em = emf.createEntityManager();
em.persist(emp);

// Hibernate way
Session session = factory.openSession();
session.save(emp);
```

In modern apps (Spring Data JPA), EntityManager is recommended for portability.

510. What is the difference between SessionFactory and EntityManagerFactory?

Answer:

- **SessionFactory (Hibernate)**
  - Heavy-weight object, created once per application.
  - Produces Session objects.
  - Hibernate-specific.
- **EntityManagerFactory (JPA)**
  - JPA's equivalent of SessionFactory.
  - Produces EntityManager objects.
  - Portable across JPA providers.

If using Hibernate standalone → use SessionFactory.
If using JPA → use EntityManagerFactory.

511. What is the difference between session.load() and session.get() in Hibernate?

Answer:

- **get()**
  - Immediately hits the database and returns the entity or null if not found.
  - Example:

    ```
    Employee emp = session.get(Employee.class, 1);
    // If not found → emp = null
    ```

- **load()**
  - Returns a **proxy object** without hitting the database initially.
  - Actual query happens when a property is accessed.
  - Throws ObjectNotFoundException if entity doesn't exist.
  - Example:

    ```
    Employee emp = session.load(Employee.class, 1);
    emp.getName(); // triggers DB hit
    ```

Use get() when you need the object right away.
Use load() when lazy loading is acceptable.

512. What is the difference between session.save() and session.persist()?

Answer:

- **save() (Hibernate-specific)**
  - Returns the generated identifier immediately.
  - Can be called outside of a transaction (not recommended).
  - Example:

    ```
    Integer id = (Integer) session.save(emp);
    ```

- **persist() (JPA-standard)**
  - Does not return ID (void method).
  - Must be called within an active transaction.
  - Example:

    ```
    entityManager.persist(emp);
    ```

Use persist() in JPA-compliant code.
Use save() if you need the identifier immediately and are tied to Hibernate.

513. What is the difference between session.update() and session.saveOrUpdate()?

Answer:

- **update()**
  - Reattaches a detached entity.
  - Fails with NonUniqueObjectException if the same entity is already in the session.
- **saveOrUpdate()**
  - Either saves a new entity (if ID not present) or updates an existing one.
  - Safer in scenarios where you are not sure about entity state.

Example:

```
// update
session.update(emp);

// saveOrUpdate
session.saveOrUpdate(emp);

saveOrUpdate() is more flexible, but may do extra checks.
update() is faster if you're sure entity is detached and unique.
```

514. What is the difference between session.merge() and session.update() in Hibernate?

Answer:

- **update()**
  - Reattaches a **detached entity** to the session.
  - Throws NonUniqueObjectException if the session already has another instance with the same identifier.
- **merge()**
  - Copies the state of the detached entity into a **new managed instance** in the session.
  - Doesn't throw exception if another instance with the same ID is already present.
  - Returns the managed instance.

Example:

```
Employee detachedEmp = ...; // detached state

// update
session.update(detachedEmp);

// merge
Employee managedEmp = (Employee) session.merge(detachedEmp);
```

Use update() when you're sure the entity is not already associated with session.
Use merge() when you want a safe operation even with multiple instances.

515. What is the difference between session.flush() and session.clear()?

Answer:

- **flush()**
  - Synchronizes session state with the database (executes pending SQL statements).
  - Does not commit the transaction.
  - Example:

    session.flush(); // push changes to DB, but still in transaction

- **clear()**
  - Detaches all entities from the session (session cache is emptied).
  - Example:

    session.clear(); // session cache is wiped out

Use flush() to force immediate DB synchronization.
Use clear() to avoid memory bloat when processing large datasets.

516. What is the difference between session.close() and session.evict()?

Answer:

- **close()**
  - Terminates the session completely.
  - All entities are detached.
  - You cannot use the session afterward.
- **evict()**
  - Removes a specific object from the session cache.
  - Other objects remain managed.
  - Example:

    session.evict(emp); // only emp is detached

Use evict() for fine-grained cache control.
Use close() when you are done with the session.

517. What is the difference between Hibernate Session and JPA EntityManager?

Answer:

- **Session (Hibernate-specific):**
  - Hibernate's native API.
  - Provides methods like save(), update(), saveOrUpdate(), createCriteria().
  - Rich features beyond JPA standard (e.g., filters, scrollable results).
- **EntityManager (JPA standard):**
  - Standardized API under JPA spec.
  - Methods: persist(), merge(), remove(), find().
  - Portable across different JPA providers (Hibernate, EclipseLink, OpenJPA).

Example:

```
// JPA EntityManager
Employee emp = em.find(Employee.class, 1);

// Hibernate Session
Employee emp = session.get(Employee.class, 1);
```

EntityManager is portable; Session is more powerful but Hibernate-specific.

518. What is the difference between session.get() and session.byId().load()?

Answer:

- **get()**
  - Returns entity immediately.
  - Returns null if not found.
- **byId().load()**
  - Can return a proxy (lazy initialization).
  - Throws EntityNotFoundException if entity doesn't exist when accessed.

Example:

```
Employee e1 = session.get(Employee.class, 1);  // returns null if not found
Employee e2 = session.byId(Employee.class).load(2); // proxy, error if missing

Use get() when you need immediate fetch.
Use load() when you want proxy/lazy behavior.
```

519. What is Hibernate ScrollableResults and when should you use it?

Answer:

- ScrollableResults allows **scrolling through large result sets** without loading all rows into memory at once.
- Useful for batch processing with JDBC-style cursor navigation.

Example:

```
ScrollableResults results = session.createQuery("FROM Employee")
                        .scroll(ScrollMode.FORWARD_ONLY);

while (results.next()) {
   Employee emp = (Employee) results.get(0);
   process(emp);
   session.evict(emp); // free memory
}
```

Best for large datasets (millions of rows).
Prevents OutOfMemoryError by streaming data from DB.